

universidad de león

Universidad de León

Ingeniería del software seguro en la era de la IA generativa

Diseño y Programación Seguras

Francisco Javier Rodríguez Lera
Camino Fernández Llamas





Instituto I4

Escuela de Ingenierías Industrial e Informática

Edición revisada

Información editorial y licencia

Identificación de la obra

Título	Ingeniería del software seguro en la era de la IA generativa
Subtítulo	Diseño y Programación Seguras
Autoría	Francisco Javier Rodríguez Lera; Camino Fernández Llamas
Institución	Universidad de León — Máster Universitario de Investigación en Ciberseguridad
Edición	Edición revisada
Versión	1.3
Fecha de edición	7 de noviembre de 2026
Repositorio	fjrodl/disenyo-programacion-seguros

Licencia

CC BY-NC-ND 4.0 (Creative Commons Atribución-NoComercial-SinDerivadas 4.0 Internacional). Esta obra se distribuye bajo la licencia *Creative Commons Atribución-NoComercial-SinDerivadas 4.0 Internacional* (CC BY-NC-ND 4.0). Se permite su copia y distribución sin modificaciones, siempre que se reconozca la autoría y no se use con fines comerciales; no se permiten obras derivadas. El texto completo de la licencia está disponible en https://creativecommons.org/licenses/by-nc-nd/4.0/legalcode.es.

Historial de versiones

Versión	Fecha	Cambios principales
1.0	2026	Publicación inicial: caso técnico transversal de verificación y capítulo de confianza no calibrada.
1.1	2026-11-06	Revisión científica y ampliación: ciclo de vida, reversibilidad y verificación/aprobación/operación.
1.2	2026-11-06	Integración del caso transversal CSV-PDF-XLSX y renumeración.
1.3	2026-11-06	Corrección estructural: cada unidad como capítulo independiente.
Actual	2026-11-07	Revisión editorial: verificación bibliográfica, reducción de redundancias, integración SDD y consolidación de anexos.

Las fechas 1.1-1.3 son los commits de publicación del historial Git; la fecha de la edición actual es la de la revisión editorial, no la de generación del PDF.

Prefacio

La ingeniería del software seguro es, por naturaleza, una disciplina integradora. Un sistema moderno combina requisitos, diseño, implementación, verificación, despliegue y operación sobre un entorno cambiante, y lo hace mediante artefactos que deben ser correctos, robustos, portables y mantenibles al mismo tiempo. Sin embargo, la irrupción de la inteligencia artificial generativa ha alterado el reparto de ese trabajo: buena parte de la escritura de código, de la generación de pruebas y del andamiaje de proyecto puede delegarse en asistentes automáticos. Esa delegación no elimina las obligaciones de ingeniería; las desplaza hacia la especificación, la revisión y la validación, y añade superficie de ataque nueva en la propia cadena de herramientas.

Este libro nace con el objetivo de cubrir ese vacío desde una perspectiva clara: formar ingenieros e ingenieras capaces no solo de obtener código que funcione, sino de decidir qué es aceptable, razonar sobre las garantías que se pueden y no se pueden ofrecer, y sostener la responsabilidad profesional sobre el artefacto entregado cuando parte de él se ha generado con IA.

Sobre los contenidos teóricos

La primera parte del libro está dedicada a los fundamentos del desarrollo seguro en presencia de IA generativa. No pretende ser una enciclopedia exhaustiva ni un catálogo de herramientas, sino un recorrido estructurado por los conceptos que un ingeniero del software debe dominar para enfrentarse a sistemas reales.

Los capítulos iniciales recapitulan los principios clásicos de diseño seguro -que la IA no sustituye- y definen qué significa que un artefacto sea correcto, seguro, robusto, portable y mantenible, distinguiendo nociones que suelen confundirse. A partir de ahí se formalizan precondiciones, postcondiciones e invariantes como instrumento principal de control de calidad, y se recorren las familias clásicas de vulnerabilidad, con ejemplos vulnerables y corregidos tomados de un caso transversal común.

Los capítulos centrales abordan las dimensiones que la generación automática vuelve más delicadas: la portabilidad entre plataformas, la evaluación reproducible de los propios

sistemas de IA, y el marco regulatorio del Reglamento (UE) 2024/1689 y los espacios controlados de pruebas. Estos temas no se presentan como detalles de implementación, sino como decisiones de ingeniería con implicaciones directas en la robustez, la trazabilidad y la responsabilidad del sistema.

La confianza ocupa un lugar destacado en el libro. Un capítulo específico analiza el riesgo epistémico del código plausiblemente incorrecto: por qué la plausibilidad superficial no es evidencia de corrección y por qué la revisión debe ser independiente y deliberada. A continuación se modelan las amenazas de las acciones de un agente -comandos, red, paquetes, secretos, inyección de instrucciones y cadena de suministro- y se proponen límites de confianza y mecanismos de aislamiento técnico. El bloque teórico culmina con la verificación del código generado mediante contratos, invariantes, pruebas, análisis estático, fuzzing y verificación formal aplicados a un componente transversal, y con una metodología docente AI-SDLC que ordena el ciclo de vida completo, desde la especificación hasta la operación y la recuperación, reconociendo explícitamente sus límites.

Es importante señalar también lo que este libro no pretende cubrir en profundidad. No se abordan aquí los algoritmos internos de los modelos de lenguaje, el entrenamiento de sistemas de IA, la criptografía aplicada, la seguridad de redes o la auditoría legal exhaustiva del Reglamento europeo. Estos temas pertenecen a asignaturas específicas, a estudios de posgrado o a contextos profesionales concretos. El objetivo de este libro no es formar especialistas en cada uno de esos ámbitos, sino proporcionar una base sólida para integrarlos correctamente dentro de un ciclo de vida de software seguro.

Sobre la parte práctica

La segunda parte del libro está organizada como una progresión de prácticas de complejidad creciente, diseñada para acompañar y reforzar los conceptos teóricos. Estas prácticas no deben entenderse como ejercicios aislados ni como recetas cerradas, sino como experiencias de ingeniería con un grado de apertura creciente, todas ellas apoyadas en un mismo caso de estudio transversal.

Las primeras prácticas tienen un carácter claramente guiado: fijan el entorno de trabajo, consolidan herramientas y establecen patrones básicos de análisis de código funcional pero vulnerable, de especificación frente a instrucciones incompletas y de construcción de un *pipeline* de validación aislado. En este tramo inicial, cada enunciado ofrece una guía detallada y, cuando es necesario, una sección específica de teoría y herramientas para facilitar la implementación.

Las prácticas intermedias mantienen objetivos funcionales observables, pero abren

progresivamente el espacio de decisiones de ingeniería: se integran capacidades más complejas, se explicita la separación entre requisito, implementación y evidencia, y se comparan enfoques de validación en niveles distintos. De este modo, la teoría no se aplica de forma estrictamente lineal, sino cuando resulta necesaria para resolver problemas concretos de integración y de verificación.

En el tramo final de la parte práctica, el enfoque se vuelve más integrador y abierto: se exige ejecutar el kit AI-SDLC sobre el caso transversal y justificar una cadena completa de evidencia, combinando los elementos trabajados en el recorrido anterior. En esta etapa, el énfasis recae menos en seguir pasos prescritos y más en argumentar decisiones, validar el comportamiento global y demostrar coherencia entre diseño, implementación y verificación.

Sobre la IA generativa y el ciclo de vida

La IA generativa ocupa un papel central en este libro, no como una herramienta más ni como una promesa comercial, sino como una infraestructura que condiciona el ciclo de vida del software y obliga a repensar dónde se sitúan la especificación, la revisión y la responsabilidad. La elección de tratarla como objeto de estudio, y no solo como ayuda a la programación, responde a su adopción generalizada en investigación e industria y a su idoneidad como marco pedagógico para razonar sobre garantías de calidad y seguridad.

A lo largo del libro, la IA se utiliza como soporte concreto para ilustrar conceptos de ingeniería del software. Nociones como corrección, contrato, invariante, puerta de calidad, aislamiento, trazabilidad o reversibilidad no se presentan de forma aislada, sino siempre conectadas con los mecanismos que permiten implementarlas y comprobarlas en sistemas reales. De este modo, la IA actúa como un puente entre la teoría y los artefactos que se entregan.

Es importante subrayar que este libro no pretende enseñar una herramienta concreta como un fin en sí mismo. No se concibe como un manual de referencia exhaustivo ni como una guía paso a paso de todas las interfaces de programación de los asistentes. En su lugar, la IA generativa se presenta como un fenómeno representativo de una infraestructura moderna de desarrollo, cuyas decisiones de diseño permiten discutir ventajas, limitaciones y compromisos de ingeniería.

El enfoque adoptado permite, además, introducir de forma natural aspectos esenciales en los sistemas reales y que a menudo quedan fuera de enfoques más superficiales: la dependencia de versión y configuración, la variabilidad entre ejecuciones, la ejecución aislada, la comunicación basada en evidencia, la gestión explícita de la responsabilidad

y la robustez frente a fallos. Estos elementos no se tratan como detalles técnicos, sino como partes integrales del diseño del software seguro.

Finalmente, el enfoque adoptado busca que el lector adquiera una comprensión transferible. Aunque los ejemplos se apoyan en asistentes y herramientas concretas, los principios de ingeniería subyacentes no dependen de una herramienta determinada. El objetivo es que el estudiante sea capaz de reconocer estos principios en otros entornos, plataformas o contextos profesionales, y de aplicar el mismo criterio de diseño y verificación más allá de la IA generativa.

Alcance y filosofía del libro

Este libro está pensado para la formación de ingenieros e ingenieras en el ámbito del diseño y la programación seguras, y su alcance es deliberadamente formativo. No busca ofrecer soluciones cerradas ni imponer una arquitectura o una herramienta concreta, sino proporcionar un lenguaje común, un marco de razonamiento y un criterio técnico que permita a los futuros profesionales enfrentarse a sistemas reales con solvencia.

Si al finalizar el curso el lector es capaz de analizar un artefacto de software, comprender sus decisiones, identificar sus limitaciones, distinguir lo que está garantizado de lo que solo se ha comprobado parcialmente y proponer mejoras fundamentadas, entonces este libro habrá cumplido su objetivo.

Índice general

Prefacio iii

Lista de acrónimos y siglas xv

1. Introducción y motivación 1

1.1. La promesa y su medida . . . 1
1.2. Hipótesis rectora . . . 1
1.3. Por qué importa en Diseño y Programación Seguras . . . 2
1.4. Objetivos de aprendizaje . . . 2
1.5. Caso transversal: de CSV a PDF/Excel . . . 3
1.6. Invariante terminológico . . . 4
1.7. Estructura del libro . . . 5
1.8. Cómo leer este libro . . . 6

2. Principios clásicos y diseño seguro 7

2.1. Heurística no es ley . . . 7
2.2. KISS: mantenerlo simple . . . 8
2.3. DRY: no te repitas . . . 9
2.4. YAGNI: no lo vas a necesitar . . . 10
2.5. Ley de Kernighan . . . 11
2.6. Ley de Brooks . . . 12
2.7. Principio de Pareto . . . 13
2.8. Ley de Conway . . . 14
2.9. Número de Dunbar . . . 15
2.10. Principios clásicos de diseño seguro: Saltzer y Schroeder . . . 16
2.10.1. Mínimo privilegio (least privilege) . . . 16
2.10.2. Valores seguros por defecto (fail-safe defaults) . . . 17
2.10.3. Economía de mecanismos (economy of mechanism) . . . 17
2.10.4. Mediación completa (complete mediation) . . . 18
2.10.5. Separación de privilegios (separation of privilege) . . . 18

2.10.6. Mínimo mecanismo común (least common mechanism) 18
2.10.7. Diseño abierto (open design) 18
2.10.8. Aceptabilidad psicológica (psychological acceptability) 18
2.11. Defensa en profundidad y ausencia de garantías absolutas 19
2.12. Diseño por contrato, invariantes y responsabilidad 19
2.13. Otros principios que requieren interpretación contextual 20
2.13.1. Principio de menor sorpresa 20
2.13.2. Ley de Postel y validación estricta 20
2.13.3. Responsabilidad única y segregación de funciones 20
2.13.4. Sustitución de Liskov y contratos de seguridad 21
2.14. Matriz de decisión para proyectos asistidos por IA 21
2.15. Ejercicios adicionales: diseño seguro con agentes 21
2.16. Síntesis: qué cambia y qué permanece con la IA 22
2.17. Caso transversal: CSV frente a formatos adicionales 23
2.17.1. Dependencias 23
2.17.2. Superficie de ataque 23
2.17.3. Costes de mantenimiento 25
2.18. Ejercicio 25

3. Correctitud y seguridad **27**
3.1. Ocho nociones de «correcto» 27
3.2. Especificación: precondiciones, postcondiciones e invariantes 29
3.3. Lógica de Hoare y sus límites 30
3.4. Pasar las pruebas no es ser seguro: el caso transversal 32
3.5. Requisitos explícitos, implícitos y especificaciones incompletas 35
3.6. Síntesis 37
3.7. Ejercicio 37

4. Vulnerabilidades clásicas **39**
4.1. Cómo se lee cada caso 39
4.2. Grupo (a): memoria y enteros 41
4.2.1. A.1. Escritura sin límite de longitud: `gets` 41
4.2.2. A.2. Desbordamiento de entero con signo 43
4.3. Grupo (b): entrada, rutas, inyección y deserialización 47
4.3.1. B.1. Validación de entrada incompleta 47
4.3.2. B.2. *Path traversal* 48
4.3.3. B.3. Inyección de comandos del sistema operativo 50

4.3.4. B.4. Inyección SQL . . . 51
4.3.5. B.5. Deserialización de datos no confiables . . . 52
4.3.6. B.6. Inyección de fórmulas en CSV . . . 54
4.4. Grupo (c): estado, recursos y cadena de suministro . . . 57
4.4.1. C.1. TOCTOU entre comprobar y usar . . . 57
4.4.2. C.2. Ficheros temporales inseguros . . . 60
4.4.3. C.3. Concurrencia sin sincronización . . . 63
4.4.4. C.4. Permisos incorrectos . . . 64
4.4.5. C.5. Errores no comprobados . . . 66
4.4.6. C.6. Fugas de recursos . . . 67
4.4.7. C.7. Secretos embebidos . . . 68
4.4.8. C.8. Dependencias vulnerables o sin mantener . . . 70
4.5. El caso transversal, familia por familia . . . 71
4.6. Síntesis . . . 71
4.7. Ejercicio . . . 73

5. Portabilidad y plataformas **75**
5.1. Sistemas operativos, API y ABI . . . 76
5.2. Rutas, separadores y canonicidad . . . 76
5.3. Enlaces simbólicos y permisos . . . 79
5.4. Descriptores de fichero y *handles* . . . 80
5.5. Temporales, fin de línea y codificación . . . 81
5.6. Unicode y modelos de caracteres . . . 83
5.7. Procesos y concurrencia . . . 84
5.8. Compiladores, comportamiento indefinido y arquitectura . . . 84
5.9. Ejemplos consolidados: válido aquí, incorrecto o inseguro allí . . . 86
5.10. Plantilla de especificación de plataforma para agentes . . . 88
5.11. Enlace con File I/O: el caso transversal CSV→PDF/Excel . . . 90
5.12. Síntesis . . . 92
5.13. Ejercicio . . . 93

6. Evaluación reproducible de sistemas de IA **95**
6.1. La falacia de «gratuito contra de pago» . . . 95
6.2. Unidad de análisis y regímenes de evaluación . . . 96
6.3. Variables controladas y variables confusas . . . 98
6.4. Diseño del protocolo . . . 99
6.5. Métricas . . . 100

6.6. Tratamiento estadístico . . . 102
6.7. Amenazas a la validez . . . 104
6.8. Plantillas de registro . . . 105
6.9. Síntesis y ejercicio . . . 105

7. El Reglamento de IA (AI Act) y los espacios controlados de pruebas 107
7.1. Ámbito, enfoque basado en el riesgo y la frontera con el software convencional 107
7.2. Los artículos 57 a 60 bis: espacios controlados de pruebas y pruebas en condiciones reales . . . 109
7.2.1. Artículo 57 - Establecimiento de los espacios controlados . . . 109
7.2.2. Artículo 58 - Disposiciones detalladas y funcionamiento . . . 110
7.2.3. Artículo 59 - Tratamiento ulterior de datos personales . . . 111
7.2.4. Artículo 60 - Pruebas en condiciones reales fuera de los espacios . 112
7.2.5. Artículo 60 bis - Supuesto añadido en 2026 . . . 112
7.3. Calendario de aplicación y regímenes transitorios . . . 113
7.4. Tres nociones que no deben confundirse . . . 114
7.5. Pruebas en condiciones reales, ejecución aislada y tratamiento de datos . . 114
7.6. Análisis crítico: qué inspira la metodología docente y qué no se transfiere 116
7.7. Caso práctico de clasificación . . . 117
7.8. Preguntas de reflexión . . . 119
7.9. Referencias oficiales y criterios de lectura . . . 120

8. Confianza no calibrada: el riesgo epistémico del código generado por IA 121
8.1. Una distinción preliminar: error técnico y riesgo epistémico . . . 121
8.2. La trampa de la plausibilidad . . . 122
8.2.1. El caso transversal . . . 123
8.3. Confianza como variable continua . . . 123
8.3.1. Cobertura de especificación . . . 124
8.3.2. Independencia de verificación . . . 124
8.3.3. Reversibilidad . . . 124
8.4. El sesgo del caso positivo y su circularidad . . . 125
8.5. Independencia de contexto como propiedad de diseño . . . 127
8.6. El generador en el modelo de amenazas . . . 128
8.7. Responsabilidad epistémica . . . 130
8.8. Qué cambia y qué permanece . . . 131
8.9. Ejercicios . . . 131
8.10. Síntesis . . . 132

9. Aislamiento técnico y riesgos de los agentes **135**
9.1. Dos objetos de riesgo: el código y las acciones . . . 135
9.2. Modelo de amenazas y límites de confianza . . . 136
9.3. Superficies de acción del agente . . . 139
9.4. Procesos restringidos, contenedores, VMs y microVMs . . . 141
9.5. Controles propuestos: privilegios mínimos y confinamiento . . . 143
9.6. Ataques de ejemplo, solo a efectos pedagógicos y defensivos . . . 145
9.7. Etiquetado epistémico y límites . . . 146
9.8. Preguntas de reflexión . . . 146
9.9. Referencias y estado de verificación . . . 147

10. Pipeline de validación automática **149**
10.1. De la generación a la evidencia . . . 149
10.2. Arquitectura del pipeline . . . 150
10.3. Etapas del pipeline . . . 150
10.4. Técnicas de análisis y sus límites . . . 154
10.5. Puertas de calidad y casos de rechazo . . . 156
10.6. Integración de cambios concurrentes . . . 158
10.7. Trazabilidad: requisito, implementación, prueba y evidencia . . . 160
10.8. Dependencias y entornos reproducibles . . . 161
10.9. El límite del pipeline verde . . . 162
10.10 Etiquetado epistémico y límites . . . 163
10.11 Validación del propio pipeline . . . 163
10.12 El kit de agentes y pipelines AI-SDLC . . . 164
10.13 Preguntas de reflexión . . . 165
10.14 Referencias y estado de verificación . . . 165

11. Verificación de código generado: contratos, invariantes y modelos de amenazas **167**
11.1. El problema de la confianza delegada . . . 167
11.2. Qué significa verificar . . . 167
11.3. Especificación como precondición de la verificación . . . 170
11.4. Contratos en código generado: dónde fallan . . . 171
11.5. Invariantes como propiedades verificables . . . 173
11.6. Modelo de amenazas como instrumento de especificación . . . 174
11.6.1. Componentes mínimos de un modelo de amenazas . . . 175
11.6.2. Amenazas al producto y al proceso de generación . . . 175
11.7. Estrategias de verificación: pruebas negativas y propiedades . . . 177

11.8. Verificación independiente y criterios de aceptación . . . 179
11.9. Caso de estudio: comparación de código humano y generado por IA . . . 181
11.10Evidencias de aceptación y trazabilidad . . . 182
11.11Verificación composicional y contratos entre componentes . . . 183
11.12Validez de las pruebas y de sus oráculos . . . 184
11.13Seguridad del proceso de generación y alcance de los cambios . . . 186
11.14Verificación proporcional al riesgo y eficacia de la revisión . . . 186
11.15Ejercicios . . . 188
11.16Síntesis . . . 188

12. Metodología docente AI-SDLC **191**
12.1. Idea rectora y desplazamiento del esfuerzo . . . 191
12.2. Desarrollo guiado por especificaciones: del prompt al proceso de ingeniería 192
12.2.1. Del prompt al proceso . . . 192
12.2.2. Fundamentos . . . 192
12.2.3. SDD y agentes de IA . . . 193
12.2.4. Implementaciones . . . 193
12.2.5. Limitaciones . . . 194
12.2.6. Integración con el ciclo de vida . . . 194
12.3. Modalidades de integración de la IA en el ciclo de vida del software . . . 195
12.3.1. Autonomía de ejecución y autoridad de aprobación . . . 198
12.4. Visión general de las fases . . . 198
12.5. Fase 1: requisitos funcionales . . . 200
12.6. Fase 2: requisitos de seguridad . . . 201
12.7. Fase 3: plataforma . . . 201
12.8. Fase 4: selección del sistema de IA . . . 202
12.9. Fase 5: generación . . . 203
12.10Fase 6: revisión . . . 204
12.11Fase 7: validación . . . 204
12.12Fase 8: atestación del aislamiento . . . 205
12.13Comprobación de las puertas y del aislamiento . . . 206
12.14Fase 9: aceptación . . . 207
12.15Fase 10: mantenimiento . . . 207
12.16De la aceptación a la operación . . . 208
12.17Gobernanza . . . 210
12.18Del método a la operación: AG-00 y P00 . . . 211

12.19 Coste total conceptual . . . 212
12.20 Límites de la metodología y etiquetado . . . 213
12.21 Preguntas de reflexión . . . 213
12.22 Referencias y estado de verificación . . . 214

13. Conclusiones **215**
13.1. La hipótesis rectora, a la vista del recorrido . . . 215
13.2. Ideas clave por bloque . . . 216
13.3. Trazabilidad objetivos de aprendizaje ↔ secciones . . . 218
13.4. Ideas abiertas y limitaciones . . . 218

Bibliografía y fuentes **223**

A. Caso de estudio transversal: ingeniería de un sistema de conversión documental **227**
A.1. Contexto y motivación . . . 227
A.2. Alcance funcional . . . 228
A.3. Requisitos no funcionales . . . 229
A.4. Arquitectura de referencia . . . 230
A.5. Modelo de datos . . . 231
A.6. Contratos e invariantes . . . 231
A.7. Modelo de amenazas del caso . . . 232
A.8. Ciclo de vida del caso . . . 233
A.9. Reversibilidad del caso . . . 233
A.10. Evolución del caso a lo largo del libro . . . 234
A.11. Comparación entre desarrollo humano y asistido por IA . . . 236
A.12. Relación con el ejemplo técnico de suma comprobada (C23) . . . 237
A.13. Limitaciones del caso . . . 237
A.14. Del prompt al proceso de ingeniería: tres aproximaciones . . . 237
A.14.1. Prompt-driven development . . . 238
A.14.2. Specification-driven development . . . 238
A.14.3. Ciclo de ingeniería asistido por IA . . . 238

B. Prácticas de laboratorio **241**
B.1. Marco común y condiciones de aislamiento . . . 242
B.2. Trazabilidad práctica ↔ objetivos de aprendizaje . . . 242
B.3. Práctica 01: código funcional pero vulnerable . . . 244
B.4. Práctica 02: *prompt* incompleto frente a especificación . . . 245

B.5. Práctica 03: *pipeline* aislado de validación 246
B.6. Comprobación negativa de las puertas de seguridad 246
B.7. Práctica 04 (opcional): ejecutar el kit AI-SDLC sobre el caso transversal . 247
B.8. Práctica técnica: verificación del componente de suma comprobada 249
B.8.1. Contrato y distinción semántica 250
B.8.2. Variantes defectuosas y discriminación por pruebas 251
B.8.3. Propiedades, fuzzing y análisis 252
B.8.4. Modelo de amenazas y verificación composicional 252
B.8.5. Validez del oráculo y pruebas de mutación 253
B.8.6. Protocolo de comparación humano-IA 253
B.8.7. Trazabilidad de requisitos, propiedades y evidencias 253
B.9. Preguntas de reflexión 254

C. Plantillas de registro (material de referencia) 257
C.1. Plantillas de registro 257

Índice alfabético 263

Lista de acrónimos y siglas

ABI Application Binary Interface (interfaz binaria de aplicaciones)

ACL Access Control List (lista de control de acceso)

Agentic SDLC SDLC agéntico

AI-assisted SDLC SDLC asistido por IA

AI-native SDLC SDLC nativo de IA

AI-SDLC AI Software Development Life Cycle (ciclo de vida del desarrollo de software asistido por IA; metodología docente, no estándar oficial)

API Application Programming Interface (interfaz de programación de aplicaciones)

ARM ARM (familia de arquitecturas de procesador)

ASVS Application Security Verification Standard (de OWASP)

BOM Byte Order Mark (marca de orden de bytes)

CERT Computer Emergency Response Team (división de codificación segura del SEI)

CPU Central Processing Unit (unidad central de proceso)

CRLF Carriage Return + Line Feed (fin de línea de Windows)

CRT C Run-Time library (biblioteca de tiempo de ejecución de C)

CSV Comma-Separated Values (valores separados por comas)

CWE Common Weakness Enumeration (catálogo de debilidades de MITRE)

DAST Dynamic Application Security Testing (análisis dinámico de seguridad)

EARS Easy Approach to Requirements Syntax (notación de requisitos)

IEC International Electrotechnical Commission

IEEE Institute of Electrical and Electronics Engineers

ISO International Organization for Standardization

JSON JavaScript Object Notation (notación de objetos de JavaScript)

LF Line Feed (fin de línea de POSIX)

LLM Large Language Model (modelo grande de lenguaje)

METR Model Evaluation & Threat Research (organización de evaluación)

MITRE MITRE Corporation (organización que mantiene el catálogo CWE)

NIST National Institute of Standards and Technology

OWASP Open Worldwide Application Security Project

PDF Portable Document Format (formato de documento portátil)

POSIX Portable Operating System Interface (familia de estándares IEEE Std 1003.1)

SAST Static Application Security Testing (análisis estático de seguridad)

SBOM Software Bill of Materials (inventario de componentes de software)

SCA Software Composition Analysis (análisis de composición de software)

SDD Spec-Driven Development (desarrollo guiado por especificaciones)

SDLC Software Development Life Cycle

SEI Software Engineering Institute (Carnegie Mellon University)

SID Security Identifier (identificador de seguridad de Windows)

SSDF Secure Software Development Framework (NIST SP 800-218)

TOCTOU Time-of-Check to Time-of-Use (condición de carrera entre comprobación y uso)

UB Undefined Behavior (comportamiento indefinido)

UE Unión Europea

UTF Unicode Transformation Format (formato de transformación Unicode)

VM Virtual Machine (máquina virtual)

XLSX Office Open XML Spreadsheet (formato de hoja de cálculo de Excel)

XML Extensible Markup Language (lenguaje de marcado extensible)

YAML YAML Ain't Markup Language (formato de serialización de datos)

1. Introducción y motivación

1.1. La promesa y su medida

Los asistentes de programación basados en inteligencia artificial generativa se han incorporado al trabajo cotidiano del desarrollador: completan funciones, traducen fragmentos entre lenguajes, proponen pruebas y andamiajes de proyecto. La promesa comercial es atractiva y fácil de resumir: *escribir software cuesta menos*. Esta afirmación, sin embargo, mezcla magnitudes distintas y exige precisión. Una cosa es el coste de *producir* líneas de código y otra, muy diferente, el coste de *entregar* software correcto, robusto, seguro, portable y mantenible.

Este libro no parte de una posición a favor ni en contra de la IA generativa. Parte de una observación operativa: la herramienta no decide qué es aceptable. La responsabilidad profesional sobre el artefacto entregado sigue recayendo en el equipo humano que lo firma. Esa es, precisamente, la razón por la que un libro de *Diseño y Programación Seguras* debe ocuparse de este fenómeno.

Nota

El término *IA generativa* designa aquí sistemas que producen texto -en particular código- a partir de un modelo entrenado. En este libro no se evalúa su calidad estética ni su productividad, sino las consecuencias de incorporarlos al ciclo de vida seguro del software.

1.2. Hipótesis rectora

Hipótesis. La IA generativa **reduce ciertos costes de implementación** -escritura inicial de código, andamiaje, traducción entre lenguajes, generación de pruebas- **sin eliminar** los problemas de correctitud, seguridad, mantenimiento, portabilidad y responsabilidad. No se asume que la IA mejore ni empeore la seguridad *per se*: desplaza el esfuerzo humano desde la escritura hacia la **especificación, la revisión y la validación**, y añade superficie de ataque nueva en la cadena de herramientas.

La hipótesis es deliberadamente asimétrica: reconoce una ganancia de productividad (etiquetada como **Evidencia** parcial y contextualizada en la literatura [15, 14]; conviene precisar que el estudio de METR de 2025 halló una *ralentización* en su contexto experimental, de modo que el efecto no es universal ni uniforme) y niega que esa ganancia se traduzca automáticamente en garantías de calidad o de seguridad. El libro se organiza para poner a prueba esa asimetría en cada etapa del desarrollo.

1.3. Por qué importa en Diseño y Programación Seguras

Si la IA asume parte de la escritura, el trabajo humano se concentra en tareas que tradicionalmente se daban por supuestas: precisar qué debe hacer el sistema, revisar críticamente lo generado y demostrar -hasta donde sea posible- que se comporta como se espera. Este desplazamiento tiene dos consecuencias que estructuran el libro:

1. **El esfuerzo se mueve hacia la especificación y la validación.** Precondiciones, postcondiciones e invariantes, antes implícitos en quien escribía el código, pasan a ser el instrumento principal de control de calidad. Sin ellos, el código generado es difícil de evaluar porque no se sabe contra qué compararlo.

2. **Aparece superficie de ataque nueva.** Un asistente de código no es solo un generador de texto: cuando se le conceden herramientas -ejecución de comandos, acceso a red, escritura de ficheros, instalación de paquetes- se convierte en un *agente*. Esa capacidad introduce riesgos propios (inyección de *prompt*, exfiltración de secretos, dependencias maliciosas en la cadena de suministro) que no existen en un editor de texto convencional.

Advertencia

No ejecute código no confiable -generado por una IA o por terceros- fuera de un entorno aislado y autorizado. Los ejemplos vulnerables de este libro son material didáctico: se estudian y, si se ejecutan, se hace únicamente en un *sandbox* sin acceso a datos reales, red ni credenciales.

1.4. Objetivos de aprendizaje

Al terminar el libro, el estudiante será capaz de:

1. **Distinguir** código funcional de código correcto, robusto, seguro, portable y mantenible.

2. **Formular** precondiciones, postcondiciones e invariantes y aplicarlos a un caso de procesamiento de archivos.
3. **Identificar** por CWE [6] familias clásicas de vulnerabilidad (memoria, enteros, validación, inyección, deserialización, TOCTOU, concurrencia, secretos, dependencias) en código producido por humanos o por IA.
4. **Explicar** por qué pasar una batería finita de pruebas no demuestra seguridad.
5. **Diseñar** un protocolo reproducible para comparar sistemas de IA controlando versión, configuración, *prompt*, herramientas y entorno.
6. **Diferenciar** un sistema de IA regulado de software convencional producido con ayuda de IA, y situar el Reglamento (UE) 2024/1689 [1] sin atribuirle obligaciones inexistentes.
7. **Distinguir** sandbox regulatorio, aislamiento técnico y *pipeline* de validación de seguridad.
8. **Modelar** amenazas de las acciones de un agente (comandos, red, paquetes, secretos, *prompt injection*, cadena de suministro) y proponer límites de confianza.
9. **Diseñar** un *pipeline* de validación con puertas de calidad y trazabilidad requisito → implementación → prueba → evidencia.
10. **Aplicar** una metodología docente AI-SDLC (no estándar oficial) y **reconocer** sus límites.

1.5. Caso transversal: de CSV a PDF/Excel

Para dar continuidad a lo largo del libro se reutiliza un mismo escenario: convertir un fichero **CSV** en **PDF** y **Excel (XLSX)** con validación previa. Es un problema deliberadamente modesto, y precisamente por ello útil: cabe en una sesión, se entiende sin contexto de dominio y expone, sin adornos, casi todos los problemas que interesan. Se elige por tres razones: (i) es realista sin ser trivial; (ii) obliga a tratar a la vez entrada no confiable, salida a disco y dependencias externas; y (iii) permite comparar, sobre el mismo enunciado, código vulnerable, código corregido y código generado por IA.

La presentación completa del sistema -contexto, requisitos, arquitectura de referencia, modelo de datos, invariantes, modelo de amenazas, ciclo de vida, reversibilidad, evolución por el libro y comparación entre desarrollo humano y asistido por IA- se desarrolla en el

Anexo A (caso de estudio transversal). Esta sección 1.5 se limita a situarlo como hilo conductor del libro; los capítulos posteriores lo utilizan sin repetir su descripción general. Los objetivos pedagógicos del caso son: dar un escenario concreto y compartido a los conceptos abstractos de especificación, verificación y riesgo; permitir comparar, sobre el mismo enunciado, código vulnerable, código corregido y código generado por IA; y servir de hilo conductor entre los capítulos de principios, correctitud, vulnerabilidades, portabilidad, confianza, verificación y metodología. El anexo se ha separado del cuerpo principal para no presentar en profundidad conceptos técnicos antes de que los capítulos correspondientes los desarrollen.

1.6. Invariante terminológico

La discusión sobre IA generativa se degrada con facilidad cuando se confunden los niveles de abstracción. Este libro mantiene una distinción estricta entre los siguientes conceptos, que se usarán con este significado en todos los capítulos del libro.

Tabla 1.1.: Distinción terminológica mantenida a lo largo del libro.

Concepto	**Qué es**	**No confundir con**
Modelo	Pesos y arquitectura (p. ej., una familia de LLM)	El producto o el asistente
Versión	Identificador concreto del modelo	«El modelo» genérico
Configuración	Temperatura, límites, *system prompt*	El modelo en sí
Agente	Modelo + herramientas + bucle de decisión	Generación aislada
Herramientas	Comandos, red, escritura, paquetes	El modelo
Entorno	SO, arquitectura, contenedor, *sandbox*	El modelo o el agente

La distinción es operativa, no erudita: sin ella no es posible reproducir un experimento (¿qué versión?, ¿qué configuración?, ¿qué herramientas tenían permiso de ejecutar?) ni atribuir responsabilidades. Esta separación se apoya en marcos de desarrollo seguro reconocidos, como el NIST SSDF [3] y las recomendaciones OWASP [4, 5], que insisten en la trazabilidad de componentes y decisiones.

1.7. Estructura del libro

Cada unidad principal del libro es un capítulo independiente que se numera de forma automática (la clase *scrbook* asigna la numeración); las referencias de este listado se generan con `\ref` para que coincidan con la numeración que asigne LaTeX. Los anexos se enumeran al final.

Capítulo 2. Principios clásicos. Recapitula los principios de diseño seguro que siguen vigentes y que la IA no sustituye.

Capítulo 3. Correctitud y seguridad. Define correctitud contextual frente a seguridad y formaliza precondiciones, postcondiciones e invariantes.

Capítulo 4. Vulnerabilidades clásicas. Recorre familias de CWE con ejemplos vulnerables y corregidos del caso transversal.

Capítulo 5. Portabilidad. Analiza cómo el código generado puede ser correcto en un entorno y fallar en otro.

Capítulo 6. Evaluación de modelos. Propone un protocolo reproducible para comparar sistemas de IA controlando las variables pertinentes.

Capítulo 7. AI Act y *sandboxes*. Sitúa el Reglamento (UE) 2024/1689 y los entornos controlados, sin atribuir obligaciones no verificadas.

Capítulo 8. Confianza no calibrada. Analiza el riesgo epistémico del código plausiblemente incorrecto y por qué la plausibilidad superficial no es evidencia de corrección.

Capítulo 9. Aislamiento técnico y agentes. Modela las amenazas de las acciones de un agente y propone límites de confianza.

Capítulo 10. Validación automática. Diseña un *pipeline* con puertas de calidad y trazabilidad.

Capítulo 11. Verificación de código generado. Formaliza contratos, invariantes y modelos de amenazas, y aplica pruebas, fuzzing, análisis y verificación formal a un componente transversal.

Capítulo 12. Metodología AI-SDLC. Presenta una metodología docente, sus pasos, su extensión hacia la operación -despliegue controlado, observabilidad, recuperación y realimentación- y sus límites explícitos.

Capítulo 13. Conclusiones. Recapitula y fija las cautelas invariantes.

Bibliografía y fuentes. Recoge las referencias empleadas y su estado de verificación.

Los anexos completan el material:

Anexo A. Caso de estudio transversal. Presenta el sistema de conversión documental CSV-PDF-XLSX que recorre el libro: contexto, alcance funcional, requisitos no funcionales, arquitectura de referencia, modelo de datos, contratos e invariantes, modelo de amenazas, ciclo de vida, reversibilidad, evolución y comparación entre desarrollo humano y asistido por IA.

Anexo B. Prácticas de laboratorio. Reúne el marco de trabajo, la trazabilidad con los objetivos y las tres prácticas obligatorias junto con la actividad complementaria opcional sobre el caso transversal, con enunciado y solución.

Anexo C. Plantillas de registro. Recoge las plantillas completas de registro (ficha de sesión, hallazgos, vulnerabilidades y formato legible por máquina) para la evaluación reproducible de sistemas de IA.

1.8. Cómo leer este libro

Toda afirmación relevante se etiqueta, cuando procede, como **Evidencia** (verificable en fuentes), **Hipótesis** (conjetura razonada aún no demostrada) o **Propuesta docente** (decisión pedagógica, no resultado científico). Esta disciplina es parte del contenido: en un campo donde abundan las afirmaciones sin respaldo, distinguir lo demostrado de lo conjeturado es una competencia profesional.

Advertencia

Superar una batería finita de pruebas **no demuestra** que un programa sea seguro. Un ejemplo «corregido» en este libro es correcto de forma *contextual*: lo es respecto de las amenazas y precondiciones declaradas, y *nunca* «universalmente seguro». Esta cautela se repite a lo largo de todo el libro.

2. Principios clásicos y diseño seguro

2.1. Heurística no es ley

La ingeniería del software acumula un repertorio de máximas que se repiten como si fuesen leyes físicas: «mantenlo simple», «no te repitas», «no lo vas a necesitar». Conviene empezar con una cautela terminológica. Casi ninguna de ellas es una ley en el sentido de la física -una regularidad universal con condiciones de validez explícitas- sino una **heurística**: una regla de decisión que suele dar buenos resultados en un rango de contextos y que admite excepciones. Otras son **observaciones** de tipo empírico o sociotécnico, recogidas en la práctica y no demostradas como teoremas. Confundir ambas categorías lleva a dos errores simétricos: descartar una heurística porque «tiene excepciones» o invocarla como argumento de autoridad incontestable.

Nota

En esta sección, cada principio se analiza con un esquema fijo: *formulación*, *naturaleza*, *problema original*, *supuestos*, *cambios con IA*, *invariantes*, *riesgos de seguridad* y *ejemplo*. El objetivo no es memorizar máximas, sino aprender a usarlas como herramientas falibles: saber *cuándo* aplican, *qué* asumen y *cómo* pueden fallar.

La pregunta que recorre la sección es la de la hipótesis rectora del capítulo: si la IA generativa reduce el coste de escribir código, ¿qué ocurre con estas heurísticas? La formulación de esta pregunta se inspira en un artículo de divulgación que plantea precisamente si «siguen vigentes las leyes elementales de ingeniería» [26]; este capítulo desarrolla esa intuición con el análisis sistemático de cada principio y su etiquetado epistémico. La respuesta que se defenderá es doble. Por un lado, **no las sustituye**: siguen describiendo tensiones reales del diseño. Por otro, **cambia su economía**: abarata unas acciones (escribir, duplicar, añadir) y encarece otras (revisar, verificar, comprender), de modo que su aplicación descuidada puede invertirse.

La columna *estatus* no pretende zanjar debates: etiqueta el grado de respaldo con el que se presenta cada enunciado en este material. Se recuerda que «Hipótesis» no significa

Tabla 2.1.: Naturaleza y ámbito de los principios analizados.

Principio	**Naturaleza**	**Ámbito**	**Estatus**
KISS	Heurística de diseño	Complejidad estructural	Propuesta docente
DRY	Heurística de diseño	Representación del conocimiento	Propuesta docente
YAGNI	Heurística de alcance	Gestión de requisitos	Propuesta docente
Kernighan	Observación (aforismo)	Comprensión del código	Hipótesis
Brooks	Observación empírica	Gestión de proyectos	Evidencia parcial
Pareto	Heurística estadística	Priorización de esfuerzo	Hipótesis
Conway	Observación sociotécnica	Arquitectura y organización	Hipótesis
Dunbar	Hipótesis científica	Tamaño de redes sociales	Hipótesis discutida

«falso», sino «no demostrado»; y que «Propuesta docente» señala una decisión pedagógica, no un resultado científico.

2.2. KISS: mantenerlo simple

Formulación. Prefiere la solución más sencilla que satisfaga el requisito, antes que una solución más general, más abstracta o más «lista» de lo necesario.

Naturaleza. Heurística de diseño. No es un teorema: existen dominios donde la complejidad es intrínseca y no puede reducirse sin perder corrección.

Problema original. La *complejidad accidental* -la que no proviene del problema, sino de cómo se resuelve- encarece el mantenimiento y oculta defectos. La máxima se atribuye a entornos de ingeniería aeronáutica; su atribución exacta debe verificarse antes de citarse formalmente.

Supuestos. (i) El coste dominante del software es el de comprenderlo y modificarlo, no el de escribirlo por primera vez. (ii) La complejidad innecesaria es evitable sin sacrificar requisitos.

Cambios con IA. La IA reduce el coste de *escribir*, lo que rebaja también el coste aparente de *añadir* complejidad. El riesgo se invierte: es fácil generar más código del necesario porque generarlo «no cuesta». La simplicidad deja de ser una restricción económica impuesta por el esfuerzo y pasa a ser una disciplina que hay que imponer deliberadamente.

Invariante. Toda línea de código es también una línea que alguien deberá comprender, revisar y asegurar. La simplicidad es una propiedad de seguridad, no solo de estilo: menos complejidad innecesaria puede reducir oportunidades de fallo, pero el número de líneas no mide por sí solo la seguridad.

Riesgos de seguridad. La sobreingeniería amplía la superficie de ataque y el número de dependencias. Un módulo «genérico» o «extensible» generado sin necesidad puede incorporar rutas de ejecución que nadie audita. La complejidad también favorece el *fallo silencioso*: ramas que nunca se prueban y que un atacante sí alcanza.

Ejemplo. En el caso transversal, KISS aconseja leer el CSV, validar y generar solo los formatos exigidos. Añadir «por si acaso» un intérprete de fórmulas o un deserializador de formatos adicionales aumenta el riesgo sin aportar valor al requisito.

Consejo

Regla práctica para el aula: antes de aceptar código generado por IA, pregúntese qué parte del mismo podría eliminarse sin incumplir un requisito. Si existen partes prescindibles, el código puede estar sobredimensionado; la ausencia de partes prescindibles no demuestra simplicidad ni seguridad.

2.3. DRY: no te repitas

Formulación. Cada pieza de conocimiento del sistema debe tener una representación única, inequívoca y autoritativa.

Naturaleza. Heurística de diseño sobre la *representación del conocimiento*, popularizada en la literatura de programación pragmática. No prohíbe la duplicación de *código* por sí misma, sino la duplicación de *decisiones*.

Problema original. Cuando una misma regla se copia en varios lugares, las copias divergen con el tiempo: se corrige una y se olvidan las demás. El resultado son defectos intermitentes y costes de mantenimiento multiplicados.

Supuestos. (i) Existe una fuente de verdad identificable. (ii) El coste de abstraer y mantener un punto único es menor que el de mantener copias divergentes. (iii) La abstracción no introduce un acoplamiento peor que la duplicación.

Cambios con IA. Los asistentes replican patrones con enorme facilidad: dado un fragmento, tienden a reproducirlo. Esto agrava el riesgo de duplicar lógica de validación en varios puntos del programa. Al mismo tiempo, la IA puede proponer abstracciones prematuras que ocultan el flujo de datos. El resultado es que DRY requiere más criterio humano, no menos.

Invariante. Las reglas de validación y de seguridad deben residir en un único punto autoritativo. Si la canonicalización de rutas o el escapado de fórmulas se duplican, tarde o temprano divergirán, y la copia más débil determinará la seguridad del conjunto.

Riesgos de seguridad. La duplicación de controles es una causa clásica de *validación inconsistente*: una ruta se canonicaliza en un sitio y no en otro; una entrada se escapa en una ruta de código y no en su gemela. Además, una abstracción mal diseñada puede concentrar el fallo: un único componente defectuoso afecta a todos los llamantes.

Ejemplo. El escapado de fórmulas de Excel debe implementarse una sola vez y reutilizarse. Duplicarlo en el generador de XLSX y en una función de exportación secundaria conduce a que una de las dos quede sin proteger.

2.4. YAGNI: no lo vas a necesitar

Formulación. No implementes funcionalidad hasta que exista una necesidad real y justificada; en particular, no la implementes por especulación sobre requisitos futuros.

Naturaleza. Heurística de gestión de alcance, asociada a las prácticas de desarrollo ágil y a la programación extrema. Es una regla de priorización, no una ley técnica.

Problema original. La funcionalidad especulativa se construye, se documenta, se prueba y se mantiene antes de que nadie la necesite; con frecuencia nunca se usa y su coste se paga durante toda la vida del sistema.

Supuestos. (i) Los requisitos cambian y no pueden preverse con fiabilidad. (ii) El coste de añadir funcionalidad más tarde es aceptable frente al de mantenerla desde el principio. (iii) Añadir después no degrada la arquitectura de forma irreversible.

Cambios con IA. La IA puede generar cualquier funcionalidad «de reserva» con muy poco esfuerzo aparente, lo que erosiona la disciplina de YAGNI. El peligro es que el andamiaje especulativo -interfaces, formatos, modos de compatibilidad- entre en el sistema sin un requisito que lo respalde.

Invariante. Cada funcionalidad debe poder trazarse a un requisito explícito. Lo que no tiene requisito no tiene usuario, y lo que no tiene usuario no tiene quien lo mantenga ni quien lo asegure.

Riesgos de seguridad. La funcionalidad no solicitada suele ser la menos revisada. Un deserializador añadido «por compatibilidad», un modo de depuración «por si acaso» o un parámetro extra «por flexibilidad» son vectores habituales. En el caso transversal, aceptar formatos adicionales sin requisito equivale a incorporar analizadores nuevos y complejos con su propia superficie de ataque.

Ejemplo. Si el requisito solo menciona CSV como entrada, no se debe implementar carga de PDF o Excel «para el futuro». Cada formato adicional es una dependencia, un analizador y un conjunto de vulnerabilidades potenciales que nadie pidió.

Advertencia

YAGNI no autoriza a ignorar requisitos de seguridad. Los controles de seguridad -límites de tamaño y tiempo, canonicalización de rutas, escapado de salidas- no son funcionalidad especulativa: son condiciones del requisito. La máxima se aplica a las funciones que *nadie ha pedido*, no a las que *la seguridad exige*.

2.5. Ley de Kernighan

Formulación. Depurar es el doble de difícil que escribir código; por tanto, si escribes el código tan hábilmente como puedes, por definición no eres lo bastante hábil para depurarlo.

Naturaleza. Observación expresada como aforismo. Resume una asimetría cognitiva percibida, pero no se apoya en una medición sistemática; aquí se etiqueta como **Hipótesis**.

Problema original. El código «demasiado listo» -compacto, con trucos y optimizaciones prematuras- es difícil de entender y, por ello, difícil de depurar y de auditar. La dificultad de escritura se paga con creces al mantener.

Supuestos. (i) Existe una asimetría entre comprender y producir código. (ii) El autor del código es también quien lo depura, o al menos comparte su modelo mental.

Cambios con IA. La asimetría se agrava. El «escritor» es ahora un modelo que no depura y que no comparte contexto con el revisor humano. El desarrollador se enfrenta a código que no escribió y que puede ser más «listo» de lo que él mismo habría producido. La comprensión del código deja de ser una consecuencia de haberlo escrito y se convierte en una tarea explícita.

Invariante. El código debe ser comprensible por la persona que habrá de mantenerlo, que puede no ser quien lo generó. La comprensibilidad es requisito de corrección y de seguridad, no un lujo.

Riesgos de seguridad. El código ofuscado o excesivamente compacto oculta fallos: un desbordamiento, una comprobación ausente o una rama inalcanzable pueden pasar inadvertidos en una revisión superficial. La dificultad de comprensión reduce la eficacia de las revisiones de seguridad.

Ejemplo. Una expresión generada que encadena varias operaciones en una sola línea puede ocultar que falta una comprobación de longitud o de rango. Reescrita en pasos explícitos, el fallo se vuelve visible.

2.6. Ley de Brooks

Formulación. Añadir personal a un proyecto de software atrasado lo retrasa aún más. En su forma ampliada, la complejidad de un sistema se divide en *esencial* (inherente al problema) y *accidental* (propia de las herramientas y del proceso), y ninguna tecnología elimina la primera.

Naturaleza. Observación empírica de gestión de proyectos, expuesta en la literatura clásica de ingeniería del software. Se presenta como **Evidencia parcial**: describe patrones observados, no una ley cuantitativa.

Problema original. La planificación de proyectos grandes ignora los costes de comunicación y de integración. Añadir personas añade canales de comunicación y tiempo de formación, que pueden superar la capacidad productiva añadida.

Supuestos. (i) El coste de comunicación crece más que linealmente con el número de participantes. (ii) Las tareas no son perfectamente paralelizables. (iii) Parte de la complejidad es irreducible.

Cambios con IA. La IA puede reducir la *complejidad accidental* (andamiaje, traducción, código repetitivo), pero no la *esencial* del dominio. No es una «bala de plata»: no convierte un problema difícil en trivial. Además introduce costes nuevos -verificación, revisión, gestión de la confianza en las herramientas y dependencia de proveedores- que no aparecen en el balance simplista «escribir cuesta menos».

Invariante. La complejidad esencial del problema -aquí, validar entradas no confiables, manejar formatos y asegurar salidas- permanece aunque se genere más código. La IA no la elimina; en el mejor caso, la hace más manejable.

Riesgos de seguridad. Si se confía en la IA como solución universal, la revisión humana se relaja precisamente donde más se necesita. La dependencia de un proveedor o de una versión concreta del modelo añade riesgo de cadena de suministro y de reproducibilidad.

Ejemplo. El caso transversal no se vuelve seguro porque se generen más funciones. La validación del CSV, la prevención de *path traversal* y el escapado de fórmulas siguen siendo trabajo esencial que alguien debe especificar y verificar.

2.7. Principio de Pareto

Formulación. Una fracción reducida de las causas produce la mayor parte de los efectos; en su forma popular, «el 80 % de los resultados proviene del 20 % de las causas».

Naturaleza. Heurística estadística con origen en el estudio de distribuciones económicas. La proporción concreta 80/20 es ilustrativa, no una constante universal; aquí se etiqueta como **Hipótesis** aplicada al software.

Problema original. La asignación de esfuerzo en contextos con recursos limitados: ¿dónde concentrar la atención para obtener el mayor beneficio? El principio responde «en la minoría crítica».

Supuestos. (i) La distribución de causas y efectos es desigual y aproximadamente estable en el dominio considerado. (ii) Existe una forma de medir y ordenar por impacto. (iii) El subconjunto crítico puede identificarse antes de invertir el esfuerzo.

Cambios con IA. La IA acelera la producción del «80» trivial -código repetitivo, andamiaje, variaciones de un patrón- pero el «20» crítico -casos límite, condiciones de carrera, validación de entradas hostiles- sigue requiriendo análisis humano. El riesgo es que el volumen generado dé una falsa sensación de progreso mientras el subconjunto crítico permanece sin atender.

Invariante. No debe confundirse el volumen de código con el valor entregado. El esfuerzo de seguridad debe dirigirse al subconjunto de causas que concentra el riesgo.

Riesgos de seguridad. Priorizar por volumen o por facilidad desplaza la atención hacia lo trivial y descuida los casos límite, que son justamente los que un atacante explota. Las rutas de error poco frecuentes suelen ser las más vulnerables.

Ejemplo. Un pequeño porcentaje de las entradas malformadas -comillas, separadores incrustados, fórmulas iniciadas por =- provoca la mayoría de los fallos de procesamiento e inyección. Ese subconjunto concentra el esfuerzo de validación.

2.8. Ley de Conway

Formulación. Las organizaciones tienden a producir sistemas cuyo diseño copia su propia estructura de comunicación: las fronteras entre módulos reflejan las fronteras entre equipos.

Naturaleza. Observación sociotécnica. Describe una tendencia organizativa, no una ley determinista; se etiqueta como **Hipótesis** en el marco de este capítulo.

Problema original. Explicar por qué la arquitectura de un sistema reproduce, con frecuencia, la estructura de la organización que lo construye, y por qué los problemas de comunicación se traducen en problemas de diseño.

Supuestos. (i) La arquitectura emerge, al menos en parte, de los canales de comunicación efectivos. (ii) Las fronteras de coordinación tienden a convertirse en fronteras de interfaz.

Cambios con IA. La IA introduce un nuevo tipo de «participante» en el flujo de trabajo: asistentes y agentes que producen código y, a veces, lo ejecutan. Si la arquitectura refleja la estructura de comunicación, cabe preguntarse cómo se refleja una organización que incluye herramientas automatizadas. Es una pregunta abierta; se formula aquí como **Hipótesis**, sin respuesta establecida.

Invariante. La arquitectura de seguridad debe reflejar las fronteras de confianza y de responsabilidad. Si los módulos se separan según quién los escribe, las fronteras de confianza pueden no coincidir con las fronteras organizativas, y la responsabilidad se difumina.

Riesgos de seguridad. Cuando las fronteras de confianza no están alineadas con los canales de comunicación, los controles se colocan en el sitio equivocado: una validación se hace en un componente porque «lo escribió el equipo de validación», no porque ahí resida el límite de confianza. La responsabilidad de revisar código generado puede quedar sin dueño claro.

Ejemplo. Si el equipo separa conceptualmente «validación» y «generación», el sistema tendrá dos módulos y una frontera clara entre ellos. Si la IA genera ambos sin que nadie defina la frontera, la validación puede quedar diluida dentro de la generación y desaparecer como control independiente.

2.9. Número de Dunbar

Formulación. Se ha propuesto que existe un límite cognitivo al número de relaciones sociales estables que un individuo puede mantener simultáneamente, con estimaciones en torno a un centenar y medio. Se conoce como «número de Dunbar».

Naturaleza. Hipótesis científica discutida, derivada de correlaciones entre el tamaño relativo del neocórtex y el tamaño de grupo en primates. **No debe extrapolarse sin reservas** a equipos de ingeniería ni a organizaciones humanas contemporáneas.

Problema original. Comprender los límites de la cohesión social en grupos de primates y su posible relación con el tamaño del cerebro.

Supuestos. (i) Existe una correlación entre capacidad cognitiva y tamaño de grupo. (ii) Tal correlación es extrapolable a humanos modernos. (iii) El número concreto tiene significado operativo fuera del contexto de estudio. Los tres supuestos son controvertidos y no deben darse por sentados.

Cambios con IA. Podría conjeturarse que las herramientas de mediación y los agentes alteran los límites de coordinación. Se trata de una especulación: no hay evidencia sólida que permita afirmar que la IA modifica el «número de Dunbar» ni que este número tenga relevancia directa para dimensionar equipos de desarrollo.

Invariante. No debe usarse el número de Dunbar como norma de diseño organizativo ni como justificación de decisiones de equipo. Su valor pedagógico es negativo: sirve como ejemplo de hipótesis atractiva pero insegura que no conviene convertir en regla.

Riesgos de seguridad. El riesgo aquí es de razonamiento, no de código: invocar un número no verificado para justificar una estructura de equipo puede ocultar problemas reales de revisión y de responsabilidad. Un equipo de revisión de seguridad mal dimensionado -por exceso o por defecto- degrada la calidad del control.

Ejemplo. No debe argumentarse que «un equipo de revisión de seguridad no debe superar las 150 personas» apelando al número de Dunbar. Si se necesita justificar el tamaño de un equipo, deben usarse criterios de carga de trabajo, competencias y trazabilidad, no una constante biológica dudosa.

Nota

El número de Dunbar ilustra una lección metodológica central del capítulo: una hipótesis popular no se convierte en evidencia por repetirse. El mismo criterio que exige verificar una CWE [6] o una edición de un estándar [3] se aplica a las máximas de gestión: hay que preguntar por las condiciones de validez.

2.10. Principios clásicos de diseño seguro: Saltzer y Schroeder

Las heurísticas de mantenimiento y organización anteriores son útiles, pero no sustituyen los principios concebidos específicamente para diseñar mecanismos de protección. Saltzer y Schroeder formularon en 1975 un conjunto de principios de diseño de sistemas de protección que sigue siendo una referencia histórica *de ingeniería*, no un catálogo de obligaciones legales. Véase Saltzer y Schroeder, *The Protection of Information in Computer Systems* (1975). La vigencia de cada principio debe analizarse frente al modelo de amenazas, las capacidades de la plataforma y los requisitos actuales.

2.10.1. Mínimo privilegio (least privilege)

Formulación. Cada proceso, usuario o componente debe operar con el conjunto mínimo de permisos necesario para realizar su función, durante el tiempo estrictamente necesario.

Problema. Una vulnerabilidad local puede convertirse en un compromiso general si el componente tiene acceso a archivos, credenciales o servicios que no necesita.

Aplicación con IA. Un agente de programación que solo debe proponer cambios no necesita, por defecto, permisos para ejecutar comandos, publicar paquetes o acceder a secretos de producción. Deben separarse generación, revisión y ejecución autorizada.

Límite. Privilegios insuficientes pueden impedir una función legítima; el conjunto mínimo se determina a partir de requisitos y amenazas, no por eliminación indiscriminada de permisos.

Ejemplo. El conversor CSV-PDF-XLSX recibe acceso de lectura a un directorio de entrada y escritura a un directorio de salida específico, no acceso de escritura al árbol completo del proyecto.

2.10.2. Valores seguros por defecto (fail-safe defaults)

Formulación. La decisión de permitir una operación debe basarse en una autorización explícita; lo no autorizado se deniega por defecto.

Aplicación con IA. Ante un parámetro de seguridad omitido por un generador, la implementación no debe inferir que cualquier recurso o destino es admisible.

Contraejemplo. Un servicio que permite exportar a cualquier ruta cuando falta la configuración de directorio de salida convierte un error de configuración en una autorización implícita.

Límite. Denegar por defecto no equivale a ignorar errores ni a detener arbitrariamente un sistema crítico: debe definirse un estado seguro acorde con la función y la disponibilidad requerida.

2.10.3. Economía de mecanismos (economy of mechanism)

Este principio recomienda que los mecanismos de protección sean tan pequeños y simples como resulte razonable. Se relaciona con KISS, pero su objeto es más preciso: facilitar la comprensión y revisión de la *base de confianza*. Una solución criptográfica improvisada de veinte líneas no es preferible a una biblioteca consolidada solo por ser más corta. La complejidad pertinente incluye dependencias, configuración, garantías y posibilidades de uso incorrecto.

2.10.4. Mediación completa (complete mediation)

Todo acceso a un objeto protegido debe someterse a las comprobaciones de autorización pertinentes. Comprobar un permiso al iniciar una sesión y asumir que sigue siendo válido para cualquier operación posterior puede ser insuficiente. En el caso transversal, validar una ruta de salida una vez no resuelve por sí mismo carreras entre comprobación y uso (TOCTOU): deben elegirse primitivas de acceso y políticas que mantengan la garantía durante la operación. La aplicación concreta requiere estudiar el sistema de archivos y la plataforma.

2.10.5. Separación de privilegios (separation of privilege)

Una operación sensible puede requerir dos condiciones independientes, en lugar de confiar en un único permiso o decisión. Por ejemplo, publicar un artefacto generado por un agente puede requerir que las pruebas de seguridad se hayan ejecutado y que una persona autorizada apruebe la publicación. No debe confundirse este principio con la separación de responsabilidades del diseño orientado a objetos: ambos pueden complementarse, pero resuelven problemas distintos.

2.10.6. Mínimo mecanismo común (least common mechanism)

Conviene reducir los mecanismos compartidos entre usuarios o componentes con diferentes niveles de confianza. Un agente que trabaja con código no confiable no debería compartir credenciales, carpetas temporales sensibles o estados persistentes con un proceso de producción. Aislar entornos reduce ciertas vías de interferencia, pero no elimina por sí solo las vulnerabilidades del sistema anfitrión.

2.10.7. Diseño abierto (open design)

La seguridad de un mecanismo no debe depender del secreto de su diseño. Los secretos operativos, como claves, sí deben protegerse. La revisión pública o independiente de una implementación puede ayudar a descubrir defectos, pero la disponibilidad del código no constituye por sí misma una prueba de seguridad. Esta idea se relaciona con el principio de Kerckhoffs en criptografía.

2.10.8. Aceptabilidad psicológica (psychological acceptability)

Los controles deben ser comprensibles y utilizables para que las personas puedan aplicarlos correctamente. Un flujo de aprobación de agentes que presenta decenas de

solicitudes indistinguibles favorece la aceptación mecánica. La interfaz debe explicar qué operación se autoriza, sobre qué recurso y con qué consecuencias.

Nota

Lectura conjunta. Los principios de Saltzer y Schroeder no son una lista de verificaciones que certifique la seguridad. Su utilidad consiste en hacer explícitas decisiones de arquitectura y de control de acceso, para poder discutirlas y someterlas a pruebas.

2.11. Defensa en profundidad y ausencia de garantías absolutas

La *defensa en profundidad* combina controles con modos de fallo diferentes: validación de entradas, privilegios mínimos, aislamiento, revisión, análisis estático, pruebas dinámicas, monitorización y procedimientos de recuperación. No equivale a multiplicar controles redundantes sin justificación. Dos verificadores que comparten el mismo supuesto equivocado pueden fallar simultáneamente.

Aplicación con IA. El código generado se trata como código no verificado, aunque compile y supere pruebas de ejemplo. Las acciones del agente también necesitan controles independientes.

Ejemplo. El conversor valida el esquema CSV, limita tamaño y recursos, restringe rutas de salida y comprueba que el documento producido no contiene fórmulas interpretables en las aplicaciones de destino previstas.

Contraejemplo. Un pipeline con diez pruebas positivas no demuestra que una entrada maliciosa sea rechazada; se necesitan pruebas negativas y un modelo explícito de amenazas.

2.12. Diseño por contrato, invariantes y responsabilidad

El *diseño por contrato* formula las obligaciones de un componente mediante precondiciones, poscondiciones e invariantes. Se relaciona con la lógica de Hoare desarrollada en la capítulo 3 (correctitud y seguridad). Aquí interesa su consecuencia práctica: una implementación generada por IA no puede evaluarse sin una especificación de lo que debe hacer y de lo que debe impedir.

Precondición. La entrada CSV satisface un límite de tamaño, un esquema declarado y una codificación aceptada.

Poscondición. El archivo de salida representa los registros admitidos sin interpretar contenido de usuario como fórmulas.

Invariante. Ninguna operación escribe fuera del directorio autorizado, incluso ante errores y ejecuciones concurrentes.

Estas propiedades no se obtienen automáticamente al redactar un contrato: deben implementarse y verificarse bajo supuestos explícitos. Una poscondición informal tampoco sustituye una prueba formal.

2.13. Otros principios que requieren interpretación contextual

2.13.1. Principio de menor sorpresa

Una interfaz debe comportarse de manera coherente con las expectativas razonables de sus usuarios y con su contrato documentado. En código generado por IA es especialmente relevante revisar opciones por defecto, efectos secundarios y funcionalidades añadidas sin petición. No debe confundirse *predecible* con *seguro*: una interfaz consistentemente insegura sigue siendo insegura.

2.13.2. Ley de Postel y validación estricta

La máxima tradicional de ser conservador al enviar y liberal al recibir puede facilitar interoperabilidad, pero una aceptación excesivamente permisiva de entradas ambiguas crea discrepancias entre analizadores y superficies de ataque. En límites de confianza, es preferible definir formatos inequívocos, rechazar entradas fuera de especificación y normalizar solo cuando la transformación esté bien definida. La máxima no debe utilizarse como justificación para aceptar cualquier entrada proporcionada por el usuario.

2.13.3. Responsabilidad única y segregación de funciones

El principio de responsabilidad única (SRP) ayuda a separar lectura, validación, transformación y escritura. Sin embargo, no obliga a crear una clase por cada operación ni sustituye las políticas de acceso. Una función de validación centralizada puede facilitar DRY, pero si implementa incorrectamente una regla de autorización, el error se propagará a todos sus consumidores. La centralización requiere pruebas específicas del componente compartido.

2.13.4. Sustitución de Liskov y contratos de seguridad

Sustituir un componente por otro que promete la misma interfaz no es seguro si debilita las poscondiciones, endurece indebidamente precondiciones o cambia garantías de confidencialidad y acceso. Un exportador alternativo generado por IA debe conservar las propiedades relevantes del contrato, no solo el tipo de retorno. Este principio se estudia aquí como aplicación del razonamiento por contratos, no como regla suficiente de seguridad.

2.14. Matriz de decisión para proyectos asistidos por IA

Tabla 2.2.: Preguntas de revisión asociadas a los principios adicionales.

Principio	**Pregunta verificable para el revisor**
Mínimo privilegio	¿Qué recursos puede leer, modificar o ejecutar el agente o programa?
Valores seguros por defecto	¿Qué ocurre si falta un permiso o una configuración?
Mediación completa	¿Se verifica cada operación sensible y se evitan carreras entre comprobación y uso?
Separación de privilegios	¿Qué acciones requieren aprobación o condiciones independientes?
Economía de mecanismos	¿Es revisable la base de confianza y están justificadas las dependencias?
Defensa en profundidad	¿Qué control sigue operando si otro falla?
Diseño por contrato	¿Qué precondiciones, poscondiciones e invariantes se comprueban?
Menor sorpresa	¿Existen efectos secundarios o capacidades no solicitadas?

2.15. Ejercicios adicionales: diseño seguro con agentes

1. Diseñe una política de permisos para un agente que modifica el conversor CSV-PDF-XLSX. Distinga lectura, escritura, red y ejecución.

2. Proponga una entrada que supere una validación inicial de ruta pero provoque un problema TOCTOU. Explique por qué la mediación completa no equivale a comprobar una cadena de texto una sola vez.

3. Defina tres propiedades de seguridad como contratos y proponga pruebas negativas para cada una. Indique qué queda sin demostrar.

4. Compare un diseño con una biblioteca consolidada y otro con criptografía propia generada por IA. Razone por qué menos líneas no implica necesariamente menor riesgo.

5. Identifique un conflicto entre DRY, separación de privilegios y responsabilidad única. Proponga una solución y explicite sus costes.

2.16. Síntesis: qué cambia y qué permanece con la IA

Las heurísticas anteriores describen tensiones que la IA generativa no crea ni resuelve, pero cuya economía modifica. La tabla 2.3 resume el desplazamiento.

Tabla 2.3.: Efecto de la IA generativa sobre cada principio.

Principio	**Lo que abarata**	**Lo que encarece o exige**
KISS	Escribir y añadir código	Imponer simplicidad deliberadamente
DRY	Reproducir patrones	Detectar y evitar duplicación de controles
YAGNI	Generar funcionalidad «de reserva»	Justificar cada funcionalidad por un requisito
Kernighan	Producir código «listo»	Comprender código ajeno antes de revisarlo
Brooks	Reducir complejidad accidental	Gestionar verificación y confianza; complejidad esencial
Pareto	Producir el «80» trivial	Atender el «20» crítico de casos límite
Conway	Generar módulos	Definir fronteras de confianza y responsabilidad
Dunbar	- (sin efecto claro)	No invocar hipótesis no verificadas

La lectura conjunta sugiere una asimetría coherente con la hipótesis rectora: la IA puede abaratar lo que se *escribe*, sin reducir proporcionalmente el esfuerzo de *comprender, especificar y verificar*. Las heurísticas siguen siendo útiles, pero su punto de aplicación se desplaza del acto de escribir al de decidir y validar.

2.17. Caso transversal: CSV frente a formatos adicionales

El caso del capítulo -convertir un CSV en PDF y XLSX con validación previa (Anexo A)- permite aplicar los principios a una decisión concreta: ¿cuántos formatos debe soportar el programa? La respuesta ingenua («cuantos más, mejor») contradice KISS y YAGNI; la respuesta descuidada («solo el mínimo») puede incumplir requisitos reales. El análisis debe separar tres dimensiones: **dependencias**, **superficie de ataque** y **costes de mantenimiento**.

2.17.1. Dependencias

Cada formato añade bibliotecas de terceros y, con ellas, versiones, licencias y vulnerabilidades potenciales. Un CSV se puede procesar con la biblioteca estándar del lenguaje, lo que reduce al mínimo la cadena de suministro. Generar PDF y XLSX suele requerir dependencias externas específicas de cada formato.

Tabla 2.4.: Comparación de alcance en el caso transversal.

Dimensión	**Solo CSV (entrada y salida)**	**CSV + PDF + XLSX**
Dependencias	Biblioteca estándar; cadena mínima	Varias bibliotecas externas; versiones y licencias
Superficie de ataque	Analizador sencillo, formato de texto	Analizadores complejos; formatos comprimidos y estructurados
Validación	Comillas, separadores, codificación	Además: fórmulas, XML, archivos comprimidos
Mantenimiento	Bajo; pocas actualizaciones	Alto; actualizaciones y compatibilidad por formato
Riesgo principal	Inyección en la salida CSV	Inyección de fórmulas, deserialización, *path traversal*
Beneficio	Cubre el requisito mínimo	Cubre requisitos de presentación y hojas de cálculo

2.17.2. Superficie de ataque

Los formatos ricos amplían la superficie de ataque de varias formas concretas:

- **Inyección de fórmulas.** Un CSV cuyos campos comienzan por `=`, `+`, `-` o `@` puede ejecutarse como fórmula al abrirse en una hoja de cálculo. Es una familia de

inyección reconocida en el catálogo CWE [6] (CWE-1236); no debe atribuirse sin más al OWASP Top 10 para aplicaciones web.

- **Deserialización de formatos estructurados.** Los formatos basados en XML (por ejemplo, ciertos documentos y hojas) pueden exponer entidades externas si el analizador no se configura de forma restrictiva. Las reglas de codificación segura de CERT [7] insisten en validar y limitar las entradas antes de procesarlas.

- **Archivos comprimidos.** Los formatos empaquetados en un contenedor comprimido pueden dar lugar a expansiones desproporcionadas (*zip bomb*) si no se imponen límites de tamaño y de tiempo.

- **Rutas de salida.** Escribir PDF o XLSX con nombres derivados de la entrada abre la puerta al *path traversal* y a condiciones de carrera TOCTOU entre validar y abrir. Las primitivas de acceso seguras, la restricción de permisos y la gestión cuidadosa de temporales deben seleccionarse según la plataforma y el modelo de amenazas.

La neutralización de prefijos (`=`, `+`, `-`, `@`) es una medida contextual: también deben considerarse TAB, CR y espacios iniciales, y el comportamiento depende del programa que abra el fichero. El identificador preciso es CWE-1236 [6].

Listing 2.1: Escapado de fórmulas antes de exportar a hoja de cálculo (ejemplo didáctico).

```
# Ejemplo didáctico: neutraliza la interpretación de formulas en hojas de calculo.
# La correccion es CONTEXTUAL: no demuestra seguridad frente a todas las entradas.
PELIGROSOS = ("=", "+", "-", "@")

def escapar_celda(valor: str) -> str:
    if valor.startswith(PELIGROSOS):
        return "'" + valor          # mitigacion contextual: verificar en el destino
    return valor
```

El listado 2.1 es un ejemplo de control *localizado*: debe residir en un único punto autoritativo (DRY) y aplicarse a toda celda exportada, no solo a algunas. Su corrección es contextual: depende del formato de destino y de la versión del programa que lo abra.

2.17.3. Costes de mantenimiento

Soportar más formatos multiplica las combinaciones que hay que probar, documentar y actualizar. Cuando una dependencia cambia de versión o introduce un cambio incompatible, el coste no se reparte de manera uniforme: cada formato añade su propia ruta de mantenimiento. Desde el punto de vista de Brooks, esto es complejidad *accidental* que la IA puede ayudar a escribir pero no a justificar. Desde YAGNI, cada formato debe trazarse a un requisito explícito; desde KISS, el programa debe contener solo lo necesario.

Consejo

Criterio de decisión para el aula: añádase un formato cuando exista un requisito que lo exija y un análisis de sus dependencias y riesgos. La facilidad con la que la IA genera el código de un formato *no* es una razón para soportarlo.

Advertencia

Ningún conjunto finito de pruebas sobre estos formatos demuestra que el programa sea seguro. Superar los casos de prueba elegidos solo indica que esos casos concretos pasan. Un ejemplo «corregido» -incluido el del listado 2.1- es correcto de forma *contextual*, nunca «universalmente seguro». Esta cautela es un invariante del capítulo.

2.18. Ejercicio

1. **Clasificación epistémica.** Para cada uno de los ocho principios iniciales, indique si lo consideraría una *ley*, una *observación* o una *heurística*, y justifique la respuesta en dos o tres líneas. ¿En qué casos la distinción cambia la decisión de diseño?

2. **Contraejemplo a una heurística.** Proponga un escenario realista en el que aplicar KISS produzca una solución *menos* segura que una alternativa más compleja. Describa el requisito, la amenaza y por qué la simplicidad falla.

3. **DRY frente a duplicación de controles.** En el caso transversal, enumere tres reglas de validación que deberían tener una representación única. Explique qué riesgo concreto aparece si cada una se duplica en dos rutas de código distintas.

4. **YAGNI y alcance.** Un compañero propone añadir soporte para PDF y Excel «porque la IA lo genera en dos minutos». Redacte una respuesta breve que distinga

el coste de *escribir* del coste de *mantener y asegurar*, apoyándose en el tabla 2.4.

5. **Dunbar con reservas.** Un informe afirma que «los equipos de revisión de seguridad no deben superar las 150 personas, según el número de Dunbar». Señale al menos dos supuestos no verificados de esa afirmación y proponga un criterio alternativo y verificable para dimensionar el equipo.

6. **Síntesis.** A partir de la tabla 2.3, escriba un párrafo que resuma qué abarata la IA y qué encarece, y enlace la conclusión con la hipótesis rectora del capítulo. Etiquete su afirmación como **Hipótesis** o **Evidencia** según corresponda.

3. Correctitud y seguridad

«El programa funciona» es una de las afirmaciones más ambiguas de la ingeniería del software. Puede significar que el fichero se compila, que produce el resultado esperado para los ejemplos que se probaron, que resiste entradas malformadas o que soporta a un adversario decidido a abusar de él. Estas lecturas describen propiedades distintas, y confundirlas es la fuente de buena parte de los fallos de seguridad reales. Esta sección separa esas propiedades, introduce el instrumental clásico para razonar sobre el comportamiento -precondiciones, postcondiciones, invariantes y la lógica de Hoare- y muestra, con el caso transversal del capítulo, que superar una batería de pruebas funcionales *no* equivale a ser seguro.

3.1. Ocho nociones de «correcto»

En el lenguaje corriente, *correcto* es una palabra única. En ingeniería conviene desdoblarla en al menos ocho nociones que se refieren a cosas diferentes. La clasificación siguiente es una **Propuesta docente**: una taxonomía operativa para el aula, no un estándar cerrado. Su utilidad es doble: obliga a precisar qué se está afirmando cuando se dice que un fragmento «funciona», y permite localizar dónde se producen los malentendidos entre quien escribe el código, quien lo revisa y quien lo explota.
Estas nociones no forman una escalera en la que cada peldaño implique el siguiente. Conviene deshacer la metáfora ascendente con tres observaciones.

1. **Compilar no es acertar.** Un programa puede compilar sin errores y no hacer lo que se espera de él. La compilación comprueba determinadas reglas *estáticas* (según el lenguaje y las opciones del compilador), pero no garantiza la corrección del algoritmo ni la ausencia de comportamiento indefinido.

2. **Pasar las pruebas no es ser correcto ni seguro.** Las pruebas funcionales son un juicio sobre un conjunto *finito* de casos elegidos. La corrección relativa a especificación aspira a una afirmación *universal* sobre el dominio; la seguridad, a resistir a un adversario que no se limita a los casos previstos.

Tabla 3.1.: Ocho nociones de corrección y qué garantiza cada una.

Noción	**Pregunta que responde**	**Cómo se comprueba**	**Qué deja fuera**
Sintaxis	¿Es un programa bien formado según la gramática?	Análisis sintáctico	Todo el significado
Compilación	¿Respeta las reglas estáticas del lenguaje?	Compilador, verificación de tipos	Corrección funcional; entradas adversarias
Funcionalidad	¿Da la salida esperada en los casos probados?	Pruebas con ejemplos	Casos no probados; seguridad
Correctitud (especificación)	¿Satisface su especificación en todo el dominio?	Demostración deductiva; aserciones parciales	Que la especificación sea la adecuada
Robustez	¿Se comporta de forma controlada ante entradas inválidas?	Pruebas negativas, *fuzzing*	Adversario dirigido; confidencialidad
Seguridad	¿Resiste a un adversario que elige entradas y entorno?	Modelado de amenazas, análisis, revisión	Todo lo no cubierto por el modelo de amenaza
Portabilidad	¿Se comporta igual en otro entorno?	Matriz de plataformas	Seguridad y corrección en el entorno original
Mantenibilidad	¿Puede comprenderse y modificarse con seguridad?	Revisión, métricas de complejidad	Corrección actual del programa

3. **La robustez no es la seguridad.** Que un programa no se caiga ante una entrada malformada no implica que un atacante no pueda obtener de él un comportamiento ventajoso. La robustez suele evaluarse ante condiciones anómalas, sean accidentales o provocadas; la seguridad exige además preservar propiedades frente a capacidades e intenciones adversarias.

La separación entre robustez y seguridad merece énfasis porque la frontera es sutil. Un programa robusto degrada de forma controlada cuando recibe un dato inesperado; un programa seguro, además, mantiene sus propiedades bajo un atacante que *elige* ese dato y el entorno para forzar la degradación. El modelado de amenazas permite delimitar esas capacidades; OWASP ASVS [5] proporciona requisitos de verificación de aplicaciones, mientras que NIST SSDF [3] orienta prácticas de desarrollo seguro. No son instrumentos equivalentes ni sustituyen por sí solos un modelo de amenaza explícito: sin él, «seguro» carece de referente y la afirmación no es verificable.

3.2. Especificación: precondiciones, postcondiciones e invariantes

La corrección relativa a especificación exige una *especificación*: una descripción del comportamiento deseado frente a la cual comparar el programa. El instrumental clásico se apoya en tres nociones.

Precondición P. Condición que debe cumplirse *antes* de ejecutar el fragmento C. Es un supuesto sobre el estado inicial o sobre la entrada: por ejemplo, «el fichero existe, es legible y su tamaño no supera un límite fijado». Si la precondición no se cumple, el fragmento no está obligado a nada.

Postcondición Q. Condición que debe cumplirse *después* de que C termine, siempre que P se cumpliera al empezar. Por ejemplo, «existe un XLSX en el directorio de salida con las mismas filas que el CSV, sin fórmulas ejecutables».

Invariante I. Propiedad que debe permanecer verdadera en puntos definidos del programa: típicamente, antes y después de cada iteración de un bucle, o a la entrada y a la salida de un módulo. Por ejemplo, «toda celda escrita hasta ahora ha sido tratada conforme a la política de interpretación de celdas definida».

La pareja (P, Q) se interpreta a menudo como un *contrato*: la precondición es una obligación del llamante, que debe establecerla; la postcondición es una obligación del

implementador, que debe garantizarla. Los invariantes son la herramienta para razonar sobre procesos repetitivos, donde el estado cambia muchas veces y una única afirmación sobre el inicio y el fin no basta.

Apliquemos las tres nociones al caso transversal (Anexo A). Una especificación razonable para la conversión de CSV a XLSX podría enunciarse así:

- **Precondición:** el fichero de entrada existe, es legible, tiene cabecera válida, usa una codificación declarada y su tamaño no excede el límite admitido.
- **Postcondición:** existe un XLSX en el directorio de salida, confinado a ese directorio, con las mismas filas y valores de datos esperados del CSV y sin que las entradas no confiables se interpreten como fórmulas, según una política comprobable.
- **Invariante del bucle de conversión:** toda fila escrita ha pasado por la rutina de validación y conversión; las celdas de entrada no confiable se escriben como datos, no como fórmulas, de acuerdo con el formato y la biblioteca utilizados.

Obsérvese que la postcondición ya incorpora un requisito de *seguridad* -el confinamiento de la salida y la neutralización de fórmulas-, no solo un requisito funcional. Esto no es casual: una especificación que solo describa el «camino feliz» será insuficiente para razonar sobre seguridad. La calidad del resultado depende de la calidad de la especificación, un punto que reaparecerá al hablar de los *prompts*. El desarrollo guiado por especificaciones y el papel de la especificación como artefacto de coordinación se tratan en la sección 12.2.

3.3. Lógica de Hoare y sus límites

La lógica de Hoare da forma matemática a la idea de contrato. Escribe la corrección como un *triple* y permite componer demostraciones de fragmentos pequeños en una demostración del programa completo.

Nota

La lógica de Hoare formaliza la corrección como un *triple* $\{P\}\,C\,\{Q\}$, donde P es la precondición, C el programa y Q la postcondición. Su lectura es: si el estado inicial satisface P y C termina normalmente, entonces el estado final satisface Q. Esta es la *correctitud parcial*: no afirma que C termine. La *correctitud total* añade la garantía de terminación. Para bucles se usa un invariante I que se conserva antes y después de cada iteración. La corrección es siempre *relativa a la especificación*: si P y Q no describen lo que de verdad se necesita, la demostración no aporta nada

sobre el problema real.

La potencia del instrumento es real: permite demostrar corrección sobre *todo* el dominio de la precondición, no sobre una muestra de casos. Es la diferencia entre «probé veinte entradas y todas dieron el resultado esperado» y «para toda entrada que cumpla P, la salida cumple Q». Su debilidad es igualmente importante, y conviene enumerarla para no vender una garantía que no es tal.

- **Relatividad a la especificación.** El teorema dice «C cumple Q», no «Q es lo que se necesita». Si la especificación es incorrecta o incompleta, la demostración es impecable y el programa sigue siendo inadecuado. La corrección formal hereda los errores de la especificación sin atenuarlos.
- **Propiedades de una sola ejecución.** La lógica de Hoare razona sobre ejecuciones individuales. Muchas propiedades de seguridad son *relacionales*: por ejemplo, la ausencia de filtración de información (*non-interference*) relaciona dos o más ejecuciones con entradas distintas, y no se expresa directamente con un triple estándar. Extender la lógica para cubrirlas exige marcos más potentes y más costosos.
- **Supone una semántica bien definida.** El razonamiento presupone qué significa cada instrucción. En C, ciertas operaciones tienen comportamiento indefinido -por ejemplo, el desbordamiento de enteros con signo o determinados accesos fuera de límites y usos de punteros colgantes-, lo que exige precisar el modelo semántico y las hipótesis de la demostración. Las reglas de codificación segura, como las de CERT [7], tratan de eliminar esas construcciones precisamente porque invalidan el razonamiento.
- **Concurrencia.** La lógica clásica se ocupa de programas secuenciales. Los programas concurrentes requieren variantes (lógica de separación concurrente, condiciones de *rely-guarantee*) con supuestos adicionales sobre el entorno y el entrelazado de hilos.
- **Coste y alcance.** La verificación completa de un sistema real es costosa. En la práctica se combinan especificaciones parciales, aserciones en puntos críticos, comprobaciones en tiempo de ejecución y pruebas. La verificación formal complementa esos métodos; no los sustituye ni garantiza, por sí sola, la seguridad del sistema completo.

En resumen: la lógica de Hoare es un resultado formal **Evidencia** en su terreno -la corrección parcial de programas secuenciales respecto de una especificación-, pero su

aplicación a la seguridad de un sistema real exige supuestos que deben declararse. El error pedagógico habitual es presentarla como si demostrara seguridad; no lo hace. Lo que demuestra es corrección *relativa a una especificación*, y la seguridad de un sistema depende, además, de su modelo de amenaza, de su entorno y de su cadena de suministro.

3.4. Pasar las pruebas no es ser seguro: el caso transversal

El punto central de la sección puede mostrarse con un ejemplo concreto. Considérese una función que convierte un CSV en un XLSX y que *supera* las pruebas funcionales que la acompañan. El listado 3.1 reproduce una versión deliberadamente vulnerable, marcada como material didáctico de ejecución únicamente en un entorno aislado y autorizado.

Listing 3.1: Conversion CSV a XLSX que supera las pruebas funcionales y sigue siendo insegura (material didactico VULNERABLE).

```
# ============================================================
# Material didactico VULNERABLE. Ejecucion SOLO en entorno aislado y autorizado.
# Este fragmento supera las pruebas del listado siguiente.
# ============================================================
import csv
import os
from openpyxl import Workbook

def convertir_csv_a_xlsx(ruta_entrada, dir_salida, nombre_salida):
    # V1. Sin limite de tamano: carga el fichero entero en memoria.
    with open(ruta_entrada, newline="", encoding="utf-8") as f:
        filas = list(csv.reader(f))

    libro = Workbook()
    hoja = libro.active
    for fila in filas:
        # V2. Formula injection: las cadenas que empiezan por = pueden
        # convertirse en formulas de XLSX mediante openpyxl.
        hoja.append(fila)

    # V3. Path traversal: nombre_salida no se canonicaliza ni se confina.
    ruta_salida = os.path.join(dir_salida, nombre_salida)
```

```
23
24      # V4. Riesgo de enlaces simbolicos: save() sigue la ruta
            indicada.
25      #     Este ejemplo NO contiene una comprobacion previa y, por
            tanto,
26      #     NO demuestra una carrera TOCTOU entre comprobacion y uso
            .
27      libro.save(ruta_salida)
28      return ruta_salida
```

Las pruebas del listado 3.2 comprueban lo esencial desde el punto de vista funcional: que el fichero de salida se crea, que las columnas son las correctas y que el número de filas coincide. Pasan todas.

Listing 3.2: Pruebas funcionales que la version vulnerable supera.

```
1  # Estas pruebas PASAN sobre el listado 1. No comprueban formula
       injection,
2  # ni path traversal, ni limites de tamano, ni el tipo de fichero
       de destino.
3  import os
4  from openpyxl import load_workbook
5
6  def test_crea_el_fichero(tmp_path):
7      entrada = tmp_path / "datos.csv"
8      entrada.write_text("nombre,importe\nAna,10\nLuis,20\n",
           encoding="utf-8")
9      salida = convertir_csv_a_xlsx(str(entrada), str(tmp_path), "
           datos.xlsx")
10     assert os.path.exists(salida)
11
12 def test_columnas_y_filas(tmp_path):
13     entrada = tmp_path / "datos.csv"
14     entrada.write_text("a,b\n1,2\n3,4\n", encoding="utf-8")
15     salida = convertir_csv_a_xlsx(str(entrada), str(tmp_path), "
           datos.xlsx")
16     hoja = load_workbook(salida).active
17     assert [c.value for c in hoja[1]] == ["a", "b"]
18     assert hoja.max_row == 3
```

Para observar V2 con un oráculo más preciso, una prueba negativa debe comprobar el *tipo* de la celda resultante, no solamente que se ha creado el archivo. Por ejemplo, una

entrada con `=1+1` puede generar una celda cuyo atributo `data_type` sea `"f"` al cargar el XLSX con `openpyxl`. Este comportamiento depende de la biblioteca y debe verificarse en la versión utilizada. La creación del fichero, por sí sola, no constituye una prueba de seguridad.

Sin embargo, las debilidades y riesgos marcados en el listado 3.1 sobrevive a esas pruebas porque *ninguna* de ellas forma parte de los casos elegidos:

- **V1 - agotamiento de recursos.** Cargar el CSV entero en memoria no molesta con ficheros de prueba pequeños, pero un fichero grande agota la memoria. Es una debilidad de disponibilidad, no de confidencialidad, y las pruebas funcionales no la detectan porque no miden el consumo de recursos.
- **V2 - inyección de fórmulas.** En este ejemplo concreto,`openpyxl` puede guardar como fórmula una cadena que comienza por `=`. Otros prefijos (como `+`, `-` o `@`) son relevantes en determinados flujos CSV y aplicaciones de hoja de cálculo, pero no deben equipararse automáticamente a la semántica de escritura XLSX de esta biblioteca. La debilidad se relaciona con CWE-1236 [6]. Las pruebas usan datos benignos y no inspeccionan el tipo de celda generado.
- **V3 - *path traversal.*** Si `nombre_salida` contiene segmentos como `../`, la escritura escapa del directorio previsto. La validación canónica de rutas y el confinamiento al directorio de salida son controles obligatorios que las pruebas no ejercitan.
- **V4 - enlaces simbólicos y posible TOCTOU en variantes.** Si un adversario controla componentes de la ruta, puede redirigir la escritura mediante enlaces simbólicos. El listado no contiene una validación previa, por lo que *no* demuestra por sí mismo un fallo TOCTOU (CWE-367); este aparecería en una variante que comprobase la ruta y la utilizase después sin preservar la identidad del objeto. El riesgo de seguimiento de enlaces simbólicos se estudia separadamente (CWE-59).

Advertencia

Pruebas verdes $\neq$ seguro. Superar una batería finita de pruebas demuestra únicamente que esos casos concretos pasan; no demuestra que el programa sea correcto ni seguro. Las pruebas funcionales cubren el *camino feliz* que el programador imaginó; la seguridad debe resistir al caso que el adversario elige. Un ejemplo «corregido» -también el de las versiones que se estudian en el capítulo 4- es correcto de forma *contextual*: lo es respecto de las amenazas y precondiciones declaradas, y *nunca* «universalmente seguro». Esta cautela es un invariante del capítulo.

La lección no es que las pruebas carezcan de valor, sino que *cubren una parte del problema*. Añadir pruebas de seguridad -entradas con fórmulas, rutas con `..`, ficheros desproporcionados y enlaces simbólicos- mejora la cobertura, pero sigue siendo finita. La verificación de seguridad combina pruebas, análisis estático y dinámico, revisión y modelado de amenazas, tal como recogen marcos como ASVS [5] e ISO/IEC 27034 [9]. Ninguno de ellos convierte el resultado en una garantía absoluta.

3.5. Requisitos explícitos, implícitos y especificaciones incompletas

La brecha anterior no es solo un problema de pruebas: es un problema de *especificación*. Los requisitos que un programa debe cumplir se dividen en dos categorías que conviene no mezclar.

Requisitos explícitos. Los que se enuncian. En el caso transversal: «leer un CSV, normalizar columnas y generar un XLSX». Un enunciado de seguridad explícito sería «no confiar en el CSV, escapar fórmulas, limitar tamaño y tiempo».

Requisitos implícitos. Los que se dan por supuestos sin escribirlos: que la salida no permita *path traversal*, que el proceso termine en tiempo y memoria acotados, que el código sea comprensible por un tercero. No aparecen en el enunciado, pero su incumplimiento constituye un fallo.

Tabla 3.2.: Requisitos explícitos e implícitos en el caso transversal.

Tipo	**Ejemplo**	**¿Lo comprueban las pruebas funcionales?**
Explícito funcional	«Leer el CSV y generar un XLSX»	Sí, con casos normales
Explícito de seguridad	«Escapar fórmulas, limitar el tamaño»	Solo si se añaden pruebas específicas
Implícito de seguridad	«No permitir *path traversal*»	No, salvo que se haga explícito
Implícito de recursos	«Terminar en tiempo y memoria acotados»	No
Implícito de mantenimiento	«Ser comprensible por un tercero»	No

La tabla 3.2 resume la situación en el caso transversal: las pruebas funcionales cubren la primera fila y, en el mejor de los casos, la segunda; las filas restantes quedan fuera del alcance habitual de la comprobación. La distinción adquiere una relevancia nueva con la IA generativa. Un *prompt* es una especificación en lenguaje natural; suele omitir requisitos implícitos y su *completitud* debe evaluarse, no presumirse. Un enunciado como «convierte este CSV en Excel» describe el requisito funcional y omite los de seguridad. El modelo generativo no conoce los requisitos implícitos del autor: completa los huecos con las opciones más plausibles según sus datos de entrenamiento, y la plausibilidad estadística no coincide necesariamente con la seguridad. La literatura sobre seguridad del código sugerido por asistentes respalda la plausibilidad de esta hipótesis, sin convertirla en un resultado medido [12, 13].

De ahí se siguen tres consecuencias para la práctica, que en este material se etiquetan con distinto estatus.

1. **(Hipótesis)** Un modelo que no recibe un requisito de seguridad en el *prompt* tiende a no incorporarlo. Es una conjetura razonada sobre el comportamiento de los asistentes, no un resultado medido aquí: se enuncia como **Hipótesis** y debe contrastarse con el protocolo del capítulo 6 antes de darla por establecida.

2. **(Inferencia metodológica)** Las pruebas derivadas exclusivamente de los requisitos explícitos pueden dejar fuera propiedades no especificadas. Si se omite el confinamiento de rutas, no hay garantía de que se diseñe una prueba para él. No se presenta aquí una medición empírica de la frecuencia con que ocurre.

3. **(Propuesta docente)** Quien usa un asistente debe escribir los requisitos de seguridad *antes* de pedir el código, y revisar el resultado contra ellos. La recomendación es pedagógica: convierte requisitos implícitos en explícitos y verificables.

La asimetría es la misma que recorre el capítulo. La IA abarata la *producción* de código, pero no *adivina* la especificación que el autor tenía en la cabeza. Cuanto más se delega la escritura, más se desplaza el trabajo crítico hacia la especificación y la validación: precisar qué debe cumplirse, y demostrar que se cumple. El instrumento para hacerlo son las precondiciones, las postcondiciones, los invariantes y su verificación, sean formales o informales.

3.6. Síntesis

- «Correcto» designa al menos ocho propiedades distintas (tabla 3.1), que no implican unas a otras: compilar no es acertar; pasar pruebas no es ser seguro ni correcto en sentido universal.
- Las precondiciones, postcondiciones e invariantes convierten el comportamiento esperado en una especificación contrastable; la lógica de Hoare le da forma matemática, pero su garantía es *relativa a la especificación* y de alcance limitado.
- El caso transversal del CSV a XLSX muestra que una función puede pasar sus pruebas funcionales y seguir siendo insegura: agotamiento de recursos, inyección de fórmulas, *path traversal* y seguimiento de enlaces simbólicos quedan fuera de los casos ensayados; TOCTOU exige además una secuencia real de comprobación y uso.
- **(Hipótesis)** Un *prompt* es una especificación incompleta; los requisitos de seguridad que no se enuncian tienden a no aparecer ni en el código ni en las pruebas.

Advertencia

Recuérdese el invariante: ningún conjunto finito de pruebas demuestra seguridad, y ningún ejemplo «corregido» es universalmente seguro. La corrección que se enseña es siempre contextual: válida respecto de un modelo de amenaza y unas precondiciones declaradas.

3.7. Ejercicio

1. **Clasificación de propiedades.** Elija tres pares de nociones del tabla 3.1 y construya un contraejemplo que satisfaga la primera y no la segunda. Explique por qué la tabla no establece una jerarquía de implicaciones entre propiedades.
2. **Especificación del caso transversal.** Redacte una precondición, una postcondición y un invariante para la conversión de CSV a XLSX. Indique explícitamente qué requisitos de seguridad ha incluido y cuáles ha decidido dejar fuera, y justifique la decisión.
3. **Límites de Hoare.** Explique, en un párrafo, por qué un triple $\{P\}\, C\, \{Q\}$ correcto no implica que C sea seguro. Apóyese en el caso del listado 3.1 y señale al menos dos propiedades que el triple no captura.

4. **Requisitos implícitos en un *prompt*.** Escriba un *prompt* mínimo para el caso transversal y, a continuación, una versión ampliada que convierta en explícitos al menos tres requisitos de seguridad. Compare qué pruebas podrían derivarse de cada versión.

5. **Pruebas y cobertura.** Enumere tres pruebas adicionales que detectarían las debilidades V2, V3 y V4 del listado 3.1. Para V4, distinga la prueba de seguimiento de enlaces simbólicos de una prueba TOCTOU, que requeriría modificar el programa para introducir una comprobación previa. Explique por qué añadirlas mejora la cobertura sin convertir el resultado en una garantía de seguridad.

4. Vulnerabilidades clásicas

La capítulo 3 mostró que superar una batería finita de pruebas no demuestra seguridad. Este capítulo pone nombre a las *familias* de fallo mediante el catálogo CWE de MITRE [6], recorriendo tres grupos: **(a)** memoria y enteros; **(b)** validación de entrada, *path traversal*, inyección y deserialización; y **(c)** estado, recursos y cadena de suministro. El caso transversal (Anexo A) reaparece en cada grupo.

Nota

Sobre los identificadores CWE. Los identificadores de esta sección se citan contra el catálogo MITRE CWE [6]. La correspondencia entre un fragmento y una CWE depende de la debilidad que efectivamente se manifiesta: el identificador no constituye una prueba de explotación. Los nombres y las asociaciones se contrastaron con el catálogo oficial (consulta 2026-10-24) al preparar esta versión. Se distingue la debilidad primaria de las consecuencias posibles.

Advertencia

Código vulnerable: material didáctico. Todos los listados marcados como vulnerables son ejemplos pedagógicos. Su compilación y ejecución deben realizarse **solo en un entorno aislado y autorizado** (contenedor o máquina virtual sin datos ni red de producción). No deben desplegarse ni ejecutarse fuera de ese entorno. La finalidad es reconocer el fallo, no reutilizar el código.

4.1. Cómo se lee cada caso

Para que la comparación entre familias sea sistemática, cada caso se presenta con la misma plantilla. Las etiquetas epistémicas del capítulo se aplican al contenido.

CWE. Identificador o identificadores que describen la debilidad, con su nombre.

Precondiciones peligrosas. Qué supuestos sobre el entorno, la entrada o el llamante hacen posible el fallo. Se recogen en un cuadro de advertencia.

Código vulnerable mínimo. Fragmento didáctico que manifiesta el fallo.

Mecanismo del fallo. La cadena causal que convierte la debilidad en un efecto observable (corrupción, ejecución, fuga, denegación de servicio).

Corrección contextual. El control que neutraliza el fallo en un contexto declarado.

Prueba positiva y negativa. Una prueba que confirma el comportamiento deseado con entrada válida y otra que intenta provocar el fallo con entrada adversaria.

Limitaciones. Qué no cubre la corrección; por qué no es «universalmente segura».

La distinción entre *evidencia*, *hipótesis* y *propuesta docente* se mantiene: las descripciones del mecanismo se apoyan en la semántica de los lenguajes y en las reglas de codificación segura (**Evidencia** en su terreno); las afirmaciones sobre el comportamiento de los asistentes generativos son **Hipótesis** a contrastar; las recomendaciones de corrección son **Propuesta docente**. La tabla 4.1 resume los tres grupos.

Tabla 4.1.: Grupos de trabajo y familias CWE tratadas en la sección.

Grupo	Familia	Identificadores CWE
(a)	Memoria y enteros	CWE-242, CWE-120, CWE-121, CWE-190
(b)	Entrada y datos	CWE-20, CWE-22, CWE-78, CWE-89, CWE-502, CWE-1236
(c)	Estado, recursos y cadena de suministro	CWE-367, CWE-59, CWE-362, CWE-732, CWE-276, CWE-377, CWE-330, CWE-400, CWE-391, CWE-252, CWE-772, CWE-404, CWE-798, CWE-321, CWE-540, CWE-1104, CWE-1395

Nota sobre los identificadores. La tabla recoge correspondencias didácticas contrastadas con las fichas oficiales de MITRE CWE (consulta 2026-10-24); no implica que todas las debilidades se produzcan simultáneamente en cada ejemplo.

Nota

Dependencia de los listados externos. Los ejemplos incluidos mediante `\ejemplolistado` se almacenan en el directorio `examples/`. Para compilar y validar los fragmentos se necesita ese árbol de ficheros: la presente sección no permite comprobar su comportamiento si solo se dispone de este archivo `.tex`.

4.2. Grupo (a): memoria y enteros

4.2.1. A.1. Escritura sin límite de longitud: `gets`

Nota

CWE-242 (*Use of Inherently Dangerous Function*). El fallo asociado de escritura fuera de límites se clasifica además como **CWE-120** (*Buffer Copy without Checking Size of Input*) y, cuando el búfer es de pila, **CWE-121** (*Stack-based Buffer Overflow*). Identificadores verificados contra el catálogo MITRE [6].

Advertencia

Precondiciones peligrosas. La función `gets` lee una línea de la entrada estándar sin conocer el tamaño del búfer de destino; basta con que el atacante proporcione una línea más larga que el búfer para escribir fuera de él. No se requiere ningún privilegio: la precondición es simplemente que la entrada no esté acotada. La función fue retirada del lenguaje C y su uso está prohibido por las reglas de codificación segura [7].

Listing 4.1: A.1 vulnerable: lectura sin cota con `gets` (material didactico VULNERABLE).

```
/*
 * ============================================================
 * Material didactico VULNERABLE.
 * Ejecucion SOLO en entorno aislado y autorizado.
 * No debe compilarse ni ejecutarse en produccion.
 * CWE-242 (gets) / CWE-120 / CWE-121.
 * ============================================================
 */
#include <stdio.h>

#define TAM 16

int main(void)
{
    char nombre[TAM];

    printf("Nombre: ");
```

```
18      gets(nombre); /* CWE-242: gets no acota la longitud leida */
19
20      printf("Hola, %s\n", nombre);
21      return 0;
22  }
```

Mecanismo del fallo.

El búfer `nombre` tiene `TAM` bytes en la pila. `gets` escribe tantos bytes como haya en la línea, más el terminador, sin comprobar `TAM`. Una entrada larga sobrepasa el búfer y sobrescribe la memoria contigua, potencialmente datos de control, según la disposición de la pila y las protecciones activas. La consecuencia típica es la corrupción del flujo de control, que puede derivar en ejecución de código si el atacante controla el contenido -la base de la explotación clásica de pila, que este mismo grupo de fallos de memoria y enteros trata desde la perspectiva del diseño seguro.

Listing 4.2: A.1 corregido contextualmente: lectura acotada con `fgets`.

```
1   /*
2    * ================================================================
3    * Material didactico CORREGIDO (correccion CONTEXTUAL).
4    * Esta version no es universalmente segura: acota la lectura,
5    * pero no valida el contenido ni sustituye un analisis de
         amenazas.
6    * ================================================================
7    */
8   #include <stdio.h>
9   #include <string.h>
10
11  #define TAM 16
12
13  int main(void)
14  {
15      char nombre[TAM];
16
17      printf("Nombre: ");
18      if (fgets(nombre, sizeof nombre, stdin) == NULL) {
19          return 1; /* fin de entrada o error */
20      }
21
22      nombre[strcspn(nombre, "\n")] = '\0';
```

```
23
24      printf("Hola, %s\n", nombre);
25      return 0;
26  }
```

Corrección contextual.

`fgets(nombre, sizeof nombre, stdin)` nunca escribe más de `sizeof nombre` bytes: recibe el tamaño del búfer y respeta un byte para el terminador. El control es *contextual*: acota la *longitud*, pero no valida el *contenido*, no gestiona líneas que excedan el búfer (quedan truncadas y el resto de la línea permanece en el flujo de entrada) y no sustituye un análisis de amenazas.

Prueba positiva y negativa.

- **Positiva:** con la entrada `Ana`, la salida es `Hola, Ana`; el programa no escribe fuera del búfer.
- **Negativa:** con una línea de, por ejemplo, cien caracteres, la versión vulnerable puede escribir fuera del búfer (sin resultado determinista), mientras la corregida limita la lectura pero debe detectar y gestionar el truncamiento. La prueba negativa debe comprobar si se produce escritura fuera del búfer (por ejemplo, con AddressSanitizer o Valgrind).

Limitaciones.

`fgets` no es una garantía universal: si el programa concatena varias lecturas sin respetar el tamaño restante, el desbordamiento reaparece. Tampoco protege frente a entradas que el programa interprete después sin validarlas.

4.2.2. A.2. Desbordamiento de entero con signo

Nota

CWE-190 (*Integer Overflow or Wraparound*). El desbordamiento aritmético de un entero con signo en C es *comportamiento indefinido*: el estándar no especifica el resultado. Identificador verificado contra MITRE [6].

Advertencia

Precondiciones peligrosas. El fallo requiere que los operandos puedan alcanzar magnitudes cuya suma o producto exceda el rango del tipo. La precondición es que el programa calcule con valores derivados de la entrada (tamaños, índices, longitudes) sin comprobar los límites. Un desbordamiento no siempre produce un error visible: puede dar un valor pequeño y positivo, apto para sortear una comprobación posterior.

Listing 4.3: A.2 vulnerable: producto sin comprobacion de rango (material didactico VULNERABLE).

```c
/*
 * ============================================================
 * Material didactico VULNERABLE.
 * Ejecucion SOLO en entorno aislado y autorizado.
 * CWE-190: desbordamiento de entero con signo (comportamiento
 * indefinido en C).
 * ============================================================
 */
#include <stdio.h>

int main(void)
{
    int ancho = 0;
    int alto = 0;

    if (scanf("%d %d", &ancho, &alto) != 2) {
        return 1;
    }

    /* Sin comprobacion: el producto puede desbordar y el
       resultado
     * queda indeterminado (UB). */
    int celdas = ancho * alto;

    printf("celdas = %d\n", celdas);
    return 0;
}
```

Mecanismo del fallo.

`ancho * alto` se evalúa en aritmética de `int`. Si el producto real excede `INT_MAX`, el resultado es indefinido: no es correcto asumir que «dará la vuelta» a un valor negativo o pequeño: el compilador puede optimizar suponiendo que ese desbordamiento no sucede. Si ese valor se usa después para dimensionar un búfer o un bucle, la comprobación de límites puede superarse y reaparecer un fallo de memoria. El desbordamiento de enteros es, por tanto, una debilidad que suele *habilitar* otras.

Listing 4.4: A.2 corregido contextualmente: comprobacion previa del producto.

```c
/*
 * ============================================================
 * Material didactico CORREGIDO (correccion CONTEXTUAL).
 * Esta version no es universalmente segura: evita el
    desbordamiento
 * para operandos no negativos, pero hay que declarar ese dominio
    y
 * considerar los signos y el tipo del resultado.
 * ============================================================
 */
#include <stdio.h>
#include <limits.h>

/* Devuelve 0 si el producto cabe en int; -1 en caso contrario.
 * Supone a >= 0 y b >= 0 (precondicion explicita). */
static int multiplicar_seguro(int a, int b, int *resultado)
{
    if (a != 0 && b > INT_MAX / a) {
        return -1;
    }
    *resultado = a * b;
    return 0;
}

int main(void)
{
    int ancho = 0;
    int alto = 0;

    if (scanf("%d %d", &ancho, &alto) != 2) {
        return 1;
```

```
30      }
31      if (ancho < 0 || alto < 0) {
32          fprintf(stderr, "solo se admiten valores no negativos\n");
33          return 1;
34      }
35
36      int celdas = 0;
37      if (multiplicar_seguro(ancho, alto, &celdas) != 0) {
38          fprintf(stderr, "desbordamiento detectado\n");
39          return 1;
40      }
41
42      printf("celdas = %d\n", celdas);
43      return 0;
44  }
```

Corrección contextual.

La función `multiplicar_seguro` comprueba el producto antes de calcularlo comparando con `INT_MAX / a`. Es una corrección contextual: solo es válida bajo la precondición declarada de operandos no negativos; con signos hay que tratar los cuatro cuadrantes, y para productos de tamaños que no caben en `int` conviene un tipo más amplio o una primitiva específica de la plataforma.

Prueba positiva y negativa.

- **Positiva:** con `100 200`, el resultado es `20000`, dentro de rango.
- **Negativa:** con `100000 100000`, la versión vulnerable produce un valor indeterminado; la corregida informa de desbordamiento y termina con error.

Limitaciones.

La comprobación cubre un único producto. Un programa con varias operaciones encadenadas necesita comprobar cada paso; y ninguna comprobación local corrige una especificación que ignore el problema.

4.3. Grupo (b): entrada, rutas, inyección y deserialización

4.3.1. B.1. Validación de entrada incompleta

Nota

CWE-20 (*Improper Input Validation*). Es una debilidad «madre»: muchas de las demás de este grupo son formas concretas de no validar. Identificador verificado contra MITRE [6].

Advertencia

Precondiciones peligrosas. El programa acepta datos de una fuente no confiable (un fichero, la red, la línea de órdenes) y los usa sin comprobar tipo, rango, longitud, codificación o formato. La precondición es que exista al menos un dato de entrada que el programa no espera.

Listing 4.5: B.1 vulnerable: conversion sin comprobar el resultado ni el rango (material didactico VULNERABLE).

```c
#include <stdlib.h>
/* atoi no informa de errores y no acota el rango. */
int n = atoi(argv[1]);    /* CWE-20: entrada no validada */
```

Mecanismo del fallo.

`atoi` devuelve `0` tanto para la entrada `0` como para una cadena no numérica, y no señala el desbordamiento. Un valor fuera de rango produce un resultado silenciosamente incorrecto que el resto del programa trata como válido.

Listing 4.6: B.1 corregido contextualmente: validacion explicita con strtol.

```c
#include <errno.h>
#include <limits.h>
#include <stdlib.h>

char *fin = NULL;
errno = 0;
long v = strtol(argv[1], &fin, 10);
if (errno != 0 || fin == argv[1] || *fin != '\0' ||
    v < 0 || v > INT_MAX) {
```

```
10      return 1;    /* entrada rechazada */
11  }
12  int n = (int)v;
```

Corrección contextual.

`strtol` permite detectar la ausencia de dígitos, los caracteres sobrantes y el desbordamiento (`errno == ERANGE`), y comprobar el rango antes de convertir. Es contextual: valida *esta* entrada para *este* uso; no convierte en seguro el resto del programa.

Prueba positiva y negativa.

Positiva: la entrada `42` produce el valor `42`. Negativa: `99999999999999999999` y `12abc` se rechazan en la versión corregida, mientras la vulnerable las acepta de forma incorrecta.

Limitaciones.

La validación debe ser coherente con el uso posterior: validar un número y luego usarlo como índice de un arreglo sin comprobar el límite superior reintroduce el fallo.

4.3.2. B.2. *Path traversal*

Nota

CWE-22 (*Improper Limitation of a Pathname to a Restricted Directory*). La resolución insegura de enlaces se relaciona con **CWE-59**. Identificadores a verificar [6].

Advertencia

Precondiciones peligrosas. El nombre de la salida (o de la entrada) procede, total o parcialmente, de una fuente no confiable y se une a un directorio base sin canonicalizar ni confinar. Basta con que el nombre contenga segmentos `../` o un enlace simbólico para escribir o leer fuera del directorio previsto. Es un punto de fallo del caso transversal: escribir el PDF o el XLSX con un nombre derivado del CSV.

Listing 4.7: B.2 vulnerable: union de rutas sin canonicalizar (material didactico VULNERABLE).

```
1  import os
```

```
2
3 def ruta_salida(dir_salida, nombre):
4     # nombre puede contener "../" y escapar del directorio
        previsto.
5     return os.path.join(dir_salida, nombre)    # CWE-22
```

Mecanismo del fallo.

`os.path.join` construye la ruta sin imponer confinamiento: si `nombre` es `../../etc/cron.d/x`, la ruta resultante sale del directorio de salida. La escritura se realiza donde el atacante elige.

Listing 4.8: B.2 corregido contextualmente: canonicalizacion y confinamiento.

```
1 import os
2
3 def ruta_salida(dir_salida, nombre):
4     base = os.path.realpath(dir_salida)
5     candidata = os.path.realpath(os.path.join(base, nombre))
6     # commonpath lanza ValueError si las rutas no comparten
        prefijo.
7     if os.path.commonpath([base, candidata]) != base:
8         raise ValueError("ruta fuera del directorio de salida")
9     return candidata
```

Corrección contextual.

Resolver la ruta con `realpath` y comprobar que permanece dentro de la base permite rechazar rutas que escapan *en el momento de la comprobación*; no confina por sí solo una apertura posterior. Es contextual: `realpath` resuelve enlaces en el momento de la comprobación y no elimina por sí solo las ventanas de carrera (véase C.1); en sistemas con enlaces simbólicos intermedios conviene además operar con descriptores de directorio.

Prueba positiva y negativa.

Positiva: `datos.xlsx` produce una ruta dentro de la base. Negativa: `../../etc/passwd` y `sub/../../../x` se rechazan.

Limitaciones.

El confinamiento por sí solo no valida el *contenido* escrito ni el tipo de fichero; un nombre válido puede seguir produciendo un resultado inseguro.

4.3.3. B.3. Inyección de comandos del sistema operativo

Nota

CWE-78 (*Improper Neutralization of Special Elements used in an OS Command*). Identificador verificado contra MITRE [6].

Advertencia

Precondiciones peligrosas. El programa construye una orden para el *shell* concatenando datos no confiables. La precondición es el uso de un intérprete de órdenes que vuelve a analizar la cadena; caracteres como `;`, `|`, `$()` o `&&` permiten encadenar órdenes arbitrarias.

Listing 4.9: B.3 vulnerable: orden construida con shell=True (material didactico VULNERABLE).

```
import subprocess

def convertir(nombre_fichero):
    # shell=True reinterpreta el nombre como parte de la orden.
    subprocess.run("convert " + nombre_fichero + " salida.pdf",
                   shell=True)    # CWE-78
```

Mecanismo del fallo.

El shell divide la cadena por sus metacaracteres. Un nombre como `a.pdf; rm -rf /` añade una segunda orden que el programa no pretendía ejecutar.

Listing 4.10: B.3 corregido contextualmente: ejecucion sin shell con lista de argumentos.

```
import subprocess

def convertir(nombre_fichero):
    # Sin shell: los argumentos no se reinterpretan.
    subprocess.run(["convert", nombre_fichero, "salida.pdf"],
                   check=True, shell=False)
```

Corrección contextual.

Pasar la orden y sus argumentos como una *lista* sin shell evita la reinterpretación. Es contextual: el programa invocado puede, a su vez, tratar el argumento como opción (por ejemplo, si empieza por `-`), de modo que sigue siendo necesario validar y, si es posible, usar rutas confinadas.

Prueba positiva y negativa.

Positiva: `entrada.csv` genera el PDF. Negativa: un nombre con `;` o `$(...)` no ejecuta órdenes adicionales en la versión corregida.

Limitaciones.

Eliminar el shell no valida el argumento ni el ejecutable: si el programa es sensible a opciones, un nombre que comience por guion puede cambiar su comportamiento.

4.3.4. B.4. Inyección SQL

Nota

CWE-89 (*Improper Neutralization of Special Elements used in an SQL Command*). Identificador verificado contra MITRE [6].

Advertencia

Precondiciones peligrosas. La consulta se compone concatenando datos no confiables en lugar de usar parámetros. La precondición es que el motor reciba la cadena final como *código* y no como *datos*; una comilla simple basta para alterar la consulta.

Listing 4.11: B.4 vulnerable: consulta concatenada (material didactico VULNERABLE).

```
1 def buscar(cursor, nombre):
2     consulta = "SELECT * FROM usuarios WHERE nombre = '" + nombre
          + "'"
3     cursor.execute(consulta)    # CWE-89
```

Mecanismo del fallo.

Un valor como `' OR '1'='1` cierra la comilla y convierte la condición en una tautología, devolviendo todas las filas. Con más imaginación, el atacante puede leer, modificar o borrar datos.

Listing 4.12: B.4 corregido contextualmente: consulta parametrizada.

```
def buscar(cursor, nombre):
    # El parametro se envia como dato, nunca como codigo.
    cursor.execute("SELECT * FROM usuarios WHERE nombre = ?", (
        nombre,))
```

Corrección contextual.

La parametrización separa código y datos: el motor compila la consulta y trata el valor como dato. Es contextual: hay que usar parámetros en *todas* las consultas y no construir SQL dinámico con identificadores (nombres de tabla o columna) sin validación por lista blanca.

Prueba positiva y negativa.

Positiva: buscar `Ana` devuelve su fila. Negativa: buscar `' OR '1'='1` devuelve el resultado tratado como un nombre literal (ninguna fila), no todas.

Limitaciones.

La parametrización protege la *inyección*; no corrige un control de acceso deficiente ni una consulta que devuelva datos que el usuario no debería ver.

4.3.5. B.5. Deserialización de datos no confiables

Nota

CWE-502 (*Deserialization of Untrusted Data*). Identificador verificado contra MITRE [6].

Advertencia

Precondiciones peligrosas. El programa deserializa datos de una fuente no confiable con un formato que permite construir objetos o ejecutar código durante

la carga. La precondición es que el atacante pueda influir en el fichero o el flujo que se deserializa. Es un punto de fallo del caso transversal cuando se admiten «formatos adicionales».

Listing 4.13: B.5 vulnerable: `pickle` sobre datos no confiables (material didactico VULNERABLE).

```
# ============================================================
# Material didactico VULNERABLE.
# Ejecucion SOLO en entorno aislado y autorizado.
# CWE-502: deserializacion de datos no confiables.
# pickle ejecuta lo que el fichero contenga durante la carga.
# ============================================================
import pickle


def cargar_perfil(ruta):
    with open(ruta, "rb") as f:
        return pickle.load(f)  # construye objetos arbitrarios
```

Mecanismo del fallo.

`pickle` no solo reconstruye datos: puede invocar métodos durante la deserialización (`__reduce__`, `__setstate__`). Un fichero manipulado ejecuta código arbitrario con los privilegios del proceso. La ejecución ocurre *al cargar*, sin que el programa llame a ninguna función sospechosa.

Listing 4.14: B.5 corregido contextualmente: formato de datos sin ejecucion.

```
# ============================================================
# Material didactico CORREGIDO (correccion CONTEXTUAL).
# Esta version no es universalmente segura: safe_load evita la
# construccion de objetos arbitrarios, pero sigue exigiendo
    limites
# de tamano, validacion del esquema y control de la fuente.
# ============================================================
import yaml


def cargar_perfil(ruta):
```

```
11      with open(ruta, "r", encoding="utf-8") as f:
12          return yaml.safe_load(f)  # solo tipos de datos basicos
```

Corrección contextual.

Sustituir `pickle` por un formato de datos que no ejecute código elimina la construcción de objetos arbitrarios. Cabe usar, por ejemplo, `safe_load` o JSON. Es contextual: `safe_load` sigue siendo un analizador que procesa entrada no confiable, por lo que hay que limitar el tamaño, validar el esquema y evitar construcciones peligrosas del propio formato.

Prueba positiva y negativa.

Positiva: un documento YAML con `nombre: Ana` produce el diccionario esperado. Negativa: un fichero con un objeto que invoca código se rechaza o se trata como dato inerte en la versión corregida, mientras la vulnerable lo ejecuta.

Limitaciones.

Cambiar de formato reduce la superficie, pero no convierte la entrada en confiable: la validación del esquema y los límites siguen siendo necesarios.

4.3.6. B.6. Inyección de fórmulas en CSV

Nota

CWE-1236 (*Improper Neutralization of Formula Elements in a CSV File*). Identificador verificado contra MITRE [6].

Advertencia

Precondiciones peligrosas. El programa exporta a un formato de hoja de cálculo celdas cuyo texto comienza por `=`, `+`, `-` o `@`, sin neutralizarlas. La precondición es que un usuario abra después el fichero con un programa que interprete esas celdas como fórmulas. La semántica exacta depende del programa y del formato: en la escritura XLSX de `openpyxl` el prefijo `=` es el caso directo; en flujos CSV y otras hojas de cálculo deben considerarse también TAB, CR y espacios iniciales. Es un punto de fallo directo del caso transversal (CSV a Excel).

Listing 4.15: B.6 vulnerable: exportacion de celdas sin neutralizar (material didactico VULNERABLE).

```
# ============================================================
# Material didactico VULNERABLE.
# Ejecucion SOLO en entorno aislado y autorizado.
# CWE-1236: inyeccion de formulas en un fichero CSV.
# Una celda que empieza por = + - @ puede ejecutarse al abrir el
# fichero en una hoja de calculo.
# ============================================================
import csv


def exportar(datos, ruta_salida):
    with open(ruta_salida, "w", newline="", encoding="utf-8") as f:
        escritor = csv.writer(f)
        for fila in datos:
            # Sin neutralizar: una celda "=1+1" o "=HYPERLINK(...)"
            # se interpreta como formula en el programa que la abra.
            escritor.writerow(fila)
```

Mecanismo del fallo.

Al abrir el fichero, la hoja de cálculo evalúa las celdas que empiezan por el signo de fórmula. Una celda como `=HYPERLINK(...)` o `=1+1` puede exfiltrar datos o alterar el documento. El fallo viaja *con los datos* y se activa en el sistema de la víctima, no en el del servidor que generó el fichero.

Listing 4.16: B.6 corregido contextualmente: prefijo de neutralizacion.

```
# ============================================================
# Material didactico CORREGIDO (correccion CONTEXTUAL).
# Esta version no es universalmente segura: neutraliza los prefijos
# peligrosos para las hojas de calculo habituales, pero el
# comportamiento depende del programa que abra el fichero y de su
# configuracion; no sustituye la validacion del contenido.
# ============================================================
import csv
```

```
9
10 PELIGROSOS = ("=", "+", "-", "@")
11
12
13 def escapar(valor):
14     texto = str(valor)
15     if texto.startswith(PELIGROSOS):
16         return "'" + texto  # el apostrofo inicial neutraliza la
               formula
17     return texto
18
19
20 def exportar(datos, ruta_salida):
21     with open(ruta_salida, "w", newline="", encoding="utf-8") as f
           :
22         escritor = csv.writer(f)
23         for fila in datos:
24             escritor.writerow([escapar(celda) for celda in fila])
```

Corrección contextual.

Anteponer un apóstrofo es una mitigación habitual para ciertos consumidores de CSV, pero su interpretación depende del programa importador y de sus opciones. No es una garantía universal de que una celda se trate como texto. Es contextual: la eficacia depende del programa y de su versión (algunos admiten otros caracteres de inicio de fórmula, y el prefijo puede mostrarse); además hay que aplicar el escapado en un único punto autoritativo y a *toda* celda.

Prueba positiva y negativa.

Positiva: `Ana` se exporta sin cambios. Negativa: una celda con `=1+1` debe permanecer como texto al importar el CSV en los programas y configuraciones declarados; se inspecciona el valor resultante y se comprueba que no se evalúa como fórmula. Para XLSX, debe verificarse además el tipo de celda en el archivo generado.

Limitaciones.

Ninguna lista de prefijos es exhaustiva para todos los programas de hoja de cálculo; el escapado reduce el riesgo pero no sustituye la validación del contenido.

4.4. Grupo (c): estado, recursos y cadena de suministro

4.4.1. C.1. TOCTOU entre comprobar y usar

Nota

CWE-367 (*Time-of-check Time-of-Use Race Condition*). Cuando la operación sigue un enlace se añade **CWE-59** (*Improper Link Resolution Before File Access*). Identificadores verificados contra MITRE [6].

Advertencia

Precondiciones peligrosas. El programa comprueba una propiedad de una ruta (por ejemplo, con `access`) y la usa después (con `fopen`), de modo que existe una ventana entre ambas operaciones. La precondición es que un atacante pueda modificar el sistema de ficheros -típicamente sustituir la ruta por un enlace simbólico- durante esa ventana. Es un punto de fallo del caso transversal entre validar y abrir.

Listing 4.17: C.1 vulnerable: ventana TOCTOU entre `access` y `fopen` (material didactico VULNERABLE).

```
/*
 * ============================================================
 * Material didactico VULNERABLE.
 * Ejecucion SOLO en entorno aislado y autorizado.
 * CWE-367 / CWE-59: condicion de carrera TOCTOU entre la
 * comprobacion con access() y el uso con fopen().
 * ============================================================
 */
#include <stdio.h>
#include <unistd.h>

int main(int argc, char **argv)
{
    if (argc < 2) {
        return 1;
    }

    /* Comprobacion (check): ?podemos escribir el fichero? */
    if (access(argv[1], W_OK) == 0) {
        /* Uso (use): la ruta puede haber cambiado entre ambas
```

```
            llamadas,
21          * por ejemplo sustituida por un enlace simbolico. */
22          FILE *f = fopen(argv[1], "w");
23          if (f != NULL) {
24              fputs("datos\n", f);
25              fclose(f);
26          }
27      }
28
29      return 0;
30  }
```

Mecanismo del fallo.

`access` comprueba los permisos de la ruta en el momento t_1. Entre t_1 y el `fopen` en t_2, el atacante puede reemplazar la ruta por un enlace a un fichero distinto. La comprobación se hizo sobre un objeto y la operación recae sobre otro: es una carrera clásica de «comprobar-antes-de-usar».

Listing 4.18: C.1 corregido contextualmente: operacion atomica con `O_NOFOLLOW`.

```
1  /*
2   * ============================================================
3   * Material didactico CORREGIDO (correccion CONTEXTUAL).
4   * Esta version no es universalmente segura: O_NOFOLLOW impide
        seguir
5   * un enlace en el ultimo componente de la ruta, pero los
        directorios
6   * intermedios pueden seguir siendo enlaces; el confinamiento
        completo
7   * requiere openat() con descriptores de directorio y validacion
        de la
8   * ruta canonica.
9   * ============================================================
10  */
11 #define _POSIX_C_SOURCE 200809L
12
13 #include <fcntl.h>
14 #include <stdio.h>
15 #include <sys/stat.h>
16 #include <unistd.h>
```

```c

int main(int argc, char **argv)
{
    if (argc < 2) {
        return 1;
    }

    /* Una unica llamada atomica: no hay ventana entre comprobar y
        usar. */
    int fd = open(argv[1],
                  O_WRONLY | O_CREAT | O_TRUNC | O_NOFOLLOW,
                  0600);
    if (fd < 0) {
        perror("open");
        return 1;
    }

    if (write(fd, "datos\n", 6) != 6) {
        close(fd);
        return 1;
    }

    close(fd);
    return 0;
}
```

Corrección contextual.

Una única llamada a `open` con `O_NOFOLLOW` evita la secuencia separada `access`/`fopen` y rechaza enlaces simbólicos en el último componente de la ruta. No elimina todas las carreras posibles sobre rutas. Es contextual: `O_NOFOLLOW` no protege los directorios intermedios, que pueden seguir siendo enlaces; para fortalecer el confinamiento deben emplearse operaciones relativas a un descriptor de directorio confiable y restricciones adecuadas en cada componente (por ejemplo, `openat2` con opciones de resolución cuando esté disponible). Una validación canónica separada no sustituye estas garantías de apertura.

Prueba positiva y negativa.

Positiva: escribir en una ruta regular dentro del directorio funciona. Negativa: si el componente final es un enlace simbólico, la apertura con `O_NOFOLLOW` se rechaza (habitualmente con `ELOOP` en Linux); la versión vulnerable puede seguirlo. La prueba debe ejecutarse sobre un árbol temporal controlado.

Limitaciones.

La protección es del último componente. El confinamiento de directorios y la comprobación del descriptor siguen siendo necesarios.

4.4.2. C.2. Ficheros temporales inseguros

Nota

CWE-377 (*Insecure Temporary File*) y **CWE-330** (*Use of Insufficiently Random Values*) cuando el nombre se genera con una fuente predecible. Identificadores a verificar [6].

Advertencia

Precondiciones peligrosas. El programa crea un fichero temporal con un nombre predecible en un directorio compartido y sin marca de exclusión. La precondición es que otro proceso pueda adivinar el nombre y crear antes el fichero (o un enlace con ese nombre), lo que da lugar a una carrera y a un posible secuestro. Es un punto de fallo del caso transversal al materializar resultados intermedios.

Listing 4.19: C.2 vulnerable: nombre predecible y creacion sin exclusion (material didactico VULNERABLE).

```
# ============================================================
# Material didactico VULNERABLE.
# Ejecucion SOLO en entorno aislado y autorizado.
# CWE-377 (fichero temporal inseguro) y CWE-330 (valores
# insuficientemente aleatorios): nombre predecible y creacion
# sin marca de exclusion.
# ============================================================
import os
import random

```

```
11
12 def ruta_temporal(dir_destino):
13     # random no es criptografico y el espacio de nombres es
       pequeno.
14     n = random.randint(0, 999999)
15     return os.path.join(dir_destino, "tmp_%d.dat" % n)
16
17
18 def escribir(dir_destino, datos):
19     ruta = ruta_temporal(dir_destino)
20     # Sin O_EXCL: otro proceso puede haber creado ya ese nombre
21     # o haberlo sustituido por un enlace simbolico.
22     with open(ruta, "w", encoding="utf-8") as f:
23         f.write(datos)
24     return ruta
```

Mecanismo del fallo.

El espacio de nombres es pequeño y `random` no es criptográfico. Otro proceso puede predecir el nombre, crear primero un enlace simbólico y desviar la escritura hacia un fichero que el atacante controla. Sin `O_EXCL`, la creación no es atómica respecto a la existencia previa.

Listing 4.20: C.2 corregido contextualmente: creacion atomica con `mkstemp`.

```
1 # ============================================================
2 # Material didactico CORREGIDO (correccion CONTEXTUAL).
3 # Esta version no es universalmente segura: mkstemp crea el
    fichero
4 # de forma atomica, con nombre impredecible y permisos
    restrictivos,
5 # pero hay que borrarlo y controlar el directorio de creacion.
6 # ============================================================
7 import os
8 import tempfile
9
10
11 def escribir(dir_destino, datos):
12     fd, ruta = tempfile.mkstemp(dir=dir_destino, prefix="tmp_",
       suffix=".dat")
13     try:
```

```
14          with os.fdopen(fd, "w", encoding="utf-8") as f:
15              f.write(datos)
16      except Exception:
17          # Cierre defensivo del descriptor: si os.fdopen fallo, fd
                sigue abierto;
18          # si fdopen ya lo consumio, os.close lanza OSError y se
                ignora.
19          try:
20              os.close(fd)
21          except OSError:
22              pass
23          os.unlink(ruta)
24          raise
25      return ruta
```

Corrección contextual.

`tempfile.mkstemp` genera un nombre impredecible, crea el fichero de forma atómica con permisos restrictivos y devuelve un descriptor. Es contextual: hay que elegir un directorio de creación controlado, borrar el fichero al terminar y no confundir la privacidad del nombre con la protección del *contenido*.

Prueba positiva y negativa.

Positiva: se crea un fichero con nombre aleatorio y se escribe el contenido. Negativa: dos llamadas no colisionan; un enlace preexistente con un nombre adivinado no intercepta la escritura.

Limitaciones.

`mkstemp` no protege el contenido ni el directorio; si el directorio es escribible por terceros, el riesgo persiste aunque el nombre sea impredecible.

4.4.3. C.3. Concurrencia sin sincronización

Nota

CWE-362 (*Concurrent Execution using Shared Resource with Improper Synchronization*, «race condition»). Identificador verificado contra MITRE [6].

Advertencia

Precondiciones peligrosas. Dos o más hilos acceden a un recurso compartido sin sincronización, y al menos uno lo modifica. La precondición es que el resultado dependa del entrelazado de las operaciones. La carrera puede producir un resultado incorrecto de forma intermitente y difícil de reproducir.

Listing 4.21: C.3 vulnerable: incremento no atomico sobre estado compartido (material didactico VULNERABLE).

```c
#include <pthread.h>

static int contador = 0;

void *hilo(void *arg)
{
    for (int i = 0; i < 100000; i++) {
        contador++;    /* CWE-362: leer-modificar-escribir sin
            sincronizar */
    }
    return NULL;
}
```

Mecanismo del fallo.

`contador++` no es atómico: consta de leer, sumar y escribir. Si dos hilos intercalan esas fases, una actualización se pierde y el total final es menor que el esperado.

Listing 4.22: C.3 corregido contextualmente: operacion atomica.

```c
#include <pthread.h>
#include <stdatomic.h>

static atomic_int contador;

```

```
6  void *hilo(void *arg)
7  {
8      for (int i = 0; i < 100000; i++) {
9          atomic_fetch_add(&contador, 1);    /* operacion atomica */
10     }
11     return NULL;
12 }
```

Corrección contextual.

Una operación atómica o un mutex hacen indivisible la actualización. Es contextual: protege *esta* variable; un diseño con varias variables relacionadas puede necesitar una sección crítica que las cubra a todas para mantener el invariante.

Prueba positiva y negativa.

Positiva: con dos hilos, el total final es exacto. Negativa: ejecutar muchas veces con varios hilos y comparar con el valor esperado revela la pérdida en la versión vulnerable (una prueba probabilística, no determinista).

Limitaciones.

Las pruebas de concurrencia son no deterministas: pasar muchas ejecuciones no demuestra ausencia de carreras. La verificación requiere razonamiento sobre el entrelazado o herramientas específicas.

4.4.4. C.4. Permisos incorrectos

Nota

CWE-732 (*Incorrect Permission Assignment for Critical Resource*) y **CWE-276** (*Incorrect Default Permissions*). Identificadores verificados contra MITRE [6].

Advertencia

Precondiciones peligrosas. El programa crea un recurso con permisos más amplios de los necesarios (por ejemplo, `0666` o `0777`), o confía en la máscara del proceso. La precondición es que otros usuarios o procesos puedan acceder al recurso y leerlo o modificarlo.

Listing 4.23: C.4 vulnerable: permisos excesivos al crear el fichero (material didactico VULNERABLE).

```
#include <fcntl.h>
/* 0666 permite lectura y escritura a cualquier usuario. */
int fd = open("datos.txt", O_CREAT | O_WRONLY, 0666);   /* CWE-732
    */
```

Mecanismo del fallo.

El modo `0666` concede acceso a todos los usuarios (limitado solo por la máscara del proceso). Un dato sensible creado así queda legible por terceros.

Listing 4.24: C.4 corregido contextualmente: privilegio minimo.

```
#include <fcntl.h>
/* 0600 limita el acceso al propietario. */
int fd = open("datos.txt", O_CREAT | O_WRONLY, 0600);   /* minimo
   privilegio */
```

Corrección contextual.

Aplicar el principio de mínimo privilegio con `0600` al crear un fichero nuevo; el modo efectivo puede restringirse por `umask`, y si el fichero ya existe, `open` no modifica sus permisos. Es contextual: los permisos deben elegirse según quién necesita el acceso; un servicio compartido puede requerir otro modo, siempre justificado.

Prueba positiva y negativa.

Positiva: el propietario lee y escribe el fichero. Negativa: otro usuario no puede leerlo ni modificarlo.

Limitaciones.

Los permisos no protegen frente a un proceso con privilegios elevados ni frente a un directorio padre accesible.

4.4.5. C.5. Errores no comprobados

Nota

CWE-391 (*Unchecked Error Condition*) y **CWE-252** (*Unchecked Return Value*). Identificadores verificados contra MITRE [6].

Advertencia

Precondiciones peligrosas. El programa ignora el valor devuelto por funciones que pueden fallar (apertura, lectura, asignación). La precondición es que la operación falle en algún escenario y el programa continúe con un estado inválido.

Listing 4.25: C.5 vulnerable: retornos ignorados (material didactico VULNERABLE).

```
#include <stdio.h>

FILE *f = fopen(ruta, "r");
fread(buf, 1, n, f);    /* CWE-252: ni fopen ni fread se comprueban */
fclose(f);
```

Mecanismo del fallo.

Si `fopen` devuelve `NULL`, el uso de `f` es comportamiento indefinido. Si `fread` lee menos de lo pedido, el programa procesa un búfer incompleto o sin inicializar.

Listing 4.26: C.5 corregido contextualmente: comprobacion de cada retorno.

```
#include <stdio.h>

FILE *f = fopen(ruta, "r");
if (f == NULL) {
    perror("fopen");
    return 1;
}
size_t leidos = fread(buf, 1, n, f);
if (leidos != n) {
    /* tratar lectura parcial o error, no continuar como si fuera completa */
}
fclose(f);
```

Corrección contextual.

Comprobar cada valor devuelto y actuar en consecuencia. Es contextual: qué hacer ante un fallo (abortar, reintentar, degradar) depende de la política del sistema; lo que no es admisible es ignorarlo.

Prueba positiva y negativa.

Positiva: con una ruta válida, la lectura se completa. Negativa: con una ruta inexistente o sin permisos, el programa informa del error en lugar de continuar con un estado inválido.

Limitaciones.

Comprobar no define la política de recuperación; un error gestionado de forma incorrecta puede seguir causando un fallo.

4.4.6. C.6. Fugas de recursos

Nota

CWE-772 (*Missing Release of Resource after Effective Lifetime*) y **CWE-404** (*Improper Resource Shutdown or Release*). Identificadores verificados contra MITRE [6].

Advertencia

Precondiciones peligrosas. Un recurso (fichero, descriptor, conexión) se adquiere y hay una ruta de ejecución que lo abandona sin liberarlo. La precondición es la existencia de un retorno anticipado o una excepción que omita la liberación. Repetido, agota descriptores o memoria.

Listing 4.27: C.6 vulnerable: retorno que omite el cierre (material didactico VULNERABLE).

```
def leer(ruta):
    f = open(ruta, "r", encoding="utf-8")
    if not f.readline():
        return None       # CWE-772: f no se cierra
    return f.read()
```

Mecanismo del fallo.

El retorno temprano abandona el objeto sin cerrarlo. En un bucle de muchas iteraciones, los descriptores se agotan y el programa falla.

Listing 4.28: C.6 corregido contextualmente: gestion de contexto.

```
def leer(ruta):
    with open(ruta, "r", encoding="utf-8") as f:
        if not f.readline():
            return None       # el contexto cierra f en cualquier
                salida
        return f.read()
```

Corrección contextual.

El gestor de contexto `with` garantiza el cierre en todas las rutas, incluidas las excepcionales. Es contextual: cubre este recurso; otros recursos (conexiones, cerrojos) requieren su propio tratamiento.

Prueba positiva y negativa.

Positiva: leer un fichero no deja descriptores abiertos. Negativa: tras muchas llamadas con retorno temprano, el número de descriptores abiertos se mantiene acotado en la versión corregida y crece en la vulnerable.

Limitaciones.

La corrección evita la fuga de *este* recurso; no detecta fugas lógicas (por ejemplo, memoria retenida en estructuras de datos).

4.4.7. C.7. Secretos embebidos

Nota

CWE-798 (*Use of Hard-coded Credentials*); según el caso, **CWE-321** (*Use of Hard-coded Cryptographic Key*) y **CWE-540** (*Information Exposure Through Source Code*). Identificadores verificados contra MITRE [6].

Advertencia

Precondiciones peligrosas. El código contiene una credencial, clave o testigo de acceso como literal. La precondición es que el código fuente sea accesible (repositorio, binario, copia) o se comparta; una vez en el historial, la rotación es obligatoria. Es un riesgo de la cadena de suministro cuando el asistente genera el literal.

Listing 4.29: C.7 vulnerable: credencial embebida en el fuente (material didactico VULNERABLE).

```
import requests

API_KEY = "sk-live-3f9a-ejemplo"    # CWE-798: nunca en el codigo

def llamar(url):
    return requests.get(url, headers={"Authorization": API_KEY})
```

Mecanismo del fallo.

El secreto forma parte del artefacto. Cualquiera que lea el código, el historial de versiones o el binario obtiene acceso. Los secretos escritos por asistentes tienden a aparecer como literales plausibles.

Listing 4.30: C.7 corregido contextualmente: secreto fuera del fuente.

```
import os
import requests

def llamar(url):
    clave = os.environ["API_KEY"]    # inyectada en tiempo de
        ejecucion
    return requests.get(url, headers={"Authorization": clave})
```

Corrección contextual.

Extraer el secreto del código y obtenerlo del entorno o de un gestor de secretos. Es contextual: las variables de entorno también pueden filtrarse (volcados, procesos, registros); lo correcto depende del modelo de amenaza y exige rotación y mínimo privilegio.

Prueba positiva y negativa.

Positiva: con la variable definida, la llamada funciona. Negativa: buscar el patrón de clave en el repositorio no encuentra literales en la versión corregida.

Limitaciones.

Sacar el secreto del código no impide que se filtre por registros, volcados o dependencias; el control debe acompañarse de rotación y auditoría.

4.4.8. C.8. Dependencias vulnerables o sin mantener

Nota

CWE-1104 (*Use of Unmaintained Third Party Components*) se refiere a componentes sin mantenimiento; **CWE-1395** (*Dependency on Vulnerable Third-Party Component*) describe la dependencia de un componente vulnerable. Son situaciones relacionadas, pero no equivalentes; la mera ausencia de versiones fijadas no demuestra ninguna de ellas [6].

Advertencia

Precondiciones peligrosas. El proyecto incorpora dependencias sin fijar versiones, sin comprobar avisos de seguridad y sin plan de actualización. La precondición es que una de esas dependencias tenga una vulnerabilidad explotable en el uso que se hace de ella. Es un riesgo de la cadena de suministro: el código propio puede ser correcto y el fallo residir en un tercero.

Listing 4.31: C.8 vulnerable: dependencias sin fijar ni revisar (material didactico VULNERABLE).

```
# requirements.txt sin versiones ni verificacion de avisos.
openpyxl
PyYAML
requests
```

Mecanismo del fallo.

Sin versiones fijadas, una instalación puede traer una versión con un fallo conocido; sin seguimiento de avisos, el fallo permanece aunque se conozca. La superficie de ataque

crece con cada dependencia añadida.

Listing 4.32: C.8 corregido contextualmente: versiones fijadas y escaneo.

```
# Versiones fijadas y escaneo de dependencias en el pipeline.
openpyxl==VERSION_REVISADA
PyYAML==VERSION_REVISADA
requests==VERSION_REVISADA
# Ademas: ejecutar un analisis de composicion de software (SCA)
# y revisar los avisos antes de cada entrega.
```

Corrección contextual.

Fijar versiones y ejecutar un análisis de composición de software (SCA) reduce el riesgo. Es contextual: fijar no basta -hay que actualizar cuando aparezca un aviso- y los marcadores `VERSION_REVISADA` son deliberadamente no ejecutables: el alumnado debe seleccionar versiones mantenidas tras consultar avisos y fijar también las transitivas. No deben copiarse como fichero de instalación real.

Prueba positiva y negativa.

Positiva: el resolvedor selecciona las versiones declaradas en el entorno ensayado; para reproducibilidad más estricta se necesitan también dependencias transitivas, hashes y condiciones del entorno. Negativa: el escaneo detecta una versión con un aviso conocido y bloquea la entrega en la versión corregida.

Limitaciones.

Ninguna lista de versiones garantiza la ausencia de vulnerabilidades futuras; la gestión de dependencias es un proceso continuo, no una configuración puntual.

4.5. El caso transversal, familia por familia

La tabla 4.2 relaciona los puntos de fallo del caso transversal -leer un CSV y generar PDF/Excel- con las familias de esta sección y el control que las neutraliza contextualmente.

4.6. Síntesis

- Las vulnerabilidades clásicas se agrupan en familias con identificadores CWE verificables (tabla 4.1): memoria y enteros; entrada, rutas, inyección y deserialización;

Tabla 4.2.: Puntos de fallo del caso transversal y controles asociados.

Punto de fallo	**CWE**	**Control contextual**
Rutas de salida no confinadas	CWE-22	Comprobar ruta y abrir de forma segura respecto a directorio confiable
Ventana entre validar y abrir	CWE-367, CWE-59	Apertura relativa a directorio confiable; `O_NOFOLLOW` solo protege el componente final
Fórmulas en celdas del CSV	CWE-1236	Tratar celdas como texto y verificar comportamiento en consumidor declarado
Temporales predecibles	CWE-377, CWE-330	`mkstemp` con directorio controlado
Deserialización de formatos adicionales	CWE-502	Formato de datos sin ejecución (`safe_load`)
Dependencias del conversor	CWE-1104/CWE-1395	Fijar versiones y escanear con SCA

Nota sobre los identificadores. La tabla sintetiza controles contextuales, no garantías universales. La asignación final de cada CWE se contrastó con la ficha oficial correspondiente (consulta 2026-10-24) y debe justificarse para la debilidad observable en el código.

y estado, recursos y cadena de suministro.

- Cada caso se describe con una plantilla común: CWE, precondiciones peligrosas, código vulnerable, mecanismo, corrección contextual, prueba positiva y negativa, y limitaciones.
- Muchas debilidades se *combinan*: un desbordamiento de entero habilita un fallo de memoria; un TOCTOU anula una validación de ruta; una dependencia vulnerable compromete código propio correcto.
- La corrección que se enseña es siempre **contextual**: válida para un modelo de amenaza y unas precondiciones declaradas, y *nunca* «universalmente segura».

Advertencia

Recuérdese el invariante del capítulo: ningún conjunto finito de pruebas demuestra seguridad, y ningún ejemplo corregido de esta sección es universalmente seguro. Las pruebas positivas y negativas ilustran el comportamiento esperado y adversario, pero la verificación de seguridad combina análisis estático y dinámico, revisión y

modelado de amenazas, según marcos como ASVS [5] y el NIST SSDF [3].

4.7. Ejercicio

1. **Clasificación por CWE.** Para cada uno de los listados vulnerables de la sección, escriba el identificador CWE principal y justifique por qué le corresponde, consultando el catálogo [6]. Señale los casos en que dos identificadores podrían aplicarse.

2. **Precondiciones.** Elija tres casos y describa, en dos líneas cada uno, la precondición exacta que hace posible el fallo. ¿Qué cambiaría si esa precondición no se cumpliera?

3. **Corrección y límites.** Tome la corrección de B.2 (*path traversal*) y explique por qué no elimina por sí sola la ventana TOCTOU de C.1. Proponga una combinación de controles.

4. **Prueba negativa.** Diseñe una prueba negativa para el caso B.6 (inyección de fórmulas) y otra para C.6 (fuga de recursos). Indique qué observa cada una y por qué pasar la prueba no demuestra seguridad.

5. **Caso transversal.** Complete la tabla 4.2 con dos puntos de fallo adicionales del escenario CSV a PDF/Excel y su control contextual.

5. Portabilidad y plataformas

La capítulo 3 (tabla 3.1) incluyó la portabilidad entre las ocho nociones de corrección y advirtió que no implica ninguna de las demás. La tesis de este capítulo es doble: la portabilidad es una propiedad *relacional* entre un programa y un conjunto *declarado* de plataformas; y las diferencias de plataforma son fuente directa de fallos de corrección y seguridad. Para la IA generativa, esto importa porque los asistentes codifican supuestos de plataforma implícitos -separador de ruta, sensibilidad a mayúsculas, tamaño de `long`- que no aparecen en el *prompt* ni en las pruebas.

Nota

Terminología. En esta sección se distingue:

- **Portabilidad de fuente:** el programa se compila o interpreta en la nueva plataforma sin cambios.
- **Portabilidad binaria:** el mismo ejecutable se ejecuta en la nueva plataforma (requiere compatibilidad de ABI, no solo de fuente).
- **Portabilidad de comportamiento:** el programa produce resultados *observacionalmente equivalentes* en ambas plataformas.
- **Portabilidad de seguridad:** el programa conserva sus propiedades de seguridad bajo el mismo modelo de amenaza en la nueva plataforma.

Las cuatro son independientes. Un programa puede ser portable de fuente y no de comportamiento; portable de comportamiento y no de seguridad. Cuando en este texto se dice «portable» sin más, se refiere a las dos últimas, que son las que importan al diseño seguro.

5.1. Sistemas operativos, API y ABI

La primera fuente de diferencias es el sistema operativo y el modelo de interfaz que expone. La familia *POSIX* (Linux, macOS, los BSD) estandariza un conjunto de llamadas -`open`, `read`, `write`, `stat`, `fork`, `exec`, señales- y una semántica común. La familia *Win32/Win64* expone un modelo distinto: `CreateFile`, `ReadFile`, `WriteFile`, `CreateProcess`, objetos con *handles* y ACL. Ambas familias se parecen en la superficie y difieren en los detalles que deciden la corrección. Las afirmaciones sobre POSIX se refieren a POSIX.1-2017/IEEE Std 1003.1 [10]; las de Win32, a la documentación de Microsoft [11].

Conviene deshacer dos simplificaciones habituales. La primera es tratar «Linux» como una plataforma única: un binario depende del núcleo, de la biblioteca C (*glibc* o *musl*), de la distribución y de la arquitectura. La segunda es tratar «Windows» como «POSIX con otra barra»: el modelo de procesos, el modelo de permisos y el modelo de caracteres son diferentes, no variantes cosméticas.

La tabla 5.1 resume las diferencias que se estudian en los apartados siguientes. Cada fila es un supuesto que un fragmento puede dar por hecho y que, en la otra familia, deja de ser cierto.

Advertencia

La portabilidad no es neutral respecto de la seguridad. Varias filas del tabla 5.1 describen diferencias que un adversario puede explotar: la insensibilidad a mayúsculas, los nombres de dispositivo reservados, el seguimiento de enlaces simbólicos o la ausencia de `O_NOFOLLOW`. Un control de seguridad que se apoya en una suposición de plataforma -«el prefijo de la ruta identifica el directorio», «este nombre no existe», «este fichero no es un enlace»- deja de ser un control cuando la suposición no se cumple.

5.2. Rutas, separadores y canonicidad

Una *ruta* es una cadena, pero no toda cadena que nombra un fichero es *la misma* cadena en todas las plataformas. El mismo objeto puede escribirse con separadores distintos, con mayúsculas distintas, con componentes `.` y `..`, con enlaces intermedios o con rutas relativas que dependen del directorio de trabajo. La operación de convertir una ruta en su forma única se denomina *canonicalización* (`realpath` en POSIX, `Path.resolve()` en bibliotecas portables).

Tabla 5.1.: Diferencias relevantes entre las familias POSIX y Win32. Los detalles dependen de la versión y deben verificarse contra la documentación vigente (A VERIFICAR).

Aspecto	**Familia POSIX**	**Familia Win32/Win64**
Separador de ruta	`/`	`\` (acepta `/` en muchas API)
Sensibilidad a mayúsculas	Sensible (típico)	Insensible (típico)
Modelo de permisos	Bits `rwx`, propietario y grupo, `umask`	ACL, tokens, SID
Bit ejecutable	Sí, en el modo del fichero	No; se decide por extensión y ACL
Enlaces simbólicos	*symlink* nativo	*symlink*/*junction*; requiere privilegio o modo desarrollador
Descriptor de fichero	Entero (*file descriptor*)	*HANDLE* opaco; la CRT ofrece un *fd* emulado
Traducción de fin de línea en modo texto	No	CRLF ↔ LF
Modelo de procesos	`fork`/`exec`, señales	`CreateProcess`, sin `fork`, eventos de consola
Nombres reservados	No	`CON`, `NUL`, `AUX`, `COM1-9`, `LPT1-9`
Longitud de ruta	Límite alto (`PATH_MAX`)	`MAX_PATH` (260) por defecto; rutas largas con opción explícita
Modelo de datos de 64 bits	LP64 (`long` de 64 bits)	LLP64 (`long` de 32 bits)
`wchar_t`	32 bits (glibc típico, UTF-32)	16 bits (UTF-16)
Fin de línea por defecto	LF (`\n`)	CRLF (`\r\n`)

La capítulo 4 trató el *path traversal* (CWE-22, a verificar contra [6]) y el TOCTOU (CWE-367) en el caso transversal. Aquí se añade la dimensión de plataforma: el mismo control de confinamiento puede ser correcto en Linux e insuficiente en Windows, o al revés, si se apoya en comparaciones de cadenas que no tienen en cuenta la plataforma.

Listing 5.1: Ruta construida y analizada con el separador de POSIX (incorrecto o fragil en Windows).

```c
/* Material didactico. Ejecucion solo en entorno aislado y
   autorizado.
   Asume el separador '/' tanto al construir como al analizar la
      ruta. */
#include <string.h>
#include <stdio.h>

/* Extrae el nombre base de una ruta asumiendo separador '/'. */
const char *nombre_base(const char *ruta)
{
    const char *base = strrchr(ruta, '/');   /* en Windows la ruta
        puede usar '\' */
    return base ? base + 1 : ruta;
}

int main(void)
{
    /* Ruta de entrada valida en POSIX; en Windows puede llegar
       con '\'. */
    const char *ruta = "datos\\entrada.csv"; /* literal con '\' */
    printf("%s\n", nombre_base(ruta));       /* imprime la ruta
       completa, no el nombre */
    return 0;
}
```

El fallo es de corrección y tiene consecuencia de seguridad. Si `nombre_base` se usa para *validar* el nombre de salida -rechazar componentes de ruta- y devuelve la ruta completa porque no encontró `/`, la validación no se aplica. La corrección contextual consiste en no analizar rutas a mano y delegar en la plataforma: `os.path.join` y `os.path.basename` en Python, `std::filesystem` en C++, o una función de la biblioteca que conozca el separador nativo.

Listing 5.2: Uso del separador nativo y de la ruta canonica (patron portable contextual).

```python
# Patron portable CONTEXTUAL. Correcto respecto de las precondiciones declaradas;
# NO es universalmente seguro.
import os

def ruta_de_salida(dir_salida, nombre):
    base = os.path.basename(nombre)                  # descarta componentes de ruta
    if not base or base in (".", ".."):
        raise ValueError("nombre de salida no valido")
    return os.path.join(dir_salida, base)            # separador nativo de la plataforma
```

Nota

Windows normaliza nombres. En Windows, los nombres que terminan en punto o en espacio pueden ser tratados como equivalentes al nombre sin ese sufijo, y ciertos nombres (`CON`, `NUL`, `AUX`, `COM1-9`, `LPT1-9`) designan dispositivos, no ficheros ordinarios. Un validador que solo rechace `..` y `/` no cubre estos casos. En Linux esos nombres son ordinarios, de modo que un programa que se comporta correctamente en Linux puede fallar -o ser evadido- en Windows. El detalle exacto depende de la versión (A VERIFICAR).

5.3. Enlaces simbólicos y permisos

En POSIX, un enlace simbólico es un objeto del sistema de archivos que apunta a otro; las llamadas lo siguen por defecto y la marca `O_NOFOLLOW` impide seguir el último componente. En Windows existen enlaces simbólicos y *junctions* con semántica y requisitos de privilegio distintos. Un control basado en `O_NOFOLLOW` es, por tanto, específico de POSIX: no tiene un equivalente directo con el mismo nombre y garantías en Win32.

El modelo de permisos es todavía más divergente. POSIX usa bits de modo (`rwx` para propietario, grupo y otros) y una máscara `umask`; el *bit ejecutable* forma parte del fichero. Windows usa listas de control de acceso (ACL) asociadas a objetos y a tokens de seguridad, y no tiene un bit ejecutable equivalente: la ejecutabilidad depende de la extensión y de las ACL. La consecuencia práctica es que un programa que «protege» un temporal fijando el modo `0600` protege de verdad en POSIX, pero no está expresando la misma garantía en Windows, donde las ACL deciden el acceso.

Listing 5.3: Apertura con `O_NOFOLLOW` (control POSIX sin equivalente directo en Win32).

```c
/* Material didactico. Ejecucion solo en entorno aislado y
   autorizado.
   O_NOFOLLOW impide seguir un enlace simbolico final en POSIX.
   En Windows no existe con la misma semantica: el mismo codigo no
       es portable. */
#include <fcntl.h>
#include <unistd.h>

int abrir_sin_seguir_enlace(const char *ruta)
{
    return open(ruta, O_RDONLY | O_NOFOLLOW | O_CLOEXEC);
}
```

Obsérvese que la corrección de 5.3 es *contextual* incluso en POSIX: protege el último componente del enlace, no los intermedios, y no elimina la ventana de carrera si entre la comprobación y la apertura cambia el directorio. En el caso transversal del CSV convertido a PDF/XLSX (sección 1.5) ya se introdujo esta cautela; aquí se subraya que, además, el control no es trasladable a Windows sin reformularlo.

5.4. Descriptores de fichero y *handles*

En POSIX, un fichero abierto se identifica por un *file descriptor*: un entero pequeño que el proceso usa en `read`, `write` y `close`. Los descriptores se heredan a través de `fork` y, salvo `O_CLOEXEC`, también a través de `exec`; esta herencia es una fuente clásica de fugas de recursos (CWE-772) y de accesos no previstos.

En Windows, el recurso es un *HANDLE* opaco gestionado por el núcleo, con su propia semántica de herencia; la biblioteca de tiempo de ejecución de C ofrece un *descriptor* emulado para compatibilidad, pero su comportamiento no coincide con el de POSIX en todos los casos. Un programa que asume que los descriptores son enteros consecutivos, o que el orden de cierre importa igual en ambas familias, puede comportarse de forma distinta.

Listing 5.4: Suposicion de que el descriptor es un entero consecutivo y predecible (no portable y fragil).

```c
/* Material didactico. Ejecucion solo en entorno aislado y
   autorizado.
   Asume que el descriptor de un fichero es un entero pequeno y
       consecutivo.
```

```
   La CRT de Windows emula descriptores, pero el valor y la
      herencia no coinciden. */
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>

int main(void)
{
    int fd = open("datos.csv", O_RDONLY);
    if (fd < 0) { return 1; }
    /* Peligroso: se asume que fd == 3 en un proceso recien
       iniciado. */
    printf("descriptor esperado: %d\n", fd);
    close(fd);
    return 0;
}
```

El ejemplo es frágil por dos motivos independientes: el valor concreto del descriptor es un detalle de la implementación (Evidencia: el estándar no lo fija), y el código no comprueba el resultado de `close`, lo que oculta errores (CWE-391/252 en el capítulo 4). La corrección contextual consiste en tratar el descriptor como un valor opaco y verificar cada operación.

5.5. Temporales, fin de línea y codificación

La creación de ficheros temporales es un punto donde las plataformas divergen en la API y en los supuestos. En POSIX, `mkstemp` crea el fichero de forma atómica con un nombre impredecible y modo restrictivo; en Windows, la CRT ofrece funciones propias (`_mktemp_s`, `GetTempFileName`) con garantías y semánticas diferentes. El directorio temporal también cambia: `TMPDIR` en POSIX, `TEMP` y `TMP` en Windows.

El *fin de línea* es otro supuesto silencioso. POSIX usa LF (`\n`); Windows usa CRLF (`\r\n`). Cuando un fichero se abre en modo texto, la biblioteca de C de Windows traduce CRLF a LF al leer y a la inversa al escribir; en POSIX no hay traducción. Un programa que cuenta bytes, calcula desplazamientos o compara el tamaño del fichero obtiene valores distintos según la plataforma y el modo de apertura.

Listing 5.5: Conteo de bytes que cambia segun el modo de apertura y la plataforma.

```
/* Material didactico. Ejecucion solo en entorno aislado y
   autorizado.
```

```c
   En modo texto, Windows traduce CRLF <-> LF; POSIX no.
   El mismo fichero tiene distinto numero de bytes leidos en cada plataforma. */
#include <limits.h>
#include <stdio.h>

long tamano_logico(const char *ruta)
{
    FILE *f = fopen(ruta, "r");            /* modo texto: traduccion dependiente de plataforma */
    if (f == NULL) { return -1; }          /* error de apertura */
    long n = 0;
    int c;
    while ((c = fgetc(f)) != EOF) {
        if (n == LONG_MAX) { fclose(f); return -1; }  /* evita desbordamiento */
        n++;
    }
    if (ferror(f)) { fclose(f); return -1; }          /* error de lectura */
    fclose(f);
    return n;                              /* en Windows con CRLF, n es menor que en binario */
}
```

La lección para el caso transversal es directa: al procesar un CSV hay que decidir explícitamente el modo (texto o binario) y la política de fin de línea, en lugar de dejar que la plataforma decida. En Python, el parámetro `newline` de `open` controla esa política; el módulo `csv` documenta que debe usarse `newline=""`.

Consejo

Regla práctica. Fije la codificación y el fin de línea *en el código*, no en la plataforma: `encoding="utf-8"` y `newline=""` al leer CSV; modo binario (`"rb"`/`"wb"`) cuando los bytes deban conservarse. Es una decisión de especificación, no un detalle de estilo.

5.6. Unicode y modelos de caracteres

El modelo de caracteres es una de las diferencias menos visibles y más peligrosas. En C, el tipo `wchar_t` tiene un tamaño que depende de la implementación: 16 bits en Windows (UTF-16) y, típicamente, 32 bits en Linux con glibc (UTF-32). Un programa que asume 2 bytes por carácter ancho calcula mal los tamaños de búfer en Linux, y uno que asume 4 bytes los calcula mal en Windows. En Python 3 la unidad de indexación es el *code point*; en Java, la unidad UTF-16; en Rust, `String` es UTF-8 y `len()` cuenta bytes. Distinguir esas unidades es imprescindible, y siguen existiendo diferencias de normalización: dos secuencias distintas pueden representar el mismo texto (NFC frente a NFD), y algunos sistemas de archivos normalizan los nombres al almacenarlos.

La capítulo 3 exigió declarar la codificación en la precondición del caso transversal. Aquí se añade que la codificación no basta: hay que fijar también la normalización y el tratamiento del BOM (*byte order mark*), porque un CSV con BOM o con nombres normalizados de forma distinta puede leerse de forma distinta según la plataforma.

Listing 5.6: Suposicion del tamano de `wchar_t` (16 bits en Windows, 32 en glibc tipico).

```c
/* Material didactico. Ejecucion solo en entorno aislado y
   autorizado.
   Asume que wchar_t ocupa 2 bytes (UTF-16, como en Windows).
   En Linux/glibc wchar_t suele ocupar 4 bytes (UTF-32). */
#include <wchar.h>
#include <stdlib.h>

wchar_t *duplicar_ancho(const wchar_t *s)
{
    size_t n = wcslen(s);
    /* Calculo que depende del tamano de wchar_t: no portable. */
    wchar_t *r = (wchar_t *)malloc((n + 1) * sizeof(wchar_t));
    if (!r) { return NULL; }
    wcscpy(r, s);
    return r;
}
```

En este caso el código es correcto porque usa `sizeof(wchar_t)`. El fallo aparece cuando se sustituye ese `sizeof` por el literal `2` o `4`, o cuando se mezclan recuentos de *bytes*, *code units* y *code points*. Distinguir esas tres unidades es la clave para no confundir longitud de cadena con tamaño de búfer.

5.7. Procesos y concurrencia

El modelo de procesos difiere: POSIX crea procesos con `fork` y los sustituye con `exec`; Windows los crea con `CreateProcess` y no tiene `fork`. Las señales POSIX (`SIGINT`, `SIGTERM`) no tienen un equivalente exacto en los eventos de consola de Windows. Un programa que gestiona la terminación mediante señales no es portable sin reescribir esa parte.

A esto se añade la *concurrencia*. Los hilos y las carreras de datos (CWE-362) existen en ambas familias, pero el modelo de memoria, la semántica de sincronización y las garantías de visibilidad difieren. Un fragmento que no provoca una carrera en una plataforma -porque el planificador o el orden de memoria lo ocultan- puede provocarla en otra. Que una carrera «no se manifieste» en las pruebas no significa que no exista: es el mismo argumento de suficiencia finita del capítulo 3.

Listing 5.7: Comando de shell construido por interpolacion (inyeccion de comandos y dependencia de plataforma).

```
# Material didactico. Ejecucion solo en entorno aislado y autorizado.
# 'rm -rf' no existe en Windows y la interpolacion de la entrada es inyeccion
# de comandos (CWE-78, a verificar contra el catalogo).
import os

def borrar_directorio(directorio):
    os.system("rm -rf " + directorio)   # no portable e inseguro
```

La corrección contextual evita el *shell* y usa una API de biblioteca con lista de argumentos (`shutil.rmtree`, `subprocess.run([...], shell=False)`), de modo que la entrada no se interpreta como sintaxis de *shell*. La plataforma sigue importando: el mismo código debe comprobar que la operación es válida en el sistema objetivo.

5.8. Compiladores, comportamiento indefinido y arquitectura

El compilador es una variable de plataforma de primer orden. Un mismo programa en C puede producir resultados distintos con `gcc`, `clang` o `MSVC`, y con distintos niveles de optimización. La causa no es la arbitrariedad, sino el estándar del lenguaje: distingue comportamiento *no especificado*, *definido por la implementación* y *indefinido*.

Comportamiento definido por la implementación. El estándar permite varias opciones

y exige documentar la elegida: si `char` es con o sin signo y el ancho de `int` y `long`. El código es *portable de fuente* pero su significado cambia.

Comportamiento no especificado. El estándar no fija una única opción ni exige documentarla; por ejemplo, el orden de evaluación de algunos operandos y de los argumentos de una llamada. El programa no debe depender de ese orden.

Comportamiento indefinido (UB). El estándar no impone ningún significado: el desbordamiento de enteros con signo, el acceso fuera de límites, la lectura de memoria no inicializada, la violación de *strict aliasing*. El compilador puede asumir que el UB no ocurre y optimizar en consecuencia, de modo que el programa «funciona» con una configuración y falla con otra. Las reglas de codificación segura, como las de CERT [7], existen en buena parte para eliminar estas construcciones.

La arquitectura añade sus propios supuestos: el ancho de palabra, el alineamiento, el *endianness* y el modelo de datos (LP64 frente a LLP64). Un cálculo que asume que `long` es de 64 bits es correcto en Linux de 64 bits y puede desbordar en Windows de 64 bits, donde `long` es de 32 bits.

Listing 5.8: Suposicion del ancho de `long` y del modelo de datos (LP64 frente a LLP64).

```
/* Material didactico. Ejecucion solo en entorno aislado y
   autorizado.
   LP64 (Linux/macOS 64-bit): long = 64 bits.
   LLP64 (Windows 64-bit):    long = 32 bits.
   Un calculo que asume 64 bits puede desbordar en Windows. */
#include <limits.h>
#include <stdio.h>

long bytes_totales(long elementos, long tamano)
{
    /* Producto con posible desbordamiento de entero con signo (
       CWE-190). */
    return elementos * tamano;
}
```

Listing 5.9: Signo de `char` definido por la implementacion (x86 frente a ARM y otras combinaciones).

```
/* Material didactico. Ejecucion solo en entorno aislado y
   autorizado.
```

```c
    'char' es con signo en x86 (tipico) y sin signo en ARM (tipico).
    La comparacion c < 0 tiene significados distintos segun la plataforma. */
#include <stdio.h>

int es_negativo(char c)
{
    return c < 0;   /* definido por la implementacion: puede ser siempre falso */
}
```

Nota

UB y optimización. Un programa con comportamiento indefinido puede parecer correcto en una compilación con optimización desactivada y fallar con optimización activada, porque el compilador usa el UB para justificar transformaciones. Esto no es un «error del compilador»: es el contrato del lenguaje. La corrección contextual consiste en eliminar el UB y usar aritmética comprobada, como en la versión corregida del capítulo 4.

5.9. Ejemplos consolidados: válido aquí, incorrecto o inseguro allí

La tabla 5.2 recoge los ejemplos anteriores y su correspondencia con las familias de fallo del capítulo 4. La tercera columna indica la consecuencia al trasladar el fragmento a la otra plataforma.

Advertencia

Invariante. Que un fragmento funcione en una plataforma no demuestra que sea correcto ni seguro en otra, ni siquiera que sea seguro en la suya. La portabilidad de seguridad exige declarar el entorno y verificar el control *en ese entorno*. Como en el resto del capítulo, superar una batería finita de pruebas no demuestra seguridad.

Tabla 5.2.: Ejemplos de código válido en una plataforma e incorrecto o inseguro en otra.

Ejemplo (listado)	**Válido en**	**Consecuencia en la otra plataforma**
Ruta con separador `/` (5.1)	POSIX	El análisis falla con \; la validación no se aplica
`O_NOFOLLOW` (5.3)	POSIX	No existe con la misma semántica; puede seguirse el enlace (CWE-59)
Descriptor entero (5.4)	Supuesto implícito	Valor y herencia no coinciden; recursos no cerrados (CWE-772)
Conteo en modo texto (5.5)	POSIX	Windows traduce CRLF; los recuentos y desplazamientos cambian
Variante con literal 2/4 para `wchar_t` (5.6)	Windows	En glibc es de 4 bytes; búfer mal dimensionado si se usa el literal
Shell interpolado (5.7)	POSIX con `sh`	En Windows no existe el comando; además, inyección (CWE-78)
`long` de 64 bits (5.8)	LP64	En LLP64 es de 32 bits; posible desbordamiento (CWE-190)
Signo de `char` (5.9)	x86 (con signo)	En ARM suele ser sin signo; comparación siempre falsa

5.10. Plantilla de especificación de plataforma para agentes

Las diferencias anteriores son requisitos, y un requisito que no se escribe no se verifica. Por eso esta sección propone -como **Propuesta docente**- una plantilla de especificación de plataforma destinada a quien delega la generación de código en un agente. La plantilla no es un estándar oficial: es una lista de comprobación que obliga a enunciar los supuestos que, de otro modo, el agente resolvería por su cuenta.

El listado 5.10 presenta la plantilla como texto rellenable. El tabla 5.3 explica cada campo, un ejemplo de valor y el fallo que ayuda a prevenir. La plantilla se entrega *antes* de pedir el código y se usa *después* para revisarlo, igual que los requisitos de seguridad del capítulo 3.

Listing 5.10: Plantilla de especificación de plataforma para agentes (rellenable).

```
PLANTILLA DE ESPECIFICACION DE PLATAFORMA (para agentes de codigo)
================================================================
[P1]  Sistema operativo y version:
[P2]  Arquitectura y modelo de datos:
[P3]  Compilador o interprete y version:
[P4]  Lenguaje, estandar y version:
[P5]  APIs y ABI objetivo:
[P6]  Bibliotecas permitidas (nombre y version):
[P7]  Sistema de archivos, rutas y canonicidad:
[P8]  Permisos, cuentas y aislamiento:
[P9]  Recursos y limites (CPU, memoria, tiempo, red):
[P10] Codificacion, locale y fin de linea:
[P11] Amenazas consideradas y fuera de alcance:
[P12] Criterios de aceptacion y evidencia requerida:
================================================================
```

La plantilla convierte supuestos implícitos en explícitos. Es la misma operación que la capítulo 3 describió para el *prompt*: un enunciado como «convierte este CSV en Excel» no menciona la plataforma, y el agente completa el hueco con la opción más frecuente en sus datos de entrenamiento. La **Hipótesis** de que esa opción tiende a ser la del entorno más representado -y no la del entorno objetivo- debe contrastarse con el protocolo de la capítulo 6; aquí no se mide, se enuncia.

Tabla 5.3.: Campos de la plantilla de plataforma, ejemplo y fallo que previene.

Campo	**Ejemplo de valor**	**Fallo que previene**
P1 SO y versión	Linux x86-64, núcleo y distribución	APIs ausentes o con semántica distinta
P2 Arquitectura	`x86-64`, LP64, *little-endian*	Suposición de ancho de palabra o *endianness*
P3 Compilador	`gcc` 13, `-O2 -Wall`	Dependencia de UB o de una extensión del compilador
P4 Lenguaje	C11, Python 3.12	Funciones retiradas o características no disponibles
P5 APIs y ABI	POSIX.1-2017; no Win32	Uso de primitivas no trasladables
P6 Bibliotecas	`openpyxl`, `reportlab` (versión fijada)	Dependencias implícitas o vulnerables (CWE-1395)
P7 Rutas	separador nativo, canonicalización obligatoria	*Path traversal*, nombres reservados, mayúsculas
P8 Permisos	cuenta sin privilegios, modo `0600`	Escritura no autorizada, temporales expuestos
P9 Recursos	256 MiB, 30 s, sin red	Agotamiento de recursos, exfiltración
P10 Codificación	UTF-8 sin BOM, LF, `newline=""`	Confusión de bytes, fin de línea inconsistente
P11 Amenazas	entradas no confiables, enlaces simbólicos	Control sin modelo de amenaza (ASVS, SSDF)
P12 Aceptación	pruebas positivas y negativas + SAST	«Pasa las pruebas» confundido con seguridad

5.11. Enlace con File I/O: el caso transversal CSV→PDF/Excel

El caso transversal reúne casi todas las dimensiones anteriores en un solo flujo (presentado en el Anexo A). El tabla 5.4 las ordena: cada fila es una decisión de plataforma que afecta a la corrección o a la seguridad de la conversión.

Tabla 5.4.: Dimensiones de portabilidad en el caso transversal CSV→PDF/Excel.

Dimensión	**En el caso transversal**	**Riesgo si se ignora**
Rutas y separadores	Construir la salida con `os.path.join` y canonicalizar	*Path traversal* (CWE-22); validación que no aplica
Temporales	`mkstemp`/`NamedTemporaryFile` con modo restrictivo	Nombre predecible (CWE-377); exposición del temporal
Enlaces simbólicos	No seguir enlaces al abrir; evitar TOCTOU	CWE-59, CWE-367
Fin de línea	`newline=""` al leer CSV; LF o CRLF explícito	Recuentos y campos distintos según plataforma
Permisos	Modo `0600` en POSIX; ACL equivalentes en Windows	Lectura o escritura no autorizada (CWE-732/276)
Codificación	UTF-8 declarado; BOM y normalización decididos	Confusión de caracteres; corrupción de datos
Nombres válidos	Rechazar reservados y sufijos con punto o espacio	Evasión de validación en Windows
Bibliotecas PDF/Excel	Fijar versión y comprobar avisos	Dependencia vulnerable (CWE-1395)

El listado 5.11 muestra un patrón portable para escribir la salida. Es *contextual*: correcto respecto de las precondiciones declaradas -sin confiar en el CSV, confinando la salida al directorio permitido, usando un temporal seguro y fijando la codificación- y *no* universalmente seguro. No cubre, por ejemplo, el análisis de la biblioteca de PDF ni el escapado de fórmulas, que se tratan en el capítulo 4.

Listing 5.11: Patron portable contextual para escribir la salida del caso transversal. Se escribe a traves del descriptor, sin reabrir el temporal por nombre.

```
# Patron portable CONTEXTUAL para el caso CSV -> XLSX/PDF.
# Correcto respecto de las precondiciones declaradas; NO
    universalmente seguro.
import os
import tempfile
```

```
5
6  def destino_seguro(dir_salida, nombre):
7      base = os.path.basename(nombre)              # sin componentes
           de ruta
8      if not base or base in (".", ".."):
9          raise ValueError("nombre de salida no valido")
10     destino = os.path.join(dir_salida, base)     # separador
           nativo
11     raiz = os.path.realpath(dir_salida)          # raiz canonica
12     destino = os.path.realpath(destino)          # destino
           canonico
13     if os.path.commonpath([raiz, destino]) != raiz:
14         raise ValueError("salida fuera del directorio permitido")
15     return destino
16
17 def escribir_salida(destino, escribir):
18     # mkstemp devuelve un descriptor ya abierto. Se escribe A
           TRAVES de ese
19     # descriptor (os.fdopen) y no se reabre el temporal por su
           nombre: asi no
20     # se reintroduce la ventana TOCTOU que el capitulo combate.
21     fd, tmp = tempfile.mkstemp(dir=os.path.dirname(destino),
22                                prefix="tmp_", suffix=".bin")
23     try:
24         with os.fdopen(fd, "wb") as f:           # escritura por
               descriptor
25             escribir(f)                          # la funcion
                   recibe el fichero abierto
26         os.replace(tmp, destino)                 # reemplazo;
               atomicidad segun plataforma
27     except BaseException:
28         try:
29             os.close(fd)                         # cierre
                   defensivo del descriptor
30         except OSError:
31             pass
32         try:
33             os.unlink(tmp)
34         finally:
35             raise
```

Nota

Limitación de este patrón. Escribir a través del descriptor evita reabrir el temporal por nombre y, con ello, la ventana TOCTOU entre validar y abrir. La garantía es *contextual*: exige que la función `escribir` escriba sobre el fichero abierto que recibe y *no* vuelva a abrirlo por su ruta; si la tarea necesitara una ruta (por ejemplo, para invocar una herramienta externa que no acepta un descriptor), la ventana reaparecería. El patrón tampoco cubre la atomicidad de `os.replace` en todas las plataformas ni el confinamiento de directorios intermedios.

Nota

Atomicidad de `os.replace`. En POSIX, `os.replace` es una operación de renombrado atómica sobre el mismo sistema de archivos. En Windows, puede fallar si el destino está abierto por otro proceso. La portabilidad de comportamiento exige probar el caso en ambas plataformas; el detalle debe verificarse contra la documentación vigente (A VERIFICAR).

5.12. Síntesis

- La portabilidad es relacional: se afirma respecto de un conjunto declarado de plataformas, y sus cuatro formas -de fuente, binaria, de comportamiento y de seguridad- son independientes (tabla 5.1).
- Las diferencias entre POSIX y Win32 afectan directamente a la seguridad: separadores y canonicidad, sensibilidad a mayúsculas, nombres reservados, enlaces simbólicos, permisos, descriptores y *handles*, fin de línea y modelo de caracteres.
- El compilador y la arquitectura introducen comportamiento definido por la implementación y comportamiento indefinido; un fragmento puede «funcionar» con una configuración y fallar con otra. Las reglas de codificación segura [7] tratan de eliminar esas construcciones.
- Los ejemplos de la tabla 5.2 muestran que la corrección en una plataforma no se traslada automáticamente a otra.
- La plantilla de plataforma convierte supuestos implícitos en requisitos explícitos y verificables; es una **Propuesta docente** alineada con la exigencia de trazabilidad del NIST SSDF [3] y OWASP ASVS [5].

- En el caso transversal, rutas, temporales, fin de línea, permisos, codificación y bibliotecas son decisiones de plataforma que determinan la seguridad del resultado.

Advertencia

Cierre. Ningún ejemplo «corregido» es universalmente seguro: la corrección es siempre contextual, válida respecto de un entorno, una especificación y un modelo de amenaza declarados.

5.13. Ejercicio

1. **Diagnóstico de plataforma.** Tome el listado 5.1 y 5.5. Para cada uno, indique en qué plataforma es válido, qué supuesto implícito contiene y qué consecuencia tiene en la otra. ¿Es un fallo de corrección o de seguridad?

2. **Plantilla aplicada.** Rellene la plantilla del listado 5.10 para el caso transversal CSV→PDF/Excel en dos entornos distintos (Linux y Windows). Señale al menos tres campos cuya respuesta cambie entre ambos.

3. **Modelo de datos.** Escriba un fragmento en C que calcule el tamaño de un fichero y explique por qué el resultado puede diferir entre LP64 y LLP64 y entre modo texto y binario. Corrija el fragmento para que sea portable.

4. **Enlaces y permisos.** Explique por qué `O_NOFOLLOW` no tiene un equivalente exacto en Win32 y qué control alternativo propondría en ese entorno para el caso transversal.

5. **Verificación.** Proponga tres pruebas que detectarían los fallos de portabilidad de la tabla 5.2 y explique por qué, aun superándolas, no se podría afirmar que el programa es «universalmente portable y seguro».

6. Evaluación reproducible de sistemas de IA

Los capítulos anteriores (capítulos 2-5) respondieron a la pregunta *«¿qué propiedad tiene este código?»*. Este capítulo responde a una distinta y previa: *«¿cómo se compara, de forma reproducible, lo que producen dos sistemas de IA?»*. Comparar sistemas de IA es un problema de *diseño experimental*: hay que declarar qué se mide, sobre qué unidad, con qué variables fijadas y con qué límites de inferencia. Sin ese control, la comparación degenera en anécdota, y una anécdota no es evidencia.

Nota

La comparación no es una propiedad del modelo. Cuando se dice «el sistema A es más seguro que el sistema B», en realidad se afirma algo sobre una *tupla* de condiciones: modelo y versión, configuración, *prompt*/contexto, herramientas habilitadas y entorno de ejecución; el número de repeticiones se registra como covariable. Cambiar cualquiera de esos elementos cambia el resultado. La consecuencia metodológica es tajante: *el objeto de la evaluación no es el modelo, sino la configuración concreta en la que se usa.* La distinción entre modelo, versión, configuración, agente, herramientas y entorno se ha fijado en el índice del capítulo y se recuerda aquí porque toda la sección depende de ella.

El capítulo no asume que la IA mejore ni empeore la seguridad *per se*; sostiene que desplaza el esfuerzo del operador desde la escritura hacia la especificación, la revisión y la validación. La evaluación es, precisamente, el instrumento que mide el *riesgo residual* de ese desplazamiento: cuánto requisito queda insatisfecho, cuánta vulnerabilidad sobrevive y a qué coste humano se detecta.

6.1. La falacia de «gratuito contra de pago»

La hipótesis de trabajo de esta sección es que **el precio de un sistema de IA no determina su seguridad**. La afirmación puede leerse como *Evidencia* parcial: no existe una relación funcional conocida entre la tarifa de un servicio y la seguridad del código

que produce; la tarifa es un atributo comercial, no una propiedad del artefacto. Pero conviene desmontar el razonamiento, porque el error que encierra es instructivo.

Supóngase que se comparan «un modelo gratuito» y «un modelo de pago» y se observa una diferencia. La comparación está *confundida*: los dos grupos difieren, casi con certeza, en más de una variable a la vez. El servicio de pago suele ofrecer una versión distinta del modelo, un *system prompt* distinto, límites de contexto mayores, acceso a herramientas, filtros de seguridad propios, cuotas de uso y regiones de despliegue diferentes. Atribuir la diferencia al precio -o a la gratuidad- es atribuirla a una variable que no se ha manipulado ni medido, mientras las variables que sí podrían explicarla quedan sin registrar.

Hay un segundo error, más sutil. La seguridad de un sistema desplegado no depende solo del modelo base, sino de la *configuración de despliegue*: el *system prompt*, los guardarraíles, los filtros de entrada y salida, los permisos de las herramientas y el *sandbox*. Dos aplicaciones que usan el mismo modelo pueden diferir radicalmente en seguridad por cómo están configuradas. El precio es, en el mejor de los casos, un indicador indirecto y ruidoso de capacidad, no un control de seguridad.

Nota

Cómo tratar el coste en el protocolo. El coste no se elimina: se *registra como covariable*. Se anota la tarifa vigente, la fecha de consulta, la moneda y las condiciones de uso, porque son datos que cambian y que condicionan la reproducibilidad del estudio. Lo que no se hace es usarlo como la variable que explica el resultado. Si se desea estudiar el efecto del precio, hay que manipularlo dejando constantes las demás variables, lo que en la práctica casi nunca es posible con servicios cerrados; reconocer esa imposibilidad es parte del rigor.

6.2. Unidad de análisis y regímenes de evaluación

Antes de medir hay que decidir *qué* se mide. Aquí aparece una distinción que el capítulo considera invariante: no es lo mismo evaluar una *generación aislada* que un *agente con herramientas*, y ninguna de las dos cosas es lo mismo que un *pipeline de validación*. Se distinguen, por tanto, tres regímenes.

Régimen A - Generación aislada. El sistema recibe un *prompt* y devuelve un artefacto (texto, código, un parche). No ejecuta nada, no escribe ficheros, no accede a la red. El observable es el artefacto y solo el artefacto. La reproducibilidad es relativamente alta porque no hay efectos secundarios, aunque persiste la no determinación por

muestreo.

Régimen B - Agente con herramientas. El modelo se inserta en un bucle de decisión con herramientas: ejecución de comandos, escritura de ficheros, acceso a red, instalación de paquetes. El observable incluye *acciones* y sus efectos sobre el entorno. Aquí la superficie de ataque crece -*prompt injection* [16, 17, 18], cadena de suministro, exfiltración de secretos- y la reproducibilidad disminuye, porque el estado del entorno influye en las decisiones posteriores. El aislamiento técnico se trata en el capítulo 9.

Régimen C - Pipeline de validación. El artefacto del régimen A, o las acciones del régimen B, se someten a un proceso de validación: análisis estático, pruebas dinámicas y *sandbox*. Es el régimen que decide si un resultado es aceptable, y se desarrolla en la capítulo 10. La evaluación de un sistema de IA no termina en el régimen A o B: termina cuando el artefacto ha pasado (o no) el pipeline.

La tabla 6.1 resume las diferencias. La conclusión operativa es que *las métricas de un régimen no son conmensurables con las de otro*: contar vulnerabilidades en un artefacto estático no es lo mismo que contar acciones peligrosas de un agente.

Tabla 6.1.: Regímenes de evaluación. Las métricas no son intercambiables entre filas.

Dimensión	**A. Generación aislada**	**B. Agente con herramientas**	**C. Pipeline de validación**
Observable	Artefacto (texto/código)	Artefacto y acciones	Resultado de SAST, pruebas y *sandbox*
Efectos secundarios	Ninguno	Escrituras, red, paquetes, procesos	Según el aislamiento del entorno
Reproducibilidad	Alta (salvo muestreo)	Media o baja	Depende de las herramientas fijadas
Amenaza principal	Vulnerabilidad en el artefacto	*Prompt injection*, cadena de suministro, secretos	Falsos positivos y negativos de las herramientas
Control asociado	Revisión de código	*Sandbox* y límites de confianza (capítulo 9)	Puertas de calidad (capítulo 10)

Advertencia

No comparar a través de regímenes. Un número obtenido en el régimen A («cuántos requisitos cumple el fragmento») no es comparable con uno del régimen B («cuántas acciones peligrosas ejecutó el agente»). Presentarlos en la misma tabla sin

declarar el régimen es un error de constructo: se está midiendo con dos instrumentos distintos algo que se etiqueta como si fuera lo mismo.

6.3. Variables controladas y variables confusas

El protocolo exige fijar y registrar ocho variables. La tabla 6.2 las enumera con el riesgo que implica dejar cada una sin control. La regla general es que *toda variable no registrada es una variable confusa en potencia.*

Tabla 6.2.: Variables controladas del protocolo y riesgo de no controlarlas.

Variable	**Cómo se fija y registra**	**Riesgo si no se controla**
Versión	Identificador exacto del modelo y fecha de uso; proveedor y punto de acceso (API/UI).	Deriva silenciosa: el proveedor actualiza el modelo y las medidas dejan de ser comparables.
Configuración	*System prompt*, temperatura, *top-p*, límite de tokens, filtros de seguridad, permisos.	La diferencia observada puede deberse a la configuración, no al modelo.
Temperatura (cuando aplique)	Se fija y se declara; si es cero, se documenta que *no garantiza* determinismo.	Introducir variabilidad no declarada en las repeticiones.
Prompt	Texto literal, sin paráfrasis; se guarda un identificador y el contenido íntegro.	Una reformulación cambia el resultado; dos sistemas reciben entradas distintas.
Contexto	Ficheros, documentos o fragmentos incluidos en la ventana; política de recuperación.	El contexto forma parte del tratamiento; omitirlo oculta parte de la intervención.
Herramientas	Lista de herramientas habilitadas, versiones, permisos y acceso a red.	Régimen B no declarado; comparar un agente con un generador puro.
Entorno	Sistema operativo, arquitectura, imagen de contenedor, versiones de ejecutables y *sandbox*.	El comportamiento depende del entorno; un fallo puede ser de plataforma (capítulo 5).
Repeticiones	Número n por celda, orden de ejecución y semilla si existe.	Un único resultado no permite estimar la variabilidad; se confunde ruido con efecto.

Nota

Variables confusas típicas que hay que registrar aunque no se manipulen. La tarifa y las condiciones del servicio; el proveedor; el punto de acceso (interfaz web frente a API); la fecha y la hora (por la deriva del modelo); la región de despliegue; las cuotas de uso; el idioma del *prompt*; y la experiencia del revisor humano que evalúa el resultado. Ninguna de ellas es el tratamiento, pero cualquiera puede correlacionar con él.

6.4. Diseño del protocolo

El protocolo se organiza en fases, del requisito al paquete de reproducibilidad. La figura 6.1 muestra la secuencia.

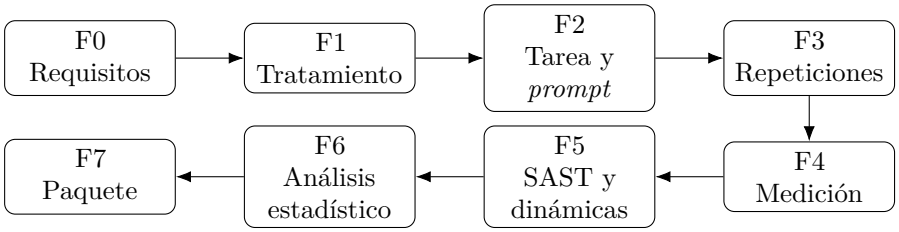


Figura 6.1.: Fases del protocolo de evaluación. Cada fase produce un registro que alimenta el paquete de reproducibilidad (F7).

F0 - Fijar los requisitos. Se congela una especificación de requisitos funcionales y de seguridad. Se reutiliza el caso transversal (leer CSV, normalizar, generar PDF y XLSX; no confiar en la entrada, evitar *path traversal*, temporales seguros, límites de tamaño y tiempo, no ejecutar contenido embebido, escapar fórmulas de Excel), presentado en el Anexo A. Los requisitos se numeran y se les asigna un identificador estable, porque las métricas se calculan sobre ellos. Los marcos de referencia - ASVS [5], SSDF [3]- sirven de guía para derivar requisitos de seguridad; el catálogo CWE [6] sirve para clasificar los fallos. Los identificadores concretos deben verificarse contra la edición vigente.

F1 - Fijar el tratamiento. Se declaran los sistemas que se van a comparar y se fija, para cada uno, el valor de las variables de la tabla 6.2. Se incluye un *brazo de control*: una solución de referencia escrita por humanos y una solución deliberadamente vulnerable, que anclan las métricas y permiten interpretar los resultados relativos.

F2 - Fijar la tarea y el *prompt*. Se escribe un *prompt* único y se usa literalmente igual para todos los sistemas y todas las repeticiones. Se registra su identificador y su contenido. Si se estudia la sensibilidad al *prompt*, se define un conjunto acotado de variantes y se trata como un factor adicional del diseño, no como una improvisación.

F3 - Repeticiones y orden. Cada celda (sistema $\times$ tarea) se ejecuta n veces. El orden se aleatoriza o se contrarresta para evitar efectos de orden y de aprendizaje del revisor. Se documenta la semilla cuando el acceso lo permita.

F4 - Medición. Se aplican las métricas del apartado siguiente sobre los artefactos y las acciones registradas. La medición es ciega al origen cuando sea posible (véase la amenaza de sesgo del evaluador en el apartado 6.7).

F5 - Análisis estático y dinámico. El artefacto entra en el pipeline: SAST y pruebas dinámicas con las versiones de herramienta fijadas. Este paso convierte el régimen A/B en régimen C.

F6 - Análisis estadístico. Se resume y se contrasta según el apartado 6.6.

F7 - Paquete de reproducibilidad. Se archivan los registros, las versiones, los *prompts* y los resultados para que un tercero pueda repetir el estudio.

6.5. Métricas

Una métrica es una *definición operativa*: no mide «seguridad» directamente, sino un *proxy* observable. La tabla 6.3 recoge las métricas exigidas por el protocolo, con su definición, su instrumento y su escala. La advertencia de constructo es constante: ninguna de ellas, por sí sola, agota la noción de seguridad.

Nota

Sobre la severidad. Una escala de severidad es una convención: solo es comparable si se fija antes de medir y se aplica igual a todos los sistemas. Si dos revisores la aplican, se mide su acuerdo. No existe una severidad «objetiva» independiente del procedimiento que la define; por eso el protocolo exige declararla y no la presenta como un hecho natural.

Tabla 6.3.: Métricas del protocolo. Todas son definiciones operativas (proxies), no la seguridad misma. Las escalas se fijan antes de ejecutar el estudio.

Métrica	**Definición operativa**	**Instrumento**	**Escala**
Requisitos satisfechos	Requisitos funcionales y de seguridad cumplidos, sobre el total de F0.	Lista de verificación y pruebas	Recuento y fracción
Vulnerabilidades confirmadas y severidad	Fallos reproducidos, clasificados por CWE [6] y por severidad declarada.	Revisión manual, SAST y reproducción dinámica	Recuento por severidad
Portabilidad (forma declarada)	Forma medida (fuente, binaria, comportamental o de seguridad) y plataformas declaradas en las que se cumple (capítulo 5).	Compilación/ejecución por forma y plataforma	Recuento por forma
Dependencias	Número, origen y versión de las dependencias; superficie de cadena de suministro.	Inventario de dependencias	Recuento y origen
Complejidad	Líneas de código, complejidad ciclomática y profundidad de anidamiento.	Herramientas de métricas de código	Medida cuantitativa
SAST	Hallazgos por regla y herramienta, normalizados por el conjunto de reglas activas.	Analizador estático	Recuento por regla
Pruebas dinámicas	Cobertura, resultados de *fuzzing* y de saneadores; pruebas superadas y fallidas.	Ejecución de pruebas	Fracción y estado
Coste	Consumo de recursos de generación y coste del servicio, con tarifa, fecha y moneda.	Registro de consumo y tarifas	Valor monetario y consumo
Tiempo de revisión	Tiempo humano para revisar y decidir sobre el artefacto.	Cronometraje con protocolo de parada	Duración

Nota sobre los identificadores CWE. La métrica «vulnerabilidades confirmadas» remite a la clasificación por CWE; los identificadores concretos están pendientes de **verificación contra el catálogo MITRE** (A VERIFICAR). La tabla C.3 recoge esa cautela para la plantilla de registro.

Advertencia

Superar una batería finita de pruebas no demuestra seguridad. Este invariante, repetido en todo el capítulo, es también un invariante de la medición. «Pruebas dinámicas superadas» es un resultado del instrumento, no una propiedad del artefacto. Un artefacto puede pasar todas las pruebas del protocolo y seguir siendo inseguro fuera del conjunto finito de casos probados. Las métricas *acotan* el riesgo observado; no lo agotan.

6.6. Tratamiento estadístico

El análisis debe ser tan explícito como el diseño. Se describen a continuación las decisiones estadísticas que el protocolo exige, marcadas como **Propuesta docente** cuando son recomendaciones metodológicas y como **Hipótesis** cuando dependen de supuestos sobre el comportamiento de los sistemas. *No se presentan resultados*: esta sección describe el método, no mediciones.

Unidad de análisis e independencia.

La unidad es la ejecución, pero las ejecuciones de una misma celda comparten configuración y no son independientes entre sí. El diseño es, por tanto, de medidas repetidas y anidado (repeticiones dentro de sistema). Tratar las repeticiones como observaciones independientes infla el tamaño de muestra efectivo y produce inferencias demasiado confiadas. **Propuesta docente:** modelar el anidamiento o, si se resume cada celda, usar la celda como unidad y reportar la dispersión entre repeticiones.

Estadística descriptiva primero.

Antes de cualquier contraste, se reporta la distribución completa de las ejecuciones, no solo un valor central. Con datos sesgados o con valores atípicos -frecuentes cuando una ejecución falla y otra no- la mediana y el rango intercuartílico son más informativos que la media; si la distribución es aproximadamente simétrica, media y desviación típica. **Propuesta docente:** reportar siempre el número de ejecuciones n, la dispersión y los valores extremos.

Tamaño de muestra y potencia.

No existe un número universal de repeticiones. La cantidad n depende de la variabilidad y del efecto que se quiera detectar, y debe justificarse antes de ejecutar. **Propuesta docente:** hacer un cálculo de potencia o un estudio piloto, y no fijar n por conveniencia posterior.

Diseño emparejado.

Cuando varios sistemas resuelven la misma tarea, la comparación emparejada reduce la varianza atribuible a la tarea. **Propuesta docente:** emplear un contraste para datos emparejados (por ejemplo, de rangos con signo o t para muestras relacionadas), eligiendo según el cumplimiento de los supuestos distribucionales, que debe comprobarse y declararse.

Tamaño del efecto e intervalos de confianza.

La significación estadística por sí sola no informa de la magnitud. **Propuesta docente:** acompañar cada contraste con una estimación del tamaño del efecto y un intervalo de confianza, y no declarar «mejor» a partir de un valor p aislado.

Comparaciones múltiples.

Evaluar muchos sistemas y muchas métricas multiplica la probabilidad de encontrar una diferencia por azar. **Propuesta docente:** preespecificar una métrica principal y, para el resto, aplicar una corrección de comparaciones múltiples o etiquetar los análisis como exploratorios.

No determinación y variabilidad entre ejecuciones.

Aun con temperatura cero, la salida puede no ser idéntica entre ejecuciones por el procesamiento por lotes, el hardware o actualizaciones del servicio. **Hipótesis:** la variabilidad residual es pequeña pero no nula; debe medirse, no supuesta. **A VERIFICAR:** el grado de determinismo depende del proveedor, la versión y la fecha. Por eso el protocolo exige repeticiones y no una sola ejecución.

Datos faltantes y fallos.

Una ejecución que falla o que no termina es un dato, no una exclusión. Descartarla sesga los resultados hacia los casos que «salieron bien». **Propuesta docente:** registrar

todos los intentos y analizarlos en la población asignada originalmente, incluyendo los fallos.

Pregistro y evitación de HARKing.

Fijar de antemano las métricas, el análisis y los criterios de exclusión evita presentar como confirmatorio un resultado encontrado a posteriori. **Propuesta docente:** redactar un pre-registro breve antes de ejecutar el estudio.

6.7. Amenazas a la validez

Toda comparación es condicional. Este apartado enumera las amenazas que el protocolo no elimina, sino que declara. Se agrupan según el tipo de validez que comprometen.

Advertencia

Validez interna. (i) *Confusión*: el precio, el proveedor o el punto de acceso varían junto con el modelo. (ii) *Deriva del modelo*: el proveedor actualiza el servicio a mitad del estudio y las medidas dejan de ser comparables. (iii) *No determinación*: la variabilidad entre ejecuciones se confunde con el efecto. (iv) *Efectos de orden y de aprendizaje*: el revisor mejora con la práctica o se cansa. (v) *Contaminación*: el sistema pudo haber visto soluciones similares en sus datos de entrenamiento.

Advertencia

Validez de constructo. «Requisitos satisfechos» es un proxy de la calidad, no la calidad. Los hallazgos de SAST no son vulnerabilidades confirmadas: incluyen falsos positivos y omiten falsos negativos. La escala de severidad es una convención. En consecuencia, ninguna métrica de la tabla 6.3 debe presentarse como una medida directa de la seguridad del sistema.

Advertencia

Validez externa. El caso transversal es *un* caso: un lenguaje, una tarea y un dominio. Los resultados no se generalizan automáticamente a otras tareas, otros lenguajes, otros equipos ni otros contextos organizativos. La validez ecológica exige preguntarse si el *prompt* y el flujo de trabajo del estudio se parecen a los del uso real.

Advertencia

Validez de conclusión estadística. Muestras pequeñas, comparaciones múltiples, observaciones no independientes y reporte selectivo producen conclusiones frágiles. Declarar n, la dispersión y las correcciones aplicadas es obligatorio para interpretar cualquier diferencia.

Advertencia

Reproducibilidad y sesgo del evaluador. Las actualizaciones silenciosas, la deriva de versiones de herramientas, la indisponibilidad de un modelo o las diferencias regionales impiden reproducir un estudio meses después. Además, el revisor que conoce el origen del artefacto -humano o IA- puede evaluarlo con expectativa. **Propuesta docente:** blanquear el origen cuando sea posible y registrar el procedimiento de revisión. En cualquier caso, un resultado *no se transfiere* a otra versión o entorno sin reevaluar.

6.8. Plantillas de registro

Las plantillas de ficha de sesión, registro de hallazgos, registro de vulnerabilidades y formato YAML se recogen en el anexo C.1 (plantillas de registro), separadas del flujo principal para facilitar la consulta independiente durante el trabajo.

6.9. Síntesis y ejercicio

Esta sección ha definido un protocolo, no un resultado. La idea central puede formularse en una frase: *lo que se compara no es el modelo, sino una configuración concreta, y solo es comparable si se fijan y registran todas sus variables.* De ahí se derivan tres consecuencias.

Primera: el precio no es una variable de seguridad. Gratuito o de pago describe una condición comercial; la seguridad depende de la versión, la configuración, el contexto, las herramientas y el entorno. Tratar el precio como causa es un error de confusión.

Segunda: hay que distinguir los regímenes. Una generación aislada, un agente con herramientas y un pipeline de validación producen observables distintos y no conmensurables. Confundirlos es un error de constructo.

Tercera: las métricas son proxies. «Requisitos satisfechos» o «vulnerabilidades confirmadas» acotan el riesgo observado; no demuestran seguridad, porque superar una batería

finita de pruebas nunca la demuestra. Toda comparación es condicional a la configuración congelada y no se transfiere a otra versión sin reevaluar.

Consejo

Ejercicio. Tomad el caso transversal del CSV a PDF/XLSX y redactad un preregistro de dos páginas: (i) enumerad los requisitos funcionales y de seguridad con identificadores estables; (ii) rellenad una ficha de sesión para dos sistemas que queráis comparar; (iii) definid la métrica principal y el número de repeticiones; (iv) enumerad al menos cinco amenazas a la validez y el modo de mitigarlas o declararlas. No ejecutéis todavía el estudio: el objetivo del ejercicio es que el protocolo quede fijado *antes* de ver ningún resultado.

7. El Reglamento de IA (AI Act) y los espacios controlados de pruebas

Material docente. No es asesoramiento jurídico. Prevalece el texto publicado en el DOUE/EUR-Lex.

Los capítulos anteriores (capítulos 2-6) han tratado los sistemas de IA como objetos técnicos, respondiendo a preguntas de *ingeniería*. Este capítulo responde a una pregunta distinta, que no se resuelve leyendo código: *¿cuándo el ordenamiento jurídico de la UE somete un sistema de IA a obligaciones específicas, y qué lugar ocupan en él los espacios controlados de pruebas?*

Nota

Fuentes y alcance. Las referencias normativas de esta sección se han revisado y comprobado contra el DOUE/EUR-Lex (consulta de 2026-11-07). El texto consolidado es un instrumento de consulta; para efectos jurídicos prevalecen los actos publicados en el Diario Oficial. Las explicaciones docentes no sustituyen un análisis jurídico de un producto concreto. Fuentes: AI Act consolidado (27-07-2026) y Reglamento modificativo (UE) 2026/1744.

7.1. Ámbito, enfoque basado en el riesgo y la frontera con el software convencional

El Reglamento no regula la informática en general ni la ingeniería del software en general. Regula *sistemas de IA* y, dentro de ellos, distingue regímenes según el riesgo que su uso previsto genera. El Reglamento combina prácticas prohibidas (artículo 5), requisitos para sistemas de alto riesgo (artículos 6 y siguientes), determinadas obligaciones de transparencia (artículo 50) y un régimen específico para modelos de IA de uso general (capítulo V). No constituye una clasificación exhaustiva de cualquier programa en cuatro categorías mutuamente excluyentes.

Enfoque basado en el riesgo. La lógica del Reglamento es proporcional: cuanto

mayor es el riesgo asociado al *uso previsto*, más exigentes son las obligaciones. Se suele describir la escala con cuatro tramos -prácticas prohibidas, sistemas de alto riesgo, sistemas sujetos a obligaciones de transparencia y sistemas de riesgo mínimo-, además de las obligaciones específicas para modelos de propósito general (artículos 5, 6, 50 y capítulo V, según la materia).

Nota

Definición operativa de «sistema de IA». El Reglamento parte de una definición de «sistema de IA» (habitualmente situada en su artículo 3) que atiende al comportamiento del sistema -capacidad de inferir, a partir de entradas, cómo generar salidas que influyen en entornos físicos o virtuales- más que al lenguaje de programación empleado. La definición es el primer filtro: si un artefacto no encaja en ella, no es un sistema de IA a efectos del Reglamento (artículo 3.1). La calificación de un caso concreto requiere examinar sus funciones y uso previsto.

La consecuencia para nuestra asignatura es una distinción que el capítulo considera invariante: **no es lo mismo un sistema de IA regulado que un programa convencional producido con ayuda de IA**. Un conversor de CSV a PDF/XLSX escrito con un asistente sigue siendo, en principio, *software convencional*: su comportamiento se especifica por reglas explícitas, no por inferencia aprendida. La IA ha intervenido en el *proceso de producción*, no en la *naturaleza* del producto. En cambio, si el mismo producto incorporase un componente que infiere predicciones o decisiones a partir de datos (por ejemplo, un clasificador que decide qué filas son anómalas), ese componente podría constituir -o formar parte de- un sistema de IA, y el conjunto podría quedar sujeto al Reglamento según su uso previsto.

Advertencia

El uso de IA no convierte automáticamente el software en un sistema de IA regulado. La herramienta que *genera* el código no determina la calificación jurídica del código generado. Lo determinante es si el artefacto resultante es un sistema de IA y cuál es su *uso previsto*. Del mismo modo, que un programa no contenga IA no lo exime de otras normas (protección de datos, ciberseguridad, responsabilidad, propiedad intelectual), que quedan fuera del ámbito de esta sección.

7.2. Los artículos 57 a 60 bis: espacios controlados de pruebas y pruebas en condiciones reales

El bloque de los artículos 57 a 60 bis introduce y ordena una figura que interesa directamente a la ingeniería: el *espacio controlado de pruebas regulatorio* (*regulatory sandbox*). Antes de analizarlo conviene fijar la terminología, porque la palabra «sandbox» se usa en el capítulo con dos sentidos que no deben confundirse.

Nota

Dos «sandboxes» distintos. (i) El *espacio controlado de pruebas regulatorio* del Reglamento es una figura *jurídico-administrativa*: un marco autorizado y supervisado por una autoridad competente para desarrollar, entrenar, validar y probar sistemas de IA, en su caso en condiciones reales, bajo un plan de pruebas y con salvaguardas. (ii) El *sandbox técnico de ejecución* (capítulo 9) es un mecanismo de *aislamiento informático* -contenedores, máquinas virtuales, permisos restringidos- que no es una figura del Reglamento. Confundirlos lleva a atribuir obligaciones legales a decisiones puramente técnicas, o a creer que ejecutar en un contenedor satisface exigencias regulatorias. No lo hace.

7.2.1. Artículo 57 - Establecimiento de los espacios controlados

El artículo 57 obliga a los Estados miembros a garantizar al menos un espacio controlado de pruebas regulatorio de IA con cobertura nacional, operativo a más tardar el **2 de agosto de 2027**. Puede establecerse conjuntamente con otros Estados o cumplirse mediante participación en un espacio existente que ofrezca cobertura nacional equivalente (art. 57.1).

Actores. Las autoridades competentes establecen y supervisan el espacio; los proveedores o futuros proveedores participan conforme a criterios de admisión. Pueden intervenir otras autoridades, en particular las de protección de datos cuando corresponda (art. 57.10).

Finalidad. Facilitar durante un tiempo limitado el desarrollo, entrenamiento, prueba y validación de sistemas de IA innovadores antes de su comercialización o puesta en servicio (art. 57.5).

Plan de pruebas. Las actividades se desarrollan conforme a un plan específico acordado entre proveedor y autoridad, con salvaguardas. Puede contemplar pruebas

supervisadas en condiciones reales.

Orientación y evidencias. La autoridad ofrece orientación sobre riesgos y requisitos regulatorios; el participante puede solicitar constancia escrita de actividades realizadas y recibe un informe de salida (art. 57.6-7).

Límites. Participar no equivale a superar una evaluación de conformidad ni elimina las competencias de supervisión o corrección. Los riesgos significativos identificados deben mitigarse (art. 57.11).

La reforma de 2026 introdujo la posibilidad de que la Oficina de IA establezca un espacio a escala de la Unión para los sistemas de su ámbito de supervisión (art. 57.3 bis). Este mecanismo no sustituye la competencia de los Estados miembros para los sistemas bajo su supervisión.

Nota

Informe de salida frente a certificación. El artículo 57.7 prevé que las evidencias y el informe de salida puedan tenerse en cuenta favorablemente en procedimientos posteriores. No establece que participar suponga, por sí mismo, autorización de comercialización, certificación de seguridad o conformidad automática. El artículo 57.12 conserva la responsabilidad civil aplicable por daños a terceros, pero prevé un tratamiento específico de las multas administrativas si se cumplen el plan, las condiciones de participación y la orientación recibida de buena fe. Esta previsión no es una inmunidad general.

7.2.2. Artículo 58 - Disposiciones detalladas y funcionamiento

El artículo 58 encomienda a la Comisión establecer mediante actos de ejecución las disposiciones detalladas de establecimiento, funcionamiento, gobernanza y supervisión. Su apartado 1 comprende criterios de admisión, procedimientos de solicitud, seguimiento, salida y terminación, así como condiciones para los participantes. La reforma de 2026 añade expresamente reglas de gobernanza, coordinación y participación de las autoridades de protección de datos cuando corresponda (art. 58.1.d).

El artículo 58.2 exige que los actos de ejecución establezcan criterios transparentes y equitativos de acceso para proveedores que cumplan las condiciones de elegibilidad y selección; no garantiza la admisión automática de cualquier solicitante. Entre las previsiones figuran la respuesta a la solicitud dentro de tres meses, el acceso gratuito

para pymes y empresas emergentes con excepciones proporcionadas, y procedimientos administrativos comprensibles y accesibles.

También fomenta la colaboración con laboratorios de ensayo, centros de investigación, organismos de normalización y otros actores, y el desarrollo de instrumentos para evaluar exactitud, robustez, ciberseguridad e impactos sobre derechos fundamentales (art. 58.2).

Nota

Supervisión y responsabilidad. La participación no traslada automáticamente al supervisor las responsabilidades del proveedor. Debe distinguirse la orientación regulatoria de la evaluación técnica de la solución. Las pruebas y su documentación aportan evidencias limitadas al plan, las condiciones y los resultados observados.

7.2.3. Artículo 59 - Tratamiento ulterior de datos personales

El artículo 59 regula, bajo condiciones acumulativas, el tratamiento ulterior de datos personales obtenidos lícitamente para otras finalidades, exclusivamente para desarrollar, entrenar y probar determinados sistemas de IA en un espacio controlado regulatorio.

No se trata de una autorización general de reutilización. El artículo acota las finalidades a determinados ámbitos de interés público (art. 59.1.a), entre ellos salud pública, protección ambiental, sostenibilidad energética, transporte e infraestructuras y calidad de servicios públicos. Exige, además, las salvaguardas previstas en sus restantes letras: condiciones de necesidad y protección, medidas técnicas y organizativas, documentación, conservación limitada, registros y restricciones sobre los efectos en los interesados.

Las reglas generales de protección de datos continúan siendo relevantes: el propio artículo 59.3 preserva la aplicación del Derecho de la Unión y nacional en la materia. Una eventual reutilización requiere examinar el supuesto concreto y todas las condiciones exigidas; no basta invocar el interés investigador.

Advertencia

No extrapolar el artículo 59. Esta previsión no permite subir expedientes universitarios a servicios externos, reutilizar datos personales en prácticas docentes ni probar agentes con información protegida sin una base jurídica y salvaguardas adecuadas. Un contenedor técnico no constituye el espacio regulatorio descrito por el artículo.

7.2.4. Artículo 60 - Pruebas en condiciones reales fuera de los espacios

El artículo 60 regula las pruebas en condiciones reales, fuera de los espacios controlados regulatorios, de determinados sistemas de IA de alto riesgo. La modificación de 2026 precisa el alcance para los sistemas del anexo III y los comprendidos en la legislación de armonización de la sección A del anexo I (art. 60.1-2). El artículo 60.4 contiene condiciones acumulativas: plan presentado, aprobación conforme al Derecho aplicable, registro en los supuestos exigidos, salvaguardas y límites temporales, entre otras.

Se trata de pruebas previas a la introducción en el mercado o puesta en servicio, sujetas a un plan y a condiciones específicas. Entre otras garantías, el artículo 60.4 prevé la presentación del plan, la intervención de la autoridad de vigilancia del mercado, las medidas de protección y el seguimiento de las pruebas. El artículo 61 regula el consentimiento informado de los sujetos participantes cuando sea aplicable.

El procedimiento contempla la aprobación tácita tras treinta días sin respuesta, pero el propio artículo 60.4.b advierte que, cuando el Derecho nacional no prevea tal mecanismo, sigue siendo necesaria la autorización. Por ello no puede enseñarse una regla general de «silencio positivo» sin atender a la legislación aplicable.

Nota

Prueba en condiciones reales frente a ejecución aislada. La primera introduce deliberadamente el sistema en situaciones reales bajo salvaguardas; la segunda restringe sus efectos sobre el exterior. Son dimensiones distintas: una prueba real puede emplear medidas de aislamiento parcial, pero ningún aislamiento informático sustituye la supervisión y los requisitos jurídicos de la prueba real.

7.2.5. Artículo 60 bis - Supuesto añadido en 2026

El Reglamento (UE) 2026/1744 incorporó el artículo 60 bis (identificado como artículo 60a en la versión inglesa). Permite que los Estados miembros establezcan marcos para determinadas pruebas en condiciones reales de sistemas de alto riesgo relacionados con productos comprendidos en la sección B del anexo I. Su aplicación depende del marco que el Estado miembro adopte y de las salvaguardas establecidas (art. 60 bis.1-6).

Consecuencia docente: la clasificación de un sistema de alto riesgo no basta para decidir qué procedimiento de pruebas en condiciones reales resulta aplicable. Es necesario identificar el sector, el anexo y el marco jurídico correspondiente.

7.3. Calendario de aplicación y regímenes transitorios

El artículo 113 distingue la entrada en vigor de la aplicación de sus disposiciones. La consolidación consultada incorpora la modificación de 2026; las fechas siguientes corresponden a la consolidación de 27 de julio de 2026 y se han comprobado contra el DOUE/EUR-Lex (consulta de 2026-11-07):

- **1 de agosto de 2024:** entrada en vigor del Reglamento.
- **2 de febrero de 2025:** aplicación de los capítulos I y II, con excepciones específicas incorporadas en 2026.
- **2 de agosto de 2025:** aplicación de las partes indicadas en el artículo 113.b, entre ellas el capítulo V.
- **2 de agosto de 2026:** fecha general de aplicación.
- **2 de diciembre de 2026:** aplicación de determinadas prohibiciones añadidas al artículo 5 (art. 113.a).
- **2 de agosto de 2027:** fecha límite de operatividad del espacio regulatorio nacional (art. 57.1).
- **2 de diciembre de 2027:** aplicación de los requisitos del capítulo III, secciones 1-3, para sistemas de alto riesgo del artículo 6.2 y anexo III, con la excepción indicada en el artículo 113.c.
- **2 de agosto de 2028:** aplicación correspondiente a sistemas de alto riesgo del artículo 6.1 y anexo I, con la misma precisión normativa.

Los artículos 102-110 se aplican desde el 27 de julio de 2026 (art. 113.d). Además, el artículo 111 establece reglas transitorias para determinados sistemas y modelos previamente comercializados. Una tabla de fechas no sustituye el examen de esas reglas cuando se estudia un caso real.

Advertencia

Fechas diferentes, obligaciones diferentes. La entrada en vigor, la aplicación general, la aplicación de requisitos de alto riesgo y el plazo de establecimiento de sandboxes regulatorios son hitos distintos. No debe concluirse que todo el AI Act empezó a exigirse íntegramente el 2 de agosto de 2026. Antes de utilizar estas

fechas para un proyecto real debe consultarse la versión vigente de EUR-Lex.

7.4. Tres nociones que no deben confundirse

La siguiente tabla diferencial -que el estudiante debe poder reproducir- separa la figura regulatoria (A) de los dos mecanismos técnico-metodológicos del capítulo (B y C). Las tres filas *no* son tres versiones de lo mismo: A pertenece al Derecho; B y C pertenecen a la ingeniería. Solo A representa aquí una figura regulatoria específica; B y C también pueden ser relevantes para cumplir otras obligaciones jurídicas o contractuales.

Advertencia

No leer la tabla de arriba como equivalencias. Que las tres nociones compartan la palabra «prueba» no las hace sustituibles. En particular, ejecutar el caso transversal del CSV en un contenedor (B) no crea un espacio controlado regulatorio (A), y aprobar un *pipeline* de SAST y pruebas (C) no equivale a una autorización administrativa (A). La columna A se rige por el Reglamento; las columnas B y C se describen aquí como decisiones de ingeniería, sin perjuicio de otras obligaciones legales que puedan afectar a su implantación.

7.5. Pruebas en condiciones reales, ejecución aislada y tratamiento de datos

Conviene desarrollar la diferencia que la tabla sólo apunta, porque es la que más errores provoca en el aula. Las tres preguntas que hay que separar son: *¿dónde se ejecuta?*, *¿sobre qué datos?* y *¿bajo qué obligaciones?*

¿Dónde se ejecuta? En la *ejecución aislada* el sistema se confina: sin red, con sistema de ficheros restringido, sin acceso a datos personales reales. En la *prueba en condiciones reales* el sistema opera en el entorno de producción o en condiciones equivalentes, precisamente para observar su comportamiento allí. El aislamiento reduce el riesgo de daño; las condiciones reales lo asumen de forma controlada y con salvaguardas.

¿Sobre qué datos? La ejecución aislada se apoya, como regla, en datos sintéticos, anonimizados o ficticios. Las pruebas en condiciones reales y el supuesto del artículo 59 pueden implicar datos personales, lo que activa el régimen de protección de datos y las garantías específicas del Reglamento (artículos 59-61, según el supuesto).

Tabla 7.1.: Diferencia entre espacio controlado regulatorio, sandbox técnico y pipeline de validación. Solo la columna A designa una figura del Reglamento (UE) 2024/1689; las columnas B y C son mecanismos técnicos o metodológicos.

Dimensión	**A. Sandbox regulatorio de IA**	**B. Sandbox técnico de ejecución**	**C. Pipeline de validación de seguridad**
Naturaleza	Jurídico-administrativa	Técnica (aislamiento)	Metodológica (proceso)
Fuente	Reglamento (UE) 2024/1689, arts. 57-59 (sandbox); arts. 60, 60 bis (60a en la versión inglesa) y 61 (pruebas reales)	Decisión de ingeniería; no es una figura del Reglamento	Prácticas de desarrollo seguro (SSDF, capítulo 10)
Quién lo establece u opera	Autoridad competente del Estado miembro	El propio equipo de desarrollo	El equipo de desarrollo y su integración continua
Finalidad	Desarrollar, entrenar, validar y probar bajo un plan de pruebas, en su caso en condiciones reales	Contener los efectos de ejecutar código no fiable	Decidir si un artefacto es aceptable mediante puertas de calidad
Base de la autorización	Solicitud y admisión conforme al art. 58	Permisos y configuración del entorno	Criterios de aceptación definidos por el proyecto
Obligatoriedad	La obligación de garantizar cobertura recae en los Estados; la participación depende de solicitud y selección	Ninguna por sí misma	Ninguna por sí misma
Salida	Informe de salida y constancia de actividades, cuando proceda	Comportamiento observado en aislamiento	Resultado binario o graduado: pasa o no pasa
Límite principal	No exime ni certifica; no suspende el ordenamiento	No satisface por sí solo exigencias regulatorias	Superar pruebas finitas no demuestra seguridad

¿Bajo qué obligaciones? Solo esas figuras generan las obligaciones específicas de esas figuras; el Reglamento impone además otras obligaciones (arts. 5, 6, 8-15, 16-27, 50, cap. V, etc.). El aislamiento técnico y el *pipeline* de validación son instrumentos del proveedor para gestionar su propio riesgo; no son obligaciones que el Reglamento imponga como tales.

Se mantiene aquí el invariante del capítulo: *superar una batería finita de pruebas no demuestra seguridad*. Ni una prueba en condiciones reales, ni un espacio controlado, ni un *pipeline* verde convierten un sistema en «seguro». A lo sumo, acreditan que en las condiciones probadas no se observó un fallo determinado. La seguridad es una propiedad que se argumenta, no un certificado que se obtiene.

7.6. Análisis crítico: qué inspira la metodología docente y qué no se transfiere

El interés de los artículos 57 a 60 bis para una asignatura de diseño y programación seguras es *analógico*, no normativo. La lectura crítica exige separar con nitidez lo que puede inspirar una metodología docente de lo que **no** puede presentarse como exigencia legal.

Ideas que pueden inspirar una metodología docente (Propuesta docente).

- **El plan de pruebas como artefacto explícito.** Antes de probar, se declara qué se va a probar, con qué criterios y con qué límites. Es exactamente lo que la capítulo 6 exige para un protocolo de evaluación reproducible: el pre-registro.
- **La supervisión y la trazabilidad.** Un marco de prueba regulado deja evidencia de lo probado. Trasladado al aula, esto inspira el registro de sesión y la puerta de calidad: cada artefacto con su huella, cada decisión con su justificación.
- **La separación entre probar y comercializar.** Probar en un marco controlado no es lanzar el producto. La idea se traduce en la distinción entre el entorno de validación y el de producción.
- **La proporcionalidad al riesgo.** El esfuerzo de prueba y de salvaguarda debe corresponder al daño potencial. En el caso transversal, no es lo mismo un CSV de juguete que un fichero que llega de un tercero no confiable.

Ideas que NO pueden transferirse como exigencias legales (Hipótesis/evidencia deliberadamente negativa).

- **No se puede afirmar que el aula necesite un «sandbox regulatorio».** Un sandbox regulatorio lo establece una autoridad competente conforme al Reglamento; un laboratorio universitario no lo es. Llamar «sandbox regulatorio» a un contenedor de prácticas es un uso impropio del término.

- **No se puede reutilizar datos personales al amparo del artículo 59.** La eventual habilitación para el tratamiento ulterior en interés público está acotada a un marco supervisado y a una finalidad concreta. No es una licencia general.

- **No se puede invocar el artículo 60 para hacer pruebas en condiciones reales en un trabajo de curso.** Las salvaguardas -información, consentimiento, aprobación, límites- son requisitos jurídicos, no pasos opcionales.

- **No se puede tratar el *pipeline* de validación como cumplimiento.** Un pipeline verde es evidencia técnica de haber ejecutado ciertas comprobaciones; no es una declaración de conformidad ni una autorización.

Nota

Regla de traducción aula-Derecho. Antes de tomar una idea de los artículos 57 a 60 bis 60 para la práctica docente, hay que responder a dos preguntas: (1) ¿la idea es un *principio de organización* -plan, supervisión, trazabilidad, proporcionalidad- o un *mandato dirigido a un sujeto concreto*? (2) Si es un mandato, *¿quién* es el sujeto obligado: un Estado miembro, una autoridad competente o un proveedor? Solo el primer caso se traduce sin más al aula; el segundo exige verificar el articulado y no atribuir la obligación a quien no la tiene.

7.7. Caso práctico de clasificación

Considérense dos escenarios que, a primera vista, parecen el mismo, y que sin embargo pueden recibir tratamientos distintos. La advertencia metodológica es previa a la respuesta: **la clasificación depende del uso previsto y del contexto, no del lenguaje de programación ni de la herramienta con la que se escribió el código** (artículos 3 y 6 y anexos I y III, según el caso).

Escenario 1 - Asistente que genera código para una aplicación convencional.

Un equipo usa un asistente de IA para escribir el conversor de CSV a PDF/XLSX. El asistente propone funciones, correcciones y pruebas. El producto final es una aplicación que procesa ficheros según reglas explícitas. El asistente intervino en el proceso de producción; el producto no incorpora inferencia aprendida que decida sobre las personas. En principio, el *producto* es software convencional y su calificación como sistema de IA regulado no se activa por el mero hecho de haber sido generado con IA. Ello *no* lo exonera de otras normas ni reduce las obligaciones de seguridad y protección de datos que le sean aplicables por otras vías.

Escenario 2 - Sistema de IA integrado en un producto de alto riesgo.

El mismo equipo añade un componente que, a partir de los datos del CSV, *infiere* si un expediente debe revisarse o denegarse, y ese componente se incorpora a un producto cuyo uso previsto cae en un ámbito considerado de alto riesgo (por ejemplo, según el contexto y la finalidad). Aquí cambia la naturaleza: el producto incorpora un sistema de IA, y su uso previsto -no la técnica concreta- es lo que arrastra el régimen de obligaciones correspondiente. La calificación no depende de que el componente «parezca inteligente», sino de que sea un sistema de IA y de cuál sea su finalidad en el contexto de uso.

Tabla 7.2.: Caso práctico de clasificación. La fila no se decide por la tecnología empleada para escribir el código, sino por la naturaleza del producto y su uso previsto (artículos 3 y 6, y anexos I y III, según el caso).

Pregunta	**Escenario 1: aplicación convencional**	**Escenario 2: producto de alto riesgo**
Papel de la IA	Asiste en la producción del código	Forma parte del producto
Naturaleza del producto	Software de reglas explícitas	Sistema que infiere a partir de datos
Uso previsto	Convertir ficheros	Decidir o apoyar decisiones sobre personas
Calificación	No se activa por el uso del asistente	Puede quedar sujeta al Reglamento según el uso
Error típico	«Lo escribió una IA, luego está regulado»	«Es solo un módulo, luego no cuenta»
Consecuencia docente	El foco es la seguridad del código	El foco añade obligaciones del uso previsto

Advertencia

La frontera es el uso previsto y el contexto. Dos productos con el mismo código pueden clasificarse de forma distinta si su finalidad y su contexto de uso difieren. Por eso el material docente no debe dar una respuesta de clasificación «de memoria»: debe exigir al estudiante que *justifique* la naturaleza del sistema, su uso previsto y el contexto, y que fundamente con el articulado la clasificación que proponga.

7.8. Preguntas de reflexión

1. ¿Por qué el hecho de que un programa se haya escrito con ayuda de un asistente de IA no basta para calificarlo como sistema de IA regulado? Enunciad el criterio que sí es relevante.

2. Un equipo llama «sandbox regulatorio» a su contenedor de pruebas. ¿Qué error terminológico comete y qué consecuencias tiene ese error al razonar sobre obligaciones?

3. Comparad el *plan de pruebas* del artículo 57 con el pre-registro de la capítulo 6. ¿Qué elementos comparten y en qué se diferencian por su naturaleza jurídica o metodológica?

4. ¿Por qué la ejecución aislada y la prueba en condiciones reales persiguen fines opuestos? Dad un ejemplo del caso transversal en el que ambas sean apropiadas en momentos distintos.

5. El artículo 59 contempla un tratamiento ulterior de datos personales en un marco supervisado. ¿Por qué no puede invocarse esa previsión para reutilizar datos personales en un proyecto de curso?

6. ¿Por qué difieren las fechas de aplicación del artículo 113 y el plazo del artículo 57? ¿Qué debe comprobar un ingeniero antes de afirmar que una obligación es exigible?

7. Tomad una decisión de diseño del caso transversal (por ejemplo, rechazar filas con formato inválido) y determinad si corresponde a *software convencional* o a un *sistema de IA*, justificando el uso previsto y el contexto.

8. ¿Por qué superar un *pipeline* de validación no puede presentarse como prueba de conformidad con el Reglamento? Relacionadlo con el invariante «pruebas finitas $\neq$ seguridad».

7.9. Referencias oficiales y criterios de lectura

La referencia principal es el Reglamento (UE) 2024/1689 [1], en la versión consolidada de 27 de julio de 2026, con las modificaciones del Reglamento (UE) 2026/1744 [2]. Las afirmaciones jurídicas de este capítulo remiten a artículos y apartados concretos; el estudiante debe consultarlos antes de formular conclusiones sobre un sistema real.

- EUR-Lex: Reglamento (UE) 2024/1689, versión consolidada de 27-07-2026.
- EUR-Lex: Reglamento (UE) 2026/1744, acto modificativo.
- AI Act Explorer: herramienta secundaria de navegación, no texto normativo auténtico.
- Comisión Europea, AI Act Service Desk, art.~57 y, en el mismo portal, artículos 58, 59, 60 y 113: presentación basada en la consolidación de 27-07-2026.
- Artículos 3, 5, 6, 57-61, 111 y 113; anexos I y III, según la cuestión examinada.

Nota

Tres niveles de afirmación. (1) *Regla jurídica*: respaldada por el artículo indicado; (2) *interpretación de ingeniería*: clasificación técnica razonada de un artefacto y sus riesgos; (3) *propuesta docente*: adopción voluntaria de planes de prueba, supervisión, aislamiento y trazabilidad. El tercer nivel no debe presentarse como una obligación legal derivada del primero.

8. Confianza no calibrada: el riesgo epistémico del código generado por IA

8.1. Una distinción preliminar: error técnico y riesgo epistémico

El código incorrecto no es un fenómeno nuevo. Antes de los sistemas de generación automática, los desarrolladores producían código con errores, con supuestos tácitos erróneos y con invariantes de seguridad omitidas. La revisión de código, las pruebas y el modelado de amenazas existen precisamente porque la producción manual de código no garantiza su corrección.

Lo que distingue al código generado por modelos de lenguaje no es, por tanto, que sea incorrecto: es que es **plausiblemente incorrecto**. Esta distinción no es de grado sino de naturaleza. El código plausiblemente incorrecto supera las comprobaciones que activan la desconfianza: compila, está bien formateado, usa nombres semánticamente coherentes, pasa los casos de prueba que el desarrollador elige, y se ajusta a los patrones del proyecto. La señal que normalmente indica «este código merece revisión cuidadosa» no se activa. El defecto queda latente no porque nadie lo buscara, sino porque nadie supo que debía buscarlo.

Este es el **riesgo epistémico** del código generado: no el riesgo de que el código sea malo, sino el riesgo de no saber cuánto confiar en él. Un riesgo técnico se mitiga con herramientas técnicas. Un riesgo epistémico requiere, antes que herramientas, una teoría de qué se sabe y qué no se sabe sobre el artefacto que se está evaluando.

Nota

La hipótesis rectora de este capítulo es que la plausibilidad superficial del código generado por IA aumenta la probabilidad de confianza no calibrada respecto al código escrito manualmente. Esta afirmación se etiqueta como **Hipótesis**: hay evidencia parcial que la respalda -incluyendo los patrones documentados en la práctica del equipo de Claude Code [19]- pero no se ha demostrado de manera sistemática y controlada.

8.2. La trampa de la plausibilidad

El cerebro humano infiere la calidad de un artefacto a partir de señales de superficie. En el código escrito por un colega experimentado, estas señales están correlacionadas con comprensión real: si el código está bien estructurado, usa abstracciones adecuadas y maneja los casos de error explícitamente, es porque quien lo escribió pensó en ellos. La señal superficial es un proxy razonablemente fiable de la señal profunda.

Con el código generado por un modelo de lenguaje, esta correlación se rompe. El modelo optimiza la señal superficial directamente: aprende de código existente que fue considerado bueno, y reproduce sus propiedades formales. El resultado es código que *parece* correcto porque comparte las propiedades de forma del código correcto, sin que eso implique que comparte sus propiedades de fondo.

Formulación técnica. Un modelo de lenguaje aprende la distribución de probabilidad sobre secuencias de tokens en código considerado de calidad. Genera código que es probable bajo esa distribución. La probabilidad bajo la distribución no implica corrección respecto a una especificación no observada por el modelo.

Consecuencia práctica. El revisor humano, al ver código bien formateado que compila y pasa los tests visibles, activa un modo de revisión de baja intensidad. La revisión de alta intensidad se reserva para código que activa señales de alarma. El código generado suprime esas señales.

La *Ley de Kernighan* -véase el capítulo 2- describe una asimetría cognitiva entre escribir y depurar código. Con código generado, esa asimetría se agrava en dos dimensiones simultáneas. Primero, el «escritor» es un modelo que no comparte contexto con el revisor: no hay intención que el revisor pueda recuperar, solo un artefacto. Segundo, el código generado puede ser más «listo» -más compacto, más idiomático, más ajustado a los patrones del proyecto- de lo que el revisor habría producido. La trampa de la plausibilidad es, en parte, una trampa de la competencia aparente.

Advertencia

Un artefacto que supera las comprobaciones superficiales no es más seguro que uno que no las supera. Es más difícil de revisar con cuidado, precisamente porque las señales que activarían la revisión cuidadosa están ausentes. La plausibilidad no es una propiedad de seguridad.

8.2.1. El caso transversal

En el conversor CSV-PDF-XLSX, la trampa de la plausibilidad se manifiesta de manera concreta. Un modelo al que se le pide que genere la función de escritura XLSX produce código que:

- Importa la biblioteca correcta (`openpyxl` o equivalente).
- Itera sobre los registros de manera idiomática.
- Usa nombres de variables coherentes con el resto del proyecto.
- Genera un archivo válido que se abre correctamente en Excel o LibreOffice.

Lo que el código no hace, y que ninguna de estas señales superficiales indica, es:

- Verificar que la ruta de salida está dentro del directorio autorizado.
- Escapar los prefijos de fórmula en las celdas.
- Garantizar que no queda un archivo parcial en caso de error.
- Acotar el tiempo y la memoria de la operación.

El revisor que ve el código por primera vez, que funciona para los casos que prueba y que está bien formateado, no tiene señal de que falte nada. La omisión es invisible porque el código que *debería* estar no produce ningún artefacto observable cuando no está.

8.3. Confianza como variable continua

La confianza en un artefacto de software no es binaria. Entre «confío completamente» y «no confío en absoluto» existe un espectro continuo, y la posición en ese espectro debe calibrarse con evidencia. El problema de la confianza no calibrada es precisamente que el desarrollador toma una posición implícita -a menudo hacia el extremo de confianza alta- sin haber examinado la evidencia disponible.

Se propone un marco de tres dimensiones para calibrar la confianza en código generado. Ninguna dimensión es suficiente por sí sola; las tres deben considerarse conjuntamente.

8.3.1. Cobertura de especificación

¿Qué fracción del comportamiento esperado está enunciada de manera verificable? Un contrato que especifica el caso de éxito pero no las condiciones de error, o que enuncia invariantes informalmente pero no las hace verificables, tiene cobertura de especificación baja. La confianza en el código no puede superar la cobertura de la especificación contra la que se verifica: si no se especificó qué debe ocurrir ante una ruta maliciosa, no se puede confiar en que el código la rechaza.

8.3.2. Independencia de verificación

¿Quién verificó el código, y qué supuestos comparte con el generador? Un agente revisor que usa el mismo modelo, el mismo contexto y los mismos ejemplos que el agente generador tiene una independencia de verificación baja: ambos son instancias de la misma distribución de probabilidad. La independencia de verificación se mide por la *diversidad de supuestos* entre generador y verificador, no por el número de verificadores.

El equipo de Claude Code [19] aborda esto con subagentes de contexto fresco: el revisor no tiene acceso al historial de razonamiento del generador, solo al artefacto producido. Esta es una instancia del principio general: la independencia de contexto no es una práctica de proceso, es una **propiedad de diseño** del sistema de verificación.

8.3.3. Reversibilidad

¿Qué cuesta deshacer el cambio si falla? La reversibilidad no reduce la probabilidad de fallo; reduce el coste de un fallo que ocurra. Un cambio altamente reversible -detrás de un flag de funcionalidad, en un entorno de prueba, con un rollback de un paso- puede aceptarse con una confianza menor que un cambio irreversible -una migración de base de datos, una modificación del formato de un artefacto persistente, una API pública.
Este marco es una **Hipótesis** operativa, no un modelo validado. Su valor no es predictivo sino normativo: obliga al desarrollador a articular explícitamente las dimensiones en las que su confianza está o no está respaldada por evidencia.

Conviene no confundir tres capacidades que la reversibilidad pone en juego, porque cada una condiciona de forma distinta las consecuencias de una confianza injustificada:

Reversibilidad. Posibilidad de deshacer determinados efectos de una operación (por ejemplo, retirar un cambio de código, desactivar una funcionalidad tras un flag o deshacer una escritura concreta). No implica necesariamente restaurar el estado del sistema.

Tabla 8.1.: Marco de calibración de confianza en código generado.

Dimensión	**Confianza alta justificada**	**Confianza alta injustificada**
Cobertura de especificación	Contrato completo con invariantes de seguridad verificables	Solo caso positivo enunciado, casos de error implícitos
Independencia de verificación	Revisor con contexto fresco, supuestos distintos al generador	Revisor comparte modelo, contexto o ejemplos con el generador
Reversibilidad	Cambio detrás de flag, rollback de un paso, sin estado persistente	Migración irreversible, API pública, formato de archivo persistente

Recuperabilidad. Capacidad para restablecer un estado de servicio aceptable después de una perturbación. Puede requerir operaciones que no sean la simple inversión de la original (restauración de datos, reconciliación, compensación).

Resiliencia. Capacidad más amplia para mantener, adaptar o recuperar funciones ante perturbaciones, incluso cuando la recuperación exacta no es posible.

Estas distinciones son operativas, no equivalencias normativas universales: un cambio puede ser reversible y, aun así, dejar el sistema en un estado degradado, o ser irreversible en sus efectos externos y, sin embargo, recuperable mediante compensación. El riesgo residual que una confianza injustificada deja tras de sí depende, de forma cualitativa, de la probabilidad del fallo, de su impacto, de la facilidad para detectarlo, de la capacidad de recuperación y de la presencia de efectos irreversibles. No se trata aquí como una fórmula cuantitativa -no se definen escalas ni supuestos-, sino como una lista de dimensiones que la aceptación debe considerar.

El tratamiento *operativo* de estas capacidades -despliegue controlado, observabilidad, procedimientos de recuperación y prueba periódica de los mismos- corresponde al capítulo de metodología; véase la sección 12.16. Este capítulo se limita a situar la reversibilidad como condición de la confianza, no como un procedimiento.

8.4. El sesgo del caso positivo y su circularidad

Los modelos de lenguaje generan implementaciones que satisfacen el caso descrito en el enunciado. Esta propiedad no es un defecto del modelo: es una consecuencia de cómo aprende. El modelo ha observado código que resuelve problemas descritos de cierta

manera, y aprende a producir código que resuelve problemas descritos de esa manera. Los casos que no se describen en el enunciado no están representados en la distribución de salida.

El problema se agrava por una circularidad cognitiva que a menudo pasa inadvertida: el desarrollador que escribe el enunciado y el modelo que genera la implementación comparten, al menos parcialmente, el mismo espacio de casos imaginables. El desarrollador imagina los casos que le parecen relevantes; el modelo genera código que los maneja; el desarrollador prueba los casos que imagina; el código los pasa. La circularidad es perfecta, y los casos fuera del espacio compartido permanecen invisibles.

Nota

Esta circularidad no es exclusiva del código generado por IA. El desarrollador que escribe su propio código y luego lo prueba está sujeto a la misma dinámica: tiende a probar los casos que imaginó al escribirlo. La diferencia es que con código propio existe la posibilidad de que el proceso de escritura haya revelado casos nuevos. Con código generado, el proceso de escritura no ocurrió: solo existe el artefacto final.

Las **pruebas negativas** son la única salida sistemática de esta circularidad. Una prueba negativa no pregunta «¿hace el código lo que debe?» sino «¿rechaza el código lo que no debe aceptar?». Para diseñar pruebas negativas es necesario especificar las invariantes que deben preservarse -véase la sección 2.12- y luego construir activamente entradas que intenten violarlas. Este proceso es, por definición, externo al espacio de casos que el modelo considera naturales.

Listing 8.1: La circularidad del caso positivo y su ruptura mediante pruebas negativas (ejemplo didáctico).

```
# Lo que el modelo genera de manera natural: casos positivos
def test_exportacion_normal():
    registros = [["nombre", "valor"], ["Alice", "42"]]
    ruta = DIRECTORIO / "resultado.xlsx"
    exportar_xlsx(registros, ruta)
    assert ruta.exists()
    # Pasa. El modelo imagino este caso; el desarrollador tambien.

# Lo que requiere salir del espacio compartido: casos negativos
def test_formula_en_primera_celda():
    # Ni el modelo ni el desarrollador imaginaron esto al escribir
        el enunciado
```

```
    registros = [["=HYPERLINK(\"http://malo.ejemplo\",\"clic\")",
        "valor"]]
    ruta = DIRECTORIO / "resultado.xlsx"
    exportar_xlsx(registros, ruta)
    libro = openpyxl.load_workbook(ruta)
    celda = libro.active.cell(row=1, column=1).value
    # La prueba falla si el codigo generado no escapo el prefijo
    assert not celda.startswith("="), f"Celda no escapada: {celda!
        r}"

def test_path_traversal_en_nombre_de_archivo():
    # Tampoco estaba en el enunciado original
    ruta_maliciosa = DIRECTORIO / "../secretos.xlsx"
    with pytest.raises(ValueError):
        exportar_xlsx([["dato"]], ruta_maliciosa)
    # Si el codigo generado no verifica la ruta, este test revela
        la omision
```

El listado 8.1 no es un conjunto completo de pruebas negativas; ilustra la diferencia de origen entre los dos tipos. Los casos positivos emergen naturalmente del enunciado. Los casos negativos requieren un análisis externo de las invariantes y de los vectores de amenaza que las pueden violar.

8.5. Independencia de contexto como propiedad de diseño

La revisión es inútil si el revisor comparte los supuestos del generador. Esta afirmación parece obvia pero tiene consecuencias que no siempre se extraen.

En el desarrollo manual, la revisión por pares proporciona independencia de contexto de manera casi automática: el revisor no estaba en la sesión de programación, no vio el razonamiento intermedio, no tiene el mismo sesgo de confirmación que el autor. La independencia de contexto es una consecuencia natural de que dos personas distintas participen en el proceso.

Con el desarrollo asistido por IA, esta consecuencia natural desaparece. Si el mismo desarrollador genera el código con un agente, luego pide al mismo agente que lo revise, y finalmente pide al mismo agente que genere los tests, las tres operaciones comparten el mismo contexto, los mismos supuestos y la misma distribución de probabilidad sobre los casos relevantes. El número de participantes en el proceso es uno, no tres.

Independencia de modelo. Un revisor que usa un modelo diferente al generador tiene

independencia de modelo. Los dos modelos pueden compartir datos de entrenamiento, pero difieren en arquitectura, ajuste fino y distribución de salida. Esta independencia es parcial pero no trivial.

Independencia de contexto. Un revisor que opera sin acceso al historial de razonamiento del generador tiene independencia de contexto. Solo ve el artefacto producido y la especificación. Esta es la forma más importante de independencia para la revisión de seguridad: obliga al revisor a evaluar el artefacto por lo que es, no por lo que el generador intentaba hacer.

Independencia de enunciado. Un revisor cuyo enunciado es la especificación, no la descripción de lo que el generador hizo, tiene independencia de enunciado. La diferencia es sutil pero importante: «¿cumple este código lo que se le pidió hacer?» no es la misma pregunta que «¿cumple este código la especificación de seguridad?».

El equipo de Claude Code implementa independencia de contexto mediante subagentes con contexto fresco [19]. La literatura clásica de ingeniería del software la implementa mediante revisión por pares externa, equipos de prueba independientes y auditorías de seguridad separadas del equipo de desarrollo. El mecanismo difiere; el principio es el mismo. El detalle operativo de la revisión de un *diff* por un revisor con contexto independiente se desarrolla en la sección 11.8.

Consejo

Criterio operativo: antes de considerar que una revisión es independiente, pregúntese qué supuestos comparte el revisor con el generador. Si el revisor vio el mismo enunciado, operó en el mismo contexto o usó el mismo modelo con el mismo historial, la independencia es parcial. La independencia parcial es mejor que ninguna; pero debe reconocerse como parcial al calibrar la confianza.

8.6. El generador en el modelo de amenazas

El modelado de amenazas -véase la sección 2.10- identifica actores, vectores y activos para diseñar controles proporcionales. Con el desarrollo asistido por IA, el modelo generador debe aparecer explícitamente en el modelo de amenazas del sistema. No como un actor malicioso, sino como una **fuente de código no verificado** con características de fallo propias y predecibles.

Incluir el generador en el modelo de amenazas no implica desconfiar de él en sentido general; implica ser explícito sobre qué tipos de fallo son más probables y qué controles los mitigan. Un modelo de amenazas que omite el generador como fuente de riesgo es incompleto de la misma manera que lo sería uno que omite las dependencias de terceros o las entradas de usuario.

Tabla 8.2.: El modelo generador como fuente de riesgo en el modelo de amenazas.

Característica de fallo	**Manifestación**	**Control**
Sesgo hacia el caso positivo	Precondiciones implícitas, tratamiento de errores ausente	Especificación explícita de invariantes; pruebas negativas
Plausibilidad sin corrección	Código que pasa revisión superficial con supuestos incorrectos	Revisión con independencia de contexto; pruebas de propiedad
Replicación de patrones inseguros	Patrones del corpus de entrenamiento reproducidos con alta confianza	Revisión de seguridad especializada; análisis estático
Ausencia de modelo del sistema	Supuestos incorrectos sobre versiones, permisos, concurrencia	Especificación del entorno en el enunciado; pruebas de integración
Convergencia indefinida	Bucle generador-revisor sin terminación	Límite de rondas; revisión diferencial tras la primera pasada

La última fila de la tabla 8.2 merece atención específica. El equipo de Claude Code documenta un caso en el que un agente autor y un agente revisor interactuaron durante más de cincuenta rondas sin convergencia, porque el revisor seguía detectando problemas en código preexistente no modificado por la revisión en curso [19] (el relato completo se desarrolla en la sección 11.8). Este patrón no es una curiosidad: es una instancia de un fallo de diseño del sistema de verificación. Un proceso de revisión que no converge no es una propiedad de seguridad; es una fuente de ruido que degrada la señal y agota la atención humana.

La solución adoptada -límite de rondas, revisión diferencial, prohibición de repetir hallazgos ya publicados- es una instancia de la *economía de mecanismos* de Saltzer y Schroeder: los mecanismos de protección deben ser tan simples como sea razonable para que puedan comprenderse y revisarse. Un sistema de revisión con comportamiento emergente impredecible no es revisable.

8.7. Responsabilidad epistémica

La norma «lo integras, lo posees» (*you land it, you own it*) del equipo de Claude Code [19] tiene una lectura superficial -asignación de culpa en caso de fallo- y una lectura más profunda: es una **asignación de responsabilidad epistémica**. La persona que aprueba y fusiona un cambio debe ser capaz de articular qué sabe y qué no sabe sobre ese cambio.

Esta responsabilidad no desaparece cuando el código lo escribe un agente; se transforma. Antes, la responsabilidad incluía haber razonado sobre la implementación. Ahora incluye haber razonado sobre la *verificación*: qué comprobaciones automáticas cubrieron qué propiedades, qué quedó fuera de cobertura y qué riesgo implica esa brecha.

El criterio operativo del artículo de referencia. «Si no puedes decir qué fallarían las comprobaciones, pide una revisión real en lugar de una aprobación rápida» [19]. Esta formulación es una instancia del principio de *aceptabilidad psicológica* de Saltzer y Schroeder: los controles deben ser comprensibles para quienes los aplican. Un flujo de aprobación que el revisor no comprende no es un control de seguridad; es una ceremonia.

La aprobación automática como no-sustituto. El artículo es explícito: la aprobación de un agente nunca cuenta como la aprobación de una persona, ni en los caminos de bajo riesgo. Esto no es una política arbitraria; es una consecuencia del argumento sobre independencia de contexto: un agente aprobando el trabajo de otro agente con supuestos compartidos no añade independencia epistémica.

La cobertura como condición de la aprobación. Una aprobación válida requiere poder responder: (1) qué invariantes de seguridad debe preservar este código, (2) qué verificación cubre cada invariante, (3) qué queda sin cubrir y qué riesgo implica. Si alguna pregunta no tiene respuesta, la especificación es incompleta y la aprobación es epistémicamente injustificada.

Advertencia

La responsabilidad epistémica no puede delegarse en la herramienta que generó el código. «El agente dijo que era correcto» no es evidencia de corrección; es evidencia de que el agente no encontró lo que buscaba con los supuestos con los que lo buscó. La diferencia entre evidencia positiva y ausencia de evidencia negativa es, precisamente, el núcleo del riesgo epistémico.

8.8. Qué cambia y qué permanece

Tabla 8.3.: Efecto de la IA sobre las dimensiones epistémicas del desarrollo.

Dimensión	**Lo que abarata la IA**	**Lo que exige o encarece**
Producción de artefactos	Generar código plausible en volumen	Saber cuánto confiar en cada artefacto
Cobertura de especificación	Generar borradores de contrato a partir del código	Validar que el contrato captura las invariantes de seguridad
Verificación	Escalar la cobertura de revisión automática	Mantener independencia de contexto entre generador y revisor
Pruebas	Producir casos de prueba positivos	Diseñar casos negativos que rompan el espacio de supuestos compartidos
Responsabilidad	Producir argumentos de corrección plausibles	Articular qué sabe y qué no sabe el aprobador sobre el artefacto
Gestión del riesgo	Generar mecanismos de rollback y flags de funcionalidad	Calibrar el nivel de confianza requerido para cada nivel de reversibilidad

La tabla 8.3 refleja la misma asimetría que recorre la sección anterior (capítulo 6): la IA abarata la producción de artefactos -código, contratos, pruebas, argumentos- sin reducir proporcionalmente el coste epistémico de saber cuánto valen. Esta asimetría no es un defecto de los modelos actuales que versiones futuras resolverán; es una consecuencia estructural de que los modelos optimizan la plausibilidad bajo una distribución aprendida, y la plausibilidad no es corrección.

8.9. Ejercicios

1. **Diagnóstico de plausibilidad.** Pida a un asistente de IA que genere una función de exportación para el caso transversal sin especificar las invariantes de seguridad. Examine el código producido e identifique al menos tres propiedades que el código no garantiza pero que un revisor superficial podría asumir que garantiza. Explique por qué cada propiedad omitida no produce ningún artefacto observable en el código.

2. **Calibración de confianza.** Aplique el marco de la tabla 8.1 a un cambio reciente

en un proyecto propio o en el caso transversal. Para cada dimensión, indique si la confianza está justificada y qué evidencia respalda la respuesta. ¿Hay dimensiones en las que la confianza es alta sin respaldo explícito?

3. **Ruptura de circularidad.** Para el caso transversal, diseñe un conjunto de cinco pruebas negativas que salgan del espacio de casos que el enunciado natural sugiere. Para cada prueba, indique el vector de amenaza, la invariante que verifica y por qué ese caso no habría emergido de un enunciado típico.

4. **Independencia de verificación.** Proponga un protocolo de revisión para un equipo pequeño que usa el mismo modelo de IA para generar y para revisar código. El protocolo debe maximizar la independencia de contexto dado ese constraint. Indique qué tipos de fallo seguirán sin cobertura independientemente del protocolo.

5. **El generador en el modelo de amenazas.** Extienda la tabla 8.2 añadiendo dos características de fallo adicionales del modelo generador que sean relevantes para el caso transversal. Para cada una, proponga un control y etiquete su efectividad como **Hipótesis** o **Evidencia**.

6. **Responsabilidad epistémica.** Un compañero propone aprobar un PR generado por un agente argumentando que «pasó todos los checks automáticos y el agente revisor no encontró nada». Redacte una respuesta breve que distinga ausencia de evidencia negativa de evidencia positiva de corrección, y proponga las tres preguntas que el aprobador debe poder responder antes de fusionar el cambio.

7. **Síntesis.** A partir de la tabla 8.3, escriba un párrafo que argumente por qué la automatización del desarrollo no reduce el riesgo epistémico, sino que lo desplaza. Etiquete cada afirmación central como **Hipótesis** o **Evidencia**, y justifique la etiqueta.

8.10. Síntesis

El riesgo específico del código generado por IA no es que sea incorrecto; es que la señal que normalmente activa la revisión cuidadosa -la dificultad de producción, las marcas del razonamiento intermedio, las imperfecciones superficiales- está ausente. El código generado es plausiblemente incorrecto: supera las comprobaciones superficiales sin que eso implique corrección respecto a una especificación de seguridad.

Mitigar este riesgo requiere tres compromisos que no pueden delegarse en la herramienta generadora:

1. **Especificar antes de generar.** Las invariantes de seguridad deben enunciarse antes de que el modelo produzca el código, no inferirse a partir del código producido. Un contrato extraído del código por el mismo modelo que lo generó hereda los supuestos tácitos del generador.

2. **Verificar con independencia de contexto.** La revisión debe separarse del proceso de generación mediante diferencia de modelo, de contexto o de enunciado. La independencia parcial es mejor que ninguna; debe reconocerse y calibrarse.

3. **Calibrar la confianza explícitamente.** Antes de integrar un cambio, el aprobador debe poder articular la cobertura de especificación, el grado de independencia de verificación y el nivel de reversibilidad del cambio. Si alguna dimensión no tiene respuesta, la confianza no está calibrada.

Estos compromisos no son nuevos en ingeniería del software; son los que distinguen el desarrollo disciplinado del desarrollo descuidado. Lo que la IA cambia es la urgencia de hacerlos explícitos: a mayor velocidad de producción, mayor es el coste de no haberlos respetado.

Nota

La hipótesis rectora de este capítulo -que la plausibilidad superficial del código generado aumenta la probabilidad de confianza no calibrada- no ha sido demostrada de manera sistemática. La evidencia operativa disponible [19] la respalda parcialmente, en un contexto específico. Extrapolarla a otros contextos requiere el mismo criterio epistémico que el capítulo ha propuesto aplicar al código: especificar las condiciones de validez, verificar con independencia de contexto y calibrar la confianza de manera explícita.

9. Aislamiento técnico y riesgos de los agentes

La capítulo 7 distinguió el *espacio controlado de pruebas regulatorio* (arts. 57-58 del AI Act [1]) del *sandbox técnico de ejecución*. Este capítulo desarrolla el segundo. Cuando el agente pasa de generar texto a *actuar* -ejecutar comandos, escribir ficheros, instalar paquetes, leer secretos- el problema de seguridad cambia de naturaleza. Invariante del capítulo: superar una batería finita de pruebas no demuestra seguridad; un *pipeline* verde es, a lo sumo, la ausencia observada de fallo.

9.1. Dos objetos de riesgo: el código y las acciones

Conviene separar con nitidez dos objetos que a menudo se confunden porque comparten el mismo protagonista (el asistente o el agente) pero tienen amenazas distintas.

El primer objeto es el **código producido**: el artefacto que el asistente escribe. Sus riesgos son los del software en general -fallos de validación, deserialización insegura, *path traversal*, *TOCTOU*, desbordamientos, gestión incorrecta de secretos- y se estudian, en el caso transversal del CSV convertido a PDF y XLSX, en el capítulo 4. La pregunta que los define es: *¿qué propiedades tiene el artefacto?*

El segundo objeto son las **acciones del agente durante el desarrollo**: lo que el sistema hace con las herramientas mientras trabaja. La pregunta que las define es distinta: *¿qué puede hacer el agente, sobre qué recursos, con qué autoridad y con qué consecuencias?* Un agente que nunca ha escrito una línea vulnerable puede, aun así, exfiltrar un secreto, borrar un directorio o introducir una dependencia comprometida. Y un código impecable puede ser el resultado de un proceso de desarrollo que ha expuesto el entorno.

Tabla 9.1.: Dos objetos de riesgo. La columna izquierda describe el artefacto; la derecha, el proceso de desarrollo. Ambos son necesarios y ninguno sustituye al otro.

Dimensión	**Código producido**	**Acciones del agente**
Qué es	El artefacto escrito (fichero, módulo, *script*)	El comportamiento del agente mientras trabaja
Pregunta guía	¿Qué propiedades tiene el artefacto?	¿Qué puede hacer el agente y con qué autoridad?
Momento	Después de generar	Durante el desarrollo
Ejemplo en el caso transversal	Fórmula de Excel no escapada; ruta no canónica	El agente abre una conexión de red o instala un paquete
Marco de análisis	Familias CWE [6] (p. ej., CWE-22, CWE-367, CWE-502, CWE-798 y CWE-400)	Modelo de amenazas y límites de confianza
Control típico	Revisión, SAST, pruebas, validación de entrada	Aislamiento, privilegios mínimos, auditoría, aprobaciones
Límite	Superar pruebas finitas no demuestra seguridad	El aislamiento reduce el daño; no certifica seguridad

Nota

Etiquetado epistémico. La distinción entre el riesgo del artefacto y el riesgo del proceso es **Evidencia** en el terreno de la ingeniería: son objetos de análisis diferentes, con marcos y controles diferentes. Que un agente sea «el mismo sistema» en ambos casos no implica que las dos preguntas se respondan igual.

9.2. Modelo de amenazas y límites de confianza

Un *modelo de amenazas* es una declaración explícita y acotada de qué se protege, de quién se protege, con qué capacidades y bajo qué supuestos. No es una lista de vulnerabilidades conocidas: es el encuadre que permite decidir qué controles son pertinentes y cuáles son inútiles. Su valor docente es que obliga a escribir los supuestos, y por tanto a hacer visibles los que se están dando por ciertos sin comprobarlos.

Activos que se protegen.

- **Integridad del repositorio**: el historial y el código fuente, que no deben modificarse sin autorización.
- **Confidencialidad de los secretos**: credenciales, claves de API, *tokens* y datos personales que el entorno pueda contener.

- **Integridad del sistema anfitrión**: los ficheros, procesos y servicios fuera del ámbito de la tarea.

- **Disponibilidad del servicio**: los recursos compartidos (CPU, memoria, disco, red) que un agente descontrolado puede agotar.

- **Integridad de la cadena de suministro**: el origen y la autenticidad de las dependencias y de las herramientas usadas.

Adversario y capacidades.

En este modelo el adversario relevante no es únicamente un atacante externo. Es también el *contenido no fiable* que el agente procesa -un CSV de un tercero, un *README* de un repositorio, una página web que el agente lee- y que puede llevar instrucciones hostiles o datos manipulados. Se asume que ese contenido puede intentar influir en las decisiones del agente, y que el agente no distingue por sí mismo entre datos y órdenes. Los supuestos que hay que declarar son, al menos: qué red se permite, qué directorios se montan, qué identidad tiene el proceso, qué credenciales ve y durante cuánto tiempo.

Límites de confianza.

Un *límite de confianza* es la frontera donde el nivel de confianza cambia. En el flujo agente-herramientas-entorno hay al menos tres zonas con confianza decreciente: el *agente* (cuya fiabilidad no se presupone), las *herramientas* (que actúan sobre recursos reales) y el *entorno* (el sistema anfitrión y la red, que no deben verse afectados). La figura 9.1 representa esas zonas.

La siguiente tabla enuncia el modelo de amenazas en forma operativa: cada fila asocia un activo, una clase de amenaza, la superficie por la que se materializa y un control candidato. La columna de etiqueta epistémica indica el grado de respaldo de la fila. Las debilidades técnicas del caso transversal se pueden relacionar con CWE-22 (rutas fuera de un directorio previsto), CWE-367 (*TOCTOU*), CWE-502 (deserialización de datos no fiables), CWE-798 (credenciales embebidas) y CWE-400 (consumo incontrolado de recursos) [6]. Son asociaciones orientativas: no todas las amenazas del agente equivalen a una CWE ni estos identificadores describen por sí solos una vulnerabilidad demostrada.

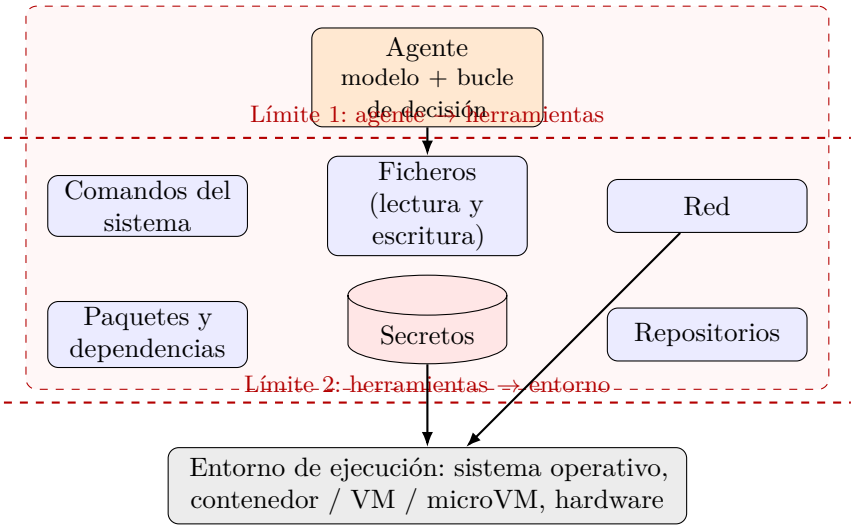


Figura 9.1.: Límite de confianza entre el agente, sus herramientas y el entorno. El rectángulo discontinuo encierra la zona aislada (agente y herramientas). El límite 1 señala lo que el agente puede invocar; el límite 2, lo que el aislamiento debe contener frente al entorno anfitrión. Las flechas hacia el entorno representan accesos que deben estar mediados por controles, no canales directos.

Tabla 9.2.: Modelo de amenazas del flujo agente-herramientas-entorno. Los identificadores CWE son referencias de clasificación, no pruebas de vulnerabilidad [6].

Activo	**Amenaza (clase)**	**Superficie**	**Control candidato**	**Etiqueta**
Repositorio	Escritura no autorizada	Herramienta de ficheros	Rama aislada; revisión obligatoria; aprobación	Evidencia
Secretos	Exfiltración	Variables de entorno; red	Minimización de secretos; red restringida	Propuesta
Anfitrión	Escape del confinamiento	Kernel; montajes	VM o microVM; sin montajes de escritura	Hipótesis
Disponibilidad	Agotamiento de recursos	CPU, memoria, disco	Cuotas de CPU/memoria; límite de tiempo	Propuesta
Cadena de suministro	Dependencia comprometida	Gestor de paquetes	Réplicas internas; verificación de artefactos	Hipótesis
Decisiones	*Prompt injection*	Contenido no fiable leído	Tratar el contenido como dato, no como orden	Propuesta

Nota

Etiquetado epistémico de la tabla. Se marca como **Evidencia** que la escritura no autorizada del repositorio es una amenaza real y que los controles de proceso -rama aislada, revisión- son prácticas consolidadas [3]. Se marca como **Propuesta docente** el conjunto de controles que se propone adoptar en el laboratorio. Se marca como **Hipótesis** que una configuración concreta de aislamiento sea suficiente frente a código hostil, porque la suficiencia no puede afirmarse sin verificación específica.

9.3. Superficies de acción del agente

Las herramientas que un agente puede invocar definen su *superficie de acción*. La lista siguiente no es exhaustiva, pero cubre las categorías que el modelo de amenazas debe declarar.

Comandos del sistema.

La herramienta más potente es la que ejecuta procesos arbitrarios. Su peligro es que la frontera entre «ejecutar la herramienta» y «ejecutar el contenido no fiable» es permeable: si un argumento del comando proviene de datos no confiables, el agente puede actuar como vector de inyección. El control no es adivinar qué comandos son peligrosos, sino restringir *qué* procesos se permiten, con qué identidad y en qué directorio.

Lectura y escritura de ficheros.

Leer permite exfiltrar; escribir permite destruir o introducir código. En el caso transversal, la escritura de salidas (PDF, XLSX) debe quedar confinada a un directorio efímero, con rutas canónicas y sin seguir enlaces simbólicos. La capítulo 4 estudió el *path traversal* y el *TOCTOU* como fallos del artefacto; aquí interesan como *acciones* del agente que un aislamiento de ficheros debe contener.

Red.

La red es el canal natural de exfiltración y de descarga de contenido no fiable. Por defecto, el aislamiento debe ser *sin red*; cuando la tarea requiere acceso, debe mediar una política explícita de destinos permitidos, no una apertura general.

Paquetes y dependencias.

Instalar paquetes durante la sesión introduce código de terceros en el proceso. Es una decisión de cadena de suministro: importa *de dónde* procede el paquete, si está fijado por versión y si su integridad se verifica. Añadir un formato o una biblioteca amplía la superficie de ataque, y esa ampliación debe justificarse frente a la alternativa de no hacerlo (equilibrio KISS/YAGNI del caso transversal).

Secretos.

Un agente con acceso a credenciales puede usarlas, pero también puede revelarlas en su salida, registrarlas en un fichero o transmitirlas. El principio de control de secretos es que el agente *no debe ver* más credenciales de las estrictamente necesarias para la tarea, y que las que ve deben ser de corta duración y de alcance mínimo.

Repositorios.

Escribir en un repositorio es una acción con efectos persistentes y compartidos. La distinción entre proponer un cambio y aplicarlo es fundamental: el agente puede preparar un cambio en una rama aislada, pero la incorporación debe requerir aprobación humana explícita.

Prompt injection.

Cuando el agente lee contenido no fiable (un CSV, un *README*, una página), ese contenido puede incluir instrucciones dirigidas a alterar su comportamiento [16, 17, 18]. La defensa conceptual no consiste en una lista de palabras prohibidas -que se elude- sino en no otorgar al contenido no fiable autoridad para decidir acciones: el contenido se trata como *dato*, y las acciones sensibles exigen una decisión que no provenga de ese dato.

Cadena de suministro.

La cadena de suministro abarca el origen del modelo, las herramientas, las dependencias y los artefactos que se descargan. Su integridad se apoya en la verificación de procedencia y en la reproducibilidad, no en la confianza por defecto.

Nota

La superficie de acción es una decisión de diseño. Cada herramienta habilitada debe corresponder a una necesidad de la tarea. Habilitar capacidades «por si acaso» amplía la superficie sin aportar requisitos, y contradice el principio de privilegios mínimos que se desarrolla más abajo. **Propuesta docente**: que el estudiante justifique por escrito cada herramienta habilitada y su ámbito de acceso.

9.4. Procesos restringidos, contenedores, VMs y microVMs

No todos los mecanismos de aislamiento ofrecen las mismas garantías. La comparación siguiente distingue cuatro niveles, de menor a mayor fortaleza del límite, y explica por qué un contenedor ordinario no basta frente a código hostil. Las afirmaciones sobre el funcionamiento general de estos mecanismos son **Evidencia** en el terreno de la ingeniería de sistemas; su suficiencia en un caso concreto es **Hipótesis** que exige verificación.

Procesos restringidos.

Se aplican al proceso del agente restricciones como conjuntos acotados de llamadas al sistema, perfiles de control de acceso obligatorio, límites de recursos y un sistema de ficheros reducido. Su ventaja es que son ligeros y se integran en el propio proceso; su límite es que comparten el núcleo con el sistema anfitrión, de modo que un fallo en la política o en el núcleo afecta a ambos.

Contenedores.

Un contenedor empaqueta un proceso con su sistema de ficheros y su espacio de nombres, y comparte el núcleo con el anfitrión. Proporciona *aislamiento de recursos y de organización*, y es excelente para reproducibilidad y empaquetado. Pero no es una frontera de seguridad frente a código que ejecuta código: comparte el núcleo, y por tanto la superficie de ataque del núcleo es común. Un contenedor *reforzado* -con políticas de llamadas al sistema, usuario no privilegiado, capacidades reducidas, sistema de ficheros de solo lectura y sin red- eleva el listón, pero sigue apoyándose en el núcleo compartido.

Máquinas virtuales.

Una máquina virtual ejecuta un núcleo propio sobre un hipervisor, de modo que el límite pasa por el hipervisor y no por el núcleo del anfitrión. Es un límite más fuerte,

con mayor coste de arranque y de recursos. La fortaleza depende de la superficie del hipervisor y de su configuración.

MicroVMs.

Las microVMs persiguen un límite comparable al de la máquina virtual -núcleo propio por instancia- con un coste de arranque y de huella menor, lo que las hace adecuadas para ejecutar cargas no fiables y efímeras. Igual que en los casos anteriores, la garantía no es absoluta y depende de la superficie del hipervisor y del endurecimiento del sistema mínimo.

Tabla 9.3.: Comparación de mecanismos de aislamiento. «Fortaleza del límite» se refiere a la dificultad de que código hostil afecte al anfitrión; no es una certificación de seguridad.

Mecanismo	**Núcleo**	**Fortaleza del límite**	**Coste / uso**	**Límite**
Proceso restringido	Compartido con el anfitrión	Baja-media (política del proceso)	Muy bajo; tareas acotadas	Un fallo de política o de núcleo afecta a ambos
Contenedor	Compartido con el anfitrión	Media si está reforzado; baja si no	Bajo; reproducibilidad	No es frontera de seguridad frente a código hostil
VM	Propio por instancia	Alta (frontera en el hipervisor)	Alto; aislamiento fuerte	Superficie del hipervisor; coste
MicroVM	Propio por instancia	Alta (frontera en el hipervisor)	Medio; cargas efímeras	Superficie del hipervisor; verificación

Advertencia

Un contenedor ordinario no es un límite de seguridad. Un contenedor comparte el núcleo con el sistema anfitrión; por tanto, una vulnerabilidad del núcleo o una configuración permisiva pueden permitir que el código confinado afecte al anfitrión. El contenedor sirve para empaquetar, reproducir y separar recursos, no para demostrar que código hostil queda contenido. Presentarlo como «sandbox seguro» por sí mismo es un error conceptual: el aislamiento hay que *construirlo y*

verificarlo, no suponerlo por el mero uso de una tecnología de contenedores.

Nota

Elección proporcional al riesgo. La elección del mecanismo debe corresponder al daño potencial de la carga. Ejecutar un fragmento de código del caso transversal generado por el asistente es una carga no fiable: no basta con «un contenedor»; hay que declarar qué se aísla y con qué límite. **Propuesta docente**: que el estudiante justifique el mecanismo elegido en función de los activos de la tabla 9.2, y no en función de la comodidad.

9.5. Controles propuestos: privilegios mínimos y confinamiento

Los controles que siguen constituyen una **propuesta docente** derivada del modelo de amenazas, no una configuración certificada. NIST SSDF [3] proporciona prácticas generales para preparar el entorno de desarrollo, proteger el software y verificarlo (familias PO, PS y PW); no prescribe un sandbox específico para agentes. OWASP ASVS [5] establece requisitos verificables para aplicaciones y servicios web, por lo que solo se aplicará aquí cuando el componente estudiado entre en ese ámbito. ISO/IEC 27034-1 [9] proporciona un marco de gestión de seguridad de aplicaciones, no garantías de aislamiento de contenedores. Fuentes: NIST SP~800-218, OWASP ASVS y ISO/IEC~27034-1.

Consejo

Privilegios mínimos. Otorgar a cada componente -agente, proceso, credencial- el mínimo privilegio necesario para la tarea, y durante el menor tiempo posible. En la práctica: usuario no privilegiado, sin capacidades elevadas, sin acceso de escritura fuera del directorio de trabajo, sin red salvo excepción justificada y con credenciales de alcance mínimo y corta duración. El privilegio que no se concede no puede ser abusado.

Aislamiento de red.

Política por defecto de *sin red*. Cuando la tarea exija acceso, permitir solo destinos declarados y necesarios, con registro de las conexiones. La red es simultáneamente canal de descarga de contenido no fiable y de exfiltración, de modo que su apertura debe justificarse como cualquier otro privilegio.

Sistema de ficheros efímero.

Montar el sistema de ficheros de la sesión como efímero y, en lo posible, de solo lectura, con un directorio de trabajo aislado y con escritura. Al terminar la tarea, el entorno se descarta, de modo que un artefacto introducido por error no persiste. Esto reduce el daño de una escritura no deseada sin sustituir a la revisión del repositorio.

Límites de CPU, memoria y tiempo.

Fijar cuotas de CPU y de memoria y un tiempo máximo de ejecución. Estos límites acotan el daño de un bucle infinito, de una asignación desmedida o de una tarea que no termina, y son condición para que el entorno sea recuperable. Un límite de tiempo es también una defensa frente a comportamientos que consumen recursos de forma encubierta.

Control de secretos.

No montar secretos que la tarea no necesite. Cuando se requieran, usar credenciales de alcance mínimo y corta duración, inyectadas en el momento del uso y no persistidas en el sistema de ficheros ni en el historial. La salida del agente debe revisarse para evitar que un secreto se filtre en un registro o en un mensaje.

Auditoría.

Registrar de forma estructurada qué acciones se solicitaron, cuáles se ejecutaron, con qué identidad y con qué resultado. La auditoría no impide el daño por sí sola, pero es la base de la trazabilidad requisito → implementación → prueba que el capítulo 10 desarrolla, y es indispensable para reconstruir un incidente.

Aprobaciones.

Introducir una puerta de aprobación humana para las acciones irreversibles o de alto impacto: escritura en el repositorio, uso de credenciales, apertura de red, instalación de paquetes. El principio es que el agente *proponga* y un humano *decida*, de modo que ninguna acción sensible dependa de una instrucción embebida en contenido no fiable.

Limpieza.

Destruir el entorno y sus recursos temporales al terminar: procesos, montajes, credenciales efímeras y ficheros temporales. La limpieza cierra el ciclo de vida y evita que un

estado intermedio se reutilice en una tarea posterior con supuestos de confianza distintos.

Advertencia

Los controles no se suman como garantías. Aplicar todos los controles anteriores reduce la superficie y acota el daño, pero no demuestra que el aislamiento sea correcto frente a cualquier código hostil. La suficiencia de una configuración concreta es una **Hipótesis** que debe probarse y auditarse para *ese* entorno y *esas* capacidades.

9.6. Ataques de ejemplo, solo a efectos pedagógicos y defensivos

Los escenarios que siguen se describen únicamente como *clases* de ataque, para razonar sobre defensas. No se incluye ningún procedimiento operativo ni referencia a un exploit concreto, y no deben ejecutarse fuera de un entorno aislado y autorizado. Se marcan como **Propuesta docente**: son ilustraciones para el análisis, no recetas.

A1 - Instrucción embebida en contenido no fiable. Un fichero de entrada contiene texto que se dirige al agente con instrucciones (*prompt injection*). *Defensa conceptual*: no conceder autoridad a las instrucciones que provienen de datos no fiables; exigir aprobación humana para acciones sensibles y verificar el origen de cada decisión.

A2 - Intento de lectura de un secreto del entorno. El agente, o el contenido que procesa, intenta leer variables de entorno o ficheros de credenciales. *Defensa conceptual*: minimizar los secretos presentes en el entorno y su alcance temporal; revisar la salida para evitar filtraciones.

A3 - Descarga de una dependencia no verificada. La sesión instala un paquete cuyo origen no se ha comprobado. *Defensa conceptual*: fijar versiones, usar réplicas internas y verificar la integridad de los artefactos; tratar la cadena de suministro como superficie de ataque.

A4 - Agotamiento de recursos. Un bucle o una tarea descontrolada consume CPU, memoria o disco. *Defensa conceptual*: cuotas de recursos y límite de tiempo de ejecución, con terminación controlada.

A5 - Escritura no autorizada en el repositorio. El agente modifica el historial o el código sin revisión. *Defensa conceptual*: rama aislada, revisión obligatoria y aprobación

humana; la auditoría permite reconstruir qué se cambió.

A6 - Intento de escape del confinamiento. El código confinado busca afectar al anfitrión aprovechando una configuración permisiva o una superficie compartida. *Defensa conceptual*: elegir un mecanismo de aislamiento proporcional al riesgo -hasta VM o microVM-, endurecer la configuración y verificar el límite.

Advertencia

Material para análisis, no para ejecución fuera de aislamiento. Los escenarios A1-A6 son material didáctico. Su ejecución solo es admisible en un entorno aislado y autorizado, con las capacidades reducidas y con la supervisión del profesorado. El objetivo es razonar sobre defensas, no reproducir ataques.

9.7. Etiquetado epistémico y límites

Nota

Resumen del etiquetado. Se marca como **Evidencia**: que el riesgo del artefacto y el riesgo del proceso son objetos distintos; que un contenedor comparte el núcleo con el anfitrión; que una máquina virtual introduce un núcleo propio; y que las prácticas de revisión y trazabilidad son consolidadas [3]. Se marca como **Propuesta docente**: la lista de controles de confinamiento y las clases de ataque A1-A6. Se marca como **Hipótesis**: que una configuración concreta de aislamiento sea suficiente frente a código hostil. En ningún caso se afirma que un mecanismo concreto «garantice» seguridad.

La consecuencia práctica es una regla de lectura: toda afirmación del tipo «ejecutarlo en un contenedor es seguro» debe leerse como «se ha aplicado un mecanismo de aislamiento cuya suficiencia no se ha verificado». El aislamiento es una reducción de riesgo argumentada y verificable, no un certificado.

9.8. Preguntas de reflexión

1. ¿Por qué el riesgo del código producido y el riesgo de las acciones del agente exigen marcos de análisis y controles distintos? Dad un ejemplo del caso transversal para cada uno.

2. Enumerad los activos que protegeríais en una sesión de agente que convierte un CSV en PDF/XLSX y asignad a cada uno un límite de confianza.
3. Un compañero afirma que «el contenedor hace que el código sea seguro». ¿Qué error conceptual comete y qué habría que verificar para sostener una afirmación más prudente?
4. ¿Por qué la red se trata como privilegio y no como capacidad por defecto? Relacionadlo con la exfiltración y con la descarga de contenido no fiable.
5. ¿Qué diferencia hay entre *proponer* un cambio en el repositorio y *aplicarlo*? ¿Por qué la aprobación humana es un control y no una formalidad?
6. ¿Por qué el control de secretos no se resuelve solo ocultando el secreto, sino limitando su alcance y su duración?
7. ¿Por qué superar una batería de pruebas de aislamiento no demuestra que el aislamiento sea correcto? Relacionadlo con el invariante del capítulo.
8. Elegid un mecanismo de aislamiento para ejecutar código generado por un asistente y justificad la elección en función de los activos y del daño potencial.

9.9. Referencias y estado de verificación

Las fuentes se emplean con alcances diferenciados:

- **Reglamento (UE) 2024/1689, arts. 57-58** [1]: define el espacio regulatorio y permite delimitarlo frente al aislamiento informático. Texto consolidado oficial (27-07-2026).
- **NIST SP 800-218, SSDF v1.1** [3]: marco de prácticas de desarrollo seguro; su alcance no es la certificación de un sandbox para agentes. Fuente oficial.
- **OWASP ASVS 5.0.0** [5]: requisitos de verificación para aplicaciones y servicios web; no constituye un estándar general de aislamiento de agentes. Fuente oficial.
- **ISO/IEC 27034-1:2011** [9]: conceptos y procesos de seguridad de aplicaciones; La vigencia de la edición debe confirmarse en la ficha oficial de ISO (pendiente de verificación; la consulta automatizada devolvió HTTP 403). Ficha oficial.
- **Catálogo CWE** [6]: taxonomía de debilidades; las correspondencias deben justificarse caso por caso. Fuente oficial.

Advertencia

Límite de la verificación. Se han comprobado los 27 identificadores CWE empleados contra las fichas de MITRE (consulta 2026-10-24) y las ediciones CERT citadas (CERT C 2016 Edition y CERT Java SE 6/7). El alcance de las fuentes ISO/IEC 27034 queda pendiente de verificación oficial. No se ha demostrado experimentalmente que una configuración concreta de contenedor, VM o microVM contenga a un adversario con capacidades definidas. La eficacia del aislamiento depende de la configuración, el anfitrión, los montajes, la red, los privilegios y las pruebas efectuadas. El cumplimiento de SSDF, ASVS o ISO/IEC 27034 tampoco equivale a una certificación de seguridad del agente. Tampoco se garantiza la resistencia a exploits del núcleo ni a canales laterales de hardware, y las cuotas de `ulimit` son por proceso, no agregadas por cgroup.

10. Pipeline de validación automática

La capítulo 9 estableció que el aislamiento reduce el daño pero no certifica seguridad, y que la revisión y la trazabilidad son prácticas consolidadas [3]. Queda abierta una pregunta de ingeniería: *¿cómo se organiza la comprobación de un artefacto generado para que la aceptación no dependa de que «parece correcto»?* La respuesta es un *pipeline de validación automática*: secuencia reproducible que produce, para cada requisito, evidencia verificable. *Validar* significa comprobar propiedades *declaradas*, no demostrar seguridad; esa distinción recorre todo el capítulo.

Nota

Etiquetado epistémico inicial. El diseño concreto del pipeline que se presenta -las etapas, las puertas y sus criterios- es una **Propuesta docente** para el laboratorio. Que las técnicas de análisis existan y tengan los límites que se describen es **Evidencia** en el terreno de la ingeniería del software. Que una configuración concreta de pipeline baste para *este* artefacto y *estas* condiciones es una **Hipótesis** que exige verificación específica. En ningún caso se afirma que superar el pipeline equivalga a la ausencia de vulnerabilidades.

10.1. De la generación a la evidencia

Un asistente generativo entrega un artefacto: un módulo, un *script*, una configuración. Ese artefacto es una *hipótesis de solución*, no una solución verificada. El pipeline es el procedimiento que somete esa hipótesis a comprobación y que *registra* el resultado de cada comprobación. Su producto no es un veredicto binario del tipo «seguro / no seguro», sino un *expediente*: qué se comprobó, con qué herramientas, sobre qué versión, con qué resultado y qué quedó sin comprobar.

Esta forma de trabajar responde a un desplazamiento que el capítulo ya ha señalado: la IA generativa traslada el esfuerzo humano desde la escritura del código hacia la especificación, la revisión y la validación. El pipeline es precisamente el instrumento que hace posible esa validación a escala, porque convierte la revisión en un proceso

repetible en lugar de en una inspección informal. Como toda herramienta, tiene un límite: automatiza lo que puede expresarse como una comprobación, y deja fuera lo que exige juicio. Ese resto es el objeto de la revisión humana, que no es un apéndice del pipeline sino su última etapa.

Advertencia

La automatización no sustituye al juicio. Un pipeline automatiza comprobaciones *expresables*: reglas, pruebas, invariantes, umbrales. No decide si un requisito de seguridad estaba bien formulado, ni si una decisión de diseño es adecuada al contexto de uso. Presentar el pipeline como sustituto de la revisión humana confunde *ejecutar comprobaciones* con *asumir responsabilidad* por el resultado.

10.2. Arquitectura del pipeline

La figura 10.1 representa la arquitectura propuesta. Las etapas se agrupan en cuatro bloques: *preparación* (especificación y generación), *análisis estático* (análisis preliminar, SAST y SCA, compilación), *ejecución dinámica* (sandbox, pruebas funcionales, pruebas basadas en propiedades, *fuzzing* y *sanitizers*) y *evidencia y decisión* (recogida de evidencias, criterios de aceptación y revisión humana). El orden no es arbitrario: cada bloque presupone el anterior y produce las condiciones del siguiente. No se ejecuta código en el sandbox antes de que la especificación declare qué se espera y qué límites se imponen.

Nota

La figura es un orden de dependencia. Que las etapas aparezcan encadenadas no significa que cada una «cubra» la anterior ni que su superación conjunta demuestre seguridad. Significa que ejecutar una etapa sin las condiciones que producen las anteriores reduce el valor de la evidencia obtenida: por ejemplo, ejecutar *fuzzing* sin límites de tiempo en el sandbox convierte la prueba en un riesgo, no en una comprobación.

10.3. Etapas del pipeline

Las etapas se describen a continuación en el orden de la figura, con la evidencia que cada una produce y el motivo por el que es una etapa y no un paso opcional.

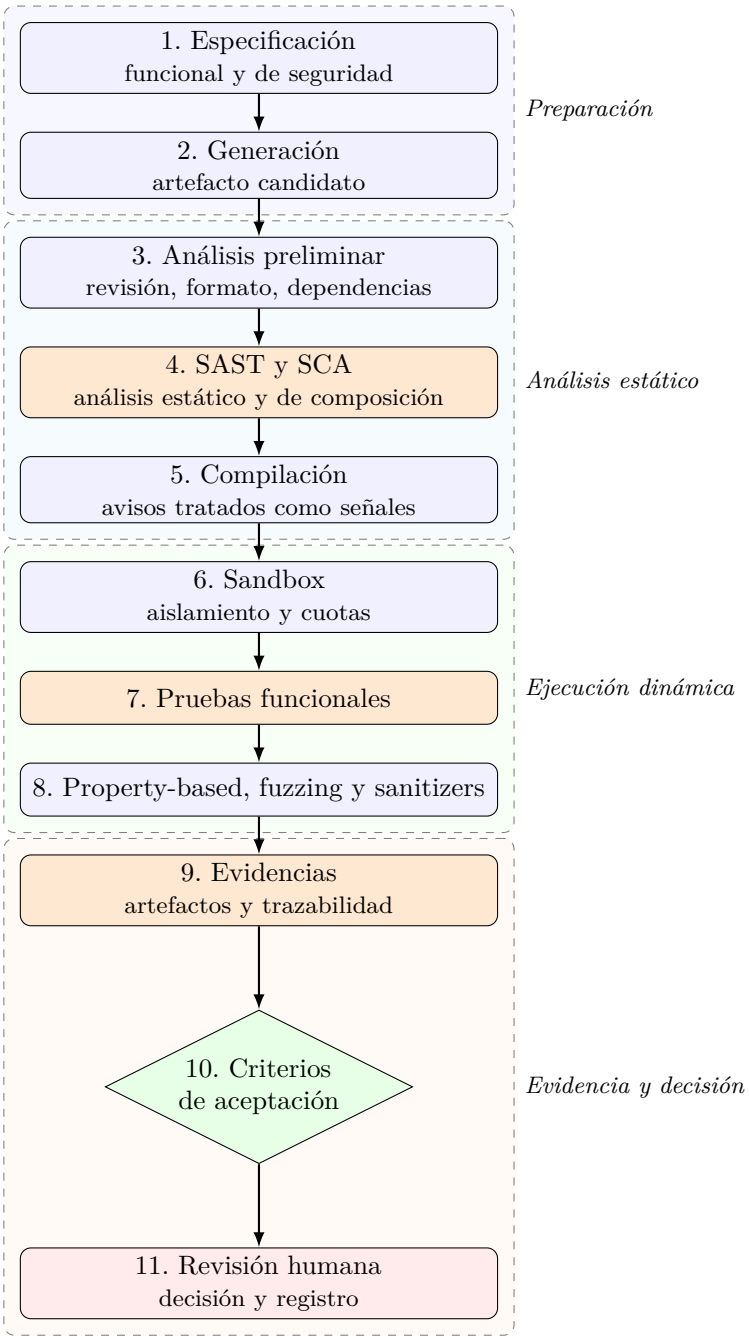


Figura 10.1.: Arquitectura del pipeline de validación automática. Las etapas en azul son transformaciones; las naranjas son *puertas de calidad* que pueden rechazar el artefacto; el rombo verde es el punto de decisión según los criterios de aceptación; la etapa roja es la revisión humana. Los rectángulos discontinuos agrupan las etapas en bloques funcionales. La figura representa un *orden de dependencia*, no una garantía de seguridad.

1. Especificación.

El pipeline no empieza en el código, sino en la especificación de lo que el artefacto debe hacer y de las propiedades que no debe violar. En el caso transversal, la especificación incluye requisitos funcionales -leer un CSV, normalizar columnas, generar PDF y XLSX- y requisitos de seguridad -no confiar en el CSV, evitar *path traversal* al escribir salidas, crear temporales seguros, limitar tamaño y tiempo y no ejecutar contenido embebido-. Sin especificación, las etapas posteriores no tienen contra qué comparar: una prueba sin propiedad declarada solo comprueba que el programa hace *algo*.

2. Generación.

El asistente produce el artefacto candidato. En esta etapa deben registrarse las condiciones de generación que el capítulo 6 señaló como variables controladas: modelo, versión, configuración, *prompt* y herramientas. Sin ese registro, la evidencia posterior no es atribuible a un artefacto concreto y la reproducibilidad se pierde.

3. Análisis preliminar.

Antes de ejecutar herramientas automáticas se realiza una revisión humana breve del artefacto: qué formato tiene, qué dependencias declara, qué supuestos introduce y si el código es legible y mantenible. Esta etapa detecta categorías que las herramientas no ven -por ejemplo, un requisito mal entendido- y evita gastar cómputo en un artefacto que no responde a la especificación.

4. SAST y SCA.

El análisis estático de código (SAST) examina el artefacto sin ejecutarlo, y el análisis de composición de software (SCA) examina sus dependencias. Ambos son *señales*, no veredictos: sus hallazgos deben ser clasificados (*triaje*) antes de decidir. La capítulo 4 estudió las familias de fallo que estas herramientas buscan; aquí interesa su condición de etapa con puerta.

5. Compilación.

Compilar es una comprobación en sí misma y, además, una condición para las etapas dinámicas. Los avisos del compilador se tratan como señales que deben justificarse, no como ruido a silenciar. Un artefacto que no compila no puede pasar al sandbox, de modo que esta etapa actúa como puerta de entrada a la ejecución. En un flujo de integración

continua (CI), estas comprobaciones -junto con la comprobación de tipos, las pruebas en distintas plataformas y la construcción y empaquetado- se ejecutan de forma automática en cada propuesta de cambio, y su fallo bloquea la integración. Las etapas descritas en esta sección son las mismas puertas de CI, expresadas como un orden verificable; la incorporación efectiva a la rama principal se estudia en la sección 10.6.

6. Sandbox.

La ejecución del artefacto se realiza en el entorno aislado descrito en el capítulo 9: sin red salvo excepción justificada, con cuotas de CPU, memoria y tiempo, con sistema de ficheros efímero y con usuario no privilegiado. El aislamiento no es una etapa de análisis, sino la condición que hace admisible ejecutar código no fiable.

7. Pruebas funcionales.

Las pruebas funcionales comprueban que el artefacto satisface los requisitos declarados para entradas representativas. Son necesarias y no suficientes: el capítulo 3 mostró un artefacto que supera sus pruebas funcionales y sigue siendo inseguro, porque las pruebas cubren los casos previstos y no los adversarios.

8. Property-based, fuzzing y sanitizers.

Este bloque amplía la comprobación dinámica. Las pruebas basadas en propiedades enuncian invariantes que deben cumplirse para *muchas* entradas generadas, en lugar de para casos escritos a mano. El *fuzzing* inyecta entradas generadas o mutadas para provocar fallos y depende de un oráculo que decida cuándo una ejecución es un fallo. Los *sanitizers* instrumentan la ejecución para detectar errores de memoria y comportamiento indefinido. Los tres aumentan la probabilidad de observar un fallo bajo las rutas ejercitadas, y ninguno garantiza haber cubierto todas las rutas.

9. Evidencias.

Aquí no se ejecuta nada nuevo: se *recoge* lo producido por las etapas anteriores -informes, registros, resultados de pruebas, artefactos de ejecución- y se asocia cada elemento con el requisito que comprueba. La evidencia sin trazabilidad es difícil de auditar; la trazabilidad sin evidencia es una afirmación sin respaldo.

10. Criterios de aceptación.

Los criterios de aceptación son las condiciones que deben cumplirse para promover el artefacto. Se expresan como puertas explícitas (sección siguiente) y su incumplimiento produce un rechazo registrado, no una excepción silenciosa.

11. Revisión humana.

La última etapa es una decisión humana informada por la evidencia. Revisa lo que la automatización no puede decidir -adecuación al contexto, requisitos mal formulados, riesgos residuales- y *asume* la responsabilidad de la promoción. Es la etapa que cierra el pipeline y la que lo distingue de una mera batería de herramientas.

10.4. Técnicas de análisis y sus límites

Las técnicas que el pipeline integra no son intercambiables y no comparten el mismo tipo de resultado. La tabla 10.1 las distingue por lo que analizan, cuándo lo hacen y las fuentes típicas de error. En herramientas de detección, los falsos positivos y falsos negativos dependen del modelo, las reglas, la configuración y la cobertura. Esta terminología no se aplica de la misma manera a un inventario SBOM ni a una demostración formal: una prueba formal válida establece la propiedad especificada bajo sus hipótesis, pero puede no representar fielmente el sistema desplegado.

Nota

SBOM no es un análisis. Un *Software Bill of Materials* es un inventario de los componentes y versiones que forman el artefacto. Su valor es de trazabilidad y de respuesta ante incidentes -saber qué se ve afectado cuando aparece una vulnerabilidad-, no de detección. Confundir el inventario con la comprobación lleva a creer que «tener un SBOM» resuelve la cadena de suministro; lo que resuelve es *poder localizar* un componente cuando la información sobre su vulnerabilidad esté disponible.

La consecuencia docente es que ninguna técnica se presenta como «la que encuentra las vulnerabilidades». El pipeline combina técnicas con sesgos de error distintos precisamente porque sus coberturas y límites son diferentes: lo que una no ve, otra puede verlo, y lo que una señala de más, otra puede descartarlo. La combinación reduce la probabilidad de que un fallo pase inadvertido; no la anula.

Tabla 10.1.: Técnicas de análisis, su objeto y las fuentes típicas de falsos positivos y falsos negativos. Las descripciones son **Evidencia** sobre el funcionamiento general de cada familia de técnicas; ninguna implica que una herramienta concreta detecte o deje de detectar un caso particular.

Técnica	**Qué analiza**	**Cuándo**	**Fuente de falso positivo**	**Fuente de falso negativo**
SAST	Código fuente o intermedio, sin ejecutarlo	Antes de compilar o ejecutar	Patrón sospechoso que no es alcanzable; análisis de flujo aproximado	Ruta no modelada; regla ausente; código generado en tiempo de ejecución
DAST	Aplicación en ejecución, sin acceso al código	Con el sistema desplegado	Escenario o sonda que provoca un comportamiento no explotable	Cobertura de rutas insuficiente; fallos lógicos no observables
SCA	Dependencias y versiones frente a catálogos de vulnerabilidades	Al resolver o actualizar dependencias	Componente marcado como vulnerable pero no alcanzable en el uso real	Vulnerabilidad no catalogada; dependencia no inventariada
SBOM	Inventario de componentes y versiones	En la generación y en cada construcción	No es una detección: la ausencia de hallazgos no implica seguridad	Inventario incompleto si la generación del propio SBOM falla
Fuzzing	Entradas generadas o mutadas y su ejecución	Sobre el binario o servicio en aislamiento	Señal de fallo espuria o atribuible al entorno de prueba	Baja cobertura de rutas; ausencia de oráculo adecuado
Análisis formal	Modelo y propiedades explícitas	Cuando hay especificación formal	No equiparable al triaje estadístico: una prueba válida demuestra una propiedad bajo hipótesis	Modelo incompleto, hipótesis falsas o implementación distinta
Sanitizers	Ejecución instrumentada para memoria y comportamiento indefinido	Durante las pruebas en el sandbox	Diagnóstico espurio o interpretación errónea, según herramienta y entorno	Ruta no ejercitada; clase de fallo no instrumentada

10.5. Puertas de calidad y casos de rechazo

Una *puerta de calidad* es una condición que debe cumplirse para que el artefacto avance. Su diseño debe ser *fail-closed*: ante duda o error, la puerta se cierra y el artefacto no avanza. Esto invierte la lógica habitual del desarrollo rápido, donde un fallo se anota y se sigue; en validación, un fallo no explicado es un motivo de rechazo. La tabla 10.2 enuncia las puertas propuestas, el criterio que las define, la evidencia que las respalda y la acción cuando no se cumplen.

Tabla 10.2.: Puertas de calidad del pipeline. La columna «Acción si no se cumple» define el caso de rechazo. La tabla es una **Propuesta docente**; los umbrales concretos deben fijarse y justificarse para cada proyecto.

Puerta	**Criterio**	**Evidencia**	**Acción si no se cumple**
Especificación	Requisitos funcionales y de seguridad declarados y revisables	Documento de especificación revisado	Rechazo: no se promueve la generación
Estático	Sin hallazgos críticos confirmados abiertos; excepciones solo por riesgo aceptado formalmente	Informe, evidencias y autorización de excepción	Bloqueo hasta corregir o autorizar excepción
Compilación	Compila sin errores; avisos tratados como señales	Registro de construcción	Rechazo del artefacto
Aislamiento	Todas las pruebas que ejecuten código no fiable se realizan dentro del sandbox configurado	Registro de invocaciones y configuración	Rechazo de la ejecución
Funcional	Las pruebas funcionales pasan	Resultados de las pruebas	Rechazo
Dinámica	Sin fallos no explicados en propiedades, *fuzzing* o *sanitizers*	Artefactos de ejecución y trazas	Bloqueo hasta explicar o corregir
Evidencia	Trazabilidad requisito → prueba → evidencia completa	Matriz de trazabilidad	Bloqueo
Aceptación y revisión humana	Se satisfacen los criterios de aceptación declarados y hay revisión humana con decisión explícita	Registro de la decisión y aprobación firmada	El artefacto no se promueve

El comportamiento *fail-closed* se ilustra mediante el siguiente *pseudocódigo Python*: las comprobaciones estáticas se ejecutan antes de introducir el artefacto en el entorno aislado, mientras que **todas** las etapas que ejecutan código candidato se invocan mediante una única interfaz de aislamiento. La función `sandbox.run` es un contrato abstracto, no una

biblioteca existente: su implementación debe comprobar las restricciones de red, usuario, recursos, montajes y tiempo descritas en el capítulo 9. La construcción también puede ejecutar código no fiable (scripts de instalación, macros, etc.); por ello se realiza dentro del aislamiento.

Listing 10.1: Orquestación conceptual fail-closed: ninguna etapa que ejecute código candidato se realiza en el anfitrión. No es código listo para producción.

```
# PSEUDOCODIGO: sandbox.run debe implementarse y auditarse.
# Cada comando se ejecuta en un entorno efimero y restringido.

def require_ok(name, result):
    if result.returncode != 0 or not result.evidence_complete:
        raise RuntimeError(f"PUERTA FALLIDA: {name}")

require_ok("especificacion", check_specification())
require_ok("sast_sca", analyze_without_running_candidate())
require_ok("riesgos", check_findings_and_approved_exceptions())

policy = load_reviewed_sandbox_policy()
for name, command in [
    ("construccion", ["python", "build.py"]),
    ("funcionales", ["python", "-m", "pytest", "tests/funcionales"
       ]),
    ("dinamicas", ["python", "run_dynamic.py"]),
]:
    result = sandbox.run(command, policy=policy)
    require_ok(name, result)

require_ok("evidencias", verify_traceability_and_artifacts())
# La promocion exige una decision humana posterior y registrada.
```

Advertencia

Contrato de aislamiento. No basta ejecutar primero un comprobador del sandbox y después lanzar `pytest` en el anfitrión. El límite de confianza debe envolver cada proceso que ejecute código no fiable, incluidos compiladores, *hooks*, pruebas, generadores y herramientas que carguen extensiones. La mera presencia de un contenedor no demuestra que se haya respetado la política.

Excepciones y aceptación del riesgo.

Una alerta puede cerrarse por corrección verificada o por falso positivo demostrado. Un riesgo confirmado que no se corrige **no** queda resuelto por añadir una explicación: requiere una excepción excepcional, limitada y documentada, autorizada por el responsable designado, con alcance, plazo, compensaciones y plan de remediación. Para la práctica docente se propone bloquear siempre los hallazgos críticos confirmados, sin excepciones automáticas.

Advertencia

Las puertas no son garantías. Que un artefacto supere todas las puertas significa que no se ha observado ningún fallo *bajo las comprobaciones ejecutadas*. No significa que el artefacto carezca de vulnerabilidades: las puertas heredan los falsos negativos de las técnicas que las respaldan. Una puerta que se supera es un dato, no una prueba de seguridad.

Casos de rechazo.

Un rechazo debe ser un resultado *registrado*, con el motivo y la evidencia asociada, y no un simple silencio. Los casos de rechazo típicos son: (i) especificación ausente o no verificable; (ii) hallazgo crítico confirmado sin corrección ni decisión autorizada; (iii) artefacto que no compila o cuyos avisos se han silenciado; (iv) ejecución fuera del sandbox o sin las cuotas declaradas; (v) pruebas funcionales que fallan; (vi) fallo reproducible o no explicado en las pruebas dinámicas; (vii) trazabilidad incompleta entre requisito, prueba y evidencia; y (viii) decisión humana que no se ha registrado. Cada rechazo debe dejar el artefacto en un estado recuperable -rama aislada, artefacto descartado- de modo que corregir no consista en reconstruir.

10.6. Integración de cambios concurrentes

Validar un artefacto de forma aislada no garantiza que el sistema resultante sea correcto cuando ese artefacto se combina con otros. Un cambio puede superar todas las puertas en su rama y, sin embargo, romper una invariante compartida al integrarse con un cambio distinto que también las superó por separado. La pregunta que responde esta etapa no es *¿pasa este cambio?*, sino *¿pasa el estado que realmente se va a integrar?*.

El problema tiene dos formas. La primera es el *conflicto textual*: dos cambios modifican las mismas líneas y el sistema de control de versiones lo detecta. La segunda, más peligrosa,

es el *conflicto semántico*: los cambios compilan y se combinan sin conflicto aparente, pero juntos violan una propiedad que cada uno preservaba por separado (por ejemplo, dos cambios que asumen de forma incompatible el orden de una operación, o que modifican el mismo contrato desde lados opuestos). Ninguna herramienta de fusión detecta el segundo por sí sola.

La práctica industrial para reducir esta ventana es la *cola de integración* (*merge queue*): los cambios que han superado las puertas se encolan y las comprobaciones se *reejecutan sobre la rama principal actualizada* con el cambio aplicado, en el orden en que se van a integrar. Así, el estado validado es el que se integrará, no el de una rama que pudo quedar desactualizada. Esta práctica es **Evidencia** de ingeniería consolidada; su implementación concreta depende de la plataforma [19, 3].

Tabla 10.3.: Del cambio aislado al sistema integrado: qué se valida en cada nivel. **Propuesta docente**.

Nivel	**Qué se valida**	**Límite**
Cambio aislado	El artefacto cumple sus propiedades en su rama	No ve incompatibilidades con otros cambios
Cambio combinado	El artefacto conserva sus propiedades junto a los ya integrados	No cubre cambios aún no integrados
Estado de integración	El resultado de aplicar el cambio sobre la rama actualizada	Reduce la ventana de integración, no demuestra corrección del conjunto

Nota

La verificación composicional de la sección 11.11 estudia la compatibilidad de contratos entre componentes dentro de un mismo artefacto. La integración de cambios concurrentes es un problema análogo, pero en el eje temporal: la compatibilidad entre versiones del sistema que evolucionan en paralelo. En ambos casos, verificar las partes por separado no establece la compatibilidad del todo.

La **limitación** debe declararse: una cola de integración reduce el intervalo en el que dos cambios pueden combinarse de forma defectuosa y valida el estado real que se integra, pero no demuestra que el conjunto sea correcto. Los conflictos semánticos que ninguna prueba ejerza seguirán siendo invisibles, y las pruebas de integración heredan los falsos negativos de las pruebas que las respaldan.

10.7. Trazabilidad: requisito, implementación, prueba y evidencia

Nota

Trazabilidad. El pipeline se sostiene sobre una cadena explícita: *requisito → implementación → prueba → evidencia*. Cada requisito debe poder seguirse hasta la parte del artefacto que lo realiza, hasta la prueba que lo comprueba y hasta la evidencia que registra el resultado. Sin esta cadena, un expediente verde no permite responder a la pregunta que importa en una auditoría: *¿qué propiedad se comprobó, con qué, y dónde está el registro?* La trazabilidad es la forma concreta que adopta la responsabilidad en el desarrollo asistido por IA.

La tabla 10.4 aplica la cadena al caso transversal. Cada fila es un requisito y cada columna avanza un eslabón. La tabla no afirma que las pruebas detecten todos los fallos: afirma que, para cada requisito, existe al menos una prueba y una evidencia asociadas, y que el vínculo puede auditarse.

Tabla 10.4.: Trazabilidad requisito → implementación → prueba → evidencia para el caso transversal (CSV a PDF/XLSX). Ejemplo de **Propuesta docente**.

Req.	Implementación	Prueba	Evidencia
R1	Lectura del CSV con validación canónica de columnas	Entradas malformadas y columnas ausentes	Registro de la prueba y artefacto de ejecución
R2	Rutas canónicas y apertura sin seguir enlaces simbólicos	Pruebas de rutas relativas, enlaces simbólicos y sustitución concurrente de entradas	Informe de la prueba de rutas
R3	Creación de temporales con funciones seguras	Colisiones y reutilización de nombres	Registro de la prueba de temporales
R4	Límites de tamaño y de tiempo de proceso	Entradas sobredimensionadas y ejecución lenta	Registro de la prueba de recursos
R5	Escapado de fórmulas antes de exportar a hoja de cálculo	Entradas que comienzan por operadores de fórmula y variantes con espacios o controles	Inspección de tipo y contenido de celdas XLSX, incluida reapertura del archivo

10.8. Dependencias y entornos reproducibles

La reproducibilidad de la validación depende de registrar y controlar el entorno. Un resultado de prueba obtenido sobre una versión de dependencia distinta de la que se usará en producción no es evidencia del artefacto que se va a desplegar. Por eso el pipeline debe fijar y registrar las dependencias -versiones exactas, no rangos abiertos- y construir el entorno de forma declarativa. La reproducibilidad del entorno es una propiedad del *proceso*, y sin ella la evidencia pierde buena parte de su valor.

Principios del entorno reproducible.

Los siguientes principios son **Evidencia** de buenas prácticas consolidadas, y su aplicación concreta es una **Propuesta docente**:

- **Fijar versiones.** Las dependencias se declaran por versión exacta y, cuando sea posible, por su huella de integridad, evitando referencias móviles que cambian con el tiempo.
- **Inventariar componentes.** El SBOM se genera en cada construcción y se conserva junto al expediente, de modo que la composición del artefacto sea consultable.
- **Construir de forma declarativa.** El entorno -sistema operativo, intérprete, bibliotecas- se describe en un fichero versionado y se reconstruye a partir de él, no se configura a mano.
- **Fijar la imagen por contenido.** Al usar contenedores, referenciarlos por su resumen criptográfico y no por una etiqueta móvil, para que la identidad del entorno sea verificable.
- **Registrar la procedencia.** Anotar el origen de cada dependencia y de cada artefacto descargado, porque la cadena de suministro es una superficie de ataque.

Nota

Reproducibilidad frente a aislamiento. Un entorno reproducible ayuda a comprobar el mismo artefacto bajo condiciones comparables; el aislamiento del capítulo 9 limita, pero no elimina, el impacto potencial de ejecutar código no fiable en el anfitrión. Son propiedades distintas y complementarias: un entorno reproducible pero no aislado puede reproducir el daño de un artefacto hostil, y un entorno aislado pero no reproducible puede dar resultados que no se repiten.

10.9. El límite del pipeline verde

Advertencia

Pipeline verde ≠ sin vulnerabilidades. Superar todas las etapas y puertas del pipeline significa que *no se ha observado ningún fallo bajo las comprobaciones ejecutadas*. No significa que el artefacto esté libre de vulnerabilidades. El pipeline es una batería *finita* de comprobaciones sobre un conjunto *declarado* de propiedades: los falsos negativos de SAST, DAST, SCA, *fuzzing*, análisis formal y *sanitizers* se heredan. Además, la especificación puede ser incompleta, las dependencias pueden cambiar y el contexto de uso puede diferir del supuesto. Un artefacto con el pipeline verde es un artefacto *validado contextualmente*, nunca un artefacto «seguro» en sentido absoluto. Este es el invariante del capítulo aplicado a la validación: superar una batería finita de pruebas no demuestra seguridad.

La lectura correcta de un resultado verde tiene tres componentes. Primero, *qué* se comprobó: la lista de puertas y las propiedades declaradas. Segundo, *cómo* se comprobó: las herramientas, sus versiones y el entorno. Tercero, *qué quedó fuera*: los falsos negativos conocidos, los requisitos no expresados y las condiciones no probadas. Un expediente que no responde a estas tres preguntas no es evidencia, sino una afirmación de confianza sin respaldo.

Consejo

Cómo presentar el resultado. En lugar de «el artefacto es seguro», que el pipeline no puede sostener, la formulación honesta es: «bajo las comprobaciones A, B y C, con las herramientas y versiones registradas, en el entorno descrito, no se observaron fallos; quedan sin comprobar las propiedades X e Y». Esta formulación conserva la información que una auditoría necesita y evita la falsa garantía.

10.10. Etiquetado epistémico y límites

Nota

Resumen del etiquetado. Se marca como **Evidencia**: que SAST analiza sin ejecutar y DAST analiza en ejecución; que SCA compara versiones con catálogos de vulnerabilidades; que un SBOM es un inventario y no una detección; que el *fuzzing* y los *sanitizers* dependen de la cobertura de rutas ejercitada; que el análisis formal depende de un modelo y de propiedades declaradas; que los resultados de las técnicas dependen de supuestos y cobertura; y que las prácticas de revisión y trazabilidad son consolidadas [3]. Se marca como **Propuesta docente**: la arquitectura concreta del pipeline, las puertas y sus criterios, el esqueleto de orquestación y la tabla de trazabilidad del caso transversal. Se marca como **Hipótesis**: que una configuración concreta de pipeline sea suficiente para *este* artefacto en *este* contexto. En ningún caso se afirma que superar el pipeline demuestre seguridad.

La distinción entre validar y demostrar tiene una consecuencia para la docencia: el pipeline se presenta como un instrumento para *reducir la incertidumbre* y *producir evidencia auditable*, no para *eliminar el riesgo*. El estudiante que comprende esta diferencia está en condiciones de usar la automatización sin confundirla con una garantía.

10.11. Validación del propio pipeline

Un pipeline también puede estar mal configurado. Para comprobar sus puertas se introducen fallos controlados y se verifica que la promoción se bloquea. Este ensayo es una **Propuesta docente**, no una certificación.

1. **Prueba fallida:** introducir una aserción deliberadamente falsa y comprobar un código de salida distinto de cero y una promoción bloqueada.

2. **Fallo de aislamiento:** deshabilitar una restricción obligatoria en un entorno de prueba y comprobar que la política rechaza la ejecución.

3. **Hallazgo confirmado:** insertar un ejemplo vulnerable controlado y verificar que la puerta SAST/SCA correspondiente lo bloquea cuando existe una regla apropiada; no asumir detección universal.

4. **Oráculo de seguridad:** generar un XLSX con entradas que comienzan por operadores de fórmula y verificar el tipo de celda y su contenido real, no solo que

el archivo existe.

5. **Evidencia ausente:** retirar un informe requerido y comprobar que la puerta de trazabilidad falla cerrada.

Advertencia

Resultado esperado. Una ejecución de prueba que falla, un informe ausente o una violación de política deben impedir la promoción. Si el pipeline termina en verde pese a esas perturbaciones, la propia infraestructura de validación requiere corrección antes de evaluar artefactos de estudiantes.

10.12. El kit de agentes y pipelines AI-SDLC

El capítulo incluye un recurso complementario, `kit_agentes_ai_sdlc_v2/`, que operacionaliza las puertas de esta sección como **Propuesta docente**. No es un estándar, no certifica seguridad y no sustituye la revisión humana.

Tabla 10.5.: Correspondencia entre las pipelines del kit y las puertas de la sección 10.5.

Pipeline del kit	Función	Puerta de esta sección
`P02_estatico`	AG-03 y AG-06: compilación, tipos y análisis estático	Estático
`P04_testing`	AG-05 y AG-07: pruebas y matriz de aceptación	Funcional y dinámica
`P05_seguridad`	AG-06 y AG-09: amenazas y dependencias	Estático y evidencia
`pipelines/run.py`	Perfiles `basic`, `extended` y `critical`; registro de evidencias	Evidencia y aceptación

El runner distingue `PASS`, `FAIL`, `SKIP` y `NOT_EVALUATED`. Un `SKIP` no es un `PASS`: con `-strict` (o en el perfil `critical`) bloquea la aceptación. La opción `-sandbox` ejecuta las comprobaciones dentro de `bubblewrap`, con el proyecto en solo lectura y `/tmp` efímero; si `bubblewrap` no está disponible, falla cerrado y no reintenta en el anfitrión. El informe de ejemplo (`reports/example_report.json`) se generó ejecutando el runner sobre `fixtures/example_project/`; no se han inventado resultados.

Advertencia

El kit no certifica. Que el runner termine en verde significa que las comprobaciones declaradas no fallaron en ese entorno, no que el artefacto sea seguro ni conforme a una norma. Las herramientas opcionales ausentes producen `SKIP`; la revisión humana sigue siendo obligatoria.

10.13. Preguntas de reflexión

1. ¿Por qué el pipeline empieza en la especificación y no en el código? Dad un ejemplo del caso transversal en el que una especificación incompleta haría inútil una prueba.
2. Explicad la diferencia entre un falso positivo y un falso negativo en SAST y en SCA. ¿Por qué ambos tipos de error son estructurales y no un defecto de una herramienta concreta?
3. ¿Por qué un SBOM no es una técnica de detección? ¿Para qué sirve entonces en un expediente de validación?
4. Un compañero afirma que «si el pipeline está verde, el código es seguro». ¿Qué error conceptual comete y cómo reformularíais la afirmación para que sea correcta?
5. ¿Qué significa que una puerta sea *fail-closed*? Dad un ejemplo de puerta y explicad qué ocurre cuando falla de forma ambigua.
6. Elegid un requisito del caso transversal y trazad su cadena completa requisito → implementación → prueba → evidencia.
7. ¿Por qué el entorno reproducible y el aislamiento son propiedades distintas? ¿Qué riesgo se corre si se confunde una con otra?
8. ¿Qué propiedades de seguridad no son automatizables con las técnicas de esta sección y por qué quedan en la revisión humana?

10.14. Referencias y estado de verificación

Las afirmaciones de esta sección sobre prácticas de desarrollo seguro y trazabilidad se apoyan en SSDF [3] y en ASVS [5]; la clasificación de familias de fallo remite al catálogo CWE [6] y a las reglas de codificación segura de CERT [7], y el marco general de seguridad de aplicaciones a ISO/IEC 27034 [9]. La referencia a riesgos de aplicación en el

contexto web remite al Top 10 de OWASP [4]. Todos estos materiales deben consultarse en su edición vigente.

Nota

Alcance de las referencias. NIST SP 800-218 (SSDF) respalda la incorporación de prácticas de desarrollo seguro y verificación a lo largo del ciclo de vida; OWASP ASVS ofrece requisitos de verificación de seguridad para aplicaciones web, pero no certifica un pipeline ni un sandbox. CWE clasifica debilidades, por ejemplo CWE-22 (rutas), CWE-367 (TOCTOU), CWE-400 (consumo no controlado de recursos) y CWE-1236 (inyección de fórmulas en hojas de cálculo). CERT C es pertinente para las reglas de programación segura en C; ISO/IEC 27034 aporta un marco de seguridad de aplicaciones. Ninguna de estas referencias prueba por sí misma la seguridad del caso transversal. Las capacidades de herramientas concretas se deben verificar en sus manuales y versiones instaladas.

11. Verificación de código generado: contratos, invariantes y modelos de amenazas

11.1. El problema de la confianza delegada

La necesidad de verificar no depende de quién haya escrito el código. Una implementación humana, una biblioteca de terceros y un fragmento generado por IA pueden incumplir los mismos requisitos. La capítulo 8 (confianza no calibrada) analiza por qué la apariencia de corrección puede confundirse con evidencia. Aquí se adopta una pregunta operativa: *¿qué propiedades deben cumplirse, con qué técnicas pueden comprobarse y qué limitaciones tiene cada comprobación?*

La comparación entre personas e IA se realiza a lo largo del capítulo sobre un criterio común: la procedencia del artefacto no constituye una prueba de corrección. Sí influye en la disponibilidad de requisitos, la trazabilidad de decisiones y la independencia de los mecanismos que generan y comprueban el código.

11.2. Qué significa verificar

Verificar consiste en obtener evidencia de que una implementación satisface propiedades previamente establecidas. Conviene distinguir la *verificación formal*, que demuestra propiedades bajo hipótesis explícitas, de la *evaluación mediante pruebas*, que observa ejecuciones seleccionadas y no establece por sí sola corrección para todas las entradas. El análisis estático, la revisión y la ejecución instrumentada aportan evidencias complementarias; no son equivalentes entre sí.

También debe distinguirse la corrección parcial (si la ejecución termina, se cumple la poscondición) de la corrección total (además, termina bajo las hipótesis dadas). La validez de cualquier conclusión queda limitada por la especificación, el modelo del entorno y la fiabilidad de las herramientas empleadas.

Tabla 11.1.: Alcance y límites de distintos mecanismos de verificación.

Mecanismo	**Evidencia que aporta**	**Límite principal**
Compilación	Conformidad con las reglas que comprueba el compilador	No prueba la corrección funcional
Pruebas de ejemplo	Comportamiento observado en ejecuciones seleccionadas	No cubren entradas no ejercitadas
Pruebas de propiedades	Comportamiento observado sobre entradas generadas	Dependen del oráculo y de la estrategia de generación
Análisis estático	Hallazgos o garantías sobre propiedades dentro de su modelo	Depende de supuestos, configuración y precisión
Revisión humana o asistida	Defectos y discrepancias detectados por inspección	No constituye por sí sola demostración exhaustiva
Verificación formal	Propiedad demostrada respecto a una especificación y unas hipótesis	No garantiza que la especificación sea la deseada

Tabla 11.2.: Comparación de las condiciones de verificación según el origen del código.

Dimensión	**Desarrollo humano**	**Desarrollo asistido por IA**
Requisitos	Pueden permanecer implícitos en decisiones del autor	Pueden quedar dispersos entre prompt, contexto y documentos
Trazabilidad	Es posible consultar decisiones al autor; sus recuerdos no son prueba	Las explicaciones generadas no acreditan por sí solas las decisiones de síntesis
Defectos	Errores de requisitos, diseño e implementación	Los mismos defectos, además de riesgos del contexto y de herramientas integradas
Pruebas	Pueden compartir sesgos con el autor	Pueden compartir supuestos con el generador y el revisor automático
Aceptación	Requiere evidencias técnicas	Requiere las mismas evidencias técnicas

La tabla 11.2 describe *posibilidades*, no diferencias universales ni tasas de fallo medidas. Una comparación empírica exigiría controlar tarea, experiencia, tiempo, herramientas y criterios de evaluación.

Verificación y validación.

La verificación responde a la pregunta *¿cumple el artefacto la especificación?*; la validación responde a una pregunta distinta, *¿satisface la necesidad real de uso?*. Un artefacto puede verificarse correctamente contra una especificación incompleta y, aun así, no resolver el problema del usuario (por ejemplo, porque un requisito se malinterpretó al formularlo). Ambas son necesarias y ninguna sustituye a la otra; la validación frente al uso se apoya en la revisión de requisitos descrita en la sección 10.3.

Cinco tipos de evidencia.

Las evidencias que respaldan una afirmación sobre el software no son intercambiables. Conviene distinguir al menos cinco tipos:

1. **Ejecución satisfactoria**: una prueba o campaña se ejecutó sin observar fallos. Respalda el comportamiento en los casos ejercitados, no la corrección general.

2. **Cumplimiento de propiedades**: se comprobó una propiedad enunciada (invariante, contrato) sobre entradas generadas o seleccionadas. Respalda la propiedad bajo la estrategia de generación y el oráculo empleados.

3. **Garantía formal**: se demostró una propiedad bajo hipótesis explícitas. Respalda la propiedad para todo el dominio del modelo, condicionada a que el modelo represente el sistema.

4. **Comportamiento en integración**: se comprobó la compatibilidad entre componentes o entre el cambio y el sistema (sección 11.11). Respalda la integración bajo las pruebas ejecutadas.

5. **Comportamiento en operación**: se observó el sistema en ejecución real. Respalda lo observado en las condiciones y ventanas cubiertas; se trata en la sección 12.16.

Una afirmación de corrección debe indicar *qué tipo* de evidencia la respalda; confundir la ejecución satisfactoria con una garantía formal, o la ausencia de fallos observados con una demostración de corrección, es el error que este capítulo previene.

11.3. Especificación como precondición de la verificación

No se puede demostrar conformidad con requisitos que no se han formulado. Esta distinción, fundamental en la teoría, se incumple con frecuencia en la práctica del desarrollo asistido por IA: se pide al modelo que «genere una función que valide entradas de usuario» sin definir qué conjunto de entradas es válido, qué ocurre con los casos límite ni qué propiedad de seguridad debe preservarse. El modelo genera algo plausible; el desarrollador lo acepta porque funciona para los casos que prueba; el defecto queda latente hasta que un atacante elige un caso que nadie eligió.

La especificación no necesita ser formal para ser útil. En el contexto del diseño por contrato -véase la sección 2.12- basta con enunciar:

- **Precondiciones:** qué debe ser verdad sobre las entradas para que la función sea aplicable.
- **Poscondiciones:** qué garantiza la función sobre sus salidas si las precondiciones se cumplen.
- **Invariantes:** qué propiedades del sistema deben preservarse antes y después de la operación, independientemente de la ruta de ejecución.

Nota

En el artículo de referencia [19], el equipo de Claude Code usa archivos `CLAUDE.md` y `REVIEW.md` como portadores de especificación operativa: convenciones del proyecto, criterios de fallo por módulo y propiedades que el revisor automático debe verificar. Este mecanismo es una instancia informal de contrato: define expectativas que el código generado debe satisfacer y que el revisor puede comprobar.

La diferencia entre una especificación útil y una inútil no es su formalismo, sino su **falsabilidad**: una especificación permite diseñar pruebas que la contradigan. «La función debe manejar entradas grandes» no es falsable. «La función debe rechazar cualquier cadena de más de 4 096 bytes con un error explícito y sin efectuar operaciones costosas sobre ella» sí lo es.

Formalización mínima del contrato.

Una tripleta de Hoare $\{P\}\ C\ \{Q\}$ expresa que, si se cumple la precondición P y el comando C termina, se cumple la poscondición Q. La terminación requiere una obligación adicional. En una operación de escritura, P puede exigir una ruta autorizada y una entrada

acotada; Q puede exigir un archivo válido o un fallo documentado sin salida parcial. Las condiciones de error y los efectos secundarios deben especificarse expresamente.

Comparación entre autores.

En código humano, el contrato puede reconstruirse mediante requisitos, revisión y consulta al autor, sin asumir que su explicación sea correcta. En código generado, el prompt no sustituye una especificación verificable, y pedir al mismo modelo que redacte el contrato a partir de su solución puede reproducir sus omisiones. En ambos casos, el contrato debe contrastarse con requisitos externos al código. El desarrollo guiado por especificaciones, que estructura el proceso en torno a esos requisitos, se trata en la sección 12.2.

11.4. Contratos en código generado: dónde fallan

Tanto una persona como un generador pueden implementar el caso nominal y omitir condiciones de fallo, límites o requisitos de seguridad. Los siguientes son modos de incumplimiento que deben buscarse en cualquier implementación, con independencia de su origen:

Precondiciones implícitas. La función asume entradas bien formadas sin verificarlo. Un generador que produce un analizador de CSV puede asumir que los campos están correctamente entrecomillados, trasladando la responsabilidad de validación al llamante sin documentarlo.

Poscondiciones parciales. La función garantiza el caso de éxito pero no especifica el estado del sistema tras un fallo. ¿Se libera el recurso abierto? ¿Queda el archivo de salida en un estado consistente?

Invariantes de seguridad omitidas. El generador no recibió la invariante como parte del enunciado y no la infiere. Una función de escritura de archivos puede no preservar la restricción de que el destino esté dentro del directorio autorizado si esa restricción no se enunció explícitamente.

Supuestos de llamada única. El código generado a menudo asume un único hilo de ejecución, sin gestionar condiciones de carrera. Una comprobación de ruta seguida de una apertura de archivo puede ser correcta en un entorno secuencial e incorrecta en uno concurrente.

Listing 11.1: Contrato explícito frente a contrato implícito en un conversor (ejemplo didáctico).

```
# Versión sin contrato explícito (lo que suele generar un modelo sin instrucciones)
def escribir_xlsx(registros, ruta_salida):
    libro = openpyxl.Workbook()
    hoja = libro.active
    for fila in registros:
        hoja.append([escapar_celda(c) for c in fila])
    libro.save(ruta_salida)   # Sin verificar que ruta_salida este dentro del directorio autorizado

# Versión ilustrativa: NO apta para produccion sin revisar temporales y rutas
# Precondicion:  registros es una lista de listas de cadenas; longitud <= MAX_FILAS
#                ruta_salida esta dentro de DIRECTORIO_SALIDA (verificado por la funcion)
# Poscondicion: se escribe un archivo XLSX en ruta_salida con todas las celdas escapadas,
#               o se lanza ValueError/OSError y no queda ningun archivo parcial
# Invariante:   no se escribe fuera de DIRECTORIO_SALIDA bajo ninguna ruta de ejecucion
def escribir_xlsx_seguro(registros, ruta_salida, directorio_salida):
    ruta = pathlib.Path(ruta_salida).resolve()
    base = pathlib.Path(directorio_salida).resolve()
    if not str(ruta).startswith(str(base) + os.sep):
        raise ValueError(f"Ruta de salida fuera del directorio autorizado: {ruta}")
    if len(registros) > MAX_FILAS:
        raise ValueError(f"Demasiados registros: {len(registros)} > {MAX_FILAS}")
    tmp = ruta.with_suffix(".tmp")
    try:
        libro = openpyxl.Workbook()
        hoja = libro.active
        for fila in registros:
            hoja.append([escapar_celda(str(c)) for c in fila])
```

```
28          libro.save(tmp)
29          tmp.replace(ruta)    # Requiere evaluar atomicidad y
                concurrencia en el entorno
30      except Exception:
31          tmp.unlink(missing_ok=True)
32          raise
```

El listado 11.1 ilustra la diferencia operativa: la versión sin contrato explícito funciona para entradas bien formadas y destinos válidos, pero no ofrece garantías en ningún otro caso. La segunda versión documenta y comprueba algunas condiciones, pero no demuestra por sí sola la poscondición anunciada ni la contención ante concurrencia o enlaces simbólicos. El ejemplo es deliberadamente incompleto: la comprobación previa de rutas puede sufrir condiciones de carrera; una ruta temporal predecible puede ser manipulada; y el renombrado no garantiza por sí solo persistencia ni atomicidad en todos los entornos. Además, debe comprobarse la extensión requerida por la biblioteca XLSX. Una implementación robusta necesita operaciones de sistema de archivos adecuadas al sistema operativo, manejo seguro de temporales y pruebas de fallos. El listado no debe reutilizarse como solución de producción sin estas correcciones.

11.5. Invariantes como propiedades verificables

Una invariante es una propiedad que debe mantenerse en los estados o puntos de ejecución definidos por su alcance. Puede ser una invariante de bucle, de representación de datos, de estado de un componente o de seguridad del sistema; no todas las invariantes son globales ni se comprueban en todos los instantes.

Las invariantes de seguridad son aquellas cuya violación tiene consecuencias de seguridad directas. Para el caso transversal del conversor CSV-PDF-XLSX (Anexo A) se pueden enunciar al menos tres:

1. **Contención de escritura.** Ninguna operación escribe fuera del directorio de salida autorizado, incluso ante entradas que contengan secuencias de path traversal (`../`, rutas absolutas, enlaces simbólicos).

2. **Ausencia de fórmulas en celdas.** Ninguna celda del archivo XLSX contiene contenido que un programa de hoja de cálculo interprete como fórmula, incluso si el CSV de entrada contiene campos que empiezan por `=`, `+`, `-` o `@`.

3. **Acotación de recursos.** El proceso termina en un tiempo acotado y consume una cantidad acotada de memoria, incluso ante archivos de entrada de tamaño máximo o

ante archivos maliciosos diseñados para provocar expansiones (por ejemplo, archivos ZIP con ratio de compresión extremo).

Para que estas invariantes sean verificables deben enunciarse con precisión suficiente para diseñar pruebas que las contradigan. «Ninguna celda contiene fórmulas» es verificable si se especifican los prefijos considerados peligrosos y el programa de destino al que se dirige el archivo. No es verificable como enunciado universal, porque depende del contexto de apertura.

Advertencia

Las invariantes de seguridad no se obtienen automáticamente al pedirle al modelo que «genere código seguro». El modelo no puede inferir la invariante si no forma parte del enunciado. Las invariantes deben especificarse, comunicarse al generador y verificarse independientemente del código producido.

Clases de invariantes y mecanismos de comprobación.

Una invariante de bucle puede justificarse por inicialización, preservación y terminación; una invariante de representación puede comprobarse en las fronteras de una estructura de datos; una propiedad temporal (por ejemplo, que toda solicitud aceptada obtenga respuesta) requiere un modelo de eventos y de progreso. Las aserciones, las pruebas basadas en propiedades, el análisis estático y los métodos formales cubren obligaciones diferentes. Ninguna técnica sustituye la definición precisa de la propiedad y de su ámbito.

Comparación entre autores.

Una persona puede omitir invariantes al diseñar; un generador puede omitirlas si el contexto no las recoge. Para reducir dependencia entre implementación y verificación, las propiedades críticas deben derivarse de requisitos y amenazas, no únicamente de la lectura del código, con independencia de quién lo produjo.

11.6. Modelo de amenazas como instrumento de especificación

Un *modelo de amenazas* es la declaración explícita de qué se protege, de quién y bajo qué supuestos (sección 9.2). No es un documento burocrático: es una herramienta de diseño que permite formular las preguntas correctas antes de generar o revisar código.

En el contexto del desarrollo asistido por IA, el modelo de amenazas cumple una función adicional: delimita qué propiedades debe verificar el revisor automático y qué

propiedades requieren verificación humana. El equipo de Claude Code [19] refleja esto en su proceso con niveles de riesgo que determinan el número de aprobaciones humanas requeridas; esa clasificación es, implícitamente, una instancia del modelo de amenazas aplicada a decisiones de proceso.

11.6.1. Componentes mínimos de un modelo de amenazas

Activos. Qué se quiere proteger: datos de usuario, disponibilidad del servicio, integridad del sistema de archivos, confidencialidad de credenciales.

Actores. Quién puede intentar comprometer los activos: usuarios externos sin autenticar, usuarios autenticados con privilegios limitados, atacantes con acceso a la red, procesos internos comprometidos.

Vectores. Cómo pueden los actores alcanzar los activos: entradas de usuario no validadas, archivos cargados, parámetros de configuración, dependencias de terceros, código generado no revisado.

Supuestos de confianza. Qué se da por supuesto como seguro y qué no: el sistema operativo anfitrión, la biblioteca estándar, las dependencias externas, el modelo de IA.

Controles existentes. Qué medidas ya mitigan las amenazas identificadas y cuáles son sus limitaciones.

11.6.2. Amenazas al producto y al proceso de generación

Debe separarse el modelo de amenazas del *producto* (inyección, errores de validación, consumo de recursos, dependencias vulnerables) del correspondiente al *proceso de desarrollo* (contaminación de contexto, instrucciones maliciosas en repositorios o documentos, filtración de secretos a herramientas y selección de dependencias inexistentes o inseguras). Las primeras amenazas son comunes al software escrito por personas y por IA; las segundas adquieren formas específicas cuando el flujo integra modelos y herramientas autónomas.

El modelo generador no necesita atribuirse como adversario intencional: puede tratarse como componente no confiable o como superficie de ataque. El resultado del análisis debe traducirse en propiedades verificables, controles y evidencias, no quedarse en una enumeración de riesgos.

Tabla 11.3.: Modelo de amenazas simplificado para el conversor CSV-PDF-XLSX.

Activo	**Vector**	**Amenaza**	**Control**
Integridad del sistema de archivos	Nombre del archivo de salida derivado de la entrada	Path traversal	Canonicalización y restricción de directorio (invariante 1)
Integridad del documento XLSX	Contenido de celdas del CSV	Inyección de fórmulas (CWE-1236 [6])	Escapado de prefijos peligrosos (invariante 2)
Disponibilidad del proceso	Archivo CSV de tamaño extremo o malformado	Consumo excesivo de recursos	Límites de tamaño y tiempo (invariante 3)
Cadena de suministro	Dependencias de terceros para PDF y XLSX	Vulnerabilidades en bibliotecas externas	Gestión de dependencias, YAGNI aplicado
Código generado	Modelo de IA como fuente de código	Supuestos tácitos erróneos, omisión de controles	Especificación explícita, pruebas negativas, revisión

Tabla 11.4.: Ejemplos de amenazas comunes y específicas del flujo asistido por IA.

Amenaza	**Ámbito**	**Control verificable**
Escritura fuera del directorio	Producto, ambos orígenes	Pruebas de rutas y revisión de operaciones de archivos
Inyección de fórmulas	Producto, ambos orígenes	Pruebas de celdas y formato XLSX resultante
Dependencia vulnerable	Producto y cadena de suministro	Inventario, análisis y política de versiones
Instrucciones hostiles en contexto	Flujo con agentes de IA	Aislamiento de instrucciones y restricciones de herramientas
Exposición de secretos a un servicio	Flujo y herramientas, ambos orígenes	Controles de acceso, registros y política de datos

11.7. Estrategias de verificación: pruebas negativas y propiedades

Las pruebas positivas verifican que el sistema hace lo que debe cuando se le dan entradas válidas. Las pruebas negativas verifican que el sistema *no* hace lo que no debe cuando se le dan entradas inválidas, malformadas o maliciosas. Esta distinción se aplica por igual a implementaciones humanas y generadas por IA: la cobertura de los casos nominales no implica cobertura de los fallos.

Formulación del problema. Un generador al que se pide «escribe pruebas para esta función» tiende a producir pruebas de los casos normales. Las pruebas de los casos anómalos requieren especificación de las invariantes que se deben preservar y de las clases de entradas que las deben violar.

Consecuencia en seguridad. Las rutas de error y los valores límite son los que los atacantes explotan. Un programa que maneja correctamente el 99,9 % de las entradas legítimas puede ser explotable a través del 0,1 % que nunca se probó.

Selección de técnicas.

Las particiones de equivalencia y los valores límite derivan casos del dominio de entrada; las pruebas basadas en propiedades contrastan invariantes sobre múltiples entradas; las metamórficas comprueban relaciones entre ejecuciones; el *fuzzing* explora entradas inesperadas; y las pruebas de mutación estiman si una batería detecta determinados cambios artificiales. Cada instrumento necesita un oráculo y una interpretación de sus límites.

Comparación entre autores.

Un desarrollador puede probar solo los casos que imaginó al implementar; un modelo que genera código y pruebas a partir del mismo contexto puede repetir las mismas omisiones. Una batería derivada previamente de requisitos y amenazas reduce esa dependencia, pero no garantiza independencia completa.

Listing 11.2: Pruebas positivas y negativas para la invariante de contención de escritura (ejemplo didáctico).

```
import pathlib, os, pytest
from conversor import escribir_xlsx_seguro

```

```
DIRECTORIO_SALIDA = pathlib.Path("/tmp/salida_test")

# --- Pruebas positivas ---
def test_escritura_valida(registros_simples):
    ruta = DIRECTORIO_SALIDA / "resultado.xlsx"
    escribir_xlsx_seguro(registros_simples, ruta,
        DIRECTORIO_SALIDA)
    assert ruta.exists()

# --- Pruebas negativas: invariante de contención de escritura ---
@pytest.mark.parametrize("ruta_mala", [
    "/tmp/fuera_del_directorio.xlsx",          # Ruta absoluta
        fuera
    "../fuera_del_directorio.xlsx",             # Path traversal
        relativo
    "/tmp/salida_test/../fuera.xlsx",           # Path traversal
        con componente válida
    "/tmp/salida_test/subdir/../../fuera.xlsx", # Traversal
        anidado
])
def test_path_traversal_rechazado(ruta_mala, registros_simples):
    with pytest.raises(ValueError, match="fuera del directorio
        autorizado"):
        escribir_xlsx_seguro(registros_simples, ruta_mala,
            DIRECTORIO_SALIDA)

# --- Pruebas negativas: invariante de ausencia de fórmulas ---
@pytest.mark.parametrize("prefijo_peligroso", ["=SUMA(A1)", "+CMD"
    , "-1+1", "@SUM(1,1)"])
def test_formula_escapada_en_celda(prefijo_peligroso):
    ruta = DIRECTORIO_SALIDA / "formulas.xlsx"
    registros = [[prefijo_peligroso, "valor_normal"]]
    escribir_xlsx_seguro(registros, ruta, DIRECTORIO_SALIDA)
    # Verificar que la celda no empieza por el caracter peligroso
        sin el prefijo de escape
    libro = openpyxl.load_workbook(ruta)
    celda = libro.active.cell(row=1, column=1).value
    assert not any(celda.startswith(p) for p in ("=", "+", "-", "@
        ")), \
        f"Celda no escapada: {celda!r}"

```

```
35 # --- Prueba negativa: invariante de acotación ---
36 def test_demasiados_registros_rechazados():
37     registros_gigantes = [["valor"] * 10] * (MAX_FILAS + 1)
38     with pytest.raises(ValueError, match="Demasiados registros"):
39         escribir_xlsx_seguro(registros_gigantes, DIRECTORIO_SALIDA
               / "grande.xlsx",
40                              DIRECTORIO_SALIDA)
```

El listado 11.2 no es un conjunto exhaustivo: ilustra la estructura de las pruebas negativas vinculadas a las invariantes enunciadas en la sección 11.5. La cobertura real depende del análisis de las clases de entradas relevantes para cada invariante, que a su vez depende del modelo de amenazas. Las pruebas negativas no demuestran seguridad; demuestran que las propiedades especificadas se preservan en los casos probados.

11.8. Verificación independiente y criterios de aceptación

Como ejemplo de organización de verificaciones en capas, la práctica del equipo de Claude Code [19] puede describirse como sigue. La figura 11.1 no pretende ser exhaustiva; representa la estructura lógica del proceso, no su implementación exacta.

1. **Especificación antes de la generación.** El desarrollador enuncia el contrato - precondiciones, poscondiciones, invariantes- y el modelo de amenazas relevante antes de pedir al modelo que genere código. Esta especificación alimenta el enunciado y los archivos de contexto (`CLAUDE.md`, `REVIEW.md`).

2. **Verificación interna por el agente.** El agente ejecuta las pruebas disponibles, el compilador y el analizador estático, y produce como artefacto la salida de esas herramientas, no solo el código. La verificación interna es una condición de entrega, no una cortesía.

3. **Revisión automática independiente.** Un agente revisor con contexto fresco examina el diff contra la especificación y el modelo de amenazas. La independencia de contexto es importante: evita que el revisor herede los supuestos del generador.

4. **Revisión de seguridad especializada.** Una revisión automática separada comprueba las propiedades de seguridad más sensibles. Los hallazgos de esta capa bloquean la integración hasta ser resueltos o justificados explícitamente.

5. **Revisión humana calibrada por riesgo.** La revisión humana se concentra en las decisiones que requieren juicio: arquitectura, cambios de alta consecuencia, casos

donde la especificación es ambigua o incompleta.

6. **Integración y detección rápida.** La integración se realiza con verificación adicional sobre la rama principal, y los mecanismos de rollback permiten revertir cambios en tiempo acotado.

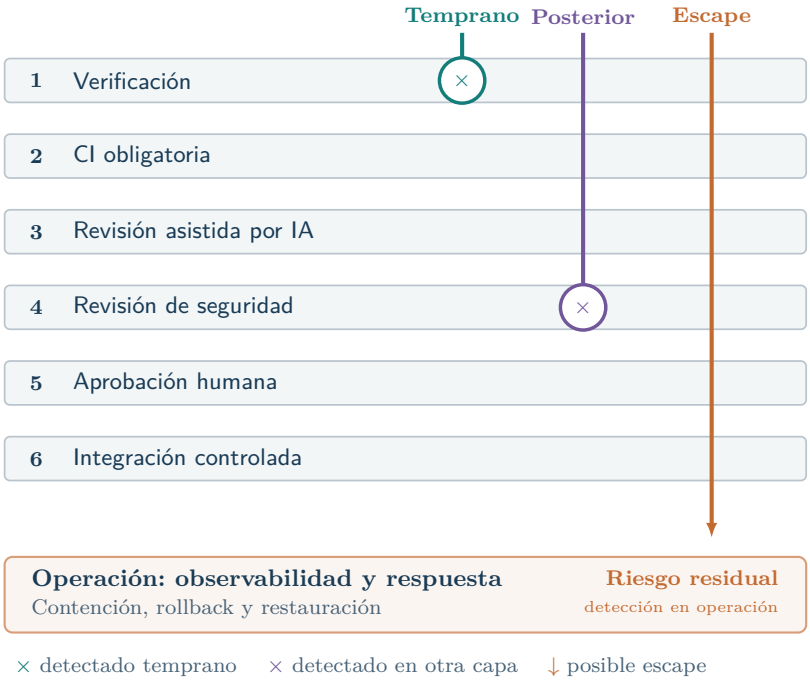


Figura 11.1.: Barreras de verificación en capas y riesgo residual: cada capa puede detectar un defecto antes o después, y lo que no se detecta pasa a la operación como riesgo residual.

Nota

El artículo de referencia documenta que, entre junio y septiembre de 2026, los *pull requests* integrados más que se duplicaron mientras la tasa de reversiones cayó más de un 40 % [19]. Este resultado no demuestra que el proceso sea generalizable sin adaptación, pero es evidencia de que la combinación de verificación en capas y calibración por riesgo puede mantener o mejorar la calidad a mayor velocidad de cambio.

La convergencia del ciclo de revisión es un problema práctico no trivial. El mismo artículo documenta un caso en el que un agente autor y un agente revisor interactuaron

durante más de cincuenta rondas sin convergencia, porque el revisor seguía detectando problemas en código preexistente que el autor no había modificado. La solución adoptada fue limitar el número de rondas, restringir la revisión posterior a los cambios incrementales y prohibir la repetición de hallazgos ya publicados. Este patrón tiene una implicación más general: la verificación automática debe diseñarse para converger; un proceso que no converge no es una propiedad de seguridad, es una fuente de ruido que degrada la señal.

11.9. Caso de estudio: comparación de código humano y generado por IA

Se propone un estudio didáctico controlado sobre el conversor CSV-PDF-XLSX. Se entrega a una persona y a un asistente de IA el mismo conjunto inicial de requisitos, el mismo entorno y el mismo tiempo disponible. Las dos implementaciones se identifican como A y B durante la evaluación. No se presupone cuál será más correcta: las diferencias observadas no pueden atribuirse causalmente al origen del código sin un diseño experimental adecuado.

Protocolo reproducible.

1. **Congelar los requisitos.** Especificar formatos admitidos, rutas autorizadas, límites de entrada, política de errores y tratamiento de fórmulas.
2. **Obtener implementaciones.** Registrar versión de dependencias, instrucciones, tiempo, herramientas y modificaciones realizadas; documentar también las condiciones de trabajo del desarrollador humano.
3. **Preparar un oráculo independiente.** Derivar contratos, invariantes y pruebas del documento de requisitos, sin consultar las implementaciones.
4. **Ejecutar la misma batería.** Aplicar compilación o análisis, pruebas nominales, límites, entradas malformadas, propiedades y escenarios del modelo de amenazas.
5. **Registrar evidencias.** Conservar comandos, resultados, versiones, cobertura pertinente, defectos reproducibles y limitaciones.

Interpretación.

La comparación debe informar qué propiedades se han demostrado, cuáles se han observado en pruebas y cuáles siguen sin evaluar. La cantidad de pruebas superadas no

Tabla 11.5.: Plantilla de comparación de las implementaciones A y B; resultados por completar tras la ejecución.

Criterio	**Implementación A**	**Implementación B**
Contrato de entradas y errores	Evidencia pendiente	Evidencia pendiente
Contención de escritura	Evidencia pendiente	Evidencia pendiente
Ausencia de fórmulas ejecutables	Evidencia pendiente	Evidencia pendiente
Límites de recursos	Evidencia pendiente	Evidencia pendiente
Pruebas negativas y de propiedades	Evidencia pendiente	Evidencia pendiente
Defectos reproducibles y severidad	Evidencia pendiente	Evidencia pendiente
Esfuerzo de verificación	Evidencia pendiente	Evidencia pendiente

constituye una medida universal de seguridad. Un caso didáctico no autoriza conclusiones estadísticas sobre desarrolladores humanos o modelos sin replicación, controles y análisis apropiados.

11.10. Evidencias de aceptación y trazabilidad

La aceptación de una entrega requiere un expediente mínimo de evidencias: versión del código, requisitos aplicables, contratos, invariantes, amenazas consideradas, resultados de herramientas, incidencias abiertas y decisión justificada sobre riesgos residuales. El expediente debe poder revisarse sin depender de afirmaciones del autor, sea una persona o un modelo.

Consejo

Antes de integrar un cambio, exigir para cada propiedad crítica: (1) una formulación comprobable; (2) el mecanismo y resultado de verificación; (3) los supuestos y límites de esa evidencia; y (4) una decisión explícita sobre las propiedades no verificadas. La ejecución satisfactoria de un agente no sustituye el registro de resultados reproducibles.

Verificar no es aprobar.

La verificación produce evidencias sobre propiedades especificadas; la aprobación decide si esas evidencias, junto con el riesgo residual y el contexto de uso, justifican aceptar un cambio. Una batería satisfactoria no es, por sí sola, una autorización. Conviene distinguir cinco decisiones que pueden recaer en la misma persona pero que *no son la misma decisión*:

1. **Resultado de la verificación**: si se satisface cada propiedad comprobada, con qué evidencia y bajo qué supuestos.
2. **Cumplimiento de criterios de aceptación**: si el conjunto de evidencias satisface los criterios declarados (sección 10.5).
3. **Autorización de integración**: si el cambio puede incorporarse al sistema, tras validar el estado que realmente se integra (sección 10.6).
4. **Autorización de despliegue**: si puede exponerse a usuarios reales, con qué mecanismos de despliegue controlado y qué condiciones operativas.
5. **Aceptación del riesgo residual**: si los riesgos no cubiertos son aceptables en el contexto de uso, con las limitaciones conocidas declaradas.

Estas decisiones pueden producirse en momentos distintos y con responsables distintos. Presentar un expediente de verificación verde como autorización automática de despliegue confunde la evidencia con la decisión.

11.11. Verificación composicional y contratos entre componentes

Una función puede satisfacer todas sus poscondiciones y, sin embargo, participar incorrectamente en un sistema. La verificación composicional exige comprobar no solo cada componente, sino también que las garantías de uno satisfacen las hipótesis del siguiente. En un contrato de tipo *assume-guarantee*, el componente C_i promete una garantía G_i siempre que su entorno cumpla una hipótesis A_i. Para conectar dos componentes es necesario justificar, entre otras obligaciones, que la salida del primero satisface la precondición del segundo; verificar cada uno aisladamente no establece esa compatibilidad.

Considérese una operación de atribución energética que devuelve valores `Decimal` con seis posiciones decimales. Aunque su cálculo esté demostrado correcto, persistir

el resultado en una columna que admite tres posiciones puede introducir redondeos incompatibles con los requisitos. De manera semejante, una enumeración que devuelve un entero puede romper un consumidor que espera una cadena. Ambos son ejemplos de desacuerdo contractual en los límites del componente, no necesariamente de un fallo del algoritmo interno. Una experiencia publicada por un desarrollador que utilizó Dafny ilustra precisamente estas clases de fallos, junto con defectos en el propio banco de pruebas[20]. Se trata de evidencia anecdótica y no permite estimar su frecuencia.

Tabla 11.6.: Obligaciones de verificación en distintos niveles de composición.

Nivel	**Obligación**	**Evidencia apropiada**
Función	Precondiciones, poscondiciones y terminación, cuando corresponda	Pruebas de propiedades, análisis o demostración
Interfaz	Tipos, unidades, precisión, codificación y semántica de errores	Pruebas de contrato y validación en límites
Integración	Compatibilidad entre dependencias, estado y transacciones	Pruebas de integración y escenarios extremos
Sistema	Requisitos de extremo a extremo y propiedades no funcionales	Pruebas de sistema, monitorización y análisis de riesgos

En el desarrollo humano y en el asistido por IA, las mismas obligaciones son necesarias. En flujos con agentes que modifican varios componentes de forma simultánea conviene, además, comprobar explícitamente la consistencia de los cambios entre archivos, esquemas, configuraciones y consumidores.

11.12. Validez de las pruebas y de sus oráculos

Una batería de pruebas superada solo constituye evidencia si realmente ejecuta las rutas relevantes y sus aserciones distinguen resultados correctos de incorrectos. El *oráculo de prueba* es el criterio que decide si una ejecución cumple la propiedad evaluada. Puede proceder de un resultado esperado, un modelo de referencia, una relación metamórfica o una especificación formal. Un oráculo incorrecto puede aceptar implementaciones incorrectas independientemente de su autoría.

Antes de aceptar una prueba, conviene comprobar que: (i) alcanza el comportamiento declarado; (ii) sus aserciones se ejecutan realmente; (iii) los dobles de prueba no sustituyen precisamente el comportamiento que se pretende comprobar; y (iv) la prueba falla al

introducir un defecto representativo. El análisis de cobertura ayuda a detectar rutas no recorridas, pero una cobertura alta no implica aserciones eficaces. Las pruebas de mutación permiten comprobar si cambios deliberados en la implementación provocan fallos observables; su resultado depende del conjunto de mutantes y de las pruebas ejecutadas.

Tabla 11.7.: Defectos de la infraestructura de verificación y controles asociados.

Defecto	**Consecuencia**	**Control**
Aserción omitida o no ejecutada	Falso resultado satisfactorio	Comprobar ejecución y fallos deliberados
Datos de prueba incompletos	Ruta relevante nunca alcanzada	Instrumentación y cobertura de ramas
Oráculo derivado del resultado observado	Se consolida el defecto como expectativa	Derivar expectativas de requisitos independientes
Prueba debilitada o eliminada	Pérdida de capacidad de detección	Revisar diferencias en pruebas y aserciones

Las herramientas de IA pueden generar tanto el código como sus pruebas. Esto resulta útil para ampliar escenarios, pero crea un riesgo de errores correlacionados cuando ambos artefactos comparten supuestos equivocados. Por ello, en propiedades críticas se recomienda definir oráculos y criterios de aceptación desde requisitos independientes, y revisar especialmente las modificaciones que suprimen pruebas, reducen aserciones o introducen *mocks* que ocultan integraciones[21].

Hallazgos de revisores automáticos.

Cuando varios revisores automáticos informan de hallazgos, la agregación de sus resultados requiere criterio. Contar cuántos coinciden (consenso) reduce el ruido de falsos positivos, pero *no* garantiza independencia: varios agentes que comparten modelo, contexto o ejemplos pueden coincidir por el mismo sesgo, no porque el hallazgo sea real. La coincidencia entre revisores es una señal, no una prueba. Cada hallazgo debe poder contrastarse con una evidencia externa -una reproducción, una prueba que lo confirme, una referencia a la propiedad violada- antes de tratarlo como defecto confirmado. La distinción entre independencia de modelo, de contexto y de enunciado se desarrolló en la sección 8.5.

11.13. Seguridad del proceso de generación y alcance de los cambios

El modelo de amenazas debe cubrir dos objetos distintos: el producto que se entrega y el entorno que lo genera. Un asistente limitado a sugerir fragmentos tiene una superficie de actuación diferente de un agente que lee incidencias, ejecuta comandos, instala dependencias y modifica archivos. En este último caso, el contenido de repositorios, solicitudes de cambio, documentación y respuestas de herramientas debe tratarse como entrada potencialmente no confiable. Una instrucción maliciosa insertada en esos materiales puede intentar alterar los objetivos del agente o inducir acciones no autorizadas[21, 22].

Las medidas técnicas incluyen ejecución aislada, privilegios mínimos, credenciales de corta duración, revisión de nuevas dependencias y restricciones de acceso a red y sistema de archivos. También deben inspeccionarse *todos* los archivos modificados: un cambio funcional aparentemente pequeño puede incluir alteraciones en pruebas, ficheros de bloqueo, scripts de compilación o configuración de integración continua. La descripción generada de una solicitud de integración no sustituye la revisión del *diff* real.

Esta amenaza no convierte automáticamente a la IA en un adversario. Distinguir error involuntario, entrada manipulada y capacidad excesiva del agente permite seleccionar controles adecuados sin atribuir intención al modelo. En el desarrollo tradicional también existen dependencias comprometidas, scripts peligrosos y modificaciones fuera de alcance; la autonomía del agente cambia las vías de exposición y la escala de posibles acciones.

11.14. Verificación proporcional al riesgo y eficacia de la revisión

La profundidad de la revisión debería depender de la criticidad de las propiedades afectadas, la exposición del componente, la amplitud del cambio y la capacidad de recuperación. No todos los cambios requieren una prueba formal, pero tampoco basta con ejecutar una batería genérica de pruebas para aceptar código que modifica autenticación, autorización, persistencia o mecanismos de despliegue. Un estándar de requisitos como OWASP ASVS puede utilizarse para concretar obligaciones de seguridad en aplicaciones web; no debe confundirse con una certificación automática derivada de ejecutar herramientas[5].

El proceso descrito por el equipo de Claude Code (sección 11.8) distribuye la revisión en capas automáticas y humanas, utiliza controles específicos según el riesgo y considera las tasas de reversión e incidentes como salvaguardas frente a una aparente reducción del

tiempo de revisión. Es una práctica industrial contextual, no una demostración de eficacia universal. Para evaluar un proceso conviene registrar al menos los defectos relevantes detectados antes del despliegue, los incidentes posteriores, los cambios revertidos, el tiempo humano invertido y la proporción de requisitos con evidencia verificable. Una mejora en una sola métrica no demuestra por sí misma mayor calidad.

La aceptación debe documentar el identificador de cada requisito, el contrato o propiedad asociada, la versión exacta del artefacto, el mecanismo de verificación, el resultado y las limitaciones conocidas. Cuando se utilicen herramientas de IA, la información sobre su intervención puede mejorar la trazabilidad del proceso, pero no reemplaza ninguna prueba técnica sobre el producto[23].

Clasificación operativa por riesgo.

Para que la profundidad de revisión sea proporcional, conviene asignar cada cambio a una categoría de riesgo y exigir un número de aprobaciones coherente con ella. La tabla siguiente es una **Propuesta docente**: los criterios y umbrales deben fijarse y justificarse en cada organización, y no se presentan como una clasificación universal.

Tabla 11.8.: Niveles de riesgo y aprobación exigida. **Propuesta docente**; los umbrales y las rutas críticas deben definirse por organización.

Nivel	**Criterios orientativos**	**Aprobación**	**Rutas que siempre exigen persona**
Bajo	Documentación, copias, cambios internos tras *flag*	Controles automáticos y puertas verdes	—
Medio	Cambios de producto ordinarios; áreas sensibles (caché, configuración)	Una persona, distinta del generador	—
Alto	Lógica de negocio crítica; cambios amplios; alta exposición	Una persona, con revisión de seguridad	—
Crítico	Autenticación, autorización, persistencia, secretos, CI/CD, despliegue	Dos personas o más; revisión de seguridad obligatoria	Sí, siempre

Dos reglas acompañan la clasificación. Primera: el nivel lo determina *el cambio*, no el autor; un cambio aparentemente pequeño en una ruta crítica no es de bajo riesgo. Segunda: la aprobación automatizada solo es razonable cuando el riesgo es bajo, las puertas están verdes y existe una capacidad de recuperación efectiva; nunca sustituye una aprobación humana exigida en una ruta crítica. La independencia de contexto de la

revisión (sección 8.5) sigue siendo necesaria con independencia del nivel.

11.15. Ejercicios

1. **Contrato.** Especifique mediante precondiciones, poscondiciones y errores una función que convierta registros CSV en XLSX. Exprese una propiedad mediante una tripleta de Hoare y aclare si demuestra terminación.

2. **Invariantes.** Distinga una invariante de representación, una de seguridad y una propiedad temporal del conversor. Asigne a cada una un mecanismo de comprobación y señale sus límites.

3. **Amenazas.** Extienda la tabla 11.3 con dos amenazas al producto y una amenaza al flujo de generación con IA. Especifique activos, límites de confianza, controles y evidencias.

4. **Pruebas.** Amplíe el listado 11.2 con pruebas de valores límite y una propiedad generativa. Explique qué fallos no detectarían y qué problemas presenta el código de ejemplo.

5. **Comparación.** Aplique el protocolo del caso de estudio a dos implementaciones de una misma función, una humana y otra asistida por IA. Complete la tabla 11.5 con resultados reproducibles, sin inferir superioridad general a partir de un caso.

6. **Aceptación.** Redacte un expediente breve de verificación: requisitos, propiedades, pruebas, hallazgos y riesgos residuales.

11.16. Síntesis

Los contratos hacen explícitas precondiciones, poscondiciones y fallos; las invariantes expresan propiedades que deben mantenerse; y los modelos de amenazas identifican escenarios que las implementaciones deben resistir. Las pruebas, el análisis y los métodos formales proporcionan evidencias de alcance distinto sobre esas obligaciones. El criterio de aceptación debe explicitar tanto las propiedades comprobadas como las limitaciones restantes.

Estos instrumentos son comunes al código humano y al generado por IA. El uso de modelos puede modificar la velocidad de producción, la trazabilidad y las dependencias entre autor y evaluador, pero no cambia qué constituye una prueba técnica de corrección. La comparación entre ambos orígenes debe basarse en requisitos, herramientas y resultados

observables, no en la plausibilidad de la implementación ni en generalizaciones sobre sus autores.

Nota

Las diferencias propuestas entre flujos humanos y asistidos por IA son hipótesis de trabajo cuando no se acompañan de datos comparativos. El caso de estudio ofrece un protocolo para obtener evidencia, no resultados experimentales inventados.

12. Metodología docente AI-SDLC

Los capítulos anteriores acumularon las piezas: correctitud y seguridad (capítulo 3), familias de fallo (capítulo 4), evaluación reproducible (capítulo 6), aislamiento (capítulo 9) y validación automática (capítulo 10). Este capítulo las integra en el **AI-Assisted Secure Development Lifecycle** (AI-SDLC): un ciclo de vida desde la especificación hasta el mantenimiento, pasando por generación asistida, revisión, validación y atestación del aislamiento.

12.1. Idea rectora y desplazamiento del esfuerzo

La hipótesis rectora del capítulo se concreta aquí en una afirmación operativa (véase su enunciado completo en el capítulo 1). La IA generativa **reduce ciertos costes de implementación** -escritura inicial, andamiaje, traducción entre lenguajes, generación de borradores de prueba- **sin eliminar** los problemas de correctitud, seguridad, mantenimiento, portabilidad y responsabilidad; conviene recordar que la evidencia empírica disponible es contextualizada (el estudio de METR de 2025, por ejemplo, midió una *ralentización* en su contexto) [15, 14]. Lo que la IA hace es *desplazar el esfuerzo humano*: desde la escritura hacia la **especificación**, la **revisión** y la **validación**. Esta observación se etiqueta como **Hipótesis** de trabajo del capítulo, no como resultado medido.

Si el esfuerzo se desplaza, la metodología docente debe desplazarse con él. Un ciclo de vida diseñado para un equipo que escribe cada línea de código no dedica la atención necesaria a formular requisitos verificables, a revisar artefactos ajenos o a construir evidencia. El AI-SDLC reordena las actividades para hacer explícitas esas tareas, y añade una que el desarrollo clásico no siempre nombra: la **selección y el gobierno del sistema de IA** empleado.

Nota

Etiquetado epistémico inicial. Se marca como **Evidencia**: que la revisión, la trazabilidad y la verificación de requisitos son prácticas de desarrollo seguro consolidadas en los marcos citados [3, 5, 9]. Se marca como **Propuesta docente**:

la secuencia de fases, sus entradas, responsables, evidencias y salidas, la tabla resumen y el modelo de gobernanza. Se marca como **Hipótesis**: que seguir esta secuencia reduzca el riesgo en un contexto dado; esa reducción no está medida aquí y no se afirma como hecho.

12.2. Desarrollo guiado por especificaciones: del prompt al proceso de ingeniería

Esta sección sitúa el *Specification-Driven Development* (SDD, desarrollo guiado por especificaciones) dentro de la evolución de los procesos de ingeniería asistidos por IA. No se presenta como una metodología definitiva ni como una garantía de calidad: se analiza cómo el recurso a especificaciones explícitas contribuye a estructurar el desarrollo, y cuáles son sus límites. La aportación contemporánea que interesa aquí no es la idea de especificar -que es tan antigua como la ingeniería de requisitos- sino el papel de la especificación como mecanismo de **coordinación y control** del trabajo realizado por agentes de IA.

12.2.1. Del prompt al proceso

Dirigir la implementación únicamente mediante instrucciones en lenguaje natural tiene límites conocidos por la ingeniería de requisitos: las instrucciones pueden ser ambiguas, incompletas o contradictorias; el contexto se pierde entre iteraciones; no queda traza explícita de qué se pidió y por qué; y el resultado es difícil de evaluar de forma sistemática. Nada de esto es exclusivo de los agentes de IA: son los mismos problemas que la ingeniería de requisitos aborda desde hace décadas. Lo que cambia con los modelos generativos es la *velocidad* con que se producen borradores y la *facilidad* con que una instrucción incompleta se convierte en código plausible (capítulo 8). El SDD propone estructurar el proceso mediante artefactos explícitos -requisitos, planificación, tareas, criterios de aceptación- que median entre la intención y la implementación. Se etiqueta como **Propuesta metodológica**: estructura el proceso, no garantiza por sí sola el resultado.

12.2.2. Fundamentos

El SDD organiza el desarrollo en torno a una *especificación* que reúne requisitos, restricciones, criterios de aceptación y casos límite; de ella derivan una planificación,

una descomposición en tareas, la implementación, las pruebas y la validación. Conviene distinguir dos sentidos de «especificación»: una especificación en *lenguaje natural estructurado* -como los requisitos numerados en notación EARS (*Easy Approach to Requirements Syntax*) que emplea Hello SDD [25]- y una especificación *formal* (contratos, precondiciones y postcondiciones verificables, como en el capítulo 3). La primera guía el desarrollo; la segunda permite razonar sobre corrección. La notación EARS es anterior al SDD y procede de la ingeniería de requisitos: el SDD la *usa*, no la origina.

Una especificación no es completa, correcta ni verificable por el mero hecho de estar documentada. La calidad de la especificación condiciona la calidad de todo lo que de ella se deriva (capítulo 3): un requisito mal formulado produce una implementación que satisface fielmente algo distinto de lo que se necesita. La gestión de cambios forma parte del proceso: cuando cambia un requisito, deben revisarse el plan, las tareas, las pruebas y las evidencias afectadas, y no solo el código.

12.2.3. SDD y agentes de IA

Las especificaciones actúan como *artefactos de coordinación* entre responsables humanos y agentes de desarrollo: separan la definición del problema de su implementación, permiten descomponer el trabajo, conservan decisiones y restricciones entre iteraciones y sirven de base para revisar artefactos intermedios (plan, tareas) antes de generar código. Esta coordinación es útil, pero **no** sustituye la verificación independiente: que varios agentes compartan una especificación no los hace independientes entre sí (sección 8.5), y la existencia de un flujo automatizado no aporta garantías de corrección (sección 11.10). La revisión de los artefactos intermedios es una oportunidad de control precisamente porque ocurre antes de que el código exista; pero esa revisión hereda las mismas limitaciones que cualquier revisión: puede pasar por alto un requisito mal entendido o una restricción ausente.

12.2.4. Implementaciones

Dos iniciativas ilustran el enfoque con diferencias metodológicas documentadas. *GitHub Spec Kit* [24] parte de una *constitución* de principios del proyecto y articula el flujo en etapas (especificar, planificar, descomponer en tareas, implementar y converger), con marcadores explícitos de ambigüedad y puertas de calidad. *Hello SDD* [25] presenta un flujo docente más ligero -constitución, especificación, clarificación, plan, tareas, implementación guiada por pruebas y validación- con requisitos en notación EARS y una plantilla de especificación reutilizable. Ambas son **documentación de prácticas**, no estándares

ni evidencia experimental; sus comandos y detalles concretos pueden cambiar, por lo que el libro describe el enfoque, no la herramienta. La diferencia relevante para el lector no está en los comandos, sino en qué artefactos hacen explícitos y cómo se encadenan: ambas coinciden en tratar la especificación como el artefacto del que derivan el resto.

12.2.5. Limitaciones

El SDD estructura el proceso, pero no garantiza la corrección. Persisten especificaciones incompletas o incorrectas, ambigüedad residual, dificultad para fijar criterios de aceptación y límites de cobertura de las pruebas; superar pruebas no equivale a satisfacer todos los requisitos (sección 11.2). El SDD no aporta garantías formales por sí mismo y puede generar una falsa confianza derivada de procesos que *parecen* rigurosos. Por ello debe distinguirse expresamente entre **Specification-Driven Development** -un proceso que estructura el desarrollo en torno a especificaciones- y **Verified Software Engineering** -la obtención de garantías mediante verificación formal, pruebas y análisis bajo supuestos declarados (capítulo 11)-. Automatizar pasos del proceso no automatiza las decisiones de aceptación: la supervisión humana sigue siendo necesaria allí donde el riesgo residual no está cubierto por evidencia. El SDD no es una solución completa a los problemas de seguridad, calidad y responsabilidad.

12.2.6. Integración con el ciclo de vida

El SDD se sitúa dentro del ciclo de ingeniería asistido por IA que desarrolla el libro. La *especificación* establece requisitos, restricciones y criterios de aceptación; la *verificación* produce evidencias sobre la conformidad con las propiedades especificadas (capítulo 11); la *aprobación* decide si esas evidencias, junto con el riesgo residual, justifican aceptar el cambio (sección 11.10); la *operación* incorpora despliegue, observabilidad y respuesta ante incidentes, y genera nueva evidencia que realimenta la especificación (sección 12.16); la *reversibilidad* permite intervenir sobre cambios y decisiones, recuperando estados anteriores cuando es técnica y operativamente viable (secciones 8.3 y A.9). La reversibilidad y la trazabilidad se tratan como propiedades transversales del proceso, no como etapas secuenciales aisladas. Todo ello se vincula con la gestión del riesgo y la responsabilidad: quien acepta un cambio asume el riesgo residual que las evidencias no cubren (sección 8.7).

Para ilustrar la diferencia entre *generar* código, *estructurar* su desarrollo y *establecer garantías* de ingeniería, la tabla 12.1 compara tres aproximaciones sobre la *misma* funcionalidad del caso transversal (Anexo A). No afirma que una aproximación produzca

necesariamente código de mayor calidad: describe qué produce cada una y qué no garantiza.

Tabla 12.1.: Tres aproximaciones al desarrollo de una misma funcionalidad del caso transversal. **Propuesta docente**. La comparación describe qué produce cada enfoque, no una jerarquía de calidad.

Aproximación	**Qué produce**	**Qué no garantiza**
A. Prompt-driven	Código a partir de instrucciones directas en lenguaje natural	Trazabilidad de requisitos, cobertura, evidencias
B. Specification-driven	Requisitos, plan, tareas, implementación y pruebas estructuradas	Que la especificación sea correcta o completa; garantías formales
C. Ciclo de ingeniería asistido por IA	B, más verificación con evidencias, aprobación, operación y reversibilidad	Que las pruebas sean exhaustivas; corrección general

12.3. Modalidades de integración de la IA en el ciclo de vida del software

Conviene distinguir cómo se integra la IA en el ciclo de vida del desarrollo de software (SDLC) antes de presentar las fases de la metodología. El ciclo de vida del desarrollo de software asistido por inteligencia artificial (**AI-SDLC**) puede entenderse como la incorporación de sistemas de IA a las actividades de especificación, diseño, implementación, verificación, integración, despliegue y mantenimiento del software [28]. Esta incorporación no presupone una metodología única ni implica necesariamente la sustitución de las prácticas convencionales de ingeniería del software. La terminología de este campo todavía está asentándose y distintas fuentes emplean etiquetas solapadas con énfasis distintos, sin una definición universal [27]. Se distinguen, a efectos docentes, cuatro modalidades.

SDLC sin IA generativa. No es un proceso completamente manual. Incluye la automatización convencional del desarrollo: integración continua, pruebas automáticas, análisis estático, metodologías ágiles, DevOps y desarrollo iterativo. La IA generativa se incorpora a este sustrato, no lo sustituye.

SDLC asistido por IA (AI-assisted SDLC). La IA asiste en actividades concretas -sugerencia de código, generación de pruebas, análisis de resultados, redacción de borradores- dentro de un proceso preexistente. No se asume que las personas escriban necesariamente la mayor parte del código, pero la organización del trabajo sigue siendo la

del proceso convencional, con herramientas de IA añadidas.

SDLC nativo de IA (AI-native SDLC). El proceso se reorganiza de forma sistemática en torno a las capacidades de la IA: los artefactos, las puertas de calidad y la coordinación se diseñan asumiendo la participación continua de sistemas de IA. Esta es la diferencia central respecto del SDLC asistido: no se añaden herramientas de IA a un proceso sin cambios, sino que se rediseña el ciclo de vida en torno a la participación de la IA [27]. No se adopta aquí ninguna definición comercial como estándar universal; se describe una orientación organizativa.

Desarrollo agéntico (Agentic SDLC). Los agentes ejecutan secuencias de acciones mediante herramientas y entornos: pueden leer y escribir ficheros, ejecutar pruebas, invocar servicios y encadenar pasos [27]. Conviene distinguir la *autonomía de ejecución* de la *autoridad de aprobación* (sección 12.3.1), y no asumir que los agentes trabajan siempre en paralelo: la coordinación puede ser secuencial y su coste no es despreciable.

Advertencia

No es una clasificación normalizada ni una escala de madurez. Estas cuatro modalidades son una distinción conceptual orientativa para el análisis docente, no una taxonomía normalizada por ningún organismo. Tampoco representan necesariamente etapas sucesivas de madurez: una organización puede usar desarrollo agéntico para unas tareas y desarrollo asistido para otras, y la adopción de una modalidad no garantiza por sí misma más productividad, más calidad ni más seguridad. La integración de IA desplaza el esfuerzo y añade riesgos; no sustituye la verificación, la aprobación ni la responsabilidad (véase el capítulo 8).

Tabla 12.2.: Modalidades de integración de la IA en el ciclo de vida del desarrollo de software (clasificación conceptual orientativa, **Propuesta docente**).

Dimensión	**SDLC sin IA generativa**	**SDLC asistido por IA**	**SDLC nativo/agéntico**
Organización	Cascada, ágil, DevOps u otras	Proceso existente con herramientas de IA	Proceso adaptado a la participación sistemática de IA

Dimensión	**SDLC sin IA generativa**	**SDLC asistido por IA**	**SDLC nativo/agéntico**
Requisitos	Elaboración humana con herramientas convencionales	IA propone y analiza requisitos	Agentes pueden transformar especificaciones en tareas ejecutables
Implementación	Humana y automatización convencional	Generación y modificación asistida	Ejecución autónoma de tareas de implementación
Pruebas	Automatizadas o manuales	IA propone pruebas y analiza resultados	Agentes pueden ejecutar ciclos de generación, prueba y corrección
Revisión	Humana y herramientas automáticas	Revisión humana apoyada por IA	Revisión multinivel con agentes y controles independientes
Aprobación	Según políticas de la organización	Según políticas de la organización	Según políticas de la organización, con controles adaptados a la autonomía
Operación	Supervisión humana y automatización convencional	IA ayuda al diagnóstico y mantenimiento	Agentes pueden ejecutar acciones operativas dentro de límites autorizados
Riesgo adicional	Riesgos convencionales del desarrollo	Dependencia de resultados generados y confianza no calibrada	Riesgos anteriores más acciones autónomas, permisos y manipulación del contexto

Las modalidades describen *cómo participa* la IA en el proceso; las diez fases que se presentan a continuación describen *cómo se organiza y controla* el trabajo. Son dimensiones complementarias: la metodología propuesta sigue siendo aplicable con distintos niveles de autonomía, porque las puertas, las evidencias y las aprobaciones no dependen de que la ejecución sea humana o agéntica.

12.3.1. Autonomía de ejecución y autoridad de aprobación

Una distinción central para el desarrollo agéntico es que **la autonomía de ejecución no implica autonomía de aprobación**. Un agente puede *generar código, ejecutar pruebas, proponer correcciones y preparar solicitudes de integración* sin estar autorizado necesariamente para *aprobar su propio trabajo, aceptar el riesgo residual, modificar políticas de seguridad, desplegar cambios críticos ni acceder a sistemas de producción.*

Conviene separar cuatro nociones que a menudo se confunden:

Capacidad técnica. Lo que el agente *puede* hacer con las herramientas y permisos disponibles en su entorno.

Permiso. Lo que el sistema *le deja* hacer según sus credenciales, alcance y política de acceso (véase el capítulo 9).

Autoridad. Lo que está *legítimamente facultado* a decidir dentro del proceso -por ejemplo, aprobar un cambio o aceptar un riesgo-.

Responsabilidad. A quién corresponde *responder* por el resultado y por el riesgo residual aceptado.

Esta separación enlaza con el riesgo epistémico del código plausiblemente incorrecto (capítulo 8), con los límites de confianza y permisos de los agentes (capítulo 9), con las puertas y evidencias del pipeline de validación (capítulo 10) y con la independencia de la revisión y los criterios de aceptación del código generado (capítulo 11). La aprobación sigue siendo una decisión humana con responsabilidad asociada, con independencia de cuánta autonomía tenga la ejecución.

12.4. Visión general de las fases

El AI-SDLC propuesto consta de diez fases. La tabla 12.3 las enumera con su objetivo y su salida principal, para dar una visión de conjunto antes de detallar cada una. El orden es un *orden de dependencia*, no una cadena rígida: cada fase presupone las condiciones de las anteriores y produce las de las siguientes, pero el proceso puede iterar -volver a requisitos al descubrir un riesgo, volver a generación al rechazar un artefacto-.

Tabla 12.3.: Visión general de las diez fases del AI-SDLC propuesto. Tabla de **Propuesta docente**. El orden indica dependencia, no obligación de recorrerlo una sola vez.

F.	**Fase**	**Objetivo**	**Salida principal**
1	Requisitos funcionales	Declarar qué debe hacer el sistema y bajo qué supuestos	Especificación funcional verificable
2	Requisitos de seguridad	Declarar qué no debe violar y qué se protege	Especificación de seguridad y modelo de amenazas
3	Plataforma	Fijar el entorno de desarrollo y de ejecución	Plataforma y cadenas de herramientas registradas
4	Selección del sistema de IA	Elegir y registrar el sistema y su configuración	Ficha de la tupla (modelo, versión, configuración, prompt/contexto, herramientas, entorno)
5	Generación	Producir el artefacto candidato asistido por IA	Artefacto candidato con condiciones de generación registradas
6	Revisión	Inspeccionar el artefacto antes de validarlo	Informe de revisión y hallazgos clasificados
7	Validación	Comprobar requisitos con el pipeline (capítulo 10)	Expediente de evidencia por requisito
8	Atestación del aislamiento	Verificar y registrar el aislamiento usado en la fase 7	Registro de configuración y evidencia del aislamiento
9	Aceptación	Decidir la promoción con evidencia y revisión humana	Decisión registrada y firmada
10	Mantenimiento	Sostener el artefacto ante el cambio	Procedimiento de actualización y trazabilidad continuada

Nota

Las fases no son un estándar ni un certificado. Que un proyecto recorra las diez fases no lo convierte en «conforme al AI-SDLC»: no existe tal conformidad, porque el AI-SDLC no es un documento normativo. Las fases son una *herramienta de enseñanza* para ordenar decisiones que en un desarrollo profesional aparecerían con otros nombres y en otros marcos.

12.5. Fase 1: requisitos funcionales

La primera fase declara *qué* debe hacer el sistema, con independencia de cómo se implemente y de qué asistente lo genere. En el caso transversal, los requisitos funcionales son leer un CSV, normalizar sus columnas y generar PDF y XLSX. El valor docente de esta fase es que fija el patrón contra el cual se juzgará todo lo demás: sin requisitos verificables, las fases posteriores no tienen contra qué comparar y una prueba solo demuestra que el programa hace *algo*.

Tabla 12.4.: Fase 1 - Requisitos funcionales. **Propuesta docente**.

Dimensión	**Contenido**
Entradas	Enunciado del problema, necesidades de los usuarios, caso transversal (CSV a PDF/XLSX; Anexo A)
Responsables	Analista de requisitos y usuario; docentes y estudiantes en el laboratorio
Evidencias	Especificación funcional escrita, criterios de aceptación funcionales, casos de uso
Riesgos	Requisitos ambiguos, incompletos o no verificables; supuestos no declarados; alcance creciente
Salida	Especificación funcional revisable, con cada requisito identificado y comprobable

Nota sobre la responsabilidad en esta fase.

La generación asistida puede ayudar a redactar la especificación, pero *decidir qué se necesita* es una tarea humana. Un requisito funcional mal formulado se propaga a todas las fases y ninguna herramienta posterior lo corrige.

12.6. Fase 2: requisitos de seguridad

La segunda fase declara *qué no debe violar* el sistema y *qué se protege*. Es la fase que separa el desarrollo seguro de la mera funcionalidad y la que alimenta el modelo de amenazas. En el caso transversal, los requisitos de seguridad son: no confiar en el CSV, evitar el *path traversal* al escribir salidas, crear temporales seguros, limitar tamaño y tiempo de proceso, no ejecutar contenido embebido y neutralizar el riesgo de fórmulas al exportar a una hoja de cálculo. El mecanismo debe definirse para los programas consumidores previstos y verificarse inspeccionando las celdas generadas, no solo comprobando que el fichero existe.

Tabla 12.5.: Fase 2 - Requisitos de seguridad. **Propuesta docente**. La clasificación de familias de fallo remite a CWE [6] y a las reglas CERT [7]; los identificadores son ejemplos de clasificación, no una prueba de vulnerabilidad.

Dimensión	**Contenido**
Entradas	Especificación funcional, activos a proteger, modelo de amenazas, marco aplicable
Responsables	Ingeniero de seguridad y analista; revisión docente
Evidencias	Requisitos de seguridad verificables, modelo de amenazas, supuestos declarados
Riesgos	Requisitos de seguridad omitidos o no verificables; confundir la funcionalidad con la protección; supuestos de confianza no comprobados
Salida	Especificación de seguridad con propiedades comprobables y límites de confianza declarados

Nota

Seguridad no es una lista de pruebas. Enunciar requisitos de seguridad no garantiza que el artefacto sea seguro. Cada requisito debe poder comprobarse, y aun comprobado permanece la advertencia del capítulo: superar una batería finita de pruebas no demuestra seguridad. Un requisito de seguridad es una *propiedad declarada a verificar*, no un certificado de ausencia de fallos.

12.7. Fase 3: plataforma

La tercera fase fija el entorno de desarrollo y de ejecución: sistema operativo, arquitectura, intérprete o compilador, gestor de dependencias, contenedores y cadenas de herramientas. Es una fase que en el desarrollo clásico suele quedar implícita y que aquí

se hace explícita porque la reproducibilidad del entorno es una condición de la validez de la evidencia, tal como estableció el capítulo 10.

Tabla 12.6.: Fase 3 - Plataforma. **Propuesta docente**.

Dimensión	**Contenido**
Entradas	Requisitos funcionales y de seguridad, restricciones de despliegue, políticas de la organización
Responsables	Ingeniero de plataforma y equipo de desarrollo; docentes en el laboratorio
Evidencias	Descripción declarativa del entorno, versiones fijadas, imagen por resumen criptográfico
Riesgos	Entorno no reproducible; dependencias móviles; divergencia entre desarrollo y producción; superficie de ataque de la cadena de suministro
Salida	Plataforma registrada y reconstruible, con dependencias y herramientas fijadas

Nota

Reproducibilidad y aislamiento son distintos. Fijar la plataforma hace verificable que la comprobación se hace sobre el mismo objeto; el aislamiento del capítulo 9 reduce el riesgo de que la ejecución de código no fiable afecte al anfitrión, bajo las condiciones declaradas y sin garantía absoluta. La plataforma es una propiedad del *proceso*; el aislamiento, del *momento de ejecución*.

12.8. Fase 4: selección del sistema de IA

La cuarta fase elige y registra el sistema de IA que asistirá al desarrollo. Aquí se aplica la distinción terminológica del capítulo: no se selecciona «un modelo», sino una *tupla* de modelo, versión, configuración, *prompt*/contexto, herramientas habilitadas y entorno (definida en el capítulo 6). La salida de la fase es una ficha que hace explícitas esas variables, porque sin ellas la evidencia posterior no es atribuible a un artefacto concreto. El *prompt*/contexto y el número de repeticiones se registran como covariables de la tupla.

Consejo

La responsabilidad humana no se delega. El sistema de IA genera, sugiere y resume, pero la decisión de usarlo, de confiar en su salida y de promover el artefacto corresponde a una persona. El registro de la tupla no es burocracia: es la condición para que esa persona pueda responder de *qué* se usó, *cómo* y con *qué*

Tabla 12.7.: Fase 4 - Selección del sistema de IA. **Propuesta docente**. La ficha registra la tupla, no «el modelo» genérico.

Dimensión	Contenido
Entradas	Necesidades del proyecto, requisitos de confidencialidad, políticas de herramientas autorizadas
Responsables	Responsable técnico y de seguridad; aprobación institucional
Evidencias	Ficha del sistema (modelo, versión, configuración, prompt/contexto, herramientas, entorno) y su justificación
Riesgos	Uso de un sistema no autorizado; configuración no registrada; exposición de datos a un servicio externo; dependencia de una versión cambiante
Salida	Sistema de IA seleccionado y documentado, con su configuración y sus límites de uso

consecuencias. La herramienta asiste; no asume.

12.9. Fase 5: generación

La quinta fase produce el artefacto candidato con asistencia de IA. Es el momento en que la hipótesis rectora se hace visible: el asistente acelera la escritura, pero su producto es una *hipótesis de solución*, no una solución verificada. La fase debe registrar las condiciones de generación -modelo, versión, configuración, *prompt* y herramientas- para preservar la reproducibilidad.

Tabla 12.8.: Fase 5 - Generación. **Propuesta docente**.

Dimensión	Contenido
Entradas	Especificaciones funcional y de seguridad, sistema de IA seleccionado, *prompts*
Responsables	Desarrollador asistido; supervisión del equipo
Evidencias	Artefacto candidato, registro de *prompts*, condiciones de generación
Riesgos	Artefacto que no responde a la especificación; código inseguro plausible; alucinación de dependencias o de interfaces; fuga de datos en el *prompt*
Salida	Artefacto candidato con su expediente de generación

Advertencia

Código generado no es código verificado. Un artefacto que compila, se ejecuta y «parece correcto» sigue siendo una hipótesis de solución. La fluidez del texto

generado no es evidencia de correctitud ni de seguridad, y las dependencias o interfaces que el asistente propone pueden no existir o no ser las adecuadas. La generación abre la fase de revisión, no la cierra.

12.10. Fase 6: revisión

La sexta fase inspecciona el artefacto antes de validarlo. Es una revisión humana estructurada -formato, dependencias declaradas, supuestos, legibilidad, adecuación a la especificación- y detecta categorías que las herramientas automáticas no ven, como un requisito mal entendido. La revisión produce hallazgos clasificados que alimentan la validación.

Tabla 12.9.: Fase 6 - Revisión. **Propuesta docente**.

Dimensión	**Contenido**
Entradas	Artefacto candidato, especificaciones funcional y de seguridad, expediente de generación
Responsables	Revisor distinto del generador; equipo de seguridad
Evidencias	Informe de revisión con hallazgos clasificados y su justificación
Riesgos	Sesgo de confianza en la salida del asistente; revisión superficial; hallazgos sin trazabilidad a un requisito
Salida	Artefacto revisado, con hallazgos clasificados (*triaje*) y acciones decididas

Nota

La revisión es humana y trazable. El producto de esta fase no es un veredicto binario, sino un informe que asocia cada hallazgo con el requisito que lo origina. Un hallazgo sin requisito asociado no puede evaluarse, y una revisión sin registro no puede auditarse.

12.11. Fase 7: validación

La séptima fase comprueba los requisitos declarados con el pipeline de validación de la capítulo 10: análisis estático y de composición, compilación, pruebas funcionales, pruebas basadas en propiedades, *fuzzing* y *sanitizers*, según la tecnología y el riesgo. **Toda etapa que ejecute código no fiable debe ejecutarse dentro del aislamiento autorizado**, incluidas la compilación con herramientas potencialmente expuestas a entradas hostiles,

las pruebas y los analizadores dinámicos. Las comprobaciones puramente documentales o estáticas pueden realizarse fuera de ese entorno si su modelo de amenazas lo permite. Su producto es un expediente de evidencia por requisito, no un veredicto de seguridad.

Tabla 12.10.: Fase 7 - Validación. **Propuesta docente**. Las puertas y su diseño *fail-closed* se detallan en el capítulo 10.

Dimensión	**Contenido**
Entradas	Artefacto revisado, requisitos verificables, puertas de calidad declaradas
Responsables	Ingeniero de validación; automatización supervisada
Evidencias	Informes de SAST/SCA, resultados de pruebas, trazas de ejecución, matriz de trazabilidad
Riesgos	Falsos negativos de las comprobaciones aplicables; especificación incompleta; umbrales mal fijados; ejecución accidental fuera del aislamiento; falsa sensación de garantía ante un expediente verde
Salida	Expediente de evidencia que vincula requisito, implementación, prueba y resultado

Advertencia

Expediente verde $\neq$ sin vulnerabilidades. Superar todas las comprobaciones de esta fase significa que *no se ha observado ningún fallo bajo las comprobaciones ejecutadas*. No significa que el artefacto esté libre de vulnerabilidades. El expediente hereda los falsos negativos de cada técnica, la especificación puede ser incompleta y el contexto de uso puede diferir del supuesto. Este es el invariante del capítulo aplicado a la validación.

12.12. Fase 8: atestación del aislamiento

La octava fase **verifica y registra las condiciones de ejecución aislada** descritas en el capítulo 9. No debe interpretarse como una primera ejecución posterior a la validación: el aislamiento es un **requisito previo y transversal** a las actividades de la fase 7 que ejecutan código no fiable. La fase 8 registra y audita esa condición de seguridad, incluida la configuración usada durante la validación:

sin red salvo excepción justificada, con cuotas de CPU, memoria y tiempo, con sistema de ficheros efímero y con usuario no privilegiado. El aislamiento no demuestra seguridad; limita las consecuencias esperables de ejecutar código que no se ha verificado por completo.

Tabla 12.11.: Fase 8 - Atestación del aislamiento. **Propuesta docente**.

Dimensión	**Contenido**
Entradas	Artefacto revisado, política de aislamiento previa a las pruebas, configuración del sandbox, cuotas y excepciones declaradas
Responsables	Ingeniero de plataforma y de seguridad; ejecución autorizada
Evidencias	Configuración del entorno registrada, registros de ejecución, recursos consumidos
Riesgos	Fuga de recursos o de datos; *escape* del aislamiento; ejecución fuera del sandbox o sin cuotas; confundir el aislamiento con una garantía de seguridad
Salida	Evidencia de que las etapas de ejecución de las fases 7 y 8 utilizaron el aislamiento autorizado, con configuración y registros asociados

Advertencia

Ejecutar código no fiable solo en aislamiento autorizado. Los ejemplos vulnerables de este material son **material didáctico** y solo deben ejecutarse en un entorno aislado y autorizado. El aislamiento reduce el daño de un artefacto hostil; no certifica que el artefacto sea seguro ni que el aislamiento sea infranqueable.

12.13. Comprobación de las puertas y del aislamiento

La validez del proceso depende también de que sus propias puertas funcionen. Una prueba que nunca se ejecuta, que se ignora al calcular el estado final o que solo comprueba la creación de un fichero no aporta la evidencia que su nombre sugiere. Por ello, antes de aceptar un pipeline docente se propone una *prueba de rechazo*: introducir de forma controlada un artefacto con una propiedad de seguridad incumplida y verificar que la puerta correspondiente lo bloquea. Debe comprobarse asimismo que un error de infraestructura, una prueba omitida o la imposibilidad de iniciar el aislamiento **no se convierten en un aprobado**.

Nota

Ejemplo transversal. Para la exportación XLSX, una prueba debe inspeccionar las celdas producidas y comprobar que el contenido procedente de un CSV no se interpreta como fórmula en los consumidores definidos. Para las rutas, una prueba debe cubrir rutas fuera del directorio autorizado y, cuando corresponda, enlaces simbólicos. El éxito de estas pruebas queda limitado al entorno y a los

casos ensayados.

12.14. Fase 9: aceptación

La novena fase decide la promoción del artefacto con base en la evidencia, en una política de aceptación de riesgos y en una revisión humana. Es la fase que cierra el ciclo y la que distingue la metodología de una mera batería de herramientas: la decisión no se deduce de un expediente verde, sino que la *asume* una persona informada por la evidencia.

Tabla 12.12.: Fase 9 - Aceptación. **Propuesta docente**.

Dimensión	**Contenido**
Entradas	Expediente de evidencia, informe de revisión, política de bloqueo y excepciones, riesgos residuales declarados
Responsables	Responsable de aceptación con autoridad para promover o rechazar
Evidencias	Decisión registrada y firmada, con su justificación y los riesgos aceptados
Riesgos	Promoción por confianza en lugar de por evidencia; riesgos residuales no declarados; decisión no registrada; presión de plazo que erosiona las puertas
Salida	Decisión de aceptación, rechazo o devolución a corrección, registrada, con su fundamento y aprobaciones exigidas

Consejo

La responsabilidad humana es el centro de la aceptación. El expediente informa, pero no decide. Quien acepta el artefacto *asume* el riesgo residual, incluido lo que quedó sin comprobar. La formulación honesta de una aceptación es la descrita en la sección 10.9: «bajo las comprobaciones A, B y C, en el entorno descrito, no se observaron fallos; se aceptan los riesgos X e Y». Esa formulación conserva la información que una auditoría necesita.

12.15. Fase 10: mantenimiento

La décima fase sostiene el artefacto ante el cambio. Las dependencias cambian, aparecen vulnerabilidades nuevas, el contexto de uso evoluciona y la superficie de ataque crece con cada formato o función añadidos. El mantenimiento incluye actualizar dependencias,

repetir la validación cuando cambia algo relevante y conservar la trazabilidad de las decisiones.

Tabla 12.13.: Fase 10 - Mantenimiento. **Propuesta docente**.

Dimensión	Contenido
Entradas	Artefacto aceptado, inventario de componentes, catálogo de vulnerabilidades vigente
Responsables	Equipo de mantenimiento; responsable de seguridad
Evidencias	Registro de actualizaciones, validaciones repetidas, trazabilidad continuada
Riesgos	Dependencia vulnerable no detectada; regresión al actualizar; pérdida de trazabilidad; ampliación no justificada de formatos y superficie de ataque
Salida	Artefacto sostenido, con su validación renovada y su trazabilidad conservada

Nota

El mantenimiento también es seguridad. Cada formato o dependencia añadidos amplían la superficie de ataque y el coste de verificación. El equilibrio KISS/YAGNI frente a los requisitos reales no es una preferencia estética, sino una decisión de seguridad: lo que no existe no puede ser vulnerable, y lo que existe debe mantenerse verificado.

12.16. De la aceptación a la operación

Las fases anteriores conducen a la aceptación, pero el ciclo no termina ahí. El software se despliega en condiciones reales y produce evidencia que ninguna comprobación previa podía aportar. **Propuesta docente**: los mecanismos concretos dependen de la plataforma, el modelo de amenazas y la criticidad del sistema.

La **evidencia operacional** responde a una pregunta distinta de la evidencia previa al despliegue: no «¿cumple la especificación bajo condiciones probadas?», sino «¿se comporta como se esperaba con datos y dependencias reales?». Puede confirmar supuestos, revelar defectos no detectados por las pruebas, motivar una nueva verificación o exigir la suspensión del cambio. La operación no es una fase terminal: es una fuente de nueva evidencia que realimenta el proceso.

Los mecanismos de recuperación deben *probarse* periódicamente, no solo documentarse. Un *rollback* no ensayado es una hipótesis, no una capacidad verificada. El análisis de incidentes realimenta el ciclo: puede traducirse en especificaciones más precisas, invariantes

Tabla 12.14.: Capacidades operativas y límites. **Propuesta docente**.

Capacidad	**Qué permite**	**Límite a declarar**
Feature flags	Activar o desactivar funcionalidades sin volver a desplegar	Añaden estados y combinaciones no probadas
Despliegue gradual	Exponer el cambio a un subconjunto antes que al total	Reduce el alcance, no la probabilidad del defecto
Observabilidad	Detectar anomalías y regresiones durante la operación	La ausencia de señales no demuestra ausencia de defectos
Rollback / desactivación	Retirar una versión o funcionalidad problemática	Retirar no garantiza recuperar el servicio ni el estado
Restauración de estado	Recuperar datos o estado previo	Una operación compensatoria no siempre reproduce el estado exacto
Efectos externos	-	Algunos efectos son irreversibles: envío de datos a terceros, exposición de información

adicionales, modelos de amenazas ampliados o criterios de aceptación más estrictos.

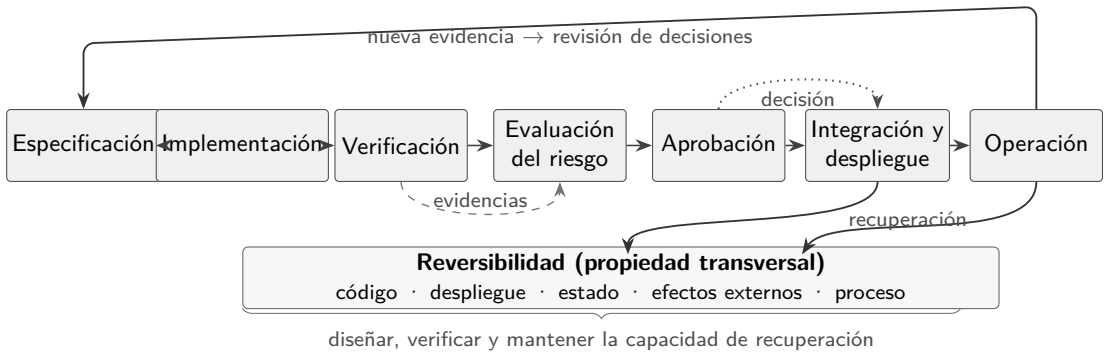


Figura 12.1.: Ciclo de ingeniería del software como abstracción: flujo de desarrollo (flechas continuas), de evidencias (punteadas), de decisiones (punteadas gruesas) y de recuperación (continuas hacia la banda), con la reversibilidad como propiedad transversal y la operación como fuente de nueva evidencia que realimenta la especificación. En procesos reales hay iteraciones y despliegues parciales; el esquema no es una secuencia rígida.

Advertencia

La operación no demuestra corrección. Que un sistema opere sin incidentes observados significa que no se han detectado fallos en las condiciones cubiertas; no que carezca de defectos. Del mismo modo que un *pipeline* verde hereda los falsos negativos de sus comprobaciones, la ausencia de incidentes hereda los puntos ciegos

de la observabilidad.

12.17. Gobernanza

Las fases anteriores describen *qué se hace*; la gobernanza describe *bajo qué reglas*. Es el conjunto de decisiones que hacen que el ciclo sea sostenible y responsable, y abarca cinco ámbitos: herramientas autorizadas, datos y confidencialidad, licencias, credenciales y responsabilidad humana. La tabla 12.15 los resume.

Tabla 12.15.: Ámbitos de gobernanza del AI-SDLC. **Propuesta docente**. Las políticas concretas deben fijarlas cada institución u organización.

Ámbito	**Decisión de gobernanza**
Herramientas autorizadas	Qué asistentes, analizadores y servicios se permite usar, en qué entornos y con qué configuraciones; qué queda prohibido
Datos y confidencialidad	Qué datos pueden introducirse en un *prompt* o enviarse a un servicio externo; clasificación de la información; minimización y anonimización
Licencias	Qué licencias tienen el código generado y las dependencias propuestas; compatibilidad con la licencia del proyecto; obligaciones de atribución
Credenciales	Cómo se gestionan claves, *tokens* y secretos; qué credenciales ve el asistente o el agente; rotación y alcance mínimo
Responsabilidad humana	Quién responde de cada decisión; revisión humana obligatoria en las puertas; registro de la decisión y de los riesgos aceptados

Nota

La gobernanza no sustituye a la técnica ni la técnica a la gobernanza. Una herramienta autorizada puede producir un artefacto inseguro, y un artefacto técnicamente correcto puede haberse producido violando confidencialidad, licencias o credenciales. Los cinco ámbitos son condiciones del proceso, no garantías del producto.

Advertencia

Datos en el *prompt* y confidencialidad. Introducir datos personales, secretos o información confidencial en un servicio externo puede constituir una fuga, incluso si el artefacto resultante es correcto. La decisión de qué puede compartirse con un sistema de IA es una decisión de gobernanza y de cumplimiento, no una decisión meramente técnica, y debe adoptarse antes de usar el sistema, no después.

Nota

Herramientas institucionalmente autorizadas. En prácticas universitarias o proyectos institucionales se emplearán únicamente los servicios y configuraciones permitidos por la política vigente de la organización. La autorización del servicio no sustituye la revisión de las categorías de datos que pueden tratarse, las condiciones contractuales, las licencias ni las medidas de seguridad aplicables.

12.18. Del método a la operación: AG-00 y P00

El kit `kit_agentes_ai_sdlc_v2/` traduce la metodología de esta sección a una secuencia operativa. `AG-00_orchestrator` coordina los roles y `P00_integral` recorre la secuencia completa: requisitos, generación, análisis estático, clean code, pruebas, seguridad, dependencias, QA, documentación, auditoría de IA y release. La secuencia es una **Propuesta docente**; no es un estándar ni una certificación.

El principio transversal es la **independencia entre generación y evaluación**: el agente que genera no puede aprobar su propio resultado, cambiar los criterios de aceptación ni modificar las pruebas protegidas. Los controles mínimos son:

1. separar permisos de escritura entre el generador y los evaluadores;
2. fijar los criterios y las pruebas de aceptación antes de generar;
3. registrar versión, comandos, herramientas, resultados y fallos;
4. bloquear la aceptación si faltan evidencias, si una prueba falla o si se alteraron los criterios sin autorización;
5. exigir una decisión final de un rol distinto del generador.

Nota

Modelos distintos no son independencia. Cambiar de modelo no sustituye la separación de permisos, la protección de las pruebas, la evidencia reproducible ni el control de cambios: la independencia es una propiedad del proceso, no del proveedor (sección 8.5).

12.19. Coste total conceptual

La asistencia de IA reduce algunos costes y desplaza otros; no los elimina. Para razonar sobre el conjunto conviene distinguir cuatro componentes del *coste total conceptual* del ciclo: el coste de **implementación**, el de **verificación**, el de **mantenimiento** y el de **riesgo**. La tabla 12.16 los describe cualitativamente.

Tabla 12.16.: Componentes del coste total conceptual del AI-SDLC. **Propuesta docente** y **Hipótesis** de razonamiento: no es una ley empírica ni una medición, y no se afirman magnitudes ni proporciones.

Componente	**Qué incluye y cómo lo afecta la IA**
Implementación	Especificar, diseñar y escribir el artefacto. La IA tiende a reducir el esfuerzo de escritura y andamiaje, pero añade el coste de formular buenos *prompts* y de seleccionar y gobernar el sistema
Verificación	Revisar, validar y construir evidencia. Tiende a *aumentar* en términos relativos: al escribir menos, se revisa y valida más, y el expediente debe ser explícito y trazable
Mantenimiento	Sostener el artefacto: actualizar dependencias, repetir validaciones, conservar trazabilidad. Crece con cada formato, dependencia o función añadidos
Riesgo	Daño esperado por fallos, fugas o incumplimientos. No desaparece: la IA puede introducir superficie nueva (dependencias inventadas, contenido no fiable, datos expuestos) y el riesgo residual debe declararse y aceptarse

Advertencia

No es una ley empírica. La tabla anterior es un marco de razonamiento (**Propuesta docente**) apoyado en una **Hipótesis** sobre el desplazamiento del esfuerzo. No afirma que la IA reduzca el coste total, ni en qué proporción, ni que los cuatro componentes sean comparables entre sí. Convertir este esquema en una predicción cuantitativa -«la IA reduce el coste en un $X\,\%$»- excedería lo que este material puede sostener: no se ha medido nada y no se ofrecen cifras.

12.20. Límites de la metodología y etiquetado

Nota

Resumen del etiquetado. Evidencia: que la revisión, la trazabilidad y la verificación de requisitos son prácticas consolidadas en los marcos citados [3, 5, 9], y que la clasificación de debilidades dispone de un catálogo como CWE [6] y de reglas de codificación segura como CERT [7]. **Propuesta docente**: las diez fases, sus entradas, responsables, evidencias y salidas; la tabla resumen; el modelo de gobernanza; y el marco de coste total conceptual. **Hipótesis**: que seguir la metodología reduzca el riesgo en un contexto dado y que la IA desplace el esfuerzo de la escritura hacia la especificación, la revisión y la validación.

Advertencia

Límites explícitos. (i) El AI-SDLC no es un estándar oficial ni una certificación. (ii) La alineación con NIST SSDF, OWASP ASVS, CWE, CERT e ISO/IEC 27034 es *conceptual*: reutiliza ideas de esos marcos, no los reproduce ni está reconocida por ellos. (iii) Superar las fases no demuestra seguridad; el invariante del capítulo se mantiene en cada fase. (iv) El coste total es un marco conceptual, no una medición. (v) Los ejemplos vulnerables del material solo se ejecutan en aislamiento autorizado. (vi) Las correspondencias concretas con controles o requisitos de cada marco deben justificarse por su identificador y versión antes de afirmar una cobertura específica.

12.21. Preguntas de reflexión

1. ¿Por qué el AI-SDLC insiste en que no es un estándar oficial? ¿Qué diferencia hay entre «alinearse conceptualmente» con NIST SSDF y «ser conforme» a él?

2. Elegid una fase y explicad qué ocurre en las fases posteriores si su salida es incompleta. Usad el caso transversal (CSV a PDF/XLSX).

3. ¿Por qué la selección del sistema de IA se define como la elección de una *tupla* y no de «un modelo»? ¿Qué evidencia se pierde si no se registra la configuración?

4. Un compañero dice: «como el pipeline está verde, puedo aceptar el artefacto sin revisarlo». ¿Qué fases del AI-SDLC está ignorando y por qué es un error?

5. ¿Por qué el coste de verificación tiende a *aumentar* en términos relativos cuando la IA escribe más código? ¿Por qué el coste de riesgo no desaparece?

6. Dad un ejemplo de decisión de gobernanza que no sea técnica y explicad por qué debe adoptarse antes de usar el sistema de IA.

7. ¿Por qué el mantenimiento es también una fase de seguridad? Relacionadlo con la superficie de ataque y con el equilibrio KISS/YAGNI.

8. ¿Qué responsabilidades humanas no se pueden delegar al sistema de IA en ninguna fase de la metodología y por qué?

9. ¿Qué inconsistencia aparece si las pruebas de la fase 7 se ejecutan fuera del aislamiento que se define en la fase 8? Proponed una secuencia operativa correcta.

10. Diseñad una prueba que demuestre que una puerta de seguridad rechaza un artefacto deliberadamente vulnerable. ¿Cómo detectaríais una prueba omitida o un fallo del sandbox?

12.22. Referencias y estado de verificación

Las afirmaciones de esta sección sobre prácticas de desarrollo seguro, verificación de requisitos y seguridad de aplicaciones se apoyan en SSDF [3], ASVS [5] e ISO/IEC 27034 [9]; la clasificación de familias de fallo remite al catálogo CWE [6] y a las reglas de codificación segura de CERT [7]. La referencia a riesgos de aplicación en el contexto web remite al Top 10 de OWASP [4]. Todos estos materiales deben consultarse en su edición vigente.

Nota

Alcance de las referencias. NIST SP 800-218 (SSDF) proporciona prácticas de desarrollo seguro; OWASP ASVS define requisitos de verificación de seguridad para aplicaciones web; CWE clasifica debilidades; CERT reúne reglas y recomendaciones de codificación segura; e ISO/IEC 27034 trata la seguridad de aplicaciones desde una perspectiva de procesos y gestión. Ninguna de estas fuentes certifica la metodología AI-SDLC aquí propuesta ni valida automáticamente un pipeline concreto. Las capacidades de cada herramienta, las versiones y las correspondencias entre controles deben verificarse para cada implantación. Fuentes de consulta: NIST SP~800-218, OWASP ASVS, CWE y SEI CERT C Coding Standard.

13. Conclusiones

Este capítulo ha recorrido un caso transversal modesto -convertir un CSV en PDF o XLSX con validación previa, presentado en el Anexo A- a lo largo de las secciones de contenido (de la introducción a las conclusiones) y tres prácticas de laboratorio y una actividad complementaria opcional. El recorrido permite ahora recapitular la hipótesis rectora, fijar las ideas clave de cada bloque, comprobar la cobertura de los objetivos de aprendizaje y declarar con honestidad lo que queda abierto.

13.1. La hipótesis rectora, a la vista del recorrido

La hipótesis rectora del capítulo se mantiene como enunciado de trabajo: la IA generativa **reduce ciertos costes de implementación** -escritura inicial de código, andamiaje, traducción entre lenguajes, generación de pruebas- **sin eliminar** los problemas de corrección, seguridad, mantenimiento, portabilidad y responsabilidad. No se ha asumido que la IA mejore ni empeore la seguridad *per se*; lo que se ha descrito es un *desplazamiento* del esfuerzo humano desde la escritura hacia la especificación, la revisión y la validación, junto con la aparición de superficie de ataque nueva en la cadena de herramientas.

Evidencia. Que la revisión, la trazabilidad y la verificación de requisitos son prácticas consolidadas de desarrollo seguro; que existen catálogos de debilidades (CWE [6]) y reglas de codificación segura (CERT [7, 8]); que el análisis estático, el análisis de composición y las pruebas tienen límites estructurales; y que un contenedor ordinario comparte el núcleo con el anfitrión. Todo ello se apoya en los marcos citados [3, 5, 9] y en la semántica de los lenguajes.

Hipótesis. Que la IA desplace el esfuerzo desde la escritura hacia la especificación, la revisión y la validación; que completar la especificación mejore el artefacto obtenido; y que seguir una metodología ordenada reduzca el riesgo en un contexto dado. Ninguna de estas afirmaciones se ha medido aquí y no se presentan como hechos probados.

Propuesta docente. La metodología AI-SDLC, las fases y sus puertas, las rúbricas de las prácticas, las correcciones contextuales propuestas y el marco de coste total conceptual son construcciones pedagógicas, no resultados científicos ni normas.

Nota

La asimetría de la hipótesis es deliberada. Reconocer una ganancia de productividad no equivale a afirmar una ganancia de seguridad. La herramienta abarata escribir; no adivina la especificación, no decide qué es aceptable y no asume la responsabilidad del artefacto entregado. Esa responsabilidad sigue siendo humana.

13.2. Ideas clave por bloque

Anexo A. Caso de estudio transversal. El conversor CSV-PDF-XLSX es un ejemplo docente que se presenta una sola vez -contexto, alcance, requisitos no funcionales, arquitectura de referencia, modelo de datos, invariantes, modelo de amenazas, ciclo de vida, reversibilidad y comparación humano-IA- para que los capítulos posteriores lo utilicen sin repetir su descripción. La conversión no es reversible; la reversibilidad del caso se refiere al proceso, no a los datos.

Capítulo 2. Principios clásicos. Las heurísticas de diseño (KISS, DRY, YAGNI, entre otras) siguen vigentes y la IA no las sustituye; una heurística admite excepciones y no es una ley. La IA cambia el modo de aplicarlas, no su validez.

Capítulo 3. Correctitud y seguridad. Conviene distinguir varias nociones de «correcto» (sintaxis, compilación, funcionalidad, correctitud relativa a especificación, robustez, seguridad, portabilidad y mantenibilidad) que no se implican entre sí. Precondiciones, postcondiciones e invariantes son el contrato que permite juzgar el artefacto, y superar pruebas funcionales no lo convierte en seguro.

Capítulo 4. Vulnerabilidades clásicas. Las familias de fallo se reconocen por su identificador CWE: memoria y enteros, validación, rutas e inyección, deserialización, estado y recursos, secretos y dependencias. Cada ejemplo vulnerable tiene una corrección *contextual*, válida para un modelo de amenaza declarado y nunca universal.

Capítulo 5. Portabilidad y plataformas. Un artefacto puede ser correcto en un entorno y fallar en otro: separadores y canonicidad de rutas, permisos, temporales, fin de línea, Unicode y comportamiento definido por la implementación. La portabilidad

es una propiedad distinta de la correctitud y de la seguridad, y no se hereda automáticamente.

Capítulo 6. Evaluación reproducible. Comparar sistemas de IA exige controlar la tupla de modelo, versión, configuración, *prompt*, herramientas y entorno, registrar las condiciones y tratar las métricas como *proxies* con amenazas a la validez declaradas. El precio de un servicio no determina su seguridad.

Capítulo 8. Confianza no calibrada. El código generado es *plausiblemente incorrecto*: supera las comprobaciones superficiales sin que eso implique corrección. El riesgo es epistémico -no saber cuánto confiar- y se mitiga con especificación previa, verificación con independencia de contexto y calibración explícita de la confianza. La hipótesis rectora se etiqueta como tal, no como resultado medido.

Capítulo 7. AI Act y espacios controlados. El uso de IA para escribir software no convierte por sí mismo el producto en un sistema de IA regulado: lo determinante es la naturaleza del artefacto y su uso previsto. Se distinguen el *sandbox* regulatorio, el aislamiento técnico y el *pipeline* de validación; no se atribuyen obligaciones inexistentes al Reglamento (UE) 2024/1689 [1].

Capítulo 9. Aislamiento técnico y agentes. El riesgo del *código producido* y el riesgo de las *acciones del agente* durante el desarrollo son objetos distintos. Se modelan las amenazas (comandos, red, paquetes, secretos, *prompt injection* [16, 17, 18], cadena de suministro) y se proponen límites de confianza; un contenedor ordinario no es, por sí solo, un límite de seguridad.

Capítulo 10. Validación automática. Un *pipeline* con puertas *fail-closed* y trazabilidad requisito → implementación → prueba → evidencia convierte un artefacto en un expediente verificable. Hereda los falsos negativos de cada técnica: un *pipeline* verde no demuestra seguridad. La integración de cambios concurrentes añade la validación del estado que realmente se integra, no solo del cambio aislado.

Capítulo 11. Verificación de código generado. Los contratos, invariantes y modelos de amenazas convierten los requisitos en propiedades comprobables. El caso transversal (`checked_add_u32`) se verifica con pruebas, propiedades, fuzzing, sanitizers, análisis estático y verificación formal: 94/94 obligaciones demostradas con Frama-C/WP. La ejecución satisfactoria de pruebas no se presenta como corrección general; las diferencias humano-IA se tratan con un protocolo neutral.

Capítulo 12. Metodología AI-SDLC. El ciclo de vida docente AI-SDLC ordena el desarrollo asistido por IA en diez fases, con gobernanza en cinco ámbitos y un marco de coste total conceptual. Se extiende hacia la *operación*: despliegue controlado, observabilidad, recuperación probada, gestión de incidentes y realimentación al proceso, con la reversibilidad (del código, del despliegue, del estado y de los efectos externos) como capacidad transversal. No es un estándar oficial ni una certificación: es una propuesta docente cuya alineación con NIST, OWASP, ISO/IEC, CWE y CERT es conceptual.

Anexo B. Prácticas de laboratorio. Las tres prácticas recorren el caso transversal; la actividad complementaria opcional (práctica 04) aplica el kit AI-SDLC desde el fallo, la especificación y la validación, en entornos aislados y con datos sintéticos. Ninguna corrección que se enseña es «universalmente segura».

13.3. Trazabilidad objetivos de aprendizaje ↔ secciones

La tabla 13.1 relaciona cada objetivo de aprendizaje con las secciones que lo cubren. Es una tabla de **Propuesta docente** y se comprueba en la auditoría final del capítulo.

13.4. Ideas abiertas y limitaciones

Este material no cierra todas las cuestiones que abre. Se distinguen a continuación las cuestiones normativas contrastadas para esta edición y las limitaciones metodológicas o técnicas que siguen abiertas:

1. **Vigencia y alcance del AI Act.** La referencia normativa de esta edición es el Reglamento (UE) 2024/1689 [1], según su texto consolidado de 27 de julio de 2026, que incorpora el Reglamento (UE) 2026/1744 [2]. Se han revisado los artículos 3, 57-60 bis y 113 en esa edición, y se han comprobado contra el DOUE/EUR-Lex (consulta de 2026-11-07). La aplicación temporal es escalonada y debe determinarse por disposición y tipo de sistema, no por una fecha única. En particular, el artículo 113 distingue los plazos del 2 de diciembre de 2027 (anexo III) y del 2 de agosto de 2028 (anexo I), con las salvedades establecidas en el propio Reglamento. La consolidación es un instrumento documental: los actos auténticos son los publicados en el DOUE. Toda aplicación a un producto concreto requiere comprobar modificaciones posteriores, el uso previsto y las circunstancias del caso. Este capítulo no constituye asesoramiento jurídico.

Tabla 13.1.: Trazabilidad entre los objetivos de aprendizaje y las secciones del capítulo. **Propuesta docente**.

Objetivo de aprendizaje	**Secciones que lo cubren**
Distinguir código funcional de código correcto, robusto, seguro, portable y mantenible	3, 4, 5
Formular precondiciones, postcondiciones e invariantes	3
Identificar familias de vulnerabilidad por CWE	4
Explicar por qué las pruebas finitas no demuestran seguridad	3, 10, B
Diseñar un protocolo reproducible que controle versión, configuración, *prompt*, herramientas y entorno	6
Reconocer el riesgo epistémico del código plausiblemente incorrecto y calibrar la confianza	8, 11
Verificar código mediante contratos, invariantes, pruebas y verificación formal	11, B
Diferenciar un sistema de IA regulado de software convencional	7
Distinguir *sandbox* regulatorio, aislamiento técnico y *pipeline* de validación	7, 9, 10
Modelar amenazas de las acciones de un agente y proponer límites de confianza	9, B
Diseñar un *pipeline* de validación con puertas y trazabilidad	10, 12, B
Distinguir verificación, aprobación, autorización de despliegue y aceptación del riesgo residual	11, 12
Integrar cambios concurrentes y validar el estado que realmente se integra	10
Desplegar de forma controlada y observar el sistema en operación	12
Tratar la reversibilidad como propiedad transversal y verificar la recuperación	8, 12
Aplicar una metodología docente AI-SDLC y reconocer sus límites	12

2. **Verificación de los CWE.** Los 27 identificadores CWE empleados en el texto y en los ejemplos se contrastaron con las fichas de MITRE (consulta 2026-10-24) y coinciden con sus títulos. Debe repetirse la comprobación si se añaden identificadores [6].

3. **Sin mediciones.** El capítulo describe protocolos, plantillas y marcos de razonamiento; no presenta resultados, cifras ni comparaciones entre sistemas reales. La hipótesis rectora no se ha medido y no se ofrecen magnitudes ni proporciones.

4. **Dependencia de versión y configuración.** Las capacidades y limitaciones precisas de los sistemas de IA y de las herramientas (Bandit, `pip-audit`, `pytest`, `bubblewrap`, entre otras) dependen de versión y configuración y quedan **A VERIFICAR**.

5. **Verificación de la literatura.** La evaluación de asistentes de código y el estudio del *prompt injection* se apoyan en las fuentes citadas [12, 13, 16, 17, 18]; su contraste externo y sus cifras deben verificarse contra las publicaciones originales.

6. **Aislamiento no verificado empíricamente.** Los escenarios de agentes y de *sandbox* se describen a nivel defensivo y conceptual; no se han ejecutado sobre un proyecto real ni se garantiza la suficiencia de una configuración concreta.

7. **Corrección contextual.** Ninguna corrección propuesta en el capítulo es «universalmente segura»: cada una es válida para un modelo de amenaza y unas precondiciones declaradas.

8. **Compilación y auditoría final.** La compilación del documento completo y la revisión de trazabilidad corresponden a la auditoría final del libro.

Nota

Etiquetado epistémico de cierre. Evidencia: las prácticas consolidadas de revisión, trazabilidad y verificación de requisitos, la existencia de catálogos de debilidades y reglas de codificación segura, y los límites estructurales de las técnicas de análisis. **Hipótesis**: que la IA desplace el esfuerzo hacia la especificación, la revisión y la validación, y que completar la especificación o configurar un *pipeline* mejore el resultado en un contexto dado. **Propuesta docente**: la metodología AI-SDLC, las prácticas, las rúbricas, las correcciones y el marco de coste total conceptual.

Advertencia

Cautela invariante del capítulo. Superar una batería finita de pruebas no demuestra seguridad. Un ejemplo «corregido» es correcto de forma *contextual*, nunca «universalmente seguro». El aislamiento reduce el daño de un artefacto hostil; no certifica que sea seguro ni que el aislamiento sea infranqueable. Los ejemplos vulnerables de este material son **material didáctico** y solo deben ejecutarse en entornos aislados y autorizados.

Bibliografía y fuentes

[1] Parlamento Europeo y Consejo de la Unión Europea. *Reglamento (UE) 2024/1689, de 13 de junio de 2024, por el que se establecen normas armonizadas en materia de inteligencia artificial*. DOUE, L 2024/1689, 12 de julio de 2024. Versión consolidada de 27 de julio de 2026 (instrumento documental): https://eur-lex.europa.eu/legal-content/ES/TXT/?uri=CELEX:02024R1689-20260727.

[2] Parlamento Europeo y Consejo de la Unión Europea. *Reglamento (UE) 2026/1744, de 8 de julio de 2026, por el que se modifica, entre otros, el Reglamento (UE) 2024/1689*. DOUE, L 2026/1744, 24 de julio de 2026. https://eur-lex.europa.eu/eli/reg/2026/1744/oj.

[3] National Institute of Standards and Technology. *Secure Software Development Framework (SSDF) Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities*. NIST SP 800-218, 2022. Edición citada: 1.1 (2022). https://doi.org/10.6028/NIST.SP.800-218.

[4] OWASP Foundation. *OWASP Top 10: The Ten Most Critical Web Application Security Risks*. Edición citada: 2025. https://top10.owasp.org/2025/.

[5] OWASP Foundation. *Application Security Verification Standard (ASVS)*. Versión estable citada: 5.0.0 (2025). https://asvs.owasp.org/downloads/.

[6] MITRE Corporation. *Common Weakness Enumeration (CWE)*. Catálogo de referencia: https://cwe.mitre.org/. Los identificadores deben justificarse para cada caso.

[7] Software Engineering Institute, Carnegie Mellon University. *SEI CERT C Coding Standard*. Referencia: https://cmu-sei.github.io/secure-coding-standards/sei-cert-c-coding-standard. Las reglas aplicables dependen del lenguaje y de la edición. Edición citada: 2016 Edition (basada en C11); las páginas de `cmu-sei.github.io` son documentación en desarrollo, no la edición publicada.

[8] Software Engineering Institute, Carnegie Mellon University. *SEI CERT Oracle Coding Standard for Java*. Referencia: https://cmu-sei.github.io/secure-coding-standards/sei-cert-oracle-coding-standard-for-java/. Las reglas aplicables dependen de la edición. Ediciones citadas: *The CERT Oracle Secure Coding Standard for Java* (Java SE 6/7) y *Java Coding Guidelines* (75 recomendaciones); la web es documentación en desarrollo.

[9] International Organization for Standardization / International Electrotechnical Commission. *ISO/IEC 27034 — Seguridad de la información: Seguridad de las aplicaciones*. Familia de normas sobre seguridad de aplicaciones; debe citarse la parte y edición específicas al aplicar requisitos.

[10] IEEE / The Open Group. *IEEE Std 1003.1-2017 (POSIX.1-2017): Standard for Information Technology-Portable Operating System Interface (POSIX)*. Edición citada: POSIX.1-2017. https://pubs.opengroup.org/onlinepubs/9699919799/.

[11] Microsoft. *Windows API documentation (Win32/Win64)*. Consulta: 2026. https://learn.microsoft.com/windows/win32/api/.

[12] Pearce, H.; Ahmad, B.; Tan, B.; Dolan-Gavitt, B.; y Karri, R. *Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions*. IEEE Symposium on Security and Privacy (SP), 2022. https://doi.org/10.1109/SP46214.2022.9833571. Estudio experimental de sugerencias de código y debilidades CWE; sus resultados no se extrapolan automáticamente a modelos o versiones posteriores.

[13] Perry, N.; Srivastava, M.; Kumar, D.; y Boneh, D. *Do Users Write More Insecure Code with AI Assistants?* ACM SIGSAC Conference on Computer and Communications Security (CCS), pp. 2785-2799, 2023. https://doi.org/10.1145/3576915.3623157. Estudio con participantes y tareas de seguridad; no constituye una estimación universal del efecto de los asistentes.

[14] Jimenez, C. E.; Yang, J.; Wettig, A.; Yao, S.; Pei, K.; Press, O.; y Narasimhan, K. *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* International Conference on Learning Representations (ICLR), 2024. https://proceedings.iclr.cc/paper_files/paper/2024/hash/edac78c3e300629acfe6cbe9ca88fb84-Abstract-Conference.html. Referencia sobre evaluación de resolución de incidencias reales; superar sus pruebas no demuestra por sí mismo ausencia de vulnerabilidades.

[15] Becker, J.; Rush, N.; Barnes, B.; y Rein, D. *Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity*. Model Evaluation & Threat Research (METR), informe experimental, 10 de julio de 2025. Prepublicación arXiv:2507.09089. Actualización de febrero de 2026. https://arxiv.org/abs/2507.09089. Estudio contextualizado (RCT con desarrolladores experimentados de 2025); no debe presentarse como efecto estable de los asistentes actuales.

[16] Greshake, K.; Abdelnabi, S.; Mishra, S.; Endres, C.; Holz, T.; y Fritz, M. *Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection*. Prepublicación arXiv:2302.12173, 2023. https://arxiv.org/abs/2302.12173.

[17] Debenedetti, E.; Zhang, J.; Balunović, M.; Beurer-Kellner, L.; Fischer, M.; y Tramèr, F. *AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents*. Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track, 2024. https://proceedings.nips.cc/paper_files/paper/2024/hash/97091a5177d8dc64b1da8bf3e1f6fb54-Abstract-Datasets_and_Benchmarks_Track.html.

[18] OWASP GenAI Security Project. *OWASP Top 10 for LLM Applications 2025*. Edición de referencia: 2025. https://genai.owasp.org/resource/owasp-top-10-for-llm-applications-2025/. No confundir con OWASP Top 10 para aplicaciones web.

[19] Osmani, A.; Lunt, M. W.; y Wang, Q. *How the Claude Code team does code review*. claude.dev Blog, 15 de octubre de 2026. https://claude.dev/blog/solving-code-review-at-anthropic/. Testimonio del equipo Claude Code sobre su proceso de revisión; no es evidencia experimental independiente.

[20] HNipps. *What tools and techniques are you using to verify AI-generated code before it hits production?*. Reddit, r/ExperiencedDevs. https://www.reddit.com/r/ExperiencedDevs/comments/1rzq738/what_tools_and_techniques_are_you_using_to_verify/. Experiencia individual; no es evidencia experimental.

[21] OWASP. *Secure Coding with AI Cheat Sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Secure_Coding_with_AI_Cheat_Sheet.html. Consultado en noviembre de 2026.

[22] OWASP. *AI Agent and MCP Security*. OWASP DevSecOps Guideline. https://owasp.github.io/DevSecOpsGuideline/2-Process/2-2-Develop/2-2-5-AI-Agent-and-MCP-Security/.

[23] OWASP SAMM. *Guidance for AI-Assisted Development*. https://owaspsamm.org/docs/guides/ai-assisted-development/. Consultado en noviembre de 2026.

[24] GitHub. *Spec Kit: toolkit for Specification-Driven Development (SDD)*. Repositorio y documento de metodología `spec-driven.md`. https://github.com/github/spec-kit. Consultado en noviembre de 2026.

[25] Moure, Brais (MoureDev). *Hello SDD: curso de Spec-Driven Development (SDD) desde cero*. Repositorio con plantillas de especificación (requisitos en notación EARS) y flujo de trabajo SDD. https://github.com/mouredev/hello-sdd. Consultado en noviembre de 2026.

[26] Bonilla, David. *¿Siguen vigentes las leyes elementales de ingeniería?* La Bonilista, número 810, 18 de octubre de 2026. https://www.bonilista.com/. Número y fecha según ejemplar facilitado por el autor; no verificables en el archivo público de La Bonilista (que alcanza 2024), por lo que no se ofrece un enlace directo al artículo. Artículo de opinión y divulgación sobre la vigencia de las heurísticas clásicas (KISS, DRY, YAGNI, ley de Kernighan, ley de Brooks, Pareto, número de Dunbar) ante la reducción del coste de escribir código con IA. Inspiró el enfoque del capítulo sobre principios clásicos; se cita como fuente de ideas, no como evidencia experimental.

[27] Endor Labs. *What is the AI-native SDLC? AI SDLC, AI-DLC, and ADLC explained*. Guía técnica (Learn), publicada el 22 de octubre de 2026 y actualizada el 27 de octubre de 2026. https://www.endorlabs.com/learn/what-is-the-ai-native-sdlc. Documentación técnica de un proveedor que distingue el SDLC asistido por IA del SDLC nativo de IA y del *Agentic Development Lifecycle* (ADLC), y advierte de que la terminología «sigue asentándose» y no tiene una definición universal. Se cita como documentación técnica, no como literatura científica revisada.

[28] Stryker, Cole (IBM Think). *La IA en el SDLC*. IBM, publicado el 14 de abril de 2026. https://www.ibm.com/es-es/think/topics/ai-in-sdlc. Artículo técnico-divulgativo que describe la integración de herramientas de IA en cada fase del SDLC tradicional y sus desafíos (sistemas probabilísticos, código plausiblemente incorrecto, necesidad de supervisión humana). Se cita como documentación técnica y divulgación, no como evidencia experimental.

A. Caso de estudio transversal: ingeniería de un sistema de conversión documental

Este capítulo utiliza un mismo sistema a lo largo de sus secciones: un *conversor documental* que lee un fichero **CSV**, valida y normaliza sus datos y genera dos salidas, un documento **PDF** y una hoja de cálculo **XLSX**. La sección 1.5 lo presentó como hilo conductor; esta sección lo describe una sola vez y con la profundidad suficiente para que las secciones posteriores puedan apoyarse en él sin repetirlo. El sistema no tiene un nombre propio en el libro: se le llama «caso transversal» o «conversor CSV-PDF-XLSX». Se conserva esa denominación.

Nota

Etiquetado epistémico del caso. El conversor es un **ejemplo docente**: un sistema modesto diseñado para exponer, sin adornos, los problemas de ingeniería que el capítulo estudia. No es un sistema industrial real y no se presentan resultados de rendimiento, de disponibilidad ni de seguridad medidos sobre él. Los fragmentos de código que aparecen en el capítulo son *ilustrativos y contextuales*; no constituyen una implementación completa y verificada del conversor. Las partes que no están desarrolladas con código se marcan explícitamente como propuestas.

A.1. Contexto y motivación

El procesamiento de datos estructurados es una tarea habitual: un informe se recibe en formato tabular, se comprueba y se transforma en un documento legible o en una hoja de cálculo manipulable. La aparente sencillez es engañosa. Leer un CSV implica decidir cómo se interpretan comillas, separadores, fin de línea y codificación; validar implica distinguir una entrada malformada de una entrada hostil; generar un PDF o un XLSX implica incorporar bibliotecas externas con sus propias superficies de ataque; y escribir ficheros implica decidir dónde y con qué permisos. Ninguna de esas decisiones es neutral para la seguridad.

El caso resulta apropiado para estudiar principios de ingeniería del software por tres razones. Primera, cabe en el enunciado de una práctica y se entiende sin contexto de dominio. Segunda, obliga a tratar a la vez *entrada no confiable*, *transformación de datos* y *salida a disco*, que son las tres fronteras donde se concentran las familias de fallo del capítulo. Tercera, permite comparar, sobre el mismo enunciado, código vulnerable, código corregido y código generado por IA, de modo que las diferencias se atribuyan a decisiones de diseño y no a sistemas distintos.

A.2. Alcance funcional

Los requisitos del conversor se reconstruyen a partir de lo que el capítulo describe. Se distinguen tres categorías: requisitos confirmados (enunciados explícitamente en el texto), requisitos propuestos (coherentes con el caso pero no desarrollados con código) y funcionalidades fuera de alcance (que el caso deliberadamente no aborda).

Tabla A.1.: Alcance funcional del caso transversal. **Propuesta docente**: la clasificación refleja lo que el capítulo desarrolla y lo que solo se enuncia.

Categoría	**Funcionalidad**	**Estado en el capítulo**
Confirmado	Leer un fichero CSV y separar sus columnas	Descrito; ejemplos de lectura en los capítulos 3 y 4
Confirmado	Validar la entrada antes de procesarla	Descrito; ejemplo de validación en el capítulo 4
Confirmado	Normalizar columnas y codificación	Descrito; decisiones de portabilidad en el capítulo 5
Confirmado	Generar un documento PDF	Descrito; la generación se cita como paso del flujo
Confirmado	Exportar a XLSX	Descrito; el escapado de fórmulas se trata en el capítulo 4
Confirmado	Gestionar errores de entrada y de escritura	Descrito; retornos comprobados en el capítulo 4
Propuesto	Configuración explícita (límites, rutas, codificación)	Enunciado como parte de la arquitectura; no desarrollado con código
Propuesto	Registro de operaciones y observabilidad	Enunciado; no desarrollado con código
Fuera de alcance	Edición colaborativa, servicio en red, interfaz gráfica	No forma parte del caso
Fuera de alcance	Conversión inversa (PDF/XLSX → CSV)	No se contempla: la conversión no es reversible

Conviene subrayar que el capítulo *no* presenta una implementación completa del conversor. Los ejemplos de los capítulos 4 y 5 son fragmentos que ilustran un fallo o una corrección

concretos; el anexo B propone actividades para construir una versión acotada. Por tanto, el alcance funcional descrito aquí es el *modelo* del sistema, no un producto terminado.

A.3. Requisitos no funcionales

Los requisitos no funcionales del caso se enuncian como criterios verificables, no como valores medidos. Cuando el capítulo no fija una cifra, aquí no se inventa: se indica la propiedad que debe cumplirse y la sección donde se desarrolla su comprobación.

Tabla A.2.: Requisitos no funcionales del caso transversal y su tratamiento en el capítulo. **Propuesta docente**: los criterios son verificables, pero no se presentan como resultados medidos.

Requisito	**Criterio verificable**	**Sección**
Seguridad	La salida no permite escribir fuera del directorio autorizado ni generar celdas interpretables como fórmulas	4, 11
Robustez	La entrada malformada se rechaza con un error explícito; el proceso no termina de forma imprevista	3, 4
Rendimiento	El proceso termina en un tiempo acotado y con memoria acotada, incluso ante entradas de tamaño máximo	11 (invariante de acotación)
Mantenibilidad	Las decisiones de diseño son explícitas y la complejidad se limita a lo necesario	2 (KISS, YAGNI)
Portabilidad	El comportamiento es el declarado en cada plataforma; las diferencias se documentan	5
Trazabilidad	Cada requisito se vincula a una comprobación y a una evidencia	10
Testabilidad	Las propiedades críticas son expresables como pruebas o invariantes	11
Recuperabilidad	Tras un fallo es posible restaurar el estado anterior de las salidas	12
Reversibilidad	Las modificaciones del código, del despliegue y del estado pueden revertirse	12

La distinción entre *requisito* y *resultado* es deliberada: un requisito de rendimiento acotado no dice cuál es el tiempo real, y un requisito de seguridad no dice que el sistema sea seguro. El capítulo mantiene esa cautela en todas las secciones.

A.4. Arquitectura de referencia

La arquitectura que el capítulo describe es una cadena de etapas: leer el CSV, validarlo, normalizarlo y generar las dos salidas, escribiéndolas en un directorio autorizado. La figura A.1 la representa. Se distinguen los componentes que el capítulo desarrolla con ejemplos de los que solo se proponen.

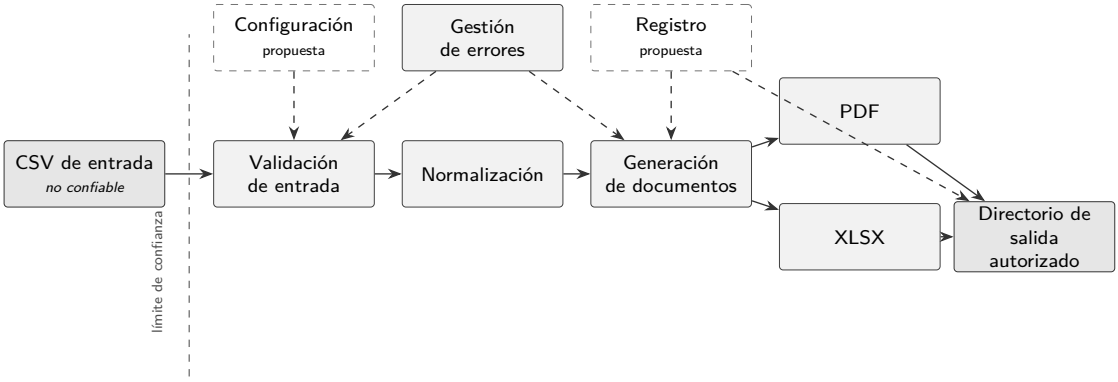


Figura A.1.: Arquitectura de referencia del caso transversal CSV–PDF–XLSX. Los componentes de trazo continuo son los descritos con ejemplos en el capítulo (lectura y validación del CSV, normalización y generación de documentos). Los de trazo discontinuo —configuración y registro— se proponen como parte de la arquitectura de referencia, pero el capítulo no los desarrolla con código; se marcan así para no presentarlos como implementados. El CSV de entrada se trata como dato no confiable y la escritura se confina al directorio de salida autorizado.

La arquitectura tiene dos rasgos que conviene fijar desde el principio. El primero es la **frontera de confianza**: el CSV de entrada se trata como dato no confiable y toda la cadena posterior debe asumir que puede ser hostil. El segundo es el **confinamiento de salida**: la escritura se limita al directorio autorizado y no sigue enlaces que permitan escapar de él. Estos dos rasgos se repiten en las secciones de vulnerabilidades, portabilidad, verificación y validación.

La arquitectura *implementada* en los ejemplos del capítulo se reduce a fragmentos: la lectura y la validación aparecen en el capítulo 4; la escritura portable, en el capítulo 5; la verificación de contratos e invariantes, en el capítulo 11. La arquitectura *propuesta* añade configuración explícita y registro de operaciones, que el capítulo enuncia pero no desarrolla con código.

A.5. Modelo de datos

El caso maneja tres representaciones de los datos, y la conversión entre ellas no conserva necesariamente toda la información.

Entrada (CSV). Texto tabular con un separador declarado, comillas y fin de línea que deben interpretarse de forma explícita. La codificación se declara (UTF-8 en el caso) y los valores se tratan como cadenas hasta que se validan.

Representación intermedia. Filas y columnas validadas y normalizadas, con los tipos decididos por la especificación y los errores de conversión tratados de forma explícita. El capítulo no fija una estructura de datos concreta; la normalización se describe como paso, no como tipo.

Salida (PDF y XLSX). El PDF es un formato de presentación; el XLSX, un formato de hoja de cálculo que interpreta ciertos prefijos como fórmulas. Ambos son salidas *terminales*: el caso no contempla la conversión inversa.

Las diferencias entre formatos son una fuente de fallos: un valor que en el CSV es una cadena puede convertirse, al exportarse a XLSX, en una fórmula que la hoja de cálculo evalúa. El capítulo trata esa diferencia como un punto de fallo concreto (capítulo 4) y no como un detalle de formato. También conviene recordar que la conversión no es reversible: el PDF no conserva necesariamente la estructura tabular original, y el XLSX puede reinterpretar los valores. Por eso la reversibilidad del caso se refiere al *proceso* (poder restaurar las salidas anteriores), no a la recuperación de los datos de entrada a partir de la salida.

A.6. Contratos e invariantes

El capítulo enuncia varias propiedades verificables del sistema. Se distinguen las que se especifican de las que solo se proponen.

Invariantes especificadas. La capítulo 11 enuncia tres invariantes de seguridad para el conversor: *contención de escritura* (ninguna operación escribe fuera del directorio autorizado), *ausencia de fórmulas en celdas* (ninguna celda del XLSX contiene contenido interpretable como fórmula) y *acotación de recursos* (el proceso termina en tiempo y memoria acotados). Son especificaciones, no resultados: el capítulo no afirma que estén verificadas sobre una implementación completa.

Contratos de entrada. La capítulo 3 formula el contrato del conversor en términos de precondiciones (qué se acepta como CSV válido), poscondiciones (qué debe cumplir la salida) e invariantes (qué debe mantenerse durante el proceso).

Propiedades propuestas. La preservación de determinados datos de entrada, la consistencia del número y orden de columnas y la ausencia de modificaciones parciales no autorizadas se proponen como criterios de aceptación del caso; el capítulo las enuncia como candidatas, no como propiedades demostradas.

La regla que el capítulo extrae de este caso es que las invariantes deben especificarse y verificarse con independencia del código producido, y que un enunciado vago («no confiar en el CSV») no es una invariante verificable hasta que se concreta en términos observables.

A.7. Modelo de amenazas del caso

Esta sección solo introduce el contexto; el desarrollo del modelo de amenazas de las acciones del agente y de los límites de confianza corresponde a la capítulo 9. Para el conversor, las superficies de ataque principales son:

- **Archivos no confiables**: un CSV malformado o diseñado para provocar expansiones o consumos excesivos.
- **Contenido malformado**: comillas, separadores y codificaciones ambiguas que distintos analizadores interpretan de forma distinta.
- **Consumo excesivo de recursos**: archivos de tamaño máximo o estructuras comprimidas con ratio extremo.
- **Rutas y nombres de archivo**: secuencias de *path traversal*, rutas absolutas y enlaces simbólicos que permiten escribir fuera del directorio autorizado.
- **Dependencias externas**: las bibliotecas de PDF y de XLSX, con sus versiones y vulnerabilidades.
- **Fórmulas en hojas de cálculo**: campos del CSV que comienzan por `=`, `+`, `-` o `@` y que la hoja de cálculo interpreta como fórmulas.
- **Información sensible**: datos personales o secretos que el CSV pueda contener y que la salida pudiera exponer.

A.8. Ciclo de vida del caso

El mismo conversor permite ilustrar las fases de un ciclo de ingeniería. La figura A.2 las relaciona con las secciones del capítulo. No todas las fases están implementadas con código: algunas aparecen solo como escenarios didácticos.

Especificación. Capítulo 3: qué significa que el conversor sea correcto y qué contrato debe cumplir.

Diseño. Capítulo 2: qué principios guían el alcance y las decisiones de diseño.

Implementación. Capítulo 4: qué familias de fallo aparecen y cómo se corrigen contextualmente.

Revisión. Capítulo 8: por qué la revisión debe ser independiente del origen del código.

Verificación. Capítulos 10 y 11: cómo se convierte la especificación en evidencia verificable.

Aprobación. Capítulo 12: qué decisión se toma a partir de la evidencia y quién la asume.

Integración. Capítulo 10: cómo se valida el estado que realmente se integra.

Despliegue y operación. Capítulo 12: despliegue controlado, observabilidad y gestión de incidentes.

Recuperación. Capítulo 12: restauración de un estado operativo aceptable.

Estas fases se presentan como un *ciclo*, no como una secuencia lineal: la evidencia obtenida en la operación realimenta la revisión de requisitos, y la reversibilidad permite volver a un estado anterior cuando una decisión resulta incorrecta. La sección 12.2 compara, sobre este mismo caso, el desarrollo guiado por instrucciones directas, el guiado por especificaciones y el ciclo de ingeniería completo.

A.9. Reversibilidad del caso

La reversibilidad se introduce aquí como propiedad transversal del proceso; su marco conceptual se desarrolla en la sección 8.3 y su tratamiento operativo en el capítulo 12. Conviene distinguir cinco operaciones que a veces se confunden bajo el mismo término:

- **Revertir una modificación del código**: deshacer un cambio en el fuente.

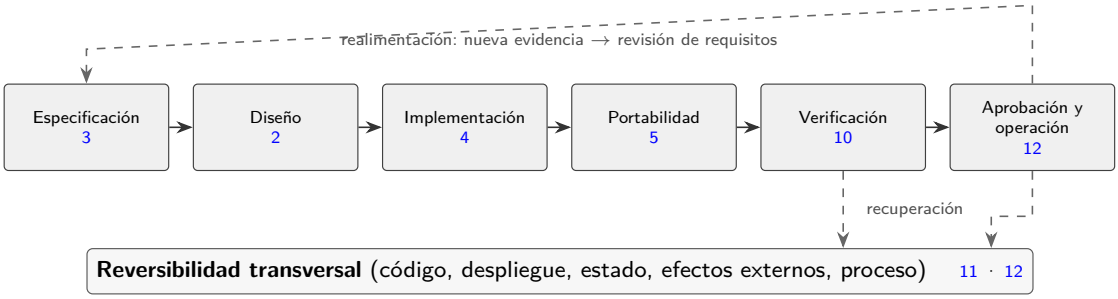


Figura A.2.: Evolución del caso transversal CSV–PDF–XLSX a lo largo del capítulo. Cada fase del ciclo de ingeniería se ilustra en una sección concreta (indicada en cada recuadro): la especificación en la sección 3, el diseño y el alcance en la sección 2, la implementación y las vulnerabilidades en la sección 4, la portabilidad en la sección 5, la verificación y la validación en la sección 10, y la aprobación y la operación en la sección 12. La reversibilidad se representa como banda transversal. La figura complementa a la del ciclo genérico (figura 12.1): aquí la atención no está en las fases en abstracto, sino en qué artefacto del caso examina cada sección.

- **Recuperar una versión anterior del sistema**: volver a una versión conocida del conversor.
- **Restaurar archivos o datos modificados**: recuperar las salidas anteriores del directorio autorizado.
- **Compensar efectos externos**: deshacer acciones fuera del sistema, como un envío de correo o una publicación.
- **Recuperar un estado operativo aceptable**: devolver el sistema a una condición en la que puede seguir operando de forma segura.

En el caso del conversor, las salidas son terminales y la conversión no es reversible: no se puede reconstruir el CSV original a partir del PDF o del XLSX. Lo que sí debe ser reversible es el *proceso*: poder restaurar las salidas anteriores y volver a una versión conocida del código. Esta distinción evita prometer una reversibilidad de datos que el formato no permite.

A.10. Evolución del caso a lo largo del libro

El conversor no aparece como una serie de versiones numeradas, sino como un mismo sistema examinado desde perspectivas distintas en cada sección. La tabla A.3 resume qué principio ilustra cada sección y qué elemento del caso se utiliza.

Tabla A.3.: Trazabilidad del caso transversal entre secciones, principios y elementos del caso. **Propuesta docente**: refleja el contenido real del capítulo.

Sección	**Principio**	**Elemento del caso**	**Evidencia o ejemplo**
1	Hilo conductor	Presentación inicial del escenario	Lista de requisitos y puntos de fallo
2	KISS, YAGNI, DRY	Alcance del conversor (cuántos formatos)	Tabla de comparación CSV frente a CSV+PDF+XLSX
3	Correctitud y especificación	Contrato del conversor	Pruebas funcionales que una versión insegura supera
4	Familias de fallo (CWE)	Cada punto de fallo del flujo	Ejemplos vulnerables y corregidos
5	Portabilidad	Rutas, temporales, fin de línea, codificación	Patrón portable de escritura de salida
6	Evaluación reproducible	El caso como unidad de análisis	Protocolo aplicado al conversor
7	Marco regulatorio	Clasificación del sistema	Caso práctico de clasificación
8	Confianza no calibrada	Código plausiblemente incorrecto	Circularidad del caso positivo
9	Aislamiento y agentes	Acciones del agente sobre el conversor	Modelo de amenazas del flujo
10	Pipeline de validación	Requisitos del conversor como entradas	Puertas de calidad y trazabilidad
11	Contratos e invariantes	Invariantes de seguridad del conversor	Contención, fórmulas, acotación
12	Ciclo de vida AI-SDLC	Fases aplicadas al caso	Aprobación, operación, reversibilidad
B	Prácticas de laboratorio	Construcción acotada del conversor	Enunciados y soluciones
13	Síntesis	Recorrido completo del caso	Ideas clave por bloque

La evolución que el capítulo describe es de *perspectiva*, no de versión: no hay una sucesión de implementaciones numeradas que el libro presente como versiones sucesivas del mismo código. Cada sección examina el mismo modelo del sistema y añade una capa de análisis. Si se quisiera presentar una evolución de versiones (por ejemplo, una versión vulnerable, una corregida y una generada por IA), sería una **propuesta docente** adicional; el capítulo no la presenta como existente.

A.11. Comparación entre desarrollo humano y asistido por IA

El caso permite estudiar dos procesos de desarrollo sobre los mismos requisitos y criterios de aceptación. No se construyen dos arquitecturas distintas: el modelo del sistema es el mismo, y lo que cambia es cómo se obtiene y se justifica cada artefacto. La capítulo 6 fija el protocolo neutral para comparar; aquí se enumeran los aspectos que pueden variar.

- **Obtención de requisitos**: un equipo humano puede elicitarlos por entrevista; un asistente puede completarlos a partir del enunciado, con el riesgo de rellenar huecos implícitos.
- **Generación de implementaciones**: ambos producen código; la diferencia está en la trazabilidad de las decisiones.
- **Trazabilidad de decisiones**: el proceso humano tiende a dejar registro de por qué se eligió una opción; el asistido puede no exponer el razonamiento.
- **Revisión**: la revisión debe ser independiente del origen del código (capítulo 8).
- **Producción de pruebas**: el asistente puede generar pruebas que reflejen el código en lugar de la especificación, con el riesgo de circularidad.
- **Independencia de la evaluación**: la evaluación debe medir el artefacto, no la procedencia (capítulo 6).
- **Documentación**: el proceso asistido puede generar documentación plausible que no corresponda a la implementación real.

El capítulo no presenta resultados experimentales de esta comparación: describe las dimensiones que un protocolo debería medir. La tesis que se mantiene es que las propiedades de seguridad del conversor dependen de las decisiones de diseño y de la verificación, no de quién escribió el código.

A.12. Relación con el ejemplo técnico de suma comprobada (C23)

El capítulo incluye además un ejemplo técnico acotado: la función de suma comprobada de enteros sin signo en C23 (`checked_add_u32`) que se verifica en la capítulo 11. Su finalidad es distinta de la del caso transversal: formalizar contratos, comprobar desbordamientos, aplicar pruebas de propiedades y demostrar verificación local y composicional. Se utiliza un componente pequeño porque las técnicas formales son más fáciles de presentar sobre una función con contrato explícito que sobre el conversor completo, cuya especificación abarca entrada, transformación y salida. No se trata de un segundo caso transversal: es un ejemplo técnico dentro de la sección de verificación.

A.13. Limitaciones del caso

- El conversor es un **ejemplo docente**, no un sistema industrial: no se presentan mediciones de rendimiento, disponibilidad ni seguridad.
- Los fragmentos de código del capítulo son ilustrativos; no constituyen una implementación completa ni verificada del sistema.
- La arquitectura incluye componentes propuestos (configuración, registro) que no se desarrollan con código.
- La conversión no es reversible: la reversibilidad del caso se refiere al proceso, no a la recuperación de los datos de entrada.
- El capítulo no presenta una serie de versiones numeradas del conversor; la evolución descrita es de perspectiva.

A.14. Del prompt al proceso de ingeniería: tres aproximaciones

Este anexo describe *un* sistema; la misma funcionalidad -convertir un CSV en PDF y XLSX con validación previa- permite, sin embargo, ilustrar tres formas de desarrollarla. El tratamiento metodológico general se encuentra en la sección 12.2; aquí se *aplica* al caso, sin repetir la exposición conceptual. La comparación describe qué produce cada aproximación y qué no garantiza; no afirma que una produzca necesariamente código de mayor calidad (tabla 12.1).

A.14.1. Prompt-driven development

En la aproximación guiada por instrucciones directas, el desarrollador describe la funcionalidad en lenguaje natural y el agente produce código. Sobre el caso transversal, una instrucción como «lee un CSV y genera un PDF y un XLSX validando los datos» basta para obtener un borrador plausible. Lo que *no* deja tras de sí es el rastro de qué se pidió exactamente: no hay requisitos enumerados, ni criterios de aceptación explícitos, ni una prueba que conecte cada requisito con su evidencia. El resultado puede ser correcto para el caso ejercitado y, aun así, no cubrir las condiciones que el sistema debe respetar (por ejemplo, la neutralización de fórmulas en el XLSX, tratada en el capítulo 4). Esta aproximación no es en sí incorrecta; simplemente no estructura la trazabilidad ni la cobertura.

A.14.2. Specification-driven development

En la aproximación guiada por especificaciones, el conversor se define primero mediante requisitos numerados (por ejemplo, en notación EARS), criterios de aceptación y casos límite; de ellos derivan un plan, una descomposición en tareas, la implementación y las pruebas. Iniciativas como Hello SDD [25] y GitHub Spec Kit [24] documentan este flujo. Su ventaja sobre la aproximación anterior es que hace *explícito* el vínculo requisito → implementación → prueba, que es la cadena de trazabilidad que el libro emplea en la sección 10.7. Lo que esta aproximación *no* garantiza es que la especificación sea correcta o completa: un requisito mal formulado produce una implementación fiel a algo distinto de lo necesario (capítulo 3). El proceso queda estructurado; la calidad de la especificación sigue siendo responsabilidad humana.

A.14.3. Ciclo de ingeniería asistido por IA

La tercera aproximación amplía la anterior con los mecanismos que el libro desarrolla en los capítulos técnicos: verificación con evidencias (capítulo 11), separación entre verificar y aprobar (sección 11.10), despliegue controlado, observabilidad y recuperación (sección 12.16), y reversibilidad como propiedad transversal (sección A.9). Sobre el caso, esto significa que la conversión no solo se especifica y se prueba, sino que se documenta qué propiedad se comprobó con qué técnica y bajo qué supuestos, quién aceptó el riesgo residual, y cómo se recuperaría un estado anterior si una decisión resultara incorrecta. Es la diferencia entre *estructurar* el desarrollo y *establecer garantías* de ingeniería; estas últimas no se automatizan por el hecho de disponer de un agente o de un flujo de trabajo. La tabla A.4 resume la diferencia aplicada al conversor. La columna «qué no garantiza»

es tan importante como la primera: ninguna de las tres aproximaciones sustituye la verificación independiente ni la supervisión humana allí donde el riesgo residual no está cubierto por evidencia.

Tabla A.4.: Tres aproximaciones aplicadas al caso transversal (conversor CSV-PDF-XLSX). **Propuesta docente**. Describe qué produce cada enfoque y qué no garantiza; no establece una jerarquía de calidad.

Aproximación	**Qué produce sobre el caso**	**Qué no garantiza**
A. Prompt-driven	Código del conversor a partir de instrucciones en lenguaje natural	Requisitos enumerados, trazabilidad, cobertura, evidencias
B. Specification-driven	Requisitos (EARS), plan, tareas, implementación y pruebas del conversor	Que la especificación sea correcta o completa; garantías formales
C. Ciclo de ingeniería asistido por IA	B, más verificación con evidencias, aprobación, operación y reversibilidad	Que las pruebas sean exhaustivas; corrección general

Las tres aproximaciones no son excluyentes ni forman una escala de madurez universal: un equipo puede adoptar la especificación estructurada sin llegar a un ciclo completo de garantías, o aplicar mecanismos de verificación a un desarrollo que no partió de una especificación formal. Lo que el libro sostiene es más modesto y más exigente: la generación de código, su estructuración y la obtención de garantías son actividades distintas, con evidencias y responsabilidades propias.

B. Prácticas de laboratorio

Este anexo reúne el marco de trabajo de las prácticas de laboratorio, su relación con los objetivos de aprendizaje y el desarrollo completo de las tres prácticas obligatorias, de la actividad complementaria opcional (práctica 04) y de la práctica técnica de verificación del componente de suma comprobada.

Las prácticas recorren el caso transversal -convertir un **CSV** en **PDF/Excel** con validación previa, presentado en el Anexo A- desde tres ángulos complementarios: localizar y explotar un fallo (práctica 01), comparar el efecto de la especificación sobre un artefacto generado (práctica 02) y construir un *pipeline* de validación con puertas de calidad (práctica 03). El material completo, con enunciado y solución separados, se encuentra en el directorio `labs/` del libro.

Las prácticas no son ejercicios aislados: cada una reutiliza el mismo escenario y enlaza con una o varias secciones teóricas. La práctica 01 trabaja las familias de fallo del capítulo 4 y la distinción entre funcional y seguro del capítulo 3; la práctica 02 aplica las variables controladas del capítulo 6 y el desplazamiento del esfuerzo hacia la especificación del capítulo 12; la práctica 03 construye el *pipeline* descrito en la capítulo 10 y ejecuta en el aislamiento del capítulo 9. Las tres terminan con una revisión humana, porque ninguna batería automática sustituye a la responsabilidad de quien acepta el artefacto.

Nota

Etiquetado epistémico de las prácticas. Se marca como **Evidencia**: que los mecanismos de fallo descritos corresponden a la semántica de los lenguajes y a reglas de codificación segura [7, 6]; que el análisis estático, el análisis de composición y las pruebas tienen los límites que aquí se exponen [3, 5]; y que la revisión y la trazabilidad son prácticas consolidadas. Se marca como **Propuesta docente**: el diseño de las prácticas, los artefactos candidatos ilustrativos, las correcciones propuestas, la arquitectura del *pipeline* con sus puertas y las rúbricas. Se marca como **Hipótesis**, a contrastar: que completar la especificación mejore el artefacto obtenido y que una configuración concreta de *pipeline* o de aislamiento baste para *este* artefacto.

B.1. Marco común y condiciones de aislamiento

Las tres prácticas obligatorias comparten un mismo régimen de trabajo, que no es una formalidad sino una condición de seguridad del propio laboratorio:

- **Aislamiento.** Las operaciones que ejecutan código no fiable se realizan dentro de una máquina virtual o un aislamiento equivalente cuya configuración haya sido comprobada, con usuario no privilegiado, directorio efímero, límites de recursos y sin acceso de red. Las entradas se montan en solo lectura; únicamente un directorio temporal de resultados puede ser escribible. Un contenedor convencional no constituye por sí solo una frontera de seguridad suficiente frente a código hostil (capítulo 9).
- **Sin servicios comerciales obligatorios.** Todo se resuelve con herramientas de código abierto y datos sintéticos: un compilador de C, Python 3, `pytest`, un analizador estático (por ejemplo, Bandit), un análisis de composición (por ejemplo, `pip-audit`) y un mecanismo de aislamiento local apropiado al sistema operativo y al modelo de amenazas (por ejemplo, una máquina virtual o `bubblewrap` en Linux, si está disponible y correctamente configurado). Las herramientas no son intercambiables sin comprobar sus garantías.
- **Sin datos reales sensibles.** Se emplean únicamente CSV sintéticos con valores ficticios; no se usan datos personales, credenciales ni secretos.
- **Agentes con permisos limitados.** Cuando una práctica permita un asistente o agente, se restringe su acceso: sin red, sin escritura fuera del directorio de trabajo, sin secretos y con aprobación humana para cualquier acción sensible. La ejecución del agente no debe conceder privilegios superiores a los del entorno que lo contiene.

Consejo

Registrar las condiciones es parte de la reproducibilidad. Antes de generar o ejecutar nada, anótese la tupla (modelo, versión, configuración, *prompt*/contexto, herramientas y entorno) definida en el capítulo 6. Sin ese registro, el resultado no es atribuible a un artefacto concreto y la comparación pierde validez.

B.2. Trazabilidad práctica ↔ objetivos de aprendizaje

La tabla B.1 relaciona cada objetivo de aprendizaje del capítulo con las prácticas que lo cubren. Es una tabla de **Propuesta docente**: la asignación es una decisión pedagógica

y se verifica en la auditoría final del capítulo.

Tabla B.1.: Trazabilidad entre las prácticas de laboratorio y los objetivos de aprendizaje. **Propuesta docente**. El símbolo • indica que la práctica cubre el objetivo.

Objetivo de aprendizaje	**P. 01**	**P. 02**	**P. 03**
Distinguir código funcional de código correcto y seguro	•	•	
Formular precondiciones, postcondiciones e invariantes		•	
Identificar familias de vulnerabilidad por CWE	•	•	
Explicar por qué las pruebas finitas no demuestran seguridad	•	•	•
Diseñar un protocolo reproducible que controle versión, configuración, *prompt*, herramientas y entorno		•	
Diferenciar un sistema de IA regulado de software convencional		•*	
Distinguir *sandbox* regulatorio, aislamiento técnico y *pipeline* de validación			•
Modelar amenazas de las acciones de un agente y proponer límites de confianza		•	•
Diseñar un *pipeline* de validación con puertas de calidad y trazabilidad			•
Aplicar una metodología docente AI-SDLC y reconocer sus límites		•	•

El símbolo • indica que la práctica cubre el objetivo; •* indica una actividad breve de análisis documental, sin simular participación en un sandbox regulatorio.

Nota

Cobertura de los objetivos 6, 7 y 10. El objetivo 6 se trabaja mediante una actividad breve de la práctica 02: comparar un conversor convencional cuyo código se obtuvo con asistencia de IA con un producto que incorpora funciones de IA, identificar su uso previsto y justificar si procede examinar la definición del artículo 3.1 y las reglas de clasificación del artículo 6 del Reglamento (UE) 2024/1689. Es un *análisis documental*, no una prueba regulatoria real. El objetivo 7 se aborda al distinguir los entornos jurídicos del capítulo 7, el aislamiento del capítulo 9 y las puertas del capítulo 10. El objetivo 10 se trabaja en las prácticas 02 y 03 mediante especificación, validación y revisión. La práctica 04 (opcional) refuerza los objetivos 5, 9 y 10 al ejecutar el kit AI-SDLC y registrar evidencias; no se añade una cuarta columna a la tabla para no alterar la asignación de las prácticas obligatorias.

Advertencia

Ejemplos vulnerables: ejecución solo en entorno aislado y autorizado. Los artefactos marcados como vulnerables en las prácticas son **material didáctico** (advertencia general en el capítulo 1). Se compilan y ejecutan **únicamente** en la máquina virtual o contenedor desechable descrito en la sección B.1, bajo supervisión del profesorado, sin acceso de red, sin datos reales y sin escritura sobre el sistema de ficheros del anfitrión. Su finalidad es reconocer el fallo y razonar sobre su defensa, no producir un programa utilizable ni reutilizar el código. No deben desplegarse ni ejecutarse fuera del laboratorio.

B.3. Práctica 01: código funcional pero vulnerable

La primera práctica parte de dos artefactos que *funcionan* -con entradas ordinarias producen el resultado esperado- pero no son seguros: una comprobación de ruta antes de usarla, con una ventana TOCTOU (CWE-367), que puede involucrar enlaces simbólicos (CWE-59), y una exportación de celdas que no neutraliza fórmulas de hoja de cálculo (CWE-1236). La aplicabilidad de CWE-59 depende de la implementación concreta. El estudiante debe localizar el fallo, reproducirlo *solo en aislamiento*, proponer una corrección contextual y explicar qué no cubre esa corrección.

Tabla B.2.: Práctica 01 - resumen. **Propuesta docente**. La clasificación CWE se refiere al mecanismo de fallo observado, no a una garantía de explotación universal [6].

Apartado	**Contenido**
Objetivo	Distinguir un programa que funciona de un programa correcto y seguro; localizar un fallo de E/S de ficheros y clasificarlo por CWE; reproducirlo en aislamiento; proponer una corrección contextual y enunciar sus límites.
Se entrega	El código fuente de los dos artefactos y de las dos versiones corregidas; el registro de comandos y salidas; la clasificación CWE con justificación.
Evidencias	Transcripción o captura de las pruebas positiva y negativa de cada artefacto; nota de trazabilidad que asocia cada fallo con el requisito del caso transversal que incumple y con el control que lo neutraliza.
Rúbrica resumida	Localización del fallo (20 %); clasificación CWE (15 %); explotación en aislamiento (25 %); corrección contextual (20 %); trazabilidad y evidencias (10 %); preguntas de reflexión (10 %).

Advertencia

La prueba negativa del fallo TOCTOU es no determinista. La ventana entre la comprobación y el uso puede no alcanzarse en cada intento. Observar el fallo en alguna ejecución *no* demuestra que sea inevitable, y no observarlo *no* demuestra que el código sea seguro. Esa asimetría es la razón concreta por la que superar una batería finita de pruebas no demuestra seguridad.

B.4. Práctica 02: *prompt* incompleto frente a especificación

La segunda práctica plantea el *mismo* problema dos veces: una vez con un *prompt* mínimo, sin requisitos de seguridad, y otra con una especificación funcional y de seguridad completa (requisitos F1-F4, de seguridad S1-S6 y restricciones R1-R3). Los dos artefactos candidatos se auditan con la misma lista de criterios y se comparan. El objetivo es poner a prueba la hipótesis rectora del capítulo: que la IA desplaza el esfuerzo hacia la especificación, la revisión y la validación, sin eliminar la responsabilidad sobre el resultado.

Tabla B.3.: Práctica 02 - resumen. **Propuesta docente**. La comparación mide *qué se ha cubierto*, no *qué riesgo queda*.

Apartado	**Contenido**
Objetivo	Comprender el desplazamiento del esfuerzo humano hacia la especificación y la validación; formular una especificación funcional y de seguridad; auditar dos artefactos con los mismos criterios y clasificar los hallazgos por CWE.
Se entrega	Los dos artefactos candidatos sin modificar, con el registro de las condiciones de generación (asistente, versión, configuración, *prompt*); la tabla de auditoría S1-S6 para cada uno; la tabla de comparación y una conclusión escrita.
Evidencias	Clasificación CWE de cada hallazgo con la línea afectada; transcripción de las pruebas P1-P5 (funcional, inyección de fórmulas, validación de columnas, confinamiento de ruta y límite de tamaño) sobre ambos artefactos.
Rúbrica resumida	Registro de condiciones (15 %); auditoría S1-S6 (25 %); clasificación CWE (15 %); comparación A/B (20 %); pruebas y evidencias (15 %); conclusión y reflexión (10 %).

Advertencia

No modificar el artefacto antes de auditarlo. La auditoría se hace sobre la salida tal como se obtuvo. Corregir antes de medir destruye la comparación: el objetivo es observar qué produce cada nivel de especificación, no arreglar la salida. Completar la especificación puede mejorar el artefacto obtenido, pero ese efecto debe medirse y no se presupone; en ningún caso garantiza seguridad; el resultado sigue siendo una *hipótesis de solución* que debe auditarse y validarse.

B.5. Práctica 03: *pipeline* aislado de validación

La tercera práctica construye un *pipeline* ejecutable que convierte un artefacto candidato en un expediente con evidencia verificable. Integra análisis estático (SAST), análisis de composición (SCA), comprobación de compilación o importación, pruebas funcionales y adversarias con `pytest`, y recogida de evidencias. **Toda etapa que ejecute código candidato no fiable**, incluida la importación y las pruebas, debe ejecutarse *dentro* del aislamiento, no después de una comprobación aislada que simplemente certifique la existencia del sandbox. Cada etapa es una *puerta* con comportamiento *fail-closed*: ante duda, fallo de la herramienta, error de infraestructura o evidencia ausente, la puerta se cierra y el artefacto no avanza. Los hallazgos críticos no se consideran resueltos por estar simplemente «justificados»: deben corregirse, identificarse como falsos positivos con evidencia o tratarse mediante una excepción formalmente autorizada, delimitada y registrada, según la política del laboratorio.

Advertencia

Un *pipeline* verde no demuestra seguridad. Superar las pruebas y puertas significa que *no se observó ningún fallo bajo esas comprobaciones*. Las pruebas son una batería finita sobre un conjunto declarado de propiedades; quedan fuera los falsos negativos heredados de cada técnica y los requisitos no expresados. El expediente es un artefacto *validado contextualmente*, nunca un artefacto «seguro» en sentido absoluto.

B.6. Comprobación negativa de las puertas de seguridad

La práctica 03 no termina cuando todos los indicadores aparecen en verde. Debe incluirse al menos una **prueba de fallo deliberado** para verificar que la orquestación

Tabla B.4.: Práctica 03 - resumen. **Propuesta docente**. La arquitectura y las puertas se detallan en el capítulo 10 (pipeline de validación).

Apartado	**Contenido**
Objetivo	Diseñar y ejecutar un *pipeline* que convierta un artefacto en un expediente con evidencia verificable; definir puertas *fail-closed*; establecer la trazabilidad requisito → implementación → prueba → evidencia; y reconocer los límites de la automatización.
Se entrega	El script de orquestación y el envoltorio de *sandbox*; la especificación `spec.md`; el informe del SAST con su triaje y el del SCA; la salida de `pytest`; y el registro de la ejecución aislada.
Evidencias	Matriz de trazabilidad completa y auditable, con oráculos que inspeccionen propiedades de seguridad y no solo la existencia de archivos; registro de la decisión; prueba de que el *pipeline* rechaza una implementación deliberadamente defectuosa; y declaración explícita de sus límites.
Rúbrica resumida	Diseño del *pipeline* (20 %); puertas *fail-closed* (20 %); SAST, SCA y triaje (15 %); pruebas y *sandbox* (20 %); trazabilidad (15 %); límites y reflexión (10 %).

realmente bloquea un artefacto que incumple una propiedad declarada. Por ejemplo, puede introducirse en una copia controlada una exportación XLSX que escriba como fórmula un valor CSV que comience por `=`. El oráculo de prueba debe inspeccionar el tipo y el contenido de la celda generada: comprobar solo que el archivo existe sería insuficiente. El *pipeline* debe devolver un estado de rechazo y conservar la evidencia de ese rechazo.

Asimismo, deben probarse un fallo de infraestructura y la ausencia de un informe obligatorio: ambos deben bloquear la aceptación. No se ejecutarán muestras maliciosas reales fuera del entorno autorizado. La elección de controles, las limitaciones del sistema operativo y el riesgo residual se documentarán en el expediente. Esta actividad enlaza con la validación del propio *pipeline* del capítulo 10.

B.7. Práctica 04 (opcional): ejecutar el kit AI-SDLC sobre el caso transversal

Objetivos.

Aplicar la secuencia `P00_integral` y el runner del kit `kit_agentes_ai_sdlc_v2/` al caso CSV→PDF/XLSX, registrar evidencias y decidir con revisión humana. Esta actividad es **opcional** y no sustituye a las prácticas 01-03.

Preparación.

Python 3.10 o superior; `bubblewrap` si se usa `-sandbox`; herramientas opcionales (`ruff`, `mypy`, `bandit`, `pip-audit`) si se usa el perfil `extended`. El proyecto debe ejecutarse en un entorno aislado o ser de confianza: el runner ejecuta los tests del proyecto.

Instrucciones.

1. Leer `README.md`, `INTEGRACION_LIBRO.md` y `PRINCIPIO_INDEPENDENCIA.md`. Fijar los criterios y las pruebas de aceptación *antes* de generar el artefacto.

2. Ejecutar el runner sobre el proyecto candidato:

Listing B.1: Ejecución del runner con perfil básico y aislamiento.

```
python pipelines/run.py --profile basic --sandbox --path
   RUTA_PROYECTO
```

3. Repetir con `-profile extended` si las herramientas opcionales están instaladas; registrar los `SKIP` y `NOT_EVALUATED` como limitaciones, no como éxitos.

4. Completar `templates/evidence.md` con requisito, comprobación, herramienta y versión, comando, resultado, log y limitación.

5. Comparar el artefacto generado con un *prompt* incompleto y otro con la especificación completa (práctica 02). La comparación debe basarse en evidencias registradas, no en impresiones.

6. Decidir la aceptación con una persona de un rol distinto del generador. Si falta evidencia, la decisión es *no aceptar*.

Evidencias.

Informe `reports/ai_sdlc_report.json`, registro de evidencias, comandos y versiones, salidas de las pruebas y decisión firmada. El ejemplo `reports/example_report.json` muestra el formato esperado.

Tabla B.5.: Rúbrica de la práctica 04 (opcional). **Propuesta docente**.

Criterio	Peso	Descriptor
Independencia	20 %	Criterios y pruebas fijados antes de generar; el generador no se autoaprueba
Ejecución y evidencias	30 %	Runner ejecutado; comandos, versiones, resultados y logs registrados
Interpretación de estados	20 %	Distingue PASS, FAIL, SKIP y NOT_EVALUATED; no confunde SKIP con éxito
Comparación de artefactos	15 %	Comparación basada en evidencias entre prompt incompleto y completo
Decisión y límites	15 %	Decisión humana justificada; límites y riesgos residuales declarados

Rúbrica orientativa.

Advertencia

El runner ejecuta código. No lo ejecutes sobre código no confiable fuera de un sandbox. Un pipeline verde no demuestra seguridad, corrección ni conformidad normativa; solo indica que las comprobaciones declaradas no fallaron en ese entorno.

Nota

Recurso complementario. `kit_agentes_ai_sdlc_v2/`, versión `2.0.0`; requisitos en `REQUISITOS_ENTORNO.md`; hashes en `SHA256SUMS`; licencia pendiente de declaración en `LICENSE.md`; revisión en `REVISION.md`. El kit es material docente y no sustituye la revisión humana.

B.8. Práctica técnica: verificación del componente de suma comprobada

Esta práctica aplica de forma conjunta las técnicas del capítulo 11 -contratos, invariantes, pruebas de propiedades, fuzzing y verificación formal- sobre un componente mínimo pero completo. Puede realizarse de forma independiente o como extensión técnica de la práctica 03.

Para mostrar cómo se aplican de forma conjunta, se desarrolla un componente mínimo pero completo: una función de suma comprobada de enteros sin signo de 32 bits. El código completo, las pruebas y los registros de ejecución están en el directorio

`examples/code_verification/` del material; aquí se presentan únicamente los fragmentos esenciales y su interpretación. Este caso se usa como ejemplo transversal de contratos, invariantes, modelos de amenazas, pruebas negativas, verificación independiente, integración y criterios de aceptación.

La interfaz es la siguiente:

Listing B.2: Interfaz del componente de suma comprobada.

```
#include <stdbool.h>
#include <stdint.h>

bool checked_add_u32(uint32_t a, uint32_t b, uint32_t *result);
```

B.8.1. Contrato y distinción semántica

El contrato funcional exige que `result` apunte a un objeto válido (precondición); si la suma matemática $a +_{\mathbb{N}} b$ es representable en `uint32_t`, la función devuelve `true` y escribe ese valor; si excede `UINT32_MAX`, devuelve `false` y *no modifica* `*result`. Fuera del contrato, cuando `result` es nulo o inválido, el comportamiento no está garantizado; en C no existe un mecanismo portable para validar punteros de forma general.

Es esencial no confundir dos semánticas. La aritmética de enteros sin signo en C es *modular* y está perfectamente definida (envoltura módulo 2^{32}): no hay comportamiento indefinido al desbordar. El contrato, en cambio, exige el *valor matemático* cuando es representable y el *rechazo* cuando no lo es. Que una expresión `a + b` se evalúe sin comportamiento indefinido no significa que su resultado cumpla el contrato. La formulación completa, con tripletas de Hoare, *frame conditions*, corrección parcial y terminación, y una representación ACSL, está en `verification/contracts.md`. La terminación es trivial por la ausencia de bucles y recursión; por tanto la corrección total se sigue de la parcial.

La implementación de referencia comprueba la representabilidad *antes* de sumar, de modo que la comprobación nunca desborda y, en caso de error, no se escribe nada:

Listing B.3: Implementación de referencia: comprobación previa a la suma.

```
bool checked_add_u32(uint32_t a, uint32_t b, uint32_t *result)
{
    if (b > UINT32_MAX - a) {   /* UINT32_MAX - a nunca desborda
       */
        return false;
    }
```

```
6       *result = a + b;              /* solo se escribe si es
            representable */
7       return true;
8   }
```

B.8.2. Variantes defectuosas y discriminación por pruebas

Para ilustrar que los principios de verificación son independientes del origen del código, se emplean dos variantes didácticas defectuosas. **No se etiquetan como humanas o generadas por IA**: el origen no puede inferirse a partir del defecto. La Candidata A usa `b >= UINT32_MAX - a` en lugar de `>`, por lo que rechaza la suma exacta `UINT32_MAX`, que es representable. La Candidata B detecta correctamente el desbordamiento pero escribe `*result` antes de comprobar, incumpliendo la preservación del estado en error. Los fragmentos completos están en `src/checked_sum_candidate_a.c` y `src/checked_sum_candidate_b.c`.

Las pruebas de valores límite ejercen la frontera exacta `UINT32_MAX` y el rechazo por encima, además de comprobar la preservación. La batería se ejecuta contra las tres implementaciones: la referencia debe pasar; las candidatas deben fallar en los casos diseñados para detectar sus defectos. La tabla B.6 resume los resultados obtenidos al ejecutar el proyecto (véase `VERIFICATION_REPORT.md`).

Tabla B.6.: Resultados de verificación de las tres implementaciones del componente.

Criterio	**Referencia**	**Candidata A**	**Candidata B**
Compila con `-Werror`	Sí	Sí	Sí
Contrato (frontera y preservación)	Pasa	Falla	Falla
Propiedades P1-P5	Pasa	Viola P1	Viola P3
ASan / UBSan	Sin hallazgos	Sin hallazgos	Sin hallazgos
Fuzzing (P1-P4)	Sin fallos	Detecta	Detecta

La lectura de la tabla ilustra una distinción central del capítulo. Las tres implementaciones *compilan sin advertencias* y pasan los sanitizers: la aritmética sin signo está definida y no hay accesos inválidos. Sin embargo, dos de ellas incumplen el contrato. Ni el compilador, ni el análisis estático, ni los sanitizers detectan estos defectos *semánticos*: solo las pruebas que ejercen las cláusulas del contrato los ponen de manifiesto. Esto confirma que «compila sin advertencias» y «pasa los sanitizers» no implican «correcto».

B.8.3. Propiedades, fuzzing y análisis

Las propiedades se enuncian y comprueban de forma generativa y reproducible: P1 (corrección del resultado representable), P2 (detección del desbordamiento), P3 (preservación del estado en error), P4 (conmutatividad) y P5 (determinismo). La generación usa un generador determinista con semilla fija y 200 000 casos por propiedad, mezclando entradas aleatorias, próximas a los límites y de magnitud baja. Es una muestra, no una prueba exhaustiva: no agota las 2^{64} parejas de entrada. El fuzzing añade un harness que comprueba automáticamente P1-P4 y aborta ante una violación; con la Candidata B se obtuvo un caso reproducible, lo que demuestra que el harness detecta el defecto de preservación. El análisis estático y los sanitizers se emplean como evidencia complementaria, no como demostración de corrección funcional. Además, las propiedades P1-P5 del componente y las propiedades I1-I3 del acumulador se demuestran formalmente con Frama-C/WP sobre variantes anotadas con ACSL: la herramienta verifica los dos comportamientos del contrato, la *frame condition*, la terminación, la conmutatividad, el determinismo y las propiedades del acumulador para todo el dominio de entrada, sin obligaciones pendientes. La demostración cubre el componente y su composición, no la concurrencia ni la validez general de punteros, que quedan fuera del contrato.

B.8.4. Modelo de amenazas y verificación composicional

El modelo de amenazas del componente (`verification/threat_model.md`) distingue entre defectos funcionales y amenazas de seguridad. Un rechazo indebido de una suma representable (Candidata A) es un defecto de corrección; si el componente se usara para calcular un tamaño de memoria, un desbordamiento no detectado podría derivar en una vulnerabilidad de memoria (CWE-190 hacia CWE-787). La clasificación depende del contexto de uso, no es automática. La matriz de amenazas traduce cada amenaza en una propiedad comprobable con su evidencia.

La verificación composicional se ilustra con un acumulador de segundo nivel que usa `checked_add_u32`. Su contrato depende del contrato inferior: solo puede preservar su estado en error si el componente inferior cumple P3. Una variante del acumulador que ignora el valor de retorno compila y pasa las pruebas nominales, pero corrompe el estado y reporta éxito tras un desbordamiento. Esto demuestra que verificar cada componente por separado no basta: es necesario comprobar la compatibilidad contractual entre componentes, tal como se discutió en la sección 11.11.

B.8.5. Validez del oráculo y pruebas de mutación

Una batería superada solo constituye evidencia si sus aserciones distinguen resultados correctos de incorrectos. Se ilustra un oráculo defectuoso que usa una implementación candidata como referencia: como comparte el defecto, la prueba «pasa» aunque la implementación sea incorrecta. Frente a él, un oráculo derivado de la especificación (suma en 64 bits) detecta el defecto. Se distingue entre oráculo derivado de especificación, derivado de implementación, metamórfico y comparación diferencial. Las pruebas de mutación introducen cambios controlados (comparación estricta por no estricta, eliminación de la condición de error, escritura antes de comprobar, detección por acarreo) y comprueban qué pruebas los detectan. El mutante por acarreo resulta *equivalente* al contrato y no se detecta, lo que se documenta explícitamente. La puntuación de mutación estima la capacidad de detección; no garantiza corrección.

B.8.6. Protocolo de comparación humano-IA

La comparación entre implementaciones de procedencia humana y asistida por IA se realiza con un protocolo independiente, no con las variantes defectuosas anteriores. El protocolo entrega la misma especificación a una persona, a un modelo de IA y a un modelo de IA que recibe además el contrato formalizado; y registra la especificación, las herramientas, el tiempo disponible, las interacciones y modificaciones, la implementación final y los resultados de verificación. No se presupone cuál será más correcta y no se atribuyen resultados a participantes reales. Los detalles están en `verification/` y en el protocolo descrito en la sección 11.9.

B.8.7. Trazabilidad de requisitos, propiedades y evidencias

La tabla B.7 relaciona cada requisito con la propiedad que lo expresa, la técnica empleada, la evidencia obtenida y su limitación. Es el expediente mínimo exigido en la sección 11.10.

Tabla B.7.: Trazabilidad de requisitos, propiedades, técnicas, evidencias y limitaciones.

Requisito	**Propiedad**	**Técnica**	**Evidencia**	**Limitación**
Aceptar suma representable	P1	Pruebas, propiedades, fuzzing	200 000 casos + fuzzing	Muestra, no exhaustiva

Requisito	**Propiedad**	**Técnica**	**Evidencia**	**Limitación**
Rechazar desbordamiento	P2	Pruebas de límites, propiedades	Frontera `UINT32_MAX`	Frontera exacta requiere caso dirigido
Preservar estado en error	P3	Pruebas, propiedades, fuzzing	Caso reproducible (B)	Ninguna para la referencia
Conmutatividad	P4	Propiedades, metamórfica	Pares de entradas	Solo entradas representables
Determinismo	P5	Propiedades	Repetición	Estado inicial equivalente
Corrección universal	P1-P5, I1-I3	Verificación formal (Frama-C/WP)	94/94 obligaciones demostradas	Concurrencia y punteros fuera de contrato
Integración del acumulador	I1-I3	Pruebas de integración	Acumulador correcto/defectuoso	Depende del contrato inferior
Validez del oráculo	-	Mutación, oráculos	Oráculo de especificación	Mutantes equivalentes no detectados

El expediente declara tanto lo comprobado como lo no comprobado. Las propiedades P1-P5 del componente y I1-I3 del acumulador se han demostrado formalmente con Frama-C/WP (94/94 obligaciones, sin pendientes) para todo el dominio de entrada. Lo que queda fuera de la demostración es explícito: concurrencia y validez general de punteros no forman parte del contrato y no se modelan. La ejecución satisfactoria de las pruebas se presenta como evidencia complementaria, no como sustituto de la demostración ni como corrección general más allá de lo que efectivamente se ha demostrado.

B.9. Preguntas de reflexión

1. ¿Por qué un programa que pasa todas las pruebas funcionales puede seguir siendo inseguro? Dad un ejemplo tomado de la práctica 01.

2. ¿Qué parte del esfuerzo se desplaza del operador al *prompt* al completar la especificación en la práctica 02, y qué parte *no* se desplaza?

3. ¿Por qué el aislamiento del entorno es una condición de las prácticas y no una formalidad? ¿Qué se protege exactamente?

4. ¿Qué significa que una puerta sea *fail-closed* en la práctica 03? ¿Qué ocurre cuando falla de forma ambigua?

5. Un compañero afirma: «el *pipeline* está verde, luego el artefacto es seguro». ¿Qué error comete y cómo reformularíais la afirmación?

6. ¿Por qué registrar la versión, la configuración y el *prompt* mejora la reproducibilidad sin garantizarla por sí solo?

7. ¿Cómo comprobaríais que una prueba de neutralización de fórmulas detecta realmente una celda interpretada como fórmula?

8. ¿Qué diferencia existe entre documentar un hallazgo crítico y aceptar formalmente su riesgo residual?

Nota

Invariante común a las prácticas (incluida la actividad opcional). Superar una batería finita de pruebas no demuestra seguridad (capítulo 3). Cada corrección que se propone es **contextual**: válida para un modelo de amenaza y unas precondiciones declaradas, y nunca «universalmente segura».

C. Plantillas de registro (material de referencia)

Este anexo recoge las plantillas completas de registro para la evaluación reproducible de sistemas de IA descritas en el capítulo 6: ficha de sesión, registro de hallazgos por requisito, registro de vulnerabilidades y formato legible por máquina, junto con el paquete de reproducibilidad. Se separan del cuerpo principal para facilitar su consulta independiente durante el trabajo experimental. La justificación metodológica y el uso de estos registros se presentan en la sección 6.8.

C.1. Plantillas de registro

Sin registros estructurados no hay reproducibilidad: un protocolo que no se puede reconstruir no es un protocolo, es una impresión. Se proponen cuatro plantillas: ficha de sesión, registro de hallazgos por requisito, registro de vulnerabilidades y un formato legible por máquina.

Ficha de sesión.

Identifica una ejecución concreta y fija todas las variables del tabla 6.2.

Registro de hallazgos por requisito.

Vincula cada requisito de F0 con la evidencia observada. Es la base de la métrica «requisitos satisfechos» y de la trazabilidad requisito → implementación → prueba → evidencia.

Registro de vulnerabilidades.

Cada vulnerabilidad confirmada se documenta por separado, con su clasificación CWE, su severidad y cómo se reprodujo. Se distingue explícitamente entre confirmada, sospechada y falso positivo.

Tabla C.1.: Ficha de sesión (plantilla). Una fila por ejecución.

Campo	**Contenido a registrar**
Identificador de sesión	Código único y estable (por ejemplo, `SUT-A-0007`).
Fecha y hora	Marca temporal en UTC de la ejecución.
Modelo y versión	Identificador exacto y punto de acceso (API/UI); proveedor.
Configuración	*System prompt* (referencia y huella), temperatura, *top-p*, límite de tokens, filtros.
Prompt	Identificador de la plantilla y contenido literal usado.
Contexto	Ficheros o fragmentos incluidos en la ventana; política de recuperación.
Herramientas	Herramientas habilitadas, versiones, permisos y acceso a red.
Entorno	Sistema operativo, arquitectura, imagen de contenedor y *sandbox*.
Repeticiones	Número n, índice de la repetición y semilla si existe.
Operador	Persona o script que lanzó la ejecución.
Observaciones	Incidencias, límites alcanzados y anomalías.

Tabla C.2.: Registro de hallazgos por requisito (plantilla).

Req.	**Tipo**	**Descripción**	**Resultado**	**Evidencia**	**Estado**
R-F-01	Funcional	Leer el CSV y normalizar columnas	Cumple / Parcial / No cumple	Fichero y línea	Abierto / Cerrado
R-S-01	Seguridad	No confiar en la entrada del CSV	Cumple / Parcial / No cumple	Prueba y traza	Abierto / Cerrado
R-S-02	Seguridad	Evitar *path traversal* al escribir salidas	Cumple / Parcial / No cumple	Revisión y prueba	Abierto / Cerrado
R-S-03	Seguridad	Crear temporales seguros	Cumple / Parcial / No cumple	Revisión y prueba	Abierto / Cerrado
R-S-04	Seguridad	Limitar tamaño y tiempo de proceso	Cumple / Parcial / No cumple	Prueba	Abierto / Cerrado
R-S-05	Seguridad	No ejecutar contenido embebido	Cumple / Parcial / No cumple	Revisión	Abierto / Cerrado
R-S-06	Seguridad	Escapar fórmulas de Excel	Cumple / Parcial / No cumple	Prueba	Abierto / Cerrado

Tabla C.3.: Registro de vulnerabilidades (plantilla).

Id.	CWE	Ubicación y descripción	Severidad	Cómo se confirma	Estado	Remedio
V-001	CWE-22	Escritura fuera del directorio de salida	Alta / Media / Baja	SAST / manual / dinámica	Confirmada / Sospechada / Falso positivo	Referencia contextual
V-002	CWE-367	TOCTOU entre validar y abrir	-	-	-	-
V-003	CWE-1236	Inyección de fórmulas en la salida XLSX	-	-	-	-

Nota

Sobre los identificadores CWE. En la tabla C.3 se muestran identificadores a modo de *plantilla*, no como resultado de una medición. Cada uno debe verificarse contra la edición vigente del catálogo [6] antes de usarse en un informe; las celdas de las filas `V-002` y `V-003` quedan incompletas precisamente para subrayar que la plantilla no contiene datos inventados.

Formato legible por máquina.

Las plantillas anteriores pueden serializarse en un formato estructurado que permita agregar los resultados sin transcripción manual. El listado C.1 muestra un esqueleto de sesión en YAML; los comentarios se escriben sin acentos para evitar problemas de codificación en `listings`.

Listing C.1: Esqueleto de registro de sesion en YAML (plantilla).

```yaml
# Plantilla de ficha de sesion. Sin datos de ejemplo inventados.
sesion:
  id: ""                      # codigo unico de la ejecucion
  timestamp_utc: ""           # marca temporal en UTC
  sistema:
    modelo: ""                # identificador exacto del modelo
    version: ""               # version concreta
    proveedor: ""             # proveedor del servicio
    punto_acceso: ""          # api o interfaz web
  configuracion:
```

```
    system_prompt_ref: ""     # referencia al texto y su huella
    temperature: null         # fijar y declarar; null si no aplica
    top_p: null
    max_tokens: null
    filtros_seguridad: ""     # ajustes declarados
  prompt:
    plantilla_id: ""          # identificador de la plantilla
    contenido_ref: ""         # ubicacion del texto literal
  contexto:
    ficheros: []              # rutas incluidas en la ventana
    recuperacion: ""          # politica de recuperacion
  herramientas:
    habilitadas: []           # lista de herramientas y versiones
    permisos: ""              # red, escritura, ejecucion
  entorno:
    so: ""
    arquitectura: ""
    imagen: ""                # contenedor o sandbox
  repeticiones:
    n: null                   # numero total
    indice: null              # indice de esta repeticion
    semilla: null
  operador: ""
  observaciones: ""
```

Paquete de reproducibilidad.

El cierre del protocolo es un archivo que permite a un tercero repetir el estudio. Debe contener, como mínimo:

- La especificación de requisitos de F0, numerada.
- El *prompt* o las plantillas usadas, literales.
- Las fichas de sesión de todas las ejecuciones, incluidas las fallidas.
- Los artefactos producidos y su huella (suma de verificación).
- Las versiones de las herramientas de análisis estático y dinámico.
- Las tablas de hallazgos y de vulnerabilidades.

- El guion de análisis estadístico y las decisiones tomadas.
- Una declaración explícita de las amenazas a la validez no resueltas.

Índice alfabético

ABI, 76
aceptación, 207
agente (de IA), 4
agentes, 135
Agentic SDLC, 195
AI-assisted SDLC, 195
AI-native SDLC, 195
AI-SDLC, *véase* ciclo de vida del desarrollo de software, 198
aislamiento, 135
alcance (gestión de), 10
alucinación, 122
análisis dinámico, 150
análisis estático, 150
API, 76
aprobación, 179
arquitectura de referencia, 230

cadena de suministro, 23, 70
canonicalización, 76
caso transversal, 3, 90
ciclo de vida del desarrollo de software, 195
clasificación de sistemas de IA, 117
codificación, 81
complejidad
  esencial y accidental, 12
comportamiento indefinido, 43, 84
comportamiento no especificado, 84
comprensibilidad del código, 11
concurrencia, 63, 84
condición de carrera, 63
confianza delegada, 167
confianza no calibrada, 123
configuración (de IA), 4
confinamiento, 143
contenedores, 141
contrato, 29, 171, 231
conversión documental, *véase* caso transversal
correctitud, 27
  funcional, 27
  parcial, 30
  total, 30
CWE, 41

DAST, 154
defensa en profundidad, 19
dependencias vulnerables, 70
desbordamiento de búfer, 41
desbordamiento de entero, 43
descriptor de fichero, 80
deserialización, 52
desplazamiento del esfuerzo, 191
despliegue, 208
diseño abierto, 16
diseño por contrato, 19
DRY, 9

duplicación de conocimiento, 9

enfoque basado en el riesgo, 107
enlaces simbólicos, 79
entorno de ejecución, 4
errores no comprobados, 66
espacio controlado de pruebas, 109
especificación, 29, 170
    incompleta, 35
especificación de plataforma, 88
evaluación reproducible, 96
evidencias, 149
evidencias de aceptación, 182

fases de la metodología, 198
ficheros temporales, 60, 81
fin de línea, 81
fugas de recursos, 67

gobernanza, 210

heurística, 7
hipótesis rectora, 1, 215

independencia de contexto, 127
integración continua, 158
inteligencia artificial generativa, 1
invariante, 19, 29, 173, 231
inyección de comandos, 50
inyección de fórmulas, 23, 32, 54
inyección de instrucciones, 139
inyección SQL, 51

KISS, 8

ley de Brooks, 12
ley de Conway, 14
ley de Kernighan, 11
ley de Postel, 20

limitaciones, 218
límites de confianza, 136
lógica de Hoare, 30

mantenimiento, 207
mediación completa, 16
menor privilegio, 16, 143
modelo (de IA), 4
modelo de amenazas, 128, 174, 232
modelo de datos, 231
máquina virtual, 141
métricas, 100

normalización, 83
número de Dunbar, 15

observabilidad, 208
oráculos, 184

path traversal, 48
permisos, 64, 79
pipeline de validación, 150, 246
pipeline verde (límite del), 162
plausibilidad, 122
portabilidad, 76
postcondición, 29
precondición, 29
principio de diseño, 7
principio de menor sorpresa, 20
principio de Pareto, 13
procesos, 84
prompt, 192
prompt (instrucción), 35
prompt injection, *véase* inyección de instrucciones
protocolo experimental, 99
pruebas
    insuficiencia de las, 32

pruebas en condiciones reales, 114
pruebas negativas, 177
prácticas de laboratorio, 244
puertas de calidad, 156

recuperación, 208
Reglamento de IA, 107
requisitos
    explícitos, 35
    implícitos, 35
responsabilidad epistémica, 130
responsabilidad única, 20
reversibilidad, 208, 233
    compensación, 208
    restauración, 208
    rollback, 208
revisión de código, 21
riesgo epistémico, 121
robustez, 27
rutas, 76
régimen transitorio, 113

Saltzer y Schroeder, 16
sandbox regulatorio, 114
SAST, 154
SBOM, 161
SCA, 161
secretos embebidos, 68
seguridad
    frente a robustez, 27
separación de privilegios, 16
sesgo del caso positivo, 125
simplicidad, 8
Spec-Driven Development, 192
superficie de ataque, 139
sustitución de Liskov, 20

TOCTOU, 57
tratamiento estadístico, 102
trazabilidad, 160

Unicode, 83
unidad de análisis, 96

validación, 167
validación de entrada, 47
validez (amenazas a la), 104
valores seguros por defecto, 16
variables controladas, 98
verificación, 167
    formal, 177, 249
    frente a pruebas, 32
    independiente, 179
verificación composicional, 183

YAGNI, 10